

АлтГТУ
ДИПЛОМНЫЙ ПРОЕКТ
группа ИСП - 21
специальность СПО 09.02.07
«Информационные системы и
программирование»
Котиков Егор Владиславович
2026 г.

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет
им. И.И. Ползунова»

Университетский технологический колледж

УДК 004

ДИПЛОМНЫЙ ПРОЕКТ
ДП 09.02.07.09

Разработка мобильного приложения для университетских точек питания

Пояснительная записка

Студент группы 1ИСП-21

Котиков Егор Владиславович

Руководитель проекта

преподаватель каф. ИСЭ

К. В. Воробьёв

Барнаул 2026

Реферат

Дипломный проект состоит из введения, трех глав, заключения и списка использованных источников литературы, выполнена 65 листах машинописного текста, содержит 24 рисунка, 22 таблицы, 2 источника информации.

Перечень ключевых слов: автоматизированная информационная система, веб-приложение, онлайн-тестирование, информационная система.

Объект исследования: процесс организации онлайн-тестирования.

Для реализации АИС выбрано собственное приложение.

В рамках данного проекта была проведена исследовательская работа, включающая анализ предметной области и существующих аналогов, выявление целевой аудитории, разработку модели бизнес-процессов в нотации IDEF0, проектирование структуры базы данных, выбор и обоснование стека технологий, реализацию веб-приложения, а также тестирование и оценку эффективности.

					ДП 09.02.07.09.000ПЗ				
Изм.	Лист	№ докум.	Подпись	Дат					
Разраб.		Котилов Е. В.			Разработка веб-сайта для создания и прохождения онлайн-тестов	Лит.	Лист	Листов	
Провер.		Воробьев С. В.				У	2	65	
Реценз.						УТК, 1ИСП-21			
Н. контр.		Воробьев С. В.							
Утверд.		Авдеев А.С.							

Содержание

ВВЕДЕНИЕ.....	5
1. АНАЛИТИЧЕСКАЯ ЧАСТЬ.....	6
1.1. Анализ предметной области.....	6
1.2. Предпроектное обследование объекта.....	7
1.2.1. Анализ целевой аудитории.....	7
1.2.2. Портреты потенциальных пользователей.....	10
1.2.3. Как сейчас решается задача веб-приложения.....	15
1.3. Анализ существующих решений.....	16
1.3.1. Обзор аналогов.....	16
1.3.2. Сравнительная таблица аналогов.....	19
1.3.3. Вывод по анализу существующих решений.....	20
2. Проектная часть.....	22
2.1. Описание процесса разработки модели предмета (объекта) исследования	22
2.1.1. Контекстная диаграмма IDEF0.....	22
2.1.2. Декомпозиция процесса в IDEF0.....	23
2.2. Требования к информационной системе.....	25
2.2.1. Функциональные требования.....	25
2.3. Проектирование баз данных.....	27
2.3.1. Концепция информационной базы.....	27
2.4. Построение функционально-алгоритмической структуры.....	29
2.4.1. Алгоритм аутентификации и авторизации пользователя.....	30
2.4.2. Алгоритм обработки заявок в друзья.....	31
2.4.3. Алгоритм прохождения теста.....	32
2.4.4. Алгоритм создания теста из пула вопросов.....	33
2.5. Проектирование пользовательского интерфейса.....	34
2.5.1. Главные экраны десктоп.....	34
2.5.2. Главные экраны мобильная версия.....	37
2.6. Выбор технологий.....	41
2.6.1. Требования к программно-аппаратной платформе.....	41

2.6.2. Описание основных технологий.....	42
2.6.3. Обоснование выбора стека технологий.....	44
3. ТЕХНИКО-РАБОЧЕЕ ПРОЕКТИРОВАНИЕ.....	46
3.1. Описание реализации.....	46
3.1.1. Архитектура приложения.....	46
3.1.2. Руководство пользователя.....	51
3.2. Тестирование.....	65
3.2.1. Порядок проведения тестирования.....	65
3.2.2. Результат тестирования.....	67
ЗАКЛЮЧЕНИЕ.....	68
СПИСОК ЛИТЕРАТУРЫ.....	70
ПРИЛОЖЕНИЕ А.....	71

Определения, обозначения и сокращения

АИС – автоматизированная информационная система;

API – Application Programming Interface (программный интерфейс приложения);

БД – база данных;

DRF – Django REST Framework;

HTML – HyperText Markup Language (язык гипертекстовой разметки);

HTTP – HyperText Transfer Protocol (протокол передачи гипертекста);

IDE – Integrated Development Environment (среда разработки);

JSON – JavaScript Object Notation (текстовый формат обмена данными);

JWT – JSON Web Token (веб-токен в формате JSON);

OS – Operating System (операционная система);

REST – Representational State Transfer (архитектурный стиль взаимодействия компонентов);

SQL – Structured Query Language (язык структурированных запросов).

Введение

В последние годы растёт спрос на инструменты дистанционного контроля знаний, самопроверки и интерактивного обучения. Одновременно широкое распространение получили неформальные тесты — психологические, профориентационные, развлекательные. Рынок онлайн-тестирования сегодня разделён на два сегмента: образовательный и развлекательный. Между ними практически нет пересечения. Пользователи вынуждены выбирать между сложными образовательными платформами и развлекательными сайтами без аналитики и модерации.

Проблема - отсутствие единой платформы, совмещающей образовательный и развлекательный сегменты, с удобными инструментами создания тестов, сохранением истории прохождений и встроенной модерацией.

Объект исследования - процесс организации онлайн-тестирования (создание тестов, прохождение, статистика, модерация).

Предмет исследования - методы и средства автоматизации этого процесса с помощью веб-приложения.

Цель работы - разработка веб-приложения «Система создания и прохождения онлайн-тестов» (ССИПОТ), объединяющего образовательный и развлекательный контент.

Задачи:

1. Провести анализ предметной области и существующих аналогов.
2. Выявить целевую аудиторию и сформулировать требования к системе.
3. Разработать модель бизнес-процессов (IDEF0).
4. Спроектировать структуру базы данных.
5. Выбрать и обосновать стек технологий.
6. Реализовать веб-приложение.
7. Провести тестирование и оценить эффективность.

1 АНАЛИТИЧЕСКАЯ ЧАСТЬ

1.1 Анализ предметной области

Разрабатываемая система относится к отрасли онлайн-образования (EdTech) и смежным развлекательным сегментам (квизы, викторины, досуг). В последние годы наблюдается активный рост спроса на инструменты дистанционного контроля знаний, самопроверки и интерактивного обучения. Одновременно с этим широкое распространение получили неформальные тесты — психологические, профориентационные, развлекательные - размещаемые на сайтах и в соцсетях.

Типовые проблемы и задачи в предметной области:

1 Разделение на «образовательные» и «развлекательные сегменты»

На текущий момент рынок онлайн-тестов сегментирован на два типа, между которыми почти нет пересечения:

- Образовательные системы онлайн-тестирования (веб-сайты созданные исключительно для образовательных целей, проверки знаний)
- Развлекательные системы онлайн-тестирования (веб-сайты созданные исключительно для развлекательных целей)

2 Отсутствие гибридных решений

На данный момент не существует популярных платформ которые совмещали бы в себе как сегмент развлечений, так и образовательный сегмент.

3 Высокий порог входа для создания теста

В условно-простых сервисах онлайн-тестирования, инструменты для разработки онлайн-тестов обычно оказываются перегруженными, либо урезанными.

Таким образом, выявленная проблема - отсутствие единой платформы, которая совмещающую в себе разные сегменты отраслей онлайн-тестирования. Пользователи вынуждены выбирать.

Целью разработки ССИПОТ «система создания и прохождения онлайн-тестов» является создания удобного веб-сайта совмещающего в себе удобные инструменты для создания онлайн тестов любых типов, удобную систему модерации, и объединяющую контент разных сегментов отрасли.

1.2 Предпроектное обследование объекта

1.2.1 Анализ целевой аудитории

Система ориентирована на широкий рынок: любые пользователи интернета, желающие создать или пройти тест. Целевая аудитория делится на три группы: авторы тестов, проходящие тесты, модераторы (см. таблица 1).

Таблица 1 — Группы целевой аудитории.

Группа	Роль в системе	Краткое описание
Авторы тестов	Авторы тестов	Пользователи, создающие контент на платформе
Проходящие тесты	Проходят готовые тесты	Основная аудитория, ради которой разрабатывается система
Модераторы	Следят за порядком	Пользователи обладающие особыми правами модерации, следящие за порядком платформы

Ниже каждая группа рассмотрена подробно.

Группа 1. Авторы тестов.

Авторы тестов - это пользователи, которые создают контент для платформы. Без них система будет пустой, поэтому важно сделать процесс создания тестов максимально удобным и понятным.

Авторов тестов можно разделить на две подгруппы (см. таблица 1.1).

Таблица 1.1 - Подгруппы авторов тестов

Подгруппа	Кому относится	Цель создания тестов
Образовательные авторы	Учителя, преподаватели, HR-специалисты	Проверка знаний, опросы, тестирования
Развлекательные	Обычные	Создание викторин, тестов «о себе»,

авторы	пользователи	опросы, опросы
---------------	--------------	----------------

Характеристика: возраст 16–55 лет, владение ПК от базового до продвинутого. Мотивация: обратная связь, статистика. Боли: перегруженные редакторы, отсутствие статистики, сложность публикации.

Группа 2. Пользователи проходящие тесты.

Пользователей проходящих тесты можно разделить на четыре подгруппы (см. таблица 1.2).

Таблица 1.2 - Подгруппы пользователей проходящих тесты

Подгруппа	Кому относится	Цель прохождения тестов
Учащиеся	Школьники, студенты	Подготовка к экзаменам, самопроверка
Любознательные	Обычные пользователи	Узнать что-то новое, проверить эрудицию
Ищущие досуг	Обычные пользователи	Пройти викторину, психологический тест, опрос и т. п.
Профессиональные	Сотрудники компаний, HR	Оценка знаний сотрудников, тестирование при приеме на работу

Характеристика: возраст 12–60 лет, владение ПК от начального. Мотивация: проверить себя, получить результат. Боли: обязательная регистрация, потеря результата, непонятный интерфейс, реклама.

Группа 3. Модераторы

Возраст 18–65 лет, необходимые навыки: внимательность, знание правил. Основные задачи: просмотр жалоб, блокировка некачественных тестов, удаление оскорбительных комментариев.

1.2.2 Портреты потенциальных пользователей

Для более детального понимания потребностей были разработаны портреты (персонажи) типичных представителей каждой группы.

Портрет 1. Автор образовательных тестов - «Анна, 34 года, учитель математики» (см. таблица 2).

Таблица 2. - Потрет автора образовательных тестов

Параметр	Значение
Возраст	34 года
Род занятий	учитель математики
Цель использование системы	Создавать проверочные тесты для учеников, отслеживать результаты каждого
Что важно	Простота создания теста, детальная статистика по каждому ученику, возможность объяснить ошибку
Что не устраивает в существующих решениях	Сложные интерфейсы (Moodle), отсутствие статистики по вопросам (Google Forms), платная подписка
Что ожидается от новой системы	Быстрый редактор, автоматический подсчет оценок, отчет по каждому ученику

Портрет 2. Автор развлекательных тестов - «Владимир, 19 лет, обычный пользователь» (см. таблица 2.1).

Таблица 2.1 - Портрет автора развлекательных тестов

Параметр	Значение
Возраст	19 лет
Род занятий	Студент, использующий сайт для создания тестов в свободное время
Цель использование системы	Создавать интересные тесты (например, «Твой любимый персонаж сериала?»)
Что важно	Простой, быстрый инструмент для создания теста, простая публикация, возможность поделиться ссылкой, комментарии к тесту
Что не устраивает в существующих решениях	На многих сайтах нужно платить за расширенный функционал, много рекламы
Что ожидается от новой системы	Бесплатный удобный инструмент для быстрого создания онлайн-тестов с использованием пула вопросов

Портрет 3. Проходящий тест (учащийся) - «Дмитрий, 16 лет, школьник» (см. таблица 2.2).

Таблица 2.2 - Портрет проходящего тест пользователя (учащийся)

Параметр	Значение
Возраст	16 лет
Род занятий	Ученик 10 класса
Цель использование системы	Готовиться к предстоящей олимпиаде

	по выбранной теме, проходить тесты которые для самопроверки
Что важно	Быстро найти нужный тест, пройти без лишних сложностей, увидеть свой результат и ошибки
Что не устраивает в существующих решениях	Нужно регистрироваться на каждом сайте, результат не сохраняется, если вышел из браузера
Что ожидается от новой системы	Можно начать без регистрации, потом зарегистрироваться и всё сохранится

Портрет 4. Проходящий тест (обычный пользователь) - «Екатерина, 25 лет, офисный работник» (см. таблица 2.3).

Таблица 2.3 - Портрет обычного пользователя проходящего тест

Параметр	Значение
Возраст	25 лет
Род занятий	Менеджер в небольшой компании
Цель использование системы	В свободное время интересуется историей и смотрит сериалы, в перерывах на работе любит проходить исторические тесты и опросы на темы сериалов
Что важно	Красивый дизайн, быстрая загрузка, удобство на телефоне (адаптив),

	быстрый доступ к тесту
Что не устраивает в существующих решениях	Навязчивая реклама, куча всплывающих окон, тесты плохо открываются на телефоне
Что ожидается от новой системы	Чистый интерфейс, работает на телефоне, быстрый доступ к интересующей информации

Портрет 5. Модератор - «Владимир, 20 лет, модератор сайта» (см. таблица 2.4).

Таблица 2.4 - Портрет модератора сайта

Параметр	Значение
Возраст	20 лет
Род занятий	Модератор сайта
Цель использование системы	Быстро обрабатывать жалобы, блокировать оскорбительные комментарии и тесты
Что важно	Удобная админ-панель
Что не устраивает в существующих решениях	Жалобы приходят в общий список, нет сортировки, много кликов для одного действия
Что ожидается от новой системы	Единая панель с жалобами

1.2.3 Как сейчас решается задача веб-приложения

Единой платформы, совмещающей образовательный и развлекательный сегменты, не существует. Пользователи используют несколько инструментов, каждый из которых решает лишь часть задачи:

- Google Forms, Яндекс.Формы - создание тестов для проверки знаний, но нет детальной статистики.
- Pikuco, Банк Тестов - создание развлекательных тестов, но нет аналитики для автора.
- Любые открытые сайты - прохождение без регистрации, но результат не сохраняется.
- Скриншоты, блокнот - сохранение истории прохождений, но неудобно и данные теряются.
- Письмо в техподдержку - жалобы на контент, но долгая обработка.
- Платные системы - получение статистики по тесту, но дорого и сложно.

Таким образом, пользователи вынуждены выбирать между образовательными платформами и развлекательными, что плохо в вопросе о удобстве. Разрабатываемая система призвана объединить лучшие черты обоих подходов.

1.3 Анализ существующих решений

1.3.1 Обзор аналогов

1. Pikuco (<https://pikuco.ru>)

Назначение: развлекательное. Сайт предлагает множество тестов-викторин, психологических тестов, опросов «для себя».

Сильные стороны:

- Удобный и современный интерфейс.
- Большая библиотека готовых тестов.

- Хорошо работает на мобильных устройствах.

Слабые стороны:

- Полное отсутствие образовательной составляющей.
- Нет возможности для авторов получить статистику по прохождением.
- Нет системы модерации комментариев и жалоб.

2. OnlineTestPad (<https://onlinetestpad.com/ru/tests>)

Назначение: Универсальная платформа с уклоном в образование. Позволяет создавать тесты, опросы, кроссворды.

Сильные стороны:

- Большой набор инструментов для создания тестов.
- Возможность настройки сложности, веса вопросов, таймера.
- Подходит для школ и вузов.

Слабые стороны:

- Интерфейс перегружен, много элементов на странице.
- Новичку сложно разобраться в настройках.
- Дизайн устаревший, неадаптивный (плохо на телефонах).

3. Oltest (<https://oltest.ru>)

Назначение: Образовательная платформа с большим количеством тестов по разным дисциплинам.

Сильные стороны:

- Огромная база тестов (1465 тестов, более 210 000 вопросов).
- Возможность случайной выборки вопросов при каждом прохождении.
- Можно скачать вопросы с ответами.

Слабые стороны:

- Визуально устаревший дизайн (напоминает сайты 2010-х годов).
- Неудобная навигация.
- Нет современного адаптивного дизайна.
- Нет системы комментариев и модерации.

4. Google Forms (<https://forms.google.com>)

Назначение: Инструмент для создания опросов и форм, часто используется для тестирования.

Сильные стороны:

- Полностью бесплатный.
- Простой и понятный интерфейс создания форм.
- Интеграция с Google Таблицами (можно собирать ответы).
- Достаточно хорошая статистика (диаграммы, средние значения).

Слабые стороны:

- Обязательная регистрация для создания тестов.
- Для прохождения теста Google Forms требует входа в аккаунт (или нужно включать специальный режим).
- Нет системы ролей (модератор, автор, пользователь).
- Нет комментариев и системы жалоб.
- Скучный дизайн, нет игровых элементов.

5. Wayground (<https://wayground.com/?lng=ru>)

Назначение: Образовательная платформа (бывшая Quizizz), ориентирована на школы и учителей.

Сильные стороны:

- Современный интерфейс.

- Мощный AI-инструментарий для создания материалов.
- Интеграция с учебными программами.
- Хорошая статистика и аналитика для учителей.

Слабые стороны:

- Обязательная регистрация для использования.
- Полностью ориентирована на образование - нет развлекательных тестов.
- Платформа платная (есть бесплатный тариф с ограничениями).
- Не подходит для обычных пользователей, которые хотят просто пройти викторину.

1.3.2 Сравнительная таблица аналогов

Сравнительная таблица аналогов (см. таблица 3)

Таблица 3 - Сравнительная таблица аналогов

Критерий	Pikuco	OnlineTestPad	Oltest	Google Forms	Wayground	ССИПОТ (проект)
Назначение	Развлечение	Образование	Образование	Опросы	Образование	Гибрид
Регистрация для создания теста	Нет	Да	Да	Да	Да	Нет
Регистрация для прохождения	Нет	Необязательно	Нет	Да	Да	Необязательно
Сохранение истории для	Нет	Нет	Нет	Нет	Нет	Да

гостей						
Статистика для автора	Нет	Базовая	Нет	Средняя	Детальная	Детальная
				я		
Система жалоб (репорты)	Да	Нет	Да	Нет	Нет	Да
Модерация и мут	Да	Нет	Да	Да	Да	Да
Образовательные тесты	Нет	Да	Да	Частично	Да	Да
				но		
Развлекательные тесты	Да	Частично	Нет	Нет	Нет	Да

1.3.3 Вывод по анализу существующих решений

Проведенный анализ позволяет сделать следующие выводы:

1. Ни один из рассмотренных аналогов не совмещает образовательный и развлекательный сегменты. Pikuso — только развлечение. OnlineTestPad, Oltest, Wayground — только образование. Google Forms — нейтральный опросник без специализации.
2. Ни одна платформа не сохраняет историю прохождений для неавторизованных пользователей. Гость всегда теряет свой результат после закрытия браузера.
3. Система модерации (жалобы, репорты, мут) отсутствует во всех рассмотренных аналогах. Комментарии либо отсутствуют, либо не модерируются.
4. Пользователям приходится выбирать: либо удобный интерфейс (Pikuso), либо функциональность (OnlineTestPad), либо статистику (Wayground, Google Forms).

Таким образом, ССИПОТ «система создания и прохождения онлайн-тестов» закрывает выявленные пробелы рынка:

- гибридный подход (образование + развлечение).
- встроенная система модерации (репорты + мут).
- детальная статистика для авторов.
- простой и современный интерфейс.

2 Проектная часть

2.1 Описание процесса разработки модели предмета (объекта) исследования

Объектом исследования является процесс организации онлайн-тестирования, включающий создание тестов, их прохождение пользователями, накопление статистики и модерацию контента. Предмет исследования - методы и средства автоматизации этого процесса с помощью веб-приложения.

Для формализации предметной области проведено моделирование бизнес-процессов. В качестве основной нотации выбрана IDEF0, позволяющая представить процесс на верхнем уровне абстракции с выделением входов, выходов, управляющих воздействий и механизмов.

2.1.1 Контекстная диаграмма IDEF0

На контекстной диаграмме (см. рисунок 1) основным бизнес-процессом является «Управление процессом онлайн-тестирования».

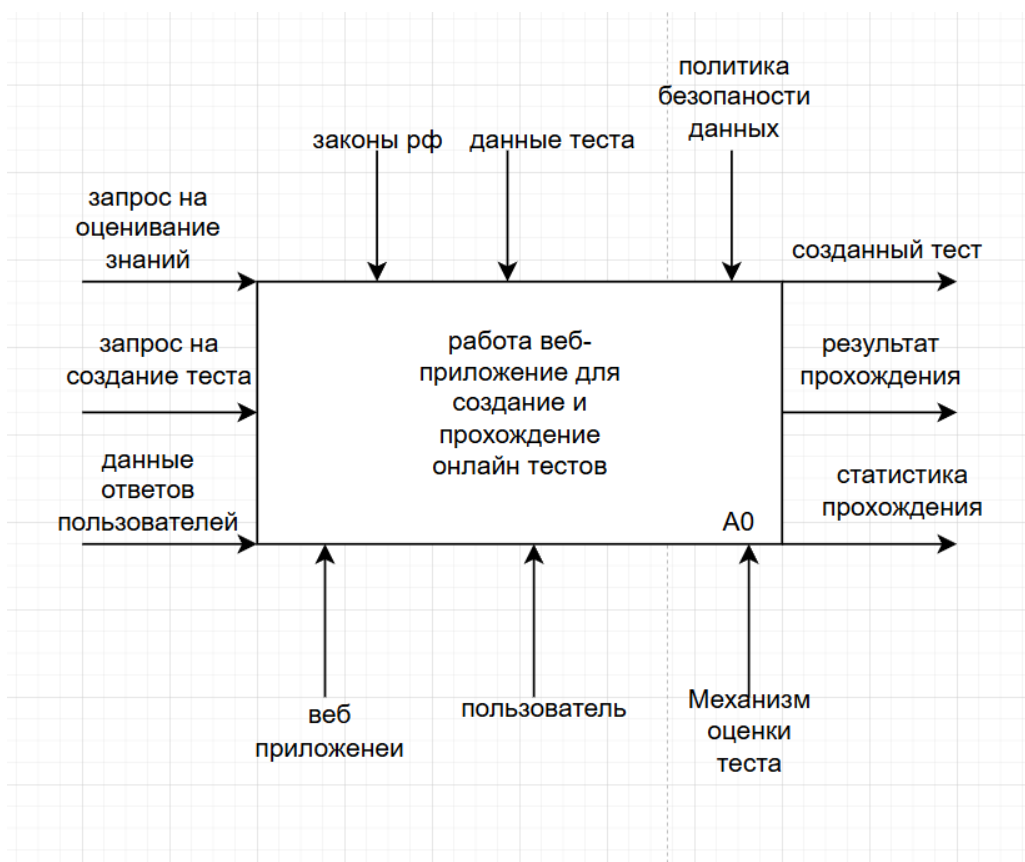


Рисунок 1 - IDEF0 диаграмма

Входные потоки: запрос на оценивание знаний, запрос на создание теста, данные ответов пользователей.

Управляющие потоки: законы рф, данные теста, политика безопасности данных.

Механизмы: веб приложение, пользователь, механизм оценки теста.

Выходные потоки: созданный тест, результат прохождения, статистика прохождения.

2.1.2 Декомпозиция процесса в IDEF0

Контекстная диаграмма декомпозируется на четыре дочерних процесса (см. рисунок 2). Декомпозиция выполнена по принципу разделения основных функциональных областей системы.

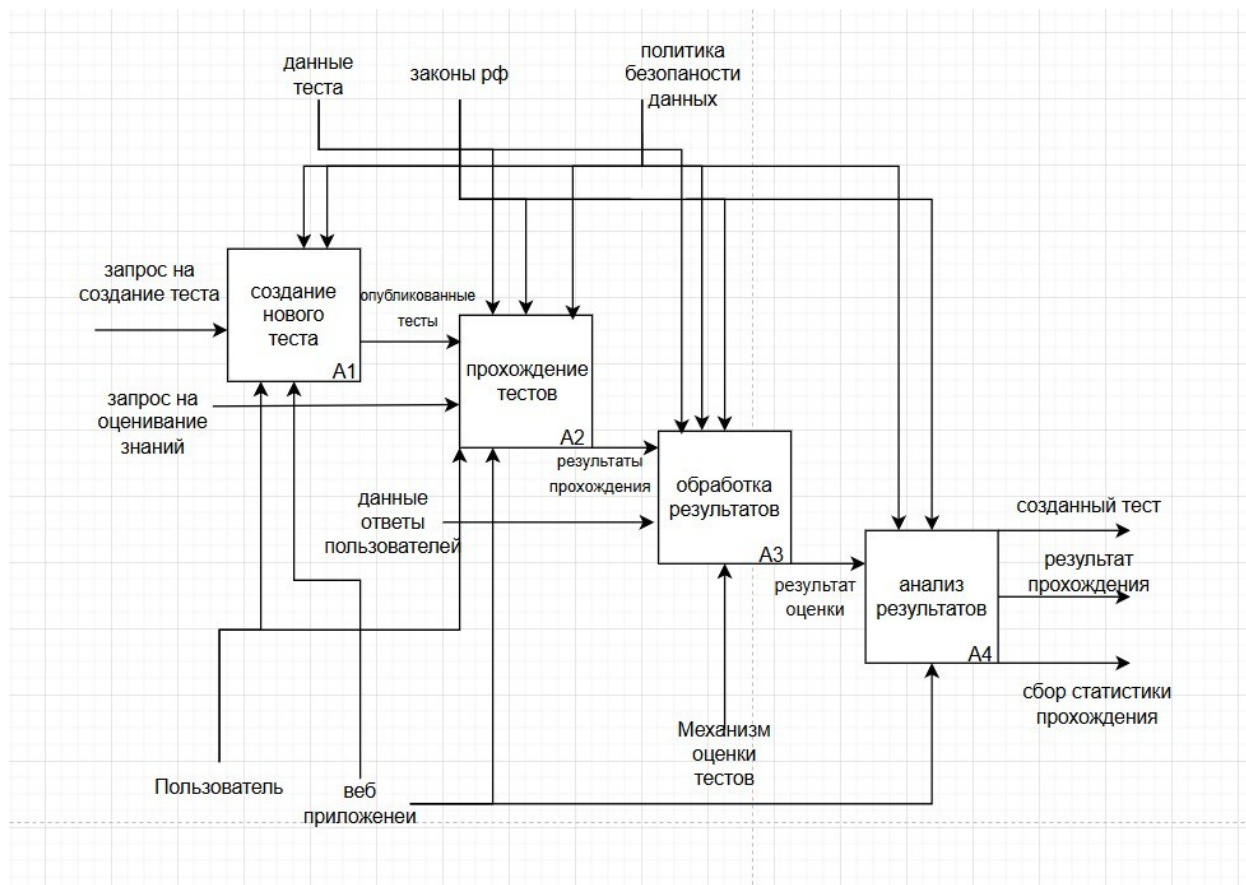


Рисунок 2 - Декомпозиция основных функциональных областей системы

Процесс 1. Создание нового теста

Обрабатывает входные потоки «запрос на создание тестов».

Управляется «законы РФ» и «политика безопасности данных».

Механизмы: веб-приложение, пользователь.

Выход: опубликованные тесты.

Процесс 2. Прохождение тестов

Обрабатывает входной поток «запрос на оценивание знаний».

Управляется «данными теста», «политикой безопасности», «законы рф».

Механизмы: веб приложение, пользователь.

Выход: «результат прохождения».

Процесс 3. Обработка результатов

Обрабатывает входной поток «результат прохождения», «данные ответов пользователей».

Управляется «данными теста», «политикой безопасности», «законы рф».

Механизмы: «механизм оценки теста».

Выход: «результат оценки».

Процесс 4. Анализ результатов

Обрабатывает входные потоки: «результат оценки».

Управляется «законами РФ» и «политикой безопасности».

Механизмы: веб-приложение.

Выход: созданный тест, результат прохождения, сбор статистики.

2.2 Требования к информационной системе

2.2.1 Функциональные требования

Функциональные требования сформулированы в виде пользовательских истории (User Stories) для каждой роли.

Роль 1. Гость (см. таблица 5)

Таблица 5 - Use Stories роль - гость

ID	User Story
US-1.1	Как гость, я хочу просматривать каталог всех доступных тестов, видеть их краткое описание (количество вопросов, автор, рейтинг), видеть комментарии, чтобы найти интересную тему для самопроверки или развлечения.

US-1.2	Как гость, я хочу пройти любой понравившийся тест и сразу увидеть свой результат (количество правильных ответов), чтобы оценить свои знания без необходимости регистрироваться.
US-1.3	Как гость, я хочу видеть, что моя статистика прохождений не сохраняется, и мне предлагают зарегистрироваться, чтобы вести историю, создавать свои тесты, оставлять комментарии, чтобы принять решение о создании аккаунта.

Роль 2. Авторизированный пользователь (см. таблица 5.1)

Таблица 5.1 - Use Stories роль - авторизированный пользователь

ID	User Story
US-2.1	Как авторизованный пользователь, я хочу создать новый тест: добавить название, описание, вопросы и варианты ответов (с указанием правильных), указать результат прохождения, чтобы поделиться своей подборкой вопросов с другими людьми.
US-2.2	Как автор теста, я хочу зайти в личный кабинет и увидеть общую статистику по моим тестам: сколько человек прошло, средний балл, какие ответы чаще всего выбирались, чтобы понимать, насколько тест интересен и сложен для аудитории.
US-2.3	Как участник, я хочу в своём профиле видеть историю всех пройденных мной тестов (какой тест, когда прошёл, мой результат), чтобы отслеживать свой прогресс в обучении.
US-2.4	Как авторизованный пользователь, я хочу оставить комментарий под чужим тестом, чтобы взаимодействовать с автором и другими участниками сообщества.

Роль 3. Модератор (см. таблица 5.2)

Таблица 5.2 - Use Stories роль - модератор

ID	User Story
US-3.1	Как модератор, я хочу видеть отдельную ленту или список «Репорты» / «Новые тесты», чтобы оперативно проверять, нет ли в тестах оскорбительного или неприемлемого контента.
US-3.2	Как модератор, я хочу иметь возможность скрыть или полностью удалить тест, который нарушает правила, блокировать доступ к комментариям для неадекватных пользователей, чтобы поддерживать качество контента и безопасную среду.
US-3.3	Как модератор, я хочу при удалении теста отправить автору уведомление с причиной блокировки, чтобы пользователь понимал, за что его контент был удалён.

2.3 Проектирование баз данных

2.3.1 Концепция информационной базы

Проектирование базы данных выполнено в соответствии с требованиями к структуре и функционированию системы. Выделены следующие подсистемы, каждая из которых опирается на соответствующие таблицы БД (см. таблица 6).

Таблица 6 — Концепция информационной базы данных

Подсистема	Таблицы в бд
Аутентификации и авторизации	Users, restrictions
Управления тестами	Tests, pools, pool_questions, questions, answers, test_questions

Прохождения тестов	Results, user_answers
Статистики и отчётов	test_statistics
Комментариев и обратной связи	Comments, reports
Модерации	Reports, restrictions
Административного управления	Users, test_statistics

Основные сущности информационной базы:

1. Пользователь (users) - хранит учётные записи всех пользователей системы с указанием их ролей
2. Вопрос (questions) - единое хранилище всех вопросов системы. Каждый вопрос имеет автора, текст, уровень сложности, пояснение. Неважно, используется ли вопрос в пуле или в тесте — он хранится здесь один раз.
3. Вариант ответа (answers) - варианты ответов для каждого вопроса. Связь с таблицей questions.
4. Пул (pools) - личная библиотека вопросов, которую создаёт пользователь. Пул — это контейнер, который группирует вопросы (связь многие ко многим через таблицу pool_questions). Один вопрос может принадлежать нескольким пулам одного пользователя. Пул принадлежит конкретному пользователю.
5. Тест (tests) - тест, создаваемый пользователем. Тест может включать любые вопросы: как из пулов автора, так и вопросы, не добавленные ни в один пул.
6. Связь теста с вопросами (test_questions) - определяет, какие вопросы входят в тест, их порядок и вес в баллах.
7. Результат прохождения (results) - фиксирует факт прохождения теста пользователем.

8. Ответ пользователя (user_answers) - сохраняет конкретный выбор пользователя на каждый вопрос.
9. Статистика тестов (test_statistics) - таблица с агрегированными показателями по каждому тесту.
10. Комментарий (comments) - обсуждение тестов.
11. Репорт (reports) - жалобы на контент.
12. Ограничение (restrictions) - блокировки пользователей.

2.4 Построение функционально-алгоритмической структуры

В разрабатываемой системе можно выделить четыре ключевых алгоритмических блока, реализующих основную функциональность: аутентификация и авторизация пользователей, управление социальными связями, создание и прохождение тестов, а также обработка результатов. Ниже представлено подробное описание каждого алгоритма.

2.4.1 Алгоритм аутентификации и авторизации пользователя

Данный алгоритм отвечает за идентификацию пользователя, выдачу токенов доступа и управление сессией. Реализован с использованием Django REST Framework и JWT-токенов (Simple JWT).

Входные данные: логин, пароль (для входа); логин, email, пароль, согласие на обработку данных (для регистрации).

Выходные данные: access-токен, refresh-токен, данные пользователя.

Последовательность шагов при регистрации:

1. Приём от клиента логина, email, пароля (с подтверждением) и флага согласия на обработку персональных данных.
2. Валидация входных данных: проверка совпадения пароля и подтверждения (password == password2), длины пароля (не менее 6 символов), наличия согласия на обработку персональных данных.
3. Нормализация email и хеширование пароля с использованием встроенного механизма Django (set_password).
4. Сохранение объекта User в базу данных.
5. Возврат клиенту статуса 201 Created с данными созданного пользователя.

Последовательность шагов при входе:

1. Приём логина и пароля из запроса.
2. Проверка наличия обоих полей - при отсутствии любого из них возвращается ошибка 400 Bad Request.
3. Вызов встроенной функции `authenticate(login, password)`, которая проверяет соответствие учётных данных.
4. При успешной аутентификации - генерация `refresh`-токена и извлечение из него `access`-токена.
5. Формирование ответа, содержащего пару токенов и сериализованные данные пользователя. При неудаче возвращается ошибка 401 Unauthorized.

2.4.2 Алгоритм обработки заявок в друзья

Данный алгоритм реализует социальную функциональность системы. Ключевой особенностью является автоматическое принятие заявки при наличии встречного запроса (механизм «взаимной подписки»).

Входные данные: ID пользователя, которому отправляется заявка; текущий авторизованный пользователь.

Выходные данные: статус обработки заявки (отправлена/принята/ошибка).

Последовательность шагов:

1. Поиск целевого пользователя по переданному ID. Если пользователь не найден - возврат ошибки 404 Not Found.
2. Проверка, что пользователь не пытается добавить самого себя. При попытке - ошибка «Нельзя добавить самого себя».
3. Проверка наличия дружеской связи между пользователями. Если связь уже существует - ошибка «Вы уже друзья».
4. Поиск существующей исходящей заявки от текущего пользователя к целевому:
 - Если заявка в статусе «ожидает» (`pending`) - ошибка «Заявка уже отправлена».
 - Если заявка ранее была отклонена (`rejected`) - обновление статуса на `pending`, повторная отправка.

5. Поиск входящей заявки от целевого пользователя к текущему со статусом pending. Если такая заявка существует:
 - Статус заявки изменяется на accepted.
 - Пользователи добавляются в списки друзей друг друга (симметричная связь через ManyToManyField).
 - Возврат сообщения «Заявка принята автоматически».
6. Если ни одно из условий не сработало - создание новой заявки со статусом pending.

2.4.3 Алгоритм прохождения теста

Данный алгоритм является наиболее сложным с точки зрения бизнес-логики. Он обрабатывает процесс прохождения теста от старта до вычисления итогового результата. Поддерживает как авторизованных, так и анонимных пользователей.

Входные данные: ID теста; для каждого вопроса — выбранный ответ (вариант или текст).

Выходные данные: результат прохождения (процент правильных ответов, статус «пройден/не пройден»).

Шаг 1 - начало теста (endpoint: /start-test):

1. Поиск объекта Test по переданному test_id. При отсутствии - ошибка 404.
2. Проверка приватности теста. Если тест закрытый (is_open = False):
 - Проверка, является ли текущий пользователь автором теста.
 - Проверка наличия валидного access_token в запросе.
 - Проверка наличия незавершённой попытки у пользователя.
 - При невыполнении любого условия - ошибка 403 Forbidden.
3. Проверка лимита попыток. Если attempts_limit > 0 и количество завершённых попыток пользователя достигло лимита - ошибка.
4. Создание объекта TestAttempt с начальными значениями: score = 0, is_passed = False, answers = {} (пустой JSON). Время начала started_at заполняется автоматически.
5. Возврат клиенту attempt_id, количества вопросов, ограничения по времени и флага is_survey.

Шаг 2 - отправка ответа на вопрос (endpoint: /submit-answer):

1. Поиск попытки TestAttempt по attempt_id.
2. Поиск вопроса Question по question_id.
3. Определение правильности ответа в зависимости от типа вопроса:
 - Если тест является опросом (is_survey = True) - любой ответ считается правильным.
 - Если вопрос с текстовым вводом (is_text_input = True) - приведение строк к нижнему регистру, удаление пробелов по краям, сравнение с эталоном.
 - Если вопрос с вариантами ответов - поиск выбранного объекта Answer и проверка флага is_correct.
4. Сохранение ответа в JSON-поле answers попытки в формате: {«question_id»: {«answer»: «текст ответа», «is_correct»: true/false, «question_text»: «текст вопроса»}}.
5. Возврат клиенту результата проверки (правильно/неправильно).

Шаг 3 - завершение теста и расчёт результата (endpoint: /finish-test):

1. Поиск попытки TestAttempt по attempt_id.
2. Расчёт результата:
 - Для опроса: score = 0, is_passed = True.
 - Для теста: подсчёт количества правильных ответов, где is_correct = True; вычисление процента: $score = (correct_answers / total_questions) * 100$; сравнение с проходным баллом: $is_passed = score \geq passing_score$.
3. Обновление полей попытки: запись score, is_passed, finished_at = текущее время.
4. Увеличение счётчиков usage_count для каждого вопроса, использованного в тесте.
5. Возврат клиенту итогового балла, статуса прохождения и детальной статистики.

2.4.4 Алгоритм создания теста из пула вопросов

Данный алгоритм описывает процесс преобразования набора вопросов в готовый тест с заданными параметрами.

Входные данные: название, описание, список ID вопросов (минимум 4), настройки (тип - тест/опрос, лимит времени, проходной балл, количество попыток, признак открытости).

Выходные данные: созданный объект Test с присвоенным ID.

1. Последовательность шагов:
2. Приём данных от клиента.
3. Валидация входных данных:
 - Проверка, что количество вопросов не менее 4.
 - Для теста (не опроса) - проверка неотрицательного значения времени и валидности проходного балла (от 0 до 100).
4. Создание объекта Test:
 - Автоматическая установка author = текущий авторизованный пользователь.
 - Если тест закрытый (is_open = False) - генерация уникального access_token.
 - Для опроса - принудительная установка time_limit = 0 и passing_score = 0.
5. Привязка вопросов к тесту: для каждого question_id из переданного списка создаётся запись TestQuestion, где order_index определяется порядком следования в списке.
6. Возврат клиенту созданного объекта Test с полями id, title, questions_count, а также access_token (для закрытых тестов).

2.5 Проектирование пользовательского интерфейса

2.5.1 Главные экраны десктоп

Первым делом была подобрана основная цветовая палитра, выполненная в коричневатых стилях, для создания эффекта строгости, но не слишком (см. рисунок 3).

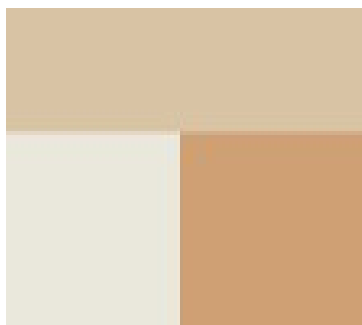


Рисунок 3 - Цветовая палитра

Далее используя модульную сетку, были разработаны макеты основных экранов (см. рис. 4 - 7):

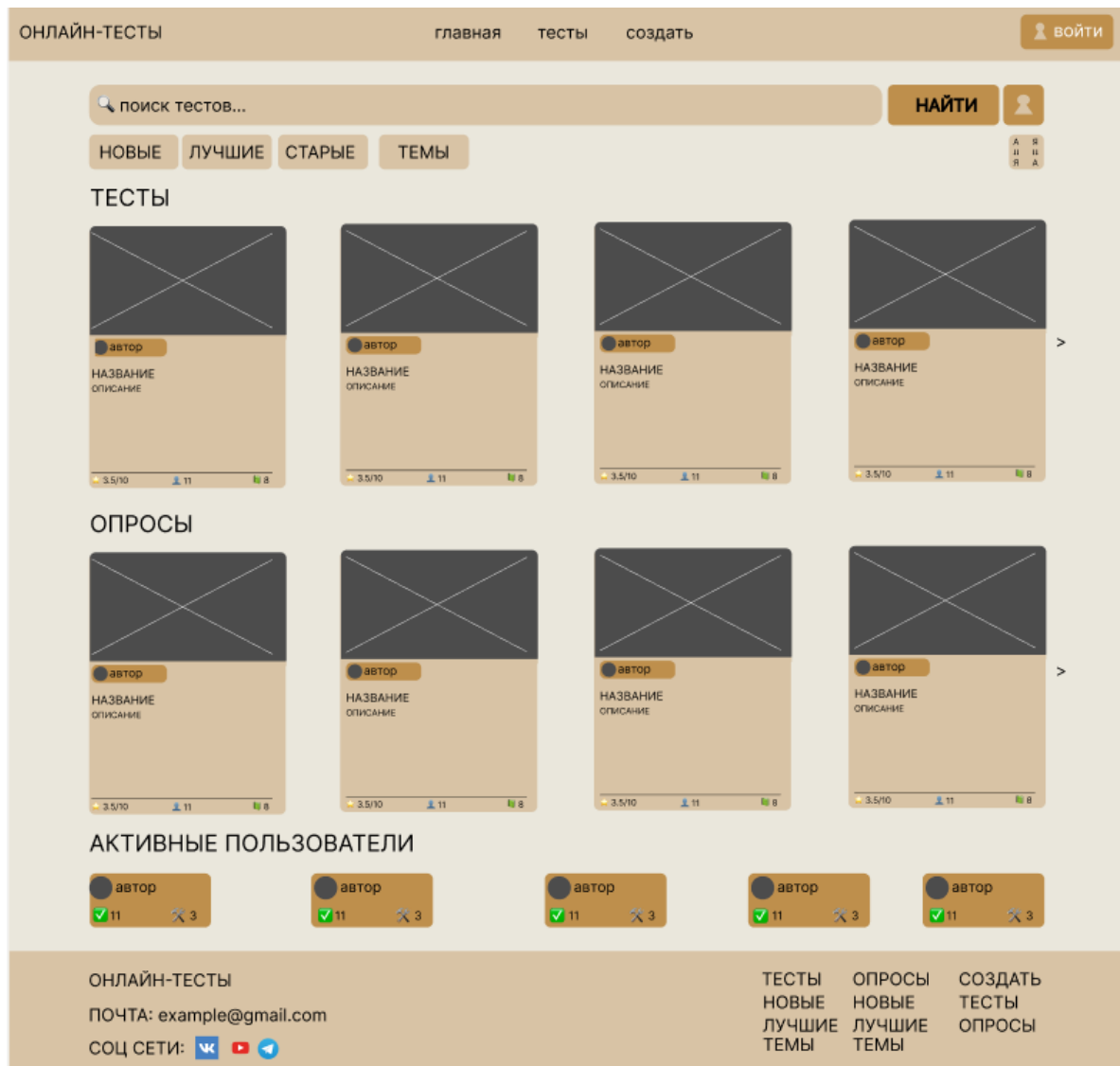


Рисунок 4 - Главная страница

ОНЛАЙН-ТЕСТЫ

главнаятестысоздать

войти

ДОБРО ПОЖАЛОВАТЬ

РЕГИСТРАЦИЯ

ЛОГИН

EMAIL

ПАРОЛЬ

ПОВТОР ПАРОЛЯ

☐ обработка персональных данных

ЗАРЕГИСТРИРОВАТЬСЯ

ВОЙТИ

ОНЛАЙН-ТЕСТЫ

ПОЧТА: example@gmail.com

СОЦ СЕТИ:   

ТЕСТЫ
НОВЫЕ
ЛУЧШИЕ
ТЕМЫ

ОПРОСЫ
НОВЫЕ
ЛУЧШИЕ
ТЕМЫ

СОЗДАТЬ
ТЕСТЫ
ОПРОСЫ

Рисунок 5 — Страница регистрация

ОНЛАЙН-ТЕСТЫ

главнаятестысоздать





имя пользователя
био

выйти

редактировать профиль

Дата регистрации: 01.01.2026 Последний заход: 01.01.2026 Друзей: 10

Статистика

Мои тесты

Друзья

СТАТИСТИКА

ТЕСТ	ТИП ТЕСТА	АВТОР	ВОПРОСОВ	РЕЗУЛЬТАТ	ДАТА
НАЗВАНИЕ	ТЕСТ	ИМЯ	19	82	01.01.2026
НАЗВАНИЕ	ОПРОС	ИМЯ	19	82	01.01.2026
НАЗВАНИЕ	ОПРОС	ИМЯ	19	82	01.01.2026

ОНЛАЙН-ТЕСТЫ

ПОЧТА: example@gmail.com

СОЦ СЕТИ:   

ТЕСТЫ
НОВЫЕ
ЛУЧШИЕ
ТЕМЫ

ОПРОСЫ
НОВЫЕ
ЛУЧШИЕ
ТЕМЫ

СОЗДАТЬ
ТЕСТЫ
ОПРОСЫ

Рисунок 6 — Страница профиля

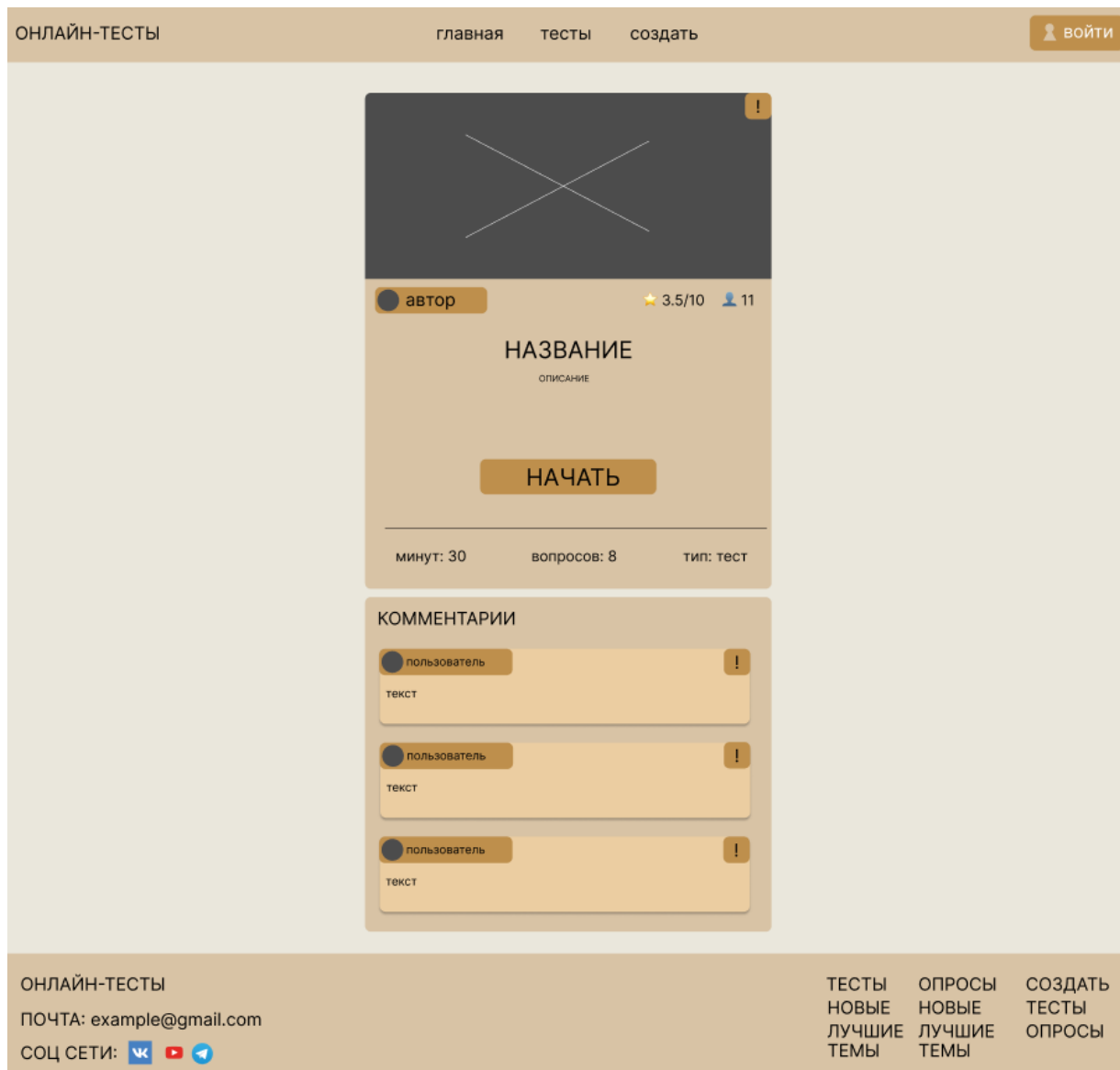


Рисунок 7 - Страница теста

2.5.2 Главные экраны мобильная версия

Была разработана мобильная адаптивная версия страниц (см. рис. 8 - 10):



Рисунок 8 - Страница теста

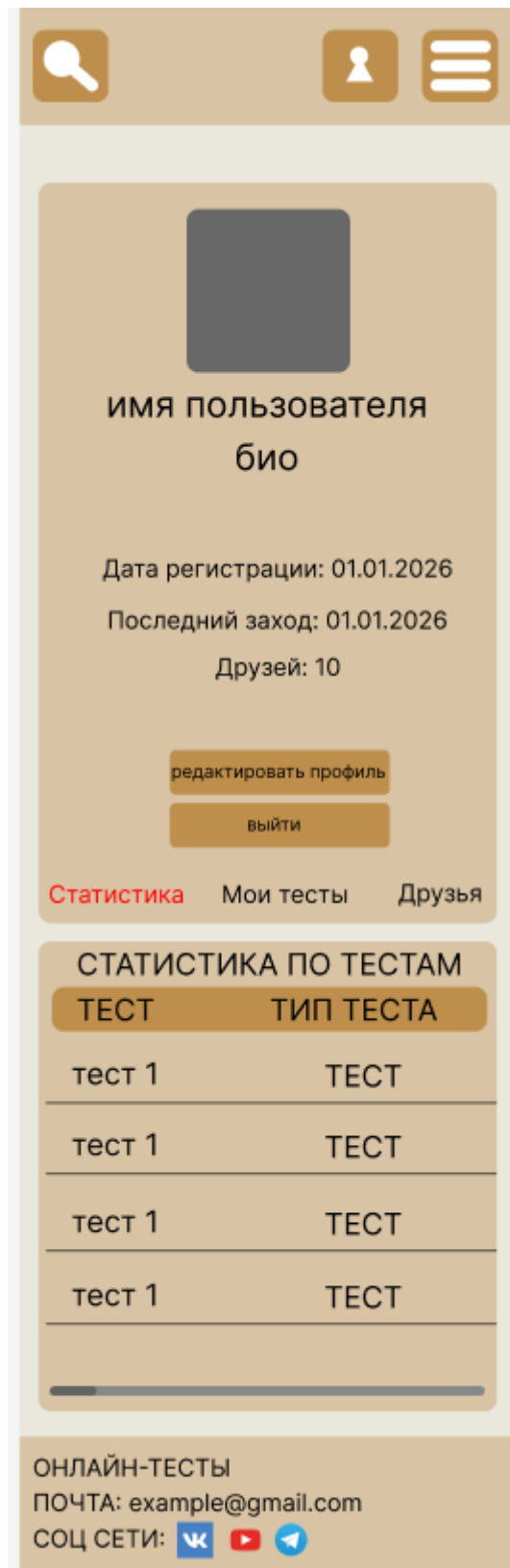


Рисунок 9 - Страница пользователя

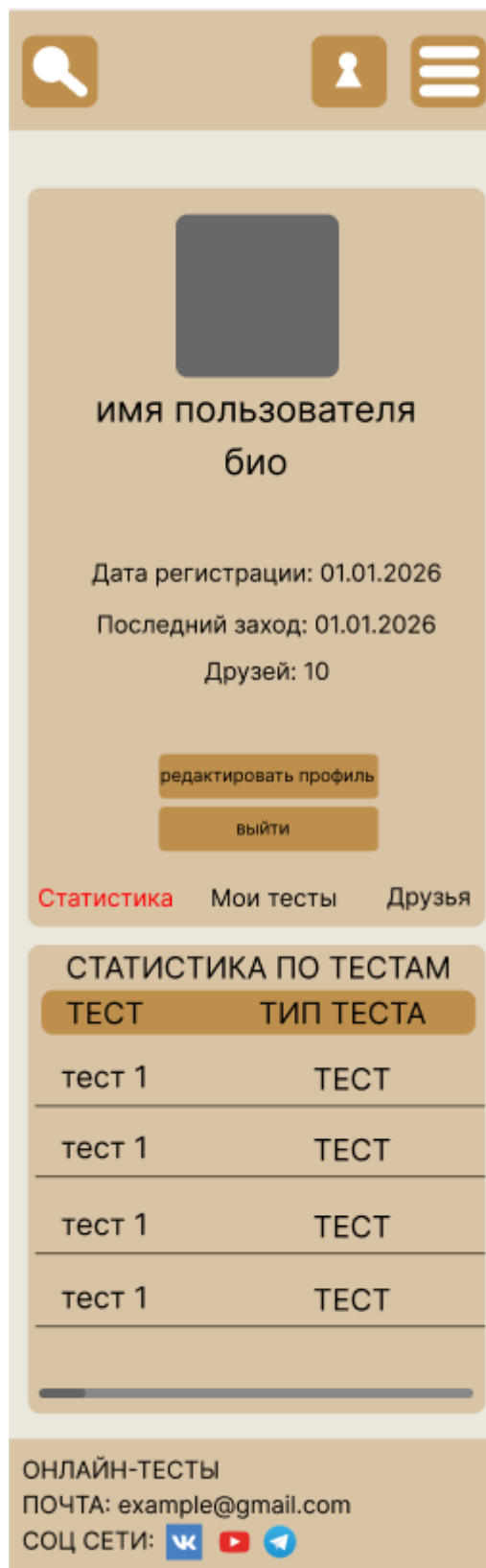


Рисунок 10 - Страница теста

2.6 Выбор технологий

2.6.1 Требования к программно-аппаратной платформе

Для пользователей (см. таблица 7).

Таблица 7 - Программно-аппаратная платформа для пользователей

Компонент	Требование	Пояснение
ОС	Любая (Windows, macOS, Linux, iOS, Android)	Веб-приложение через браузер
Браузер	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+	Современные браузеры
Разрешение	от 320px	Адаптивный дизайн
Интернет	от 1 Мбит/с	Для загрузки страниц

Для сервера (см. таблица 7.1).

Таблица 7.1 - Программно-аппаратная платформа для сервера

Компонент	Минимальные требования	Рекомендуемые требования
ОС	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS
CPU	2 vCPU	4 vCPU
RAM	2 ГБ	8 ГБ
Диск	1 ГБ SSD	50 ГБ SSD
ПО	Docker, Docker Compose,	Docker, Docker Compose, Nginx,

2.6.2 Описание основных технологий

Языки программирования (см. таблица 7.2).

Таблица 7.2 – Языки программирования

Технология	Версия	Назначение
Python	3.10+	Бэкенд (серверная логика)
TypeScript	5.0+	Фронтенд (типизированный JavaScript)

Фреемворки и библиотеки бекенд (см. таблица 7.3).

Таблица 7.3 – Фреемворки и библиотеки бекенд

Технология	Назначение
Django	Веб-фреймворк (ORM, аутентификация, админ-панель, миграции)
Django REST Framework (DRF)	Создание REST API (сериализаторы, viewset'ы, permissions)
djangorestframework-simplejwt	JWT-аутентификация

Фреемворки и библиотеки фронтенд (см. таблица 7.4).

Таблица 7.4 – Фреемворки и библиотеки фронтенд

Технология	Назначение
------------	------------

React	UI-компоненты, виртуальный DOM
React Router	Клиентская маршрутизация
Axios	HTTP-запросы к бэкенду (с перехватчиками для токенов)

Инструменты сборки и контейнеризации (см. таблица 7.5).

Таблица 7.5 – Инструменты сборки и контейнеризации

Технология	Назначение
PostgreSQL	Реляционная СУБД (ACID, внешние ключи, индексы, триггеры)

Фреймворки и библиотеки фронтенд (см. таблица 7.6).

Таблица 7.6 – Фреймворки и библиотеки бекенд

Технология	Назначение
Pip requirements.txt	+ Управление Python-зависимостями
Vite	Сборка фронтенда (быстрая горячая перезагрузка)
Docker	Контейнеризация (Django + PostgreSQL + Nginx)
Docker Compose	Оркестрация контейнеров

Контроль версий (см. таблица 7.7).

Таблица 7.7 – Контроль версий

Технология	Назначение
Git	Система контроля версий
GitHub	Хостинг репозитория

2.6.3 Обоснование выбора стека технологий

Почему Python и Django для бэкенда:

- Скорость разработки - Django предоставляет готовые компоненты: ORM, аутентификацию, админ-панель, миграции, защиту от CSRF/XSS.
- Соответствие требованиям безопасности - слепые пароли (хэширование), ролевая модель, защита от НСД.
- DRF для API - позволяет быстро создать REST API с минимальным количеством кода.
- Надёжность - Django используется в крупных проектах, имеет большое сообщество.

Почему React и TypeScript для фронтенда:

- Компонентный подход - переиспользование UI-блоков (карточки тестов, формы вопросов).
- TypeScript - статическая типизация отлавливает ошибки на этапе разработки (соответствие типов данных от API), плюс даёт возможность для дальнейшего расширения программы.
- Адаптивность - React хорошо работает с адаптивным дизайном.
- Экосистема - React Router (маршрутизация), Axios (HTTP-запросы).

Почему PostgreSQL:

- ACID - гарантия сохранности результатов тестов при сбоях.

- Поддержка сложных JOIN-запросов - необходима для статистики.
- Триггеры - автоматическое обновление таблицы test_statistics.
- Надёжность внешних ключей - предотвращение «висящих» записей.
- Бесплатность - open-source, без затрат на лицензии.

Почему Docker:

- Воспроизводимость окружения - приложение работает одинаково на всех этапах.
- Упрощение развёртывания - docker-compose up -d запускает все сервисы.
- Изоляция - каждый компонент в своём контейнере.

Почему Vite вместо Create React App (CRA):

- Быстрая сборка - Vite использует нативный ES-модули.
- Горячая перезагрузка (HMR) - мгновенное отображение изменений.

3 Техничко-рабочее проектирование

3.1 Описание реализации

3.1.1 Архитектура приложения

Разработанное веб-приложение для создания и прохождения тестов построено по классической клиент-серверной архитектуре. Серверная часть реализована на фреймворке Django (Python) с использованием Django REST Framework (DRF) для построения API, клиентская часть - на библиотеке React (TypeScript) с применением Vite в качестве сборщика. Взаимодействие между клиентом и сервером осуществляется посредством REST API с обменом данными в формате JSON. Для аутентификации пользователей используются JWT-токены (Simple JWT).

Общая архитектура приложения представлена на рисунке 11.

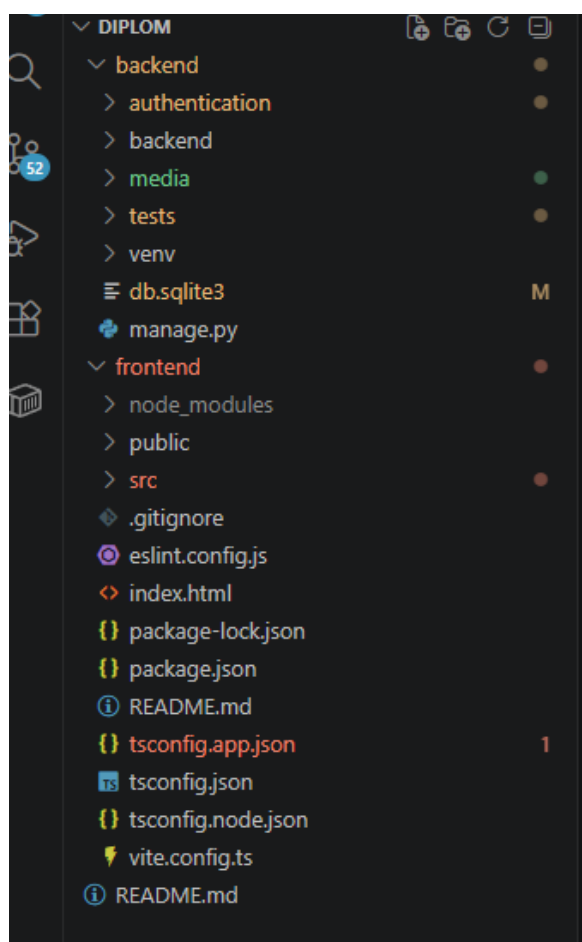


Рисунок 11 - Общая архитектура приложения

Клиентская часть (Frontend)

Клиентская часть выполнена на библиотеке React с использованием TypeScript. Выбор TypeScript обусловлен необходимостью статической типизации, которая позволяет выявлять ошибки на этапе разработки и обеспечивает самодокументируемость кода. В качестве сборщика используется Vite, обеспечивающий быструю холодную перезагрузку и мгновенное отображение изменений за счёт нативной поддержки ES-модулей.

Структура клиентской части представлена на рисунке 12.

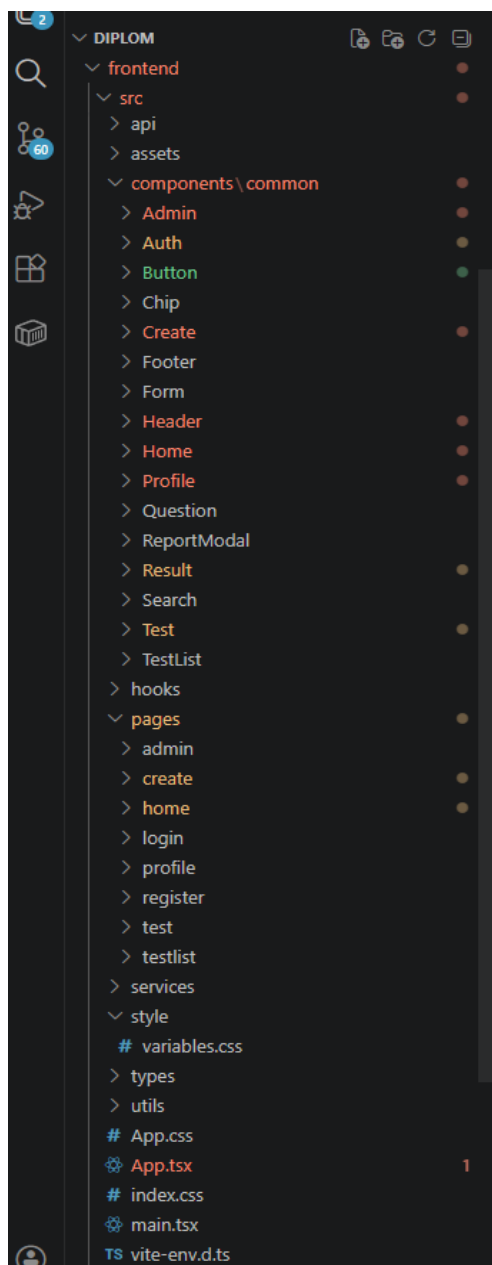


Рисунок 12 - Структура клиентской части

Основные компоненты клиентской части:

components/common/ - каталог переиспользуемых UI-компонентов:

- Button/ - кнопки различных типов (в том числе кнопка жалобы);
- Card/ - карточки для отображения тестов и пользователей;
- Chip/ - компоненты для отображения информации об авторе;
- Header/ и Footer/ - шапка и подвал сайта;
- ReportModal/ - модальное окно для отправки жалоб;
- Profile/ - компоненты профиля пользователя (ProfileHeader, ProfileTabs, ProfileStatsBlock, StatisticsTab, FriendsTab, MyTestsTab).

pages/ - каталог страниц приложения:

- home/ - главная страница;
- profile/ - страница профиля;
- test/ - страницы прохождения тестов;
- create/ - страница создания тестов;
- login/ и register/ - страницы аутентификации.

api/ - модуль взаимодействия с сервером. Содержит настройку HTTP-клиента (Axios), конфигурацию базового URL, перехватчики для добавления JWT-токенов в заголовки запросов.

hooks/ - кастомные React-хуки для переиспользования логики (useAuthForm, usePools).

types/ - глобальные TypeScript-интерфейсы (типы для тестов, пользователей).

style/ - глобальные стили с CSS-переменными, определяющими цветовую схему проекта.

utils/ - вспомогательные утилиты (работа с изображениями, форматирование дат).

Серверная часть (Backend)

Серверная часть реализована на фреймворке Django. Django обеспечивает высокую безопасность (защита от CSRF/XSS-атак, SQL-инъекций), встроенную аутентификацию, ORM для работы с базой данных, миграции для управления схемой и автоматическую админ-панель.

Структура серверной части представлена на рисунке 13.

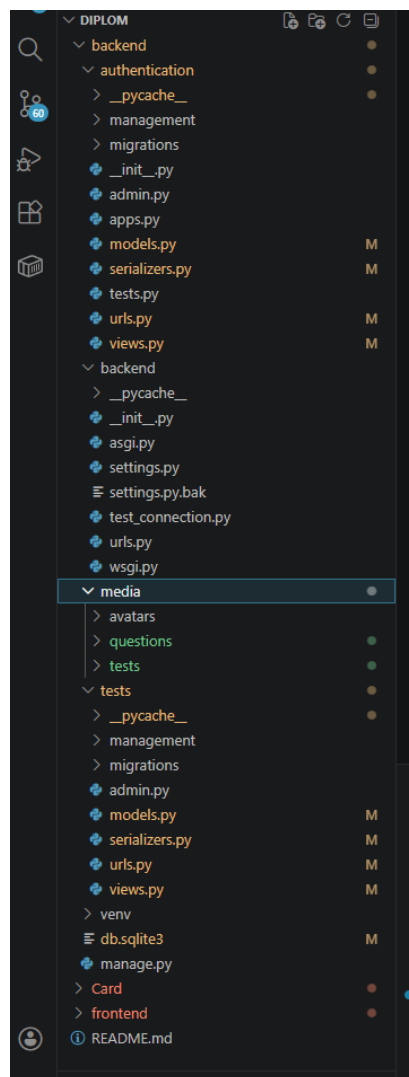


Рисунок 13 - Структура серверной части

Основные компоненты серверной части:

backend/backend/ - каталог конфигурации проекта:

- settings.py - файл конфигурации Django: подключённые приложения, настройки базы данных, конфигурация DRF (JWT-аутентификация, пагинация);
- urls.py - корневой файл маршрутизации.

backend/authentication/ - приложение для управления пользователями:

- models.py - модели: User (кастомная модель пользователя с полями login, email, avatar, bio, friends), FriendRequest (заявки в друзья);
- views.py - представления для регистрации, входа, профиля, управления друзьями;
- serializers.py - сериализаторы для преобразования моделей в JSON;
- urls.py - маршруты для эндпоинтов аутентификации.

backend/tests/ - приложение для управления тестами:

- models.py -модели: Question, Answer, Pool, Test, TestQuestion, TestAttempt, TestRating, TestComment, Report;
- views.py - представления для создания тестов, прохождения, получения результатов;
- serializers.py - сериализаторы для моделей тестов;
- urls.py - маршруты для API-эндпоинтов тестов.

database/migrations/ - миграции, управляющие схемой базы данных. Позволяют создавать и изменять таблицы, добавлять индексы и внешние ключи, обеспечивая воспроизводимость структуры на любом окружении.

management/commands/ - кастомные команды для заполнения базы данных тестовыми данными (seed_users, seed_data). Это упрощает первоначальную настройку системы и тестирование.

Взаимодействие клиента и сервера

Взаимодействие между клиентом и сервером происходит посредством AJAX-запросов через библиотеку Axios. Клиент отправляет HTTP-запросы на API-эндпоинты сервера, сервер обрабатывает их, взаимодействует с базой данных и возвращает ответ в формате JSON.

Процесс аутентификации осуществляется с использованием JWT-токенов:

1. Пользователь отправляет логин и пароль на эндпоинт `/api/auth/login/`.
2. Сервер проверяет учётные данные и возвращает access-токен и refresh-токен.
3. Клиент сохраняет токены в `localStorage` и добавляет access-токен в заголовок `Authorization` при каждом запросе.
4. При истечении срока действия access-токена клиент отправляет refresh-токен на эндпоинт `/api/auth/refresh/` для получения новой пары токенов.
5. Все защищённые маршруты (создание тестов, отправка заявок в друзья, написание комментариев) требуют наличия валидного JWT-токена, который проверяется на стороне сервера с помощью специального `middleware`.

3.1.2 Руководство пользователя

Главная страница

При переходе на сайт пользователь попадает на главную страницу, где размещены списки тестов и опросов, а также в нижней части страницы можно найти список активных пользователей. На странице присутствует поисковое поле, позволяющие находить необходимые тесты/опросы/других пользователей, а так же возможность выбирать теги, сортировать контент по желанию и другие возможности навигации. Главная страница представлена на рисунках 14-15.

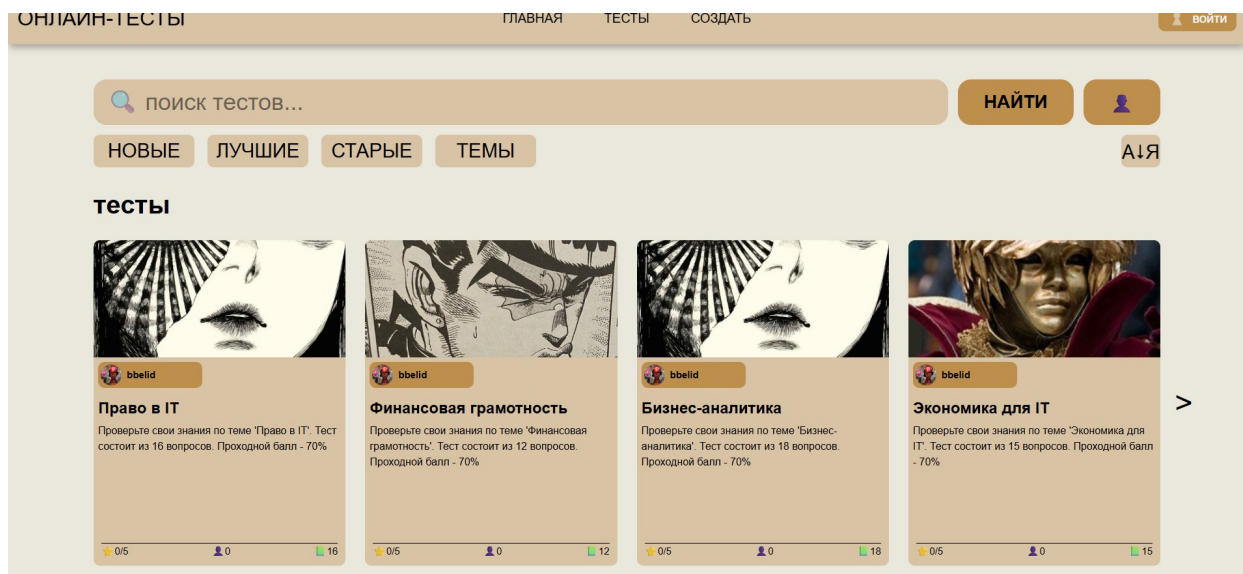


Рисунок 14 - Главная страница верхняя часть

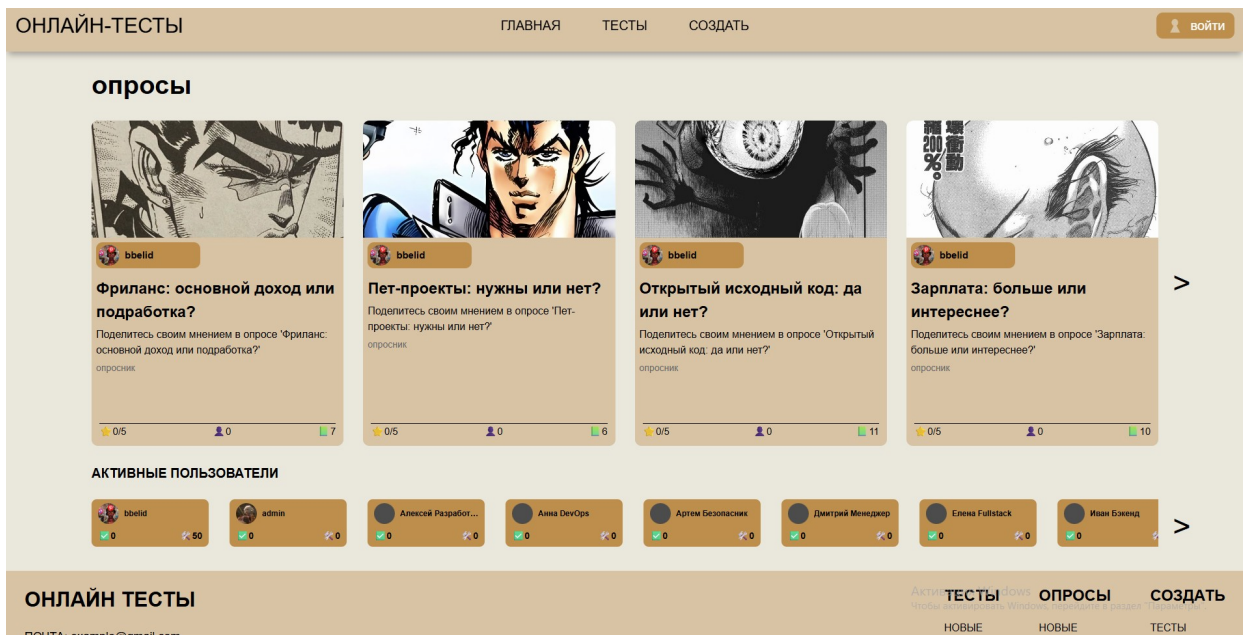


Рисунок 15 - Главная страница нижняя часть

Страница регистрации/авторизации.

Для полноценного использования веб-сайта рекомендуется пройти регистрацию, а при наличии учётной записи - авторизацию. Для регистрации необходимо указать логин, email, дважды ввести пароль, согласиться с обработкой персональных данных и нажать кнопку регистрации (см. рисунок 16). Для авторизации достаточно ввести логин и пароль и нажать кнопку «Войти» (см. рисунок 17).

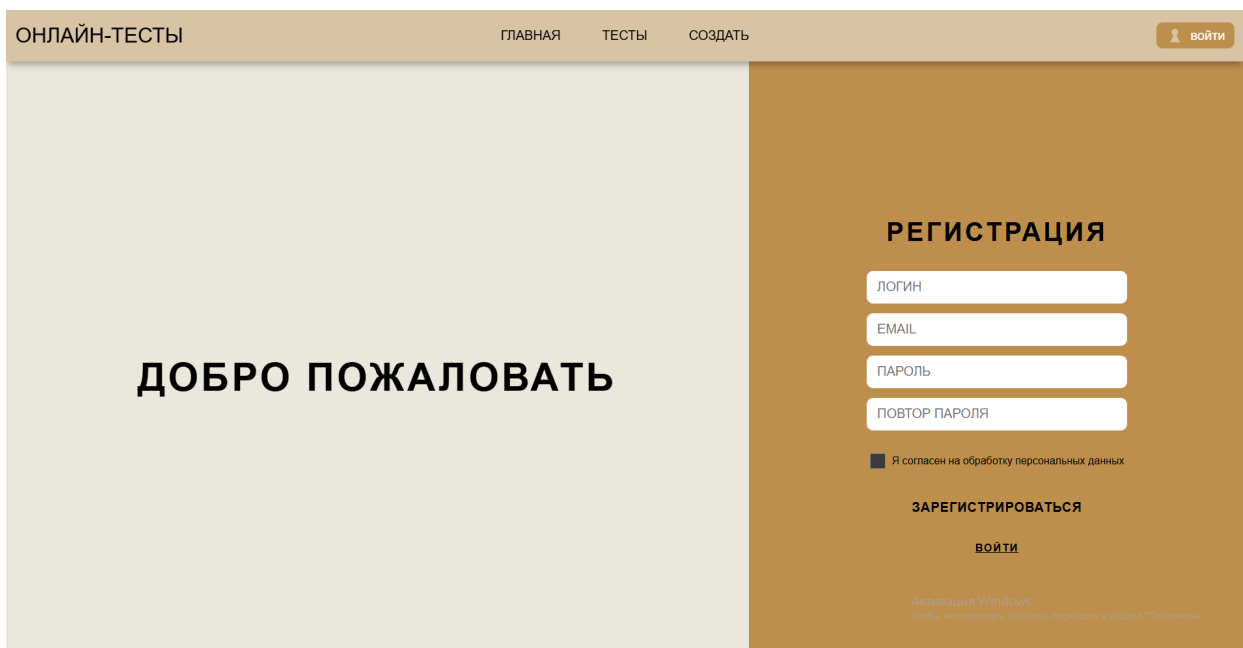


Рисунок 16 - Страница регистрации

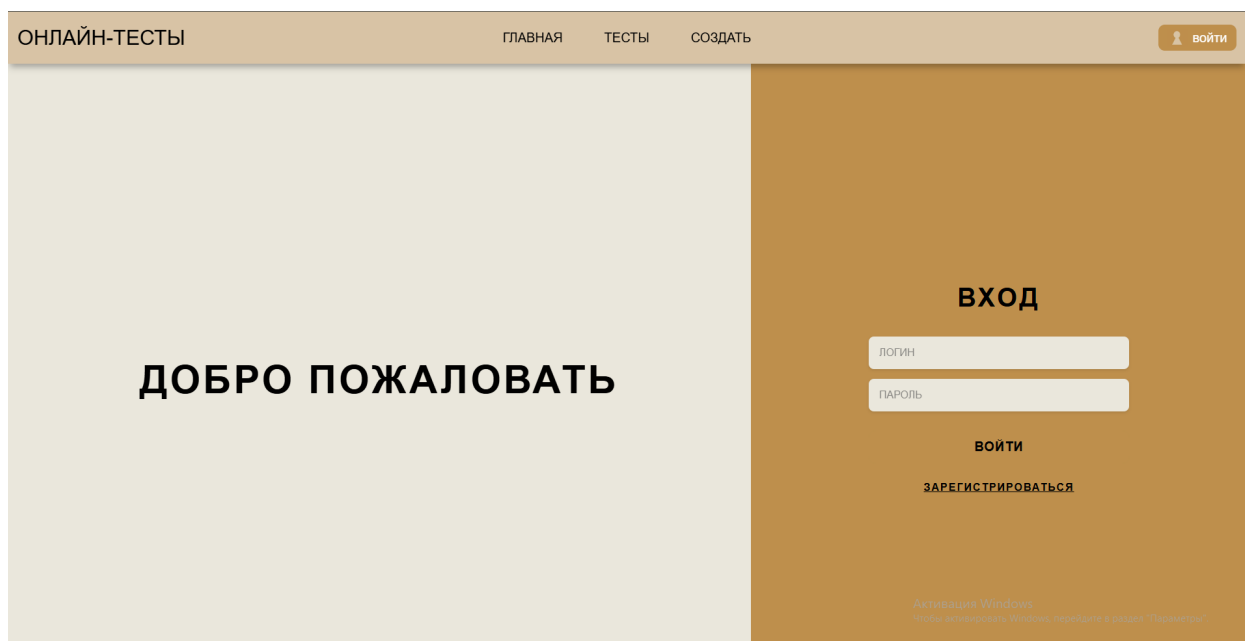


Рисунок 17 - Страница авторизации

Главная страница теста/опроса.

Для начала прохождения теста необходимо выбрать нужный тест, после чего вы будете перенаправлены на главную страницу теста. Там будет основная информация: количество вопросов, оценка теста, сколько человек прошло, ограничения по времени, количество вопросов, комментарии к тесту и т.п. (см. рисунок 18).

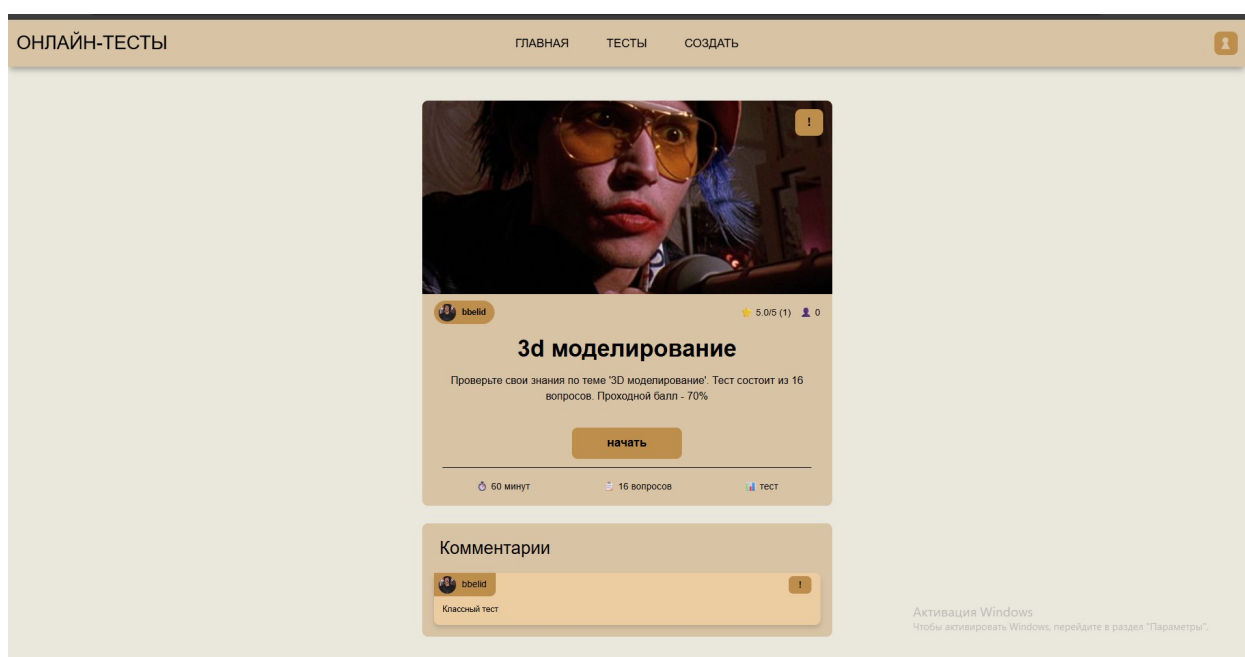


Рисунок 18 - Главная страница теста

Прохождение теста/опроса.

При прохождении теста или опроса пользователю предоставляется два типа страниц с вопросами:

- страница с вопросами, где ответы нужно выбрать из предложенного списка;
- страница с вопросами, где ответ нужно ввести самостоятельно в текстовое поле.

Если пользователь проходит тест, то у него может быть ограничение по времени, результат каждого ответа учитывается, и по окончании вычисляется итоговый результат в процентах. Также автор теста может установить ограничение на количество попыток прохождения.

При прохождении опроса ограничение по времени отсутствует, а финальный результат в виде баллов не вычисляется.

После завершения теста и расчёта оценки пользователь перенаправляется на страницу результата, где может увидеть свой результат и оставить комментарий (см. рисунки 19-21).

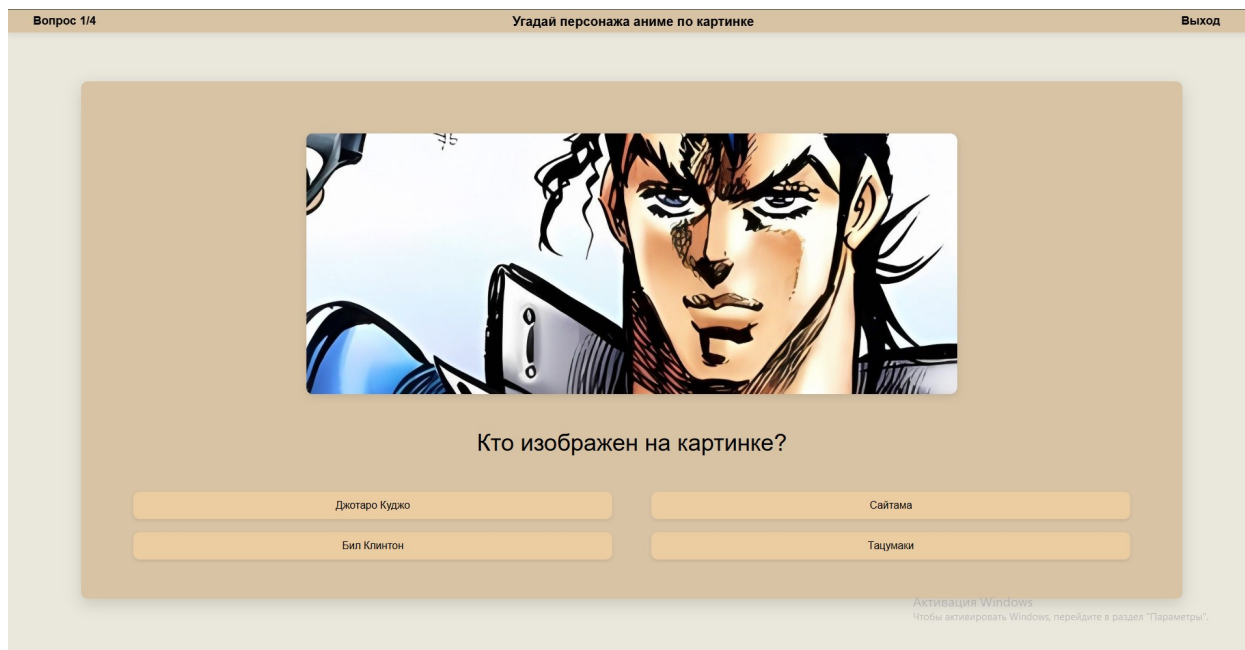


Рисунок 19 - Страница вопроса тип 1

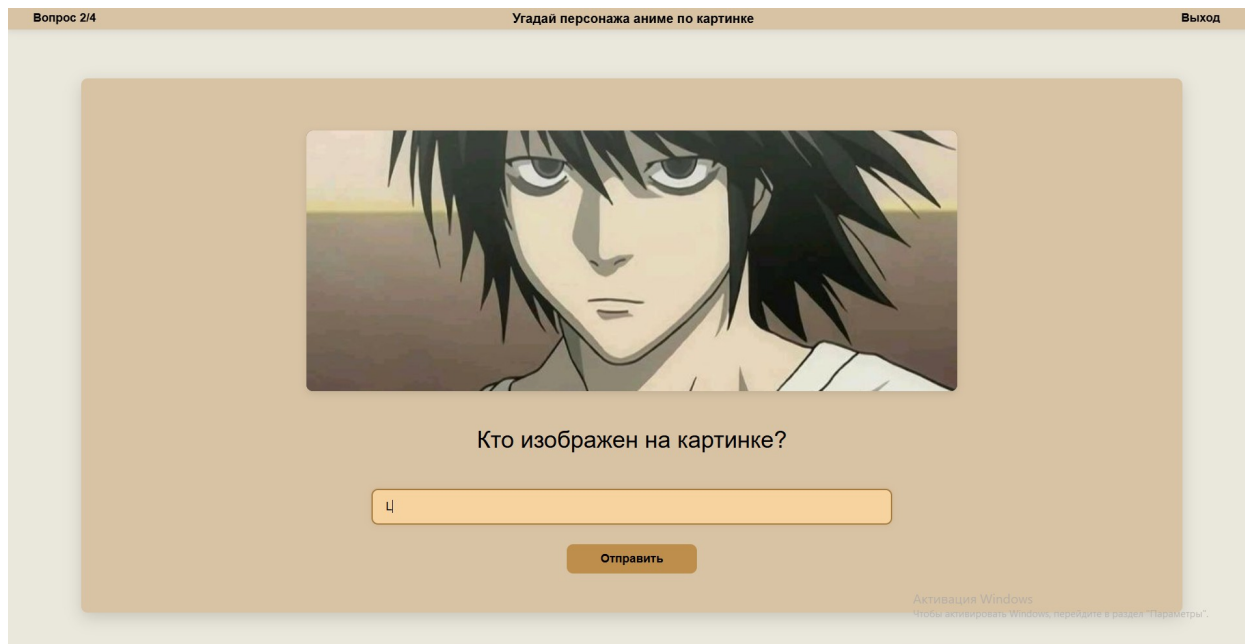


Рисунок 20 - Страница вопроса тип 2

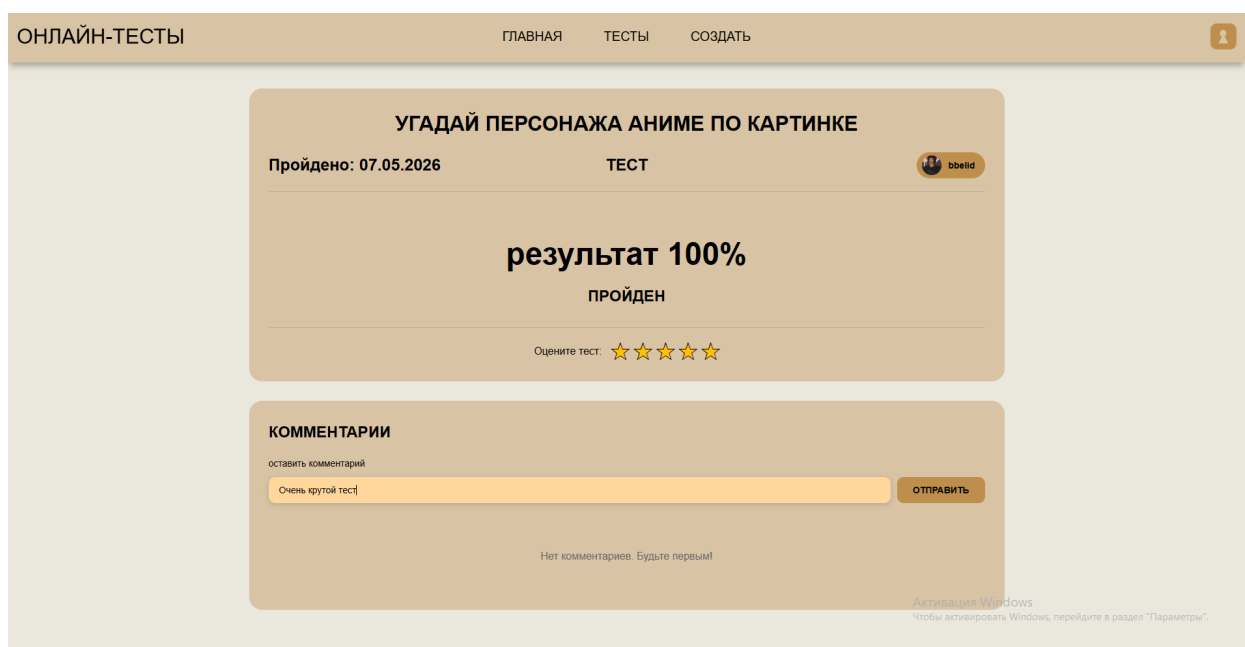


Рисунок 21 - Страница результата

Создание тестов.

После регистрации и авторизации на сайте пользователь может начать создание собственных тестов. Для этого необходимо нажать на кнопку «Создать» в шапке сайта, после чего пользователь попадает на страницу выбора типа создаваемого теста (см. рисунок 22). После выбора он переходит в меню создания. Там пользователю необходимо ввести название, описание, выбрать обложку, видимость и теги. Если создаётся тест (а не опрос), дополнительно указываются ограничение по времени, проходной балл и количество попыток. Далее идёт форма добавления вопросов

(минимум 4). Для каждого вопроса можно добавить картинку, а также необходимо добавить название и либо 4 варианта ответов с выделением правильного, либо поставить флажок «текстовый ввод» и указать правильный ответ. Вопросы также можно добавит из пула вопросов, нажав на соответствующую кнопку и выбрав нужный пул в модальном окне, либо создав новый (см. рисунки 23 — 27).



Рисунок 22 - Страница выбора типа теста



Рисунок 23 - Меню создание теста

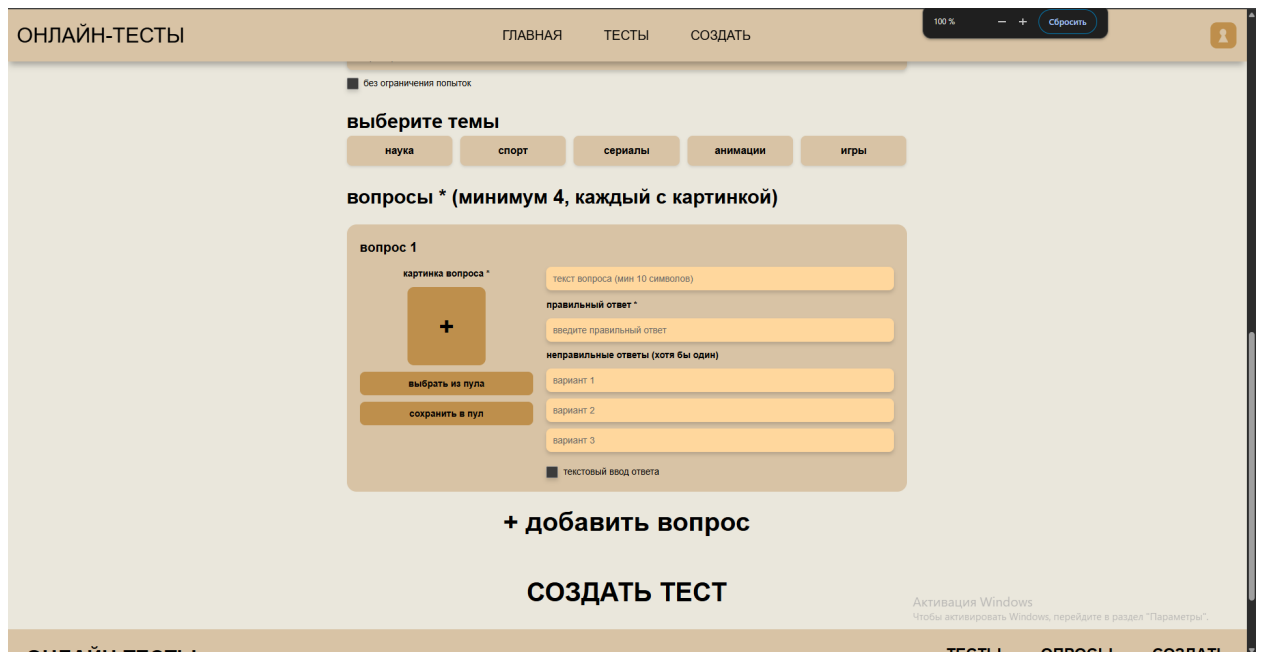


Рисунок 24 - Меню создание вопроса

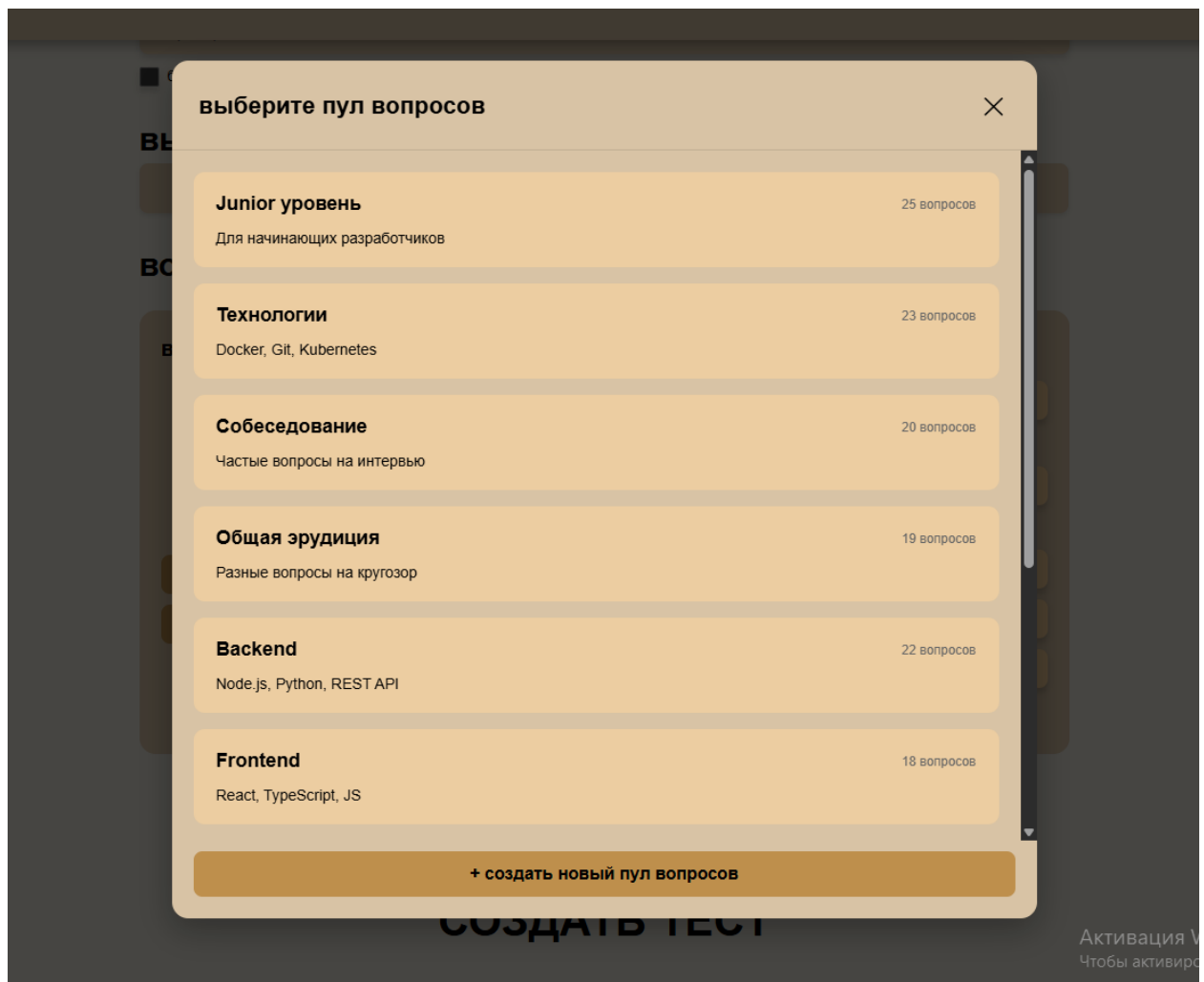


Рисунок 25 - Выбор пула вопроса (модальное окно)

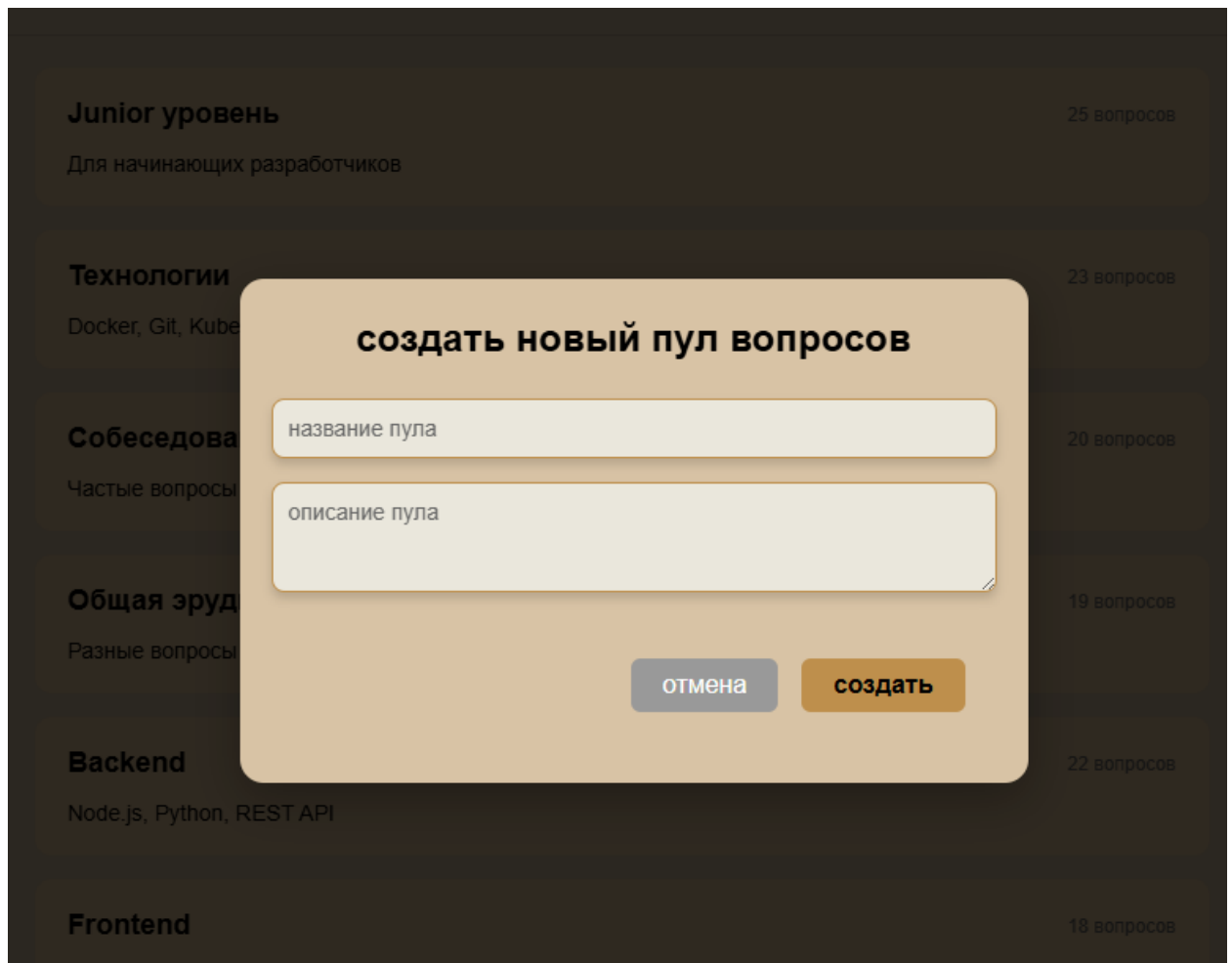


Рисунок 26 - Создание пула вопроса (модальное окно)

Профиль пользователя.

В профиле пользователь может увидеть свою полную информацию. В верхней части профиля отображаются: ник пользователя, описание профиля, дата регистрации, время последнего захода на сайт и количество друзей. Пользователь также может редактировать эти данные в модальном окне, нажав на кнопку «Редактировать профиль» (см. рисунки 27-28).

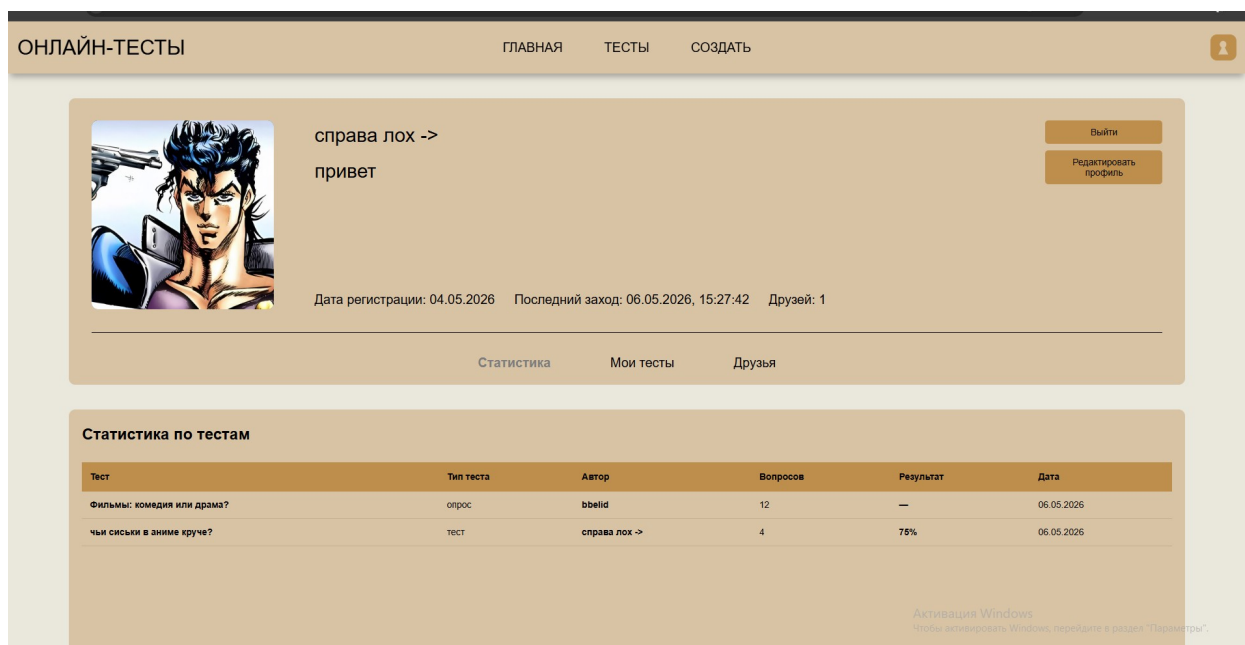


Рисунок 27 - Профиль пользователя верхняя часть

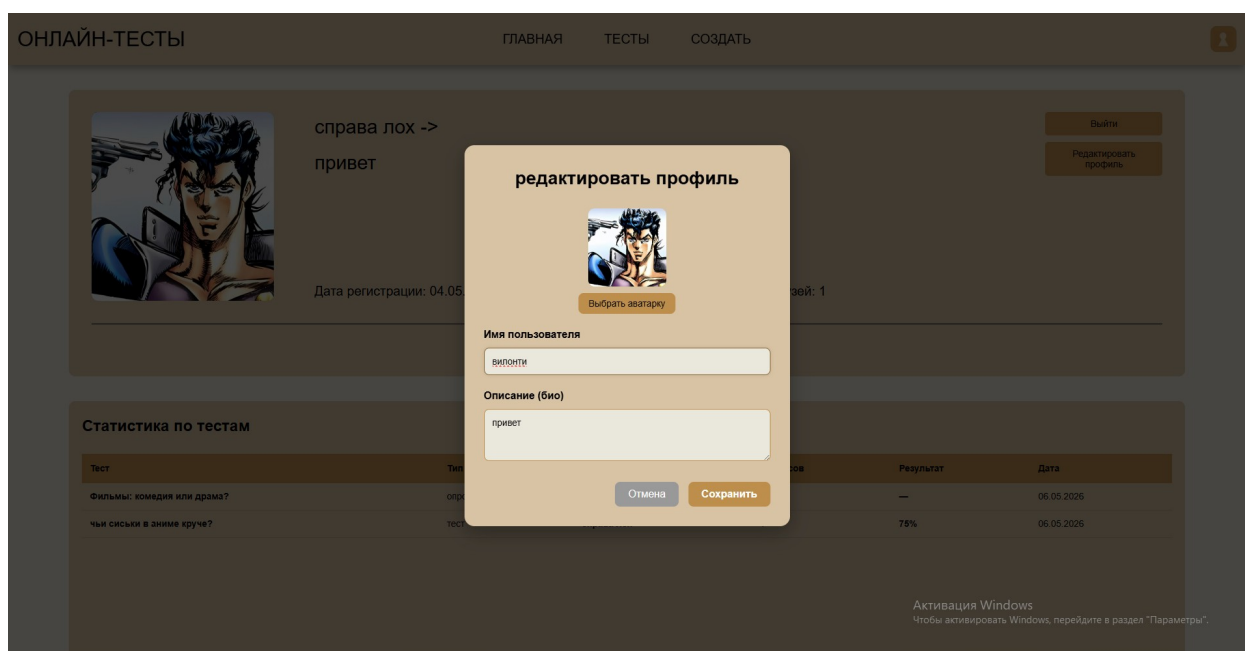


Рисунок 28 - Редактирование профиля пользователя (модальное окно)

В нижней части профиля представлены три вкладки с таблицами:

На вкладке «Статистика» отображаются все пройденные пользователем публичные тесты. Для каждого теста указано: название теста, тип теста, автор, количество вопросов, результат (если это тест, а не опрос) и дата прохождения (см. рисунок 29).

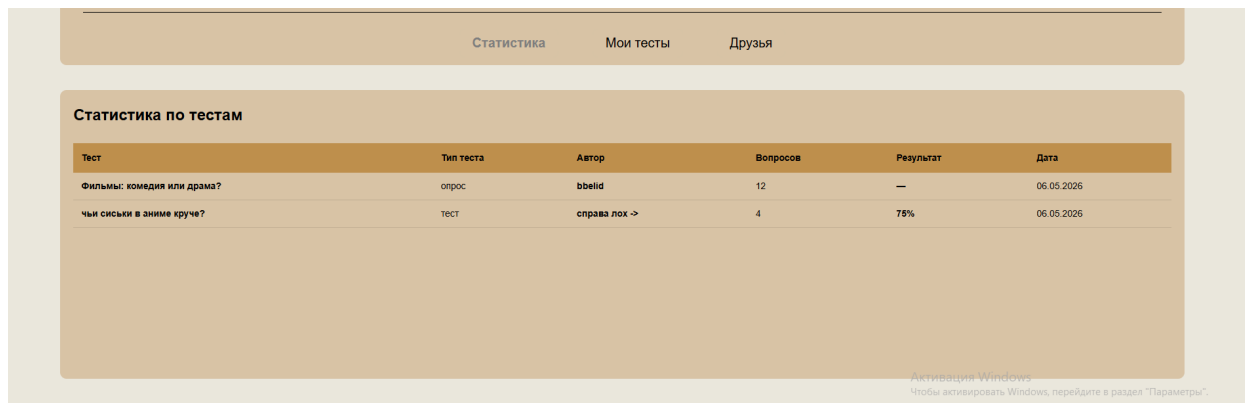


Рисунок 29 - Вкладка статистики

На вкладке «Мои тесты» отображаются созданные пользователем тесты. Для каждого теста указано: название, тип, количество вопросов, количество прохождений, рейтинг и тип доступа (публичный или приватный). При нажатии на любой из своих тестов пользователь переходит на страницу статистики этого теста, где может посмотреть, кто проходил его, на сколько баллов и сколько раз (см. рисунок 30-31).



Рисунок 30 - Вкладка созданных тестов

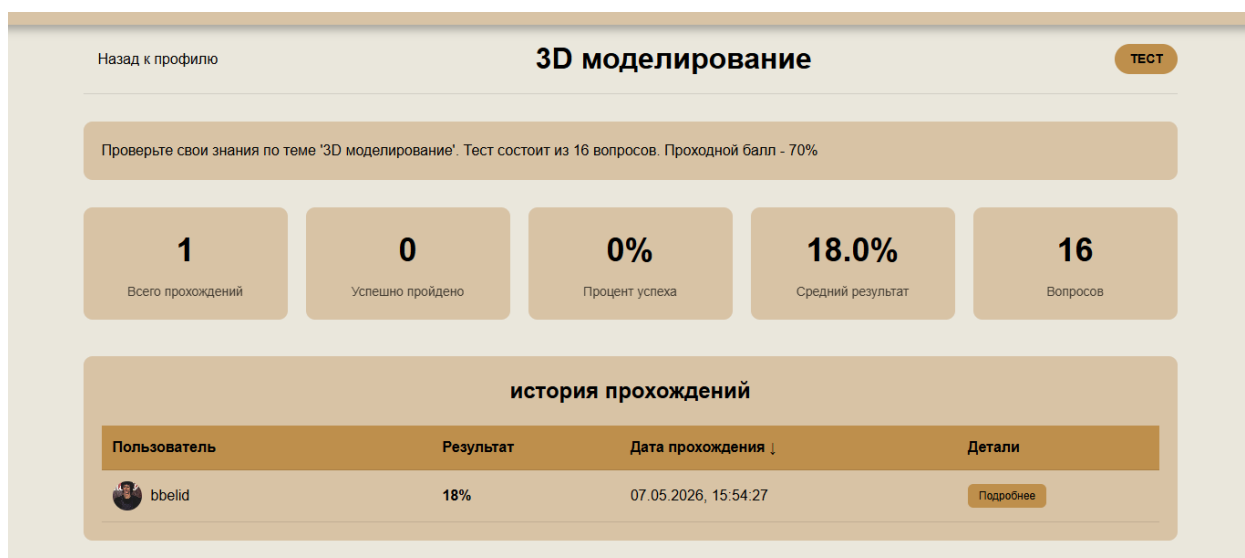


Рисунок 31 - Страница просмотра статистики

На вкладке «Друзья» пользователь видит список всех своих друзей с указанием даты их последнего захода. Он может перейти в профиль друга или удалить его из друзей. Также на этой вкладке отображаются входящие заявки в друзья, которые пользователь может принять или отклонить (см. рисунок 32).

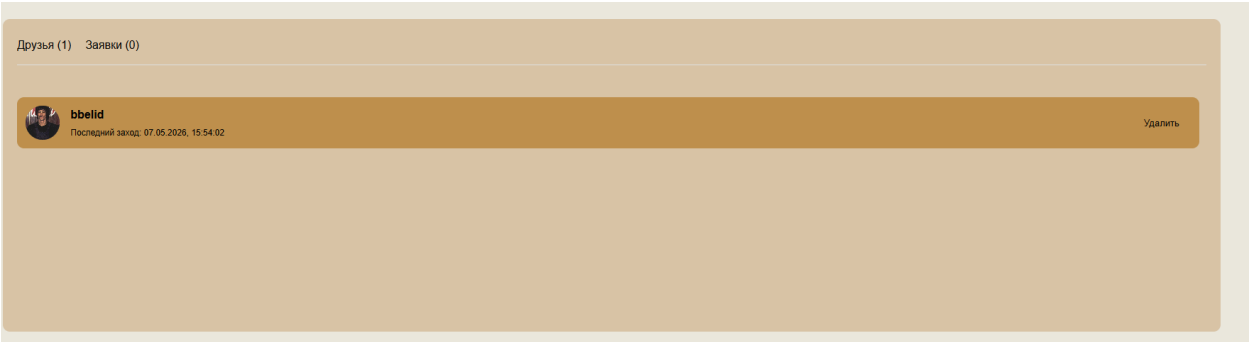


Рисунок 32 - Вкладка «друзья»

В профилях других пользователей доступны кнопка «Пожаловаться» и кнопка «Добавить в друзья» (см. рисунок 33).

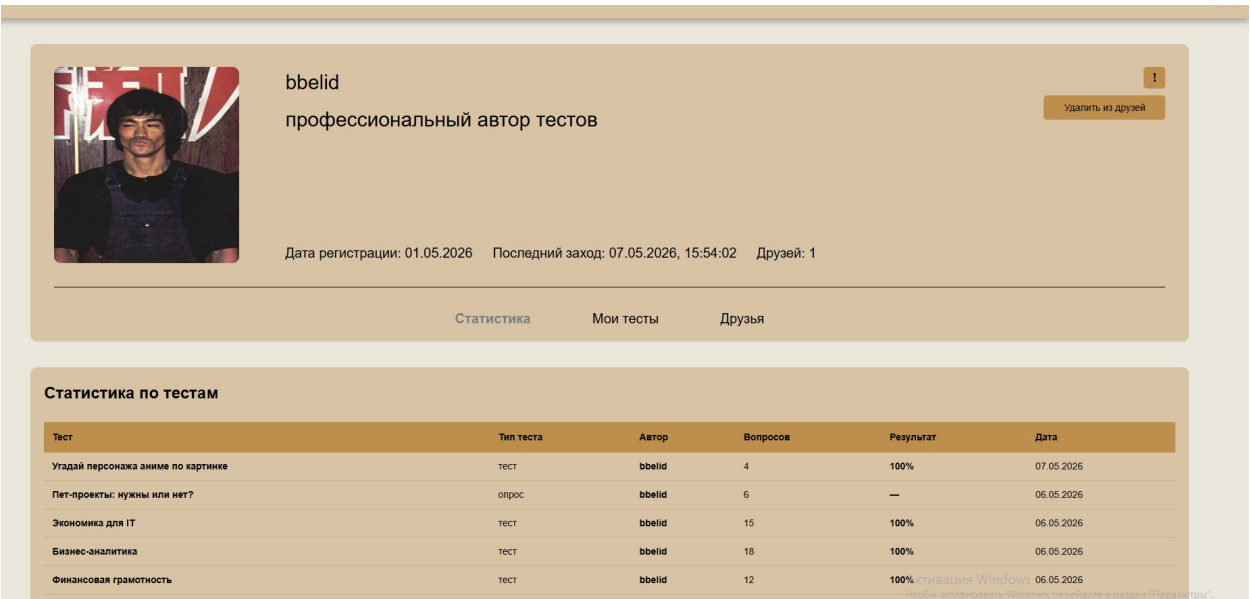


Рисунок 33 - Чужой профиль

Модерирование.

Для модератора в шапке профиля присутствует соответствующая кнопка, перекидывающая его на админ панель. В ней отображаются все репорты, сортирующиеся по вкладкам: все, на рассмотрение, проверенно, решено. Сами репорты отображают всю информацию: пользователь который подал репорт, тип репорта

РЕПОРТЫ				
Все (27) На рассмотрении (21) Проверено (2) Решено (4)				
ПОЛЬЗОВАТЕЛЬ	ТИП	ПРИЧИНА	СТАТУС	
 admin 06.05.2026, 14:07:21	Пользователь	Спам	Проверено	Перейти к профилю
ПОЛЬЗОВАТЕЛЬ	ТИП	ПРИЧИНА	СТАТУС	
 admin 06.05.2026, 14:07:07	Тест	Другое КОММЕНТАРИЙ: УЦАЦУ	На рассмотрении	Перейти к тесту
ПОЛЬЗОВАТЕЛЬ	ТИП	ПРИЧИНА	СТАТУС	
 admin 06.05.2026, 14:07:00	Комментарий	Другое КОММЕНТАРИЙ: ХУЕСОС В	Решено	Перейти к тесту
ПОЛЬЗОВАТЕЛЬ	ТИП	ПРИЧИНА	СТАТУС	
 admin 06.05.2026, 14:03:39	Тест	Другое КОММЕНТАРИЙ: 545ЦЦЦЦУН	На рассмотрении	Перейти к тесту

Рисунок 34 - Админ панель

На страницах пользователей, страницах тестов и комментариях для модератора есть кнопки удаления/бана/мута, для обычных пользователей там находятся кнопки репортов которые активируют модальные окна (см. рисунки 35-36).



Рисунок 35 - Кнопки администраторов

Рисунок 36 - Модальное окно для репорта

3.2 Тестирование

3.2.1 Порядок проведения тестирования

Тестирование проводилось в следующем порядке:

1. Модульное тестирование - проверка каждого компонента системы отдельно (авторизация, создание тестов, прохождение тестов, комментарии, рейтинг, друзья, репорты, админ-панель).
2. Интеграционное тестирование – проверка взаимодействия компонентов между собой (создание теста -> прохождение -> отображение статистики, отправка жалобы -> отображение в админ-панели и т.д.).
3. Функциональное тестирование - проверка соответствия реализованного функционала требованиям технического задания.
4. Тестирование пользовательского интерфейса - проверка корректности отображения на различных разрешениях экрана (десктоп, планшет, мобильные устройства).

5. Тестирование безопасности - проверка разграничения доступа (неавторизованные пользователи не могут создавать тесты, оставлять комментарии, ставить оценки; администратор имеет доступ к панели управления).

Все тестовые сценарии представлены в таблице 8.

Таблица 8 - Тестовые сценарии

№	Сценарий	Ожидаемый результат	Фактический результат	Статус
1	Регистрация нового пользователя с валидными данными	Пользователь создан, выполнен редирект на главную	Пользователь успешно зарегистрирован, выполнен редирект на главную страницу	Успешно
2	Авторизация с валидными данными	Вход выполнен, токены сохранены	Вход выполнен, access_token и refresh_token сохранены в localStorage	Успешно
3	Создание теста (мин. 4 вопроса)	Тест создан, отображается в списке	Тест успешно создан, отображается в разделе "Мои тесты"	Успешно
4	Прохождение теста с выбором ответов	Ответы сохраняются, результат рассчитан	Ответы сохранены, результат правильно рассчитан в процентах	Успешно
5	Прохождение теста с ограничением по времени	Таймер работает, по истечении тест завершается	Таймер корректно отсчитывает время, при истечении тест автоматически	Успешно

			завершается	
6	Добавление комментария к тесту (авторизованный)	Комментарий добавлен, отображается в списке	Комментарий успешно добавлен и отображается в блоке комментариев	Успешно
7	Отправка жалобы на тест/комментарий/пользователя	Жалоба отправлена, отображается в админ-панели	Жалоба отправлена, отображается в админ-панели со статусом "На рассмотрении"	Успешно
8	Отправка заявки в друзья и её принятие	Пользователи добавлены в друзья друг у друга	Заявка отправлена, после принятия пользователи стали друзьями	Успешно
9	Мут пользователя администратором	Замученный пользователь не может оставлять комментарии	После мута пользователь не получает сообщение "Вы замучены" и не может оставить комментарий	Успешно
10	Просмотр статистики теста автором	Отображаются все попытки прохождения	В статистике отображаются все попытки с указанием пользователя, результата и даты	Успешно

3.2.2 Результат тестирования

По итогам тестирования все функциональные требования, предъявленные к веб-приложению, выполнены в полном объёме. Выявленные в процессе тестирования ошибки были устранены. Система работает стабильно, корректно обрабатывает запросы

и отображает информацию во всех поддерживаемых браузерах и на различных разрешениях экрана.

ЗАКЛЮЧЕНИЕ

В результате выполнения дипломного проекта разработана веб-система создания и прохождения онлайн-тестов «ССИПОТ». Все поставленные задачи решены, цель работы достигнута.

Проведён анализ предметной области. Выявлены основные проблемы существующих решений: разделение на образовательные и развлекательные платформы, отсутствие детальной статистики для авторов, слабая система модерации, потеря результатов для неавторизованных пользователей. Сформулированы требования к разрабатываемой системе.

Выполнен обзор существующих аналогов: Pikuco, OnlineTestPad, Oltest, Google Forms, Wayground. Выявлены их сильные и слабые стороны. Обоснована необходимость разработки собственного гибридного решения, совмещающего образовательный и развлекательный сегменты.

Спроектирована структура базы данных для хранения информации о пользователях, вопросах, пулах, тестах, попытках прохождения, комментариях и репортах. Разработаны ключевые алгоритмы: аутентификации с использованием JWT-токенов, обработки заявок в друзья, прохождения тестов с поддержкой анонимов и ограничений, создания тестов из пула вопросов.

Обоснован выбор технологического стека: Python и Django REST Framework для бэкенда, React и TypeScript для фронтенда, PostgreSQL в качестве базы данных, Docker для контейнеризации, Vite для сборки.

Реализовано веб-приложение, включающее регистрацию и авторизацию, создание и прохождение тестов и опросов, систему комментариев и рейтингов, социальные связи, детальную статистику для авторов, систему модерации и жалоб, адаптивный интерфейс.

Проведено функциональное тестирование. Составлены тестовые сценарии, охватывающие основные функции системы. Выявленные ошибки устранены. Система показала стабильную работу.

Разработанная система может использоваться для проверки знаний, создания викторин и опросов, а также для модерации контента. Направления дальнейшего развития: мобильное приложение, система рекомендаций, экспорт статистики, интеграция с внешними платформами.

СПИСОК ЛИТЕРАТУРЫ

1. Бэнкс, А. GraphQL. Язык запросов для современных веб-приложений / А. Бэнкс. — СПб. : Питер, 2019. — 240 с. — ISBN 978-5-4461-1237-5.
2. Бибичев, С. В. Современная веб-разработка на Django / С. В. Бибичев. — М. : ДМК Пресс, 2022. — 400 с. — ISBN 978-5-97060-982-0.
3. Дакетт, Дж. HTML и CSS. Разработка и дизайн веб-сайтов / Дж. Дакетт. — М. : Эксмо, 2014. — 512 с. — ISBN 978-5-699-66890-6.
4. Дронов, В. А. React. Самоучитель / В. А. Дронов. — СПб. : БХВ-Петербург, 2022. — 464 с. — ISBN 978-5-9775-6800-5.
5. Дронов, В. А. TypeScript. Современная веб-разработка / В. А. Дронов. — СПб. : БХВ-Петербург, 2021. — 496 с. — ISBN 978-5-9775-0760-8.
6. Лоусон, Б. Изучаем React / Б. Лоусон. — СПб. : Питер, 2021. — 304 с. — ISBN 978-5-4461-1678-6.
7. Лутц, М. Изучаем Python (в 2-х томах) / М. Лутц. — 5-е изд. — СПб. : Диалектика, 2019. — Т. 1 — 832 с., Т. 2 — 928 с. — ISBN 978-5-907144-28-4.
8. Морган, Б. PostgreSQL. Основы работы с базами данных / Б. Морган. — М. : ДМК Пресс, 2020. — 320 с. — ISBN 978-5-97060-816-8.
9. Рубио, Д. Django 3.0. Практика создания веб-сайтов на Python / Д. Рубио. — М. : ДМК Пресс, 2020. — 300 с. — ISBN 978-5-97060-873-1.
10. Себеста, Р. У. Языки программирования. Принципы и парадигмы / Р. У. Себеста. — 5-е изд. — М. : Вильямс, 2020. — 800 с. — ISBN 978-5-8459-1962-8.
11. Стоян, С. TypeScript. Быстрая разработка веб-приложений / С. Стоян. — СПб. : Питер, 2022. — 320 с. — ISBN 978-5-4461-1924-4.
12. Файн, Я. JavaScript: подробное руководство / Я. Файн. — 2-е изд. — СПб. : Питер, 2022. — 464 с. — ISBN 978-5-4461-1905-3.

13. Холзнер, С. Docker для разработчика / С. Холзнер. — СПб. : БХВ-Петербург, 2021. — 256 с. — ISBN 978-5-9775-6789-3.
14. Хэдлок, Н. React. Полный курс / Н. Хэдлок. — М. : ДМК Пресс, 2021. — 450 с. — ISBN 978-5-97060-903-5.
15. Чиннем, С. TypeScript для профессионалов / С. Чиннем. — М. : ДМК Пресс, 2021. — 360 с. — ISBN 978-5-97060-904-2.

Приложение А

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное
учреждение высшего образования
«Алтайский государственный технический университет им. И. И. Ползунова»

УТВЕРЖДАЮ
Заведующий кафедрой ИСЭ
_____ Авдеев А. С.
подпись ФИО

ЗАДАНИЕ №9 НА ВЫПОЛНЕНИЕ ДИПЛОМНОГО ПРОЕКТА

по специальности 09.02.07 «Информационные системы и программирование»

студенту группы 1ИСП-21 Котикову Егору Владиславовичу

Тема Разработка веб-сайта для создания и прохождения онлайн тестов

Утверждена приказом ректора от 30.03.2026 № Л-733

Срок выполнения задания 26.05.2026

Задание принял к исполнению:

Котилов Егор Владиславович
подпись ФИО

Барнаул 2026 г.

1 ИСХОДНЫЕ ДАННЫЕ

На основании анализа существующих систем онлайн-тестирования (Pikuso, OnlineTestPad, Oltest, Google Forms, Wayground) и выявленных недостатков (разделение на образовательные и развлекательные сегменты, отсутствие детальной статистики для авторов, слабая система модерации, потеря результатов для неавторизованных пользователей) разработать веб-приложение «Система создания и прохождения онлайн-тестов» (ССИПОТ), совмещающее образовательный и развлекательный контент. Приложение должно включать: регистрацию и авторизацию пользователей с использованием JWT-токенов, создание тестов и опросов (не менее 4 вопросов), прохождение тестов с ограничением по времени и подсчетом результата в процентах, систему комментариев и рейтингов, социальные связи (друзья, заявки), детальную статистику для авторов (количество прохождений, средний балл, ответы пользователей), систему модерации и жалоб (репорты), адаптивный интерфейс (десктоп и мобильные устройства), а также административную панель для управления контентом и пользователями.

2 СОДЕРЖАНИЕ РАЗДЕЛОВ РАБОТЫ

Наименование разделов работы и их содержание	Трудоем кость, %	Срок выполнения	Руководитель (Ф.И.О., подпись)
1. Аналитическая часть	25		Воробьев К.В.
1.1 Анализ предметной области (онлайн-тестирование, образовательные и развлекательные сегменты, выявление проблем)	6	28.04.2026	Воробьев К.В.
1.2 Предпроектное обследование (анализ целевой аудитории, портреты	7	29.04.2026	Воробьев К.В.

пользователей, анализ текущих решений)			
1.3 Анализ существующих решений (обзор аналогов Pikuco, OnlineTestPad, Oltest, Google Forms, Wayground, сравнительная таблица, выводы)	11	30.04.2026	Воробьев К.В.
2. Проектная часть	30		Воробьев К.В.
2.1 Моделирование бизнес-процессов (контекстная диаграмма IDEF0, декомпозиция процесса)	6	02.05.2026	Воробьев К.В.
2.2 Требования к информационной системе (функциональные требования, User Stories для гостя, пользователя и модератора)	5	02.05.2026	Воробьев К.В.
2.3 Проектирование базы данных (концептуальная модель, сущности, связи)	6	03.05.2026	Воробьев К.В.
2.4 Построение функционально-алгоритмической структуры (алгоритмы аутентификации, создания тестов, прохождения тестов)	5	04.05.2026	Воробьев К.В.
2.5 Проектирование пользовательского интерфейса (десктопная и мобильная версии, 36 экранов)	4	04.05.2026	Воробьев К.В.
2.6 Выбор технологий (требования к платформе, описание стека Django/React/PostgreSQL, обоснование)	4	06.05.2026	Воробьев К.В.
3. Техничко-рабочее	45		Воробьев К.В.

проектирование			
3.1 Архитектура приложения (клиент-серверная, REST API, JWT-аутентификация, структура бэкенда и фронтенда)	12	07.05.2026	Воробьев К.В.
3.2 Руководство пользователя (главная страница, регистрация, прохождение тестов, создание тестов, профиль, модерация)	18	12.05.2026	Воробьев К.В.
3.3 Тестирование (модульное, интеграционное, функциональное, UI, безопасности, тестовые сценарии, результаты)	15	13.05.2026	Воробьев К.В.

3 СПИСОК РЕКОМЕНДОВАННЫХ ИСТОЧНИКОВ

3.1. По научно-технической литературе просмотреть реферативные журналы

_____ за
последние _____ года и научно-технические журналы _____
_____ за последние _____ года.

3.2. По нормативной литературе просмотреть указатели государственных и
отраслевых стандартов за последний год.

3.3. Патентный поиск провести за _____ лет по странам

Руководитель дипломного проекта: Воробьев К.В. _____
Ф.И.О. _____ подпись

Приложение А

backend:

```
===== \asgi.py ===== """ ASGI config for backend project.
```

It exposes the ASGI callable as a module-level variable named `application`.

For more information on this file,

see <https://docs.djangoproject.com/en/6.0/howto/deployment/asgi/> """

```
import os
```

```
from django.core.asgi import get_asgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
```

```
application = get_asgi_application()
```

```
===== \settings.py ===== from pathlib import Path from datetime import  
timedelta import os
```

```
BASE_DIR = Path(file).resolve().parent.parent
```

```
MEDIA_URL = '/media/' MEDIA_ROOT = os.path.join(BASE_DIR, 'media')
```

Максимальный размер загружаемого файла (5MB)

```
DATA_UPLOAD_MAX_MEMORY_SIZE = 5242880
```

```
SECRET_KEY = 'django-insecure-298@lp=r1fl28m45qen9=3i190%o@j+7^#g%4-z-  
pkgfpljl'
```

```
DEBUG = True
```

```
ALLOWED_HOSTS = []
```

```
INSTALLED_APPS =
```

```
[ 'django.contrib.admin', 'django.contrib.auth', 'django.contrib.contenttypes', 'django.contrib  
sessions', 'django.contrib.messages', 'django.contrib.staticfiles', 'authentication', 'tests', 're  
st_framework', 'rest_framework_simplejwt.token_blacklist', 'corsheaders', ]
```

```
MIDDLEWARE =
```

```
[ 'corsheaders.middleware.CorsMiddleware', 'django.middleware.security.SecurityMiddle  
ware', 'django.contrib.sessions.middleware.SessionMiddleware', 'django.middleware.com  
mon.CommonMiddleware', 'django.middleware.csrf.CsrfViewMiddleware', 'django.contrib  
auth.middleware.AuthenticationMiddleware', 'django.contrib.messages.middleware.Mes  
sageMiddleware', 'django.middleware.clickjacking.XFrameOptionsMiddleware', ]
```



```

ROOT_URLCONF = 'backend.urls'

TEMPLATES = [ { 'BACKEND':
'django.template.backends.django.DjangoTemplates', 'DIRS': [], 'APP_DIRS':
True, 'OPTIONS': { 'context_processors':
[ 'django.template.context_processors.request', 'django.contrib.auth.context_processors.aut
h', 'django.contrib.messages.context_processors.messages', ], }, }, ]

WSGI_APPLICATION = 'backend.wsgi.application'

DATABASES = { 'default': { 'ENGINE': 'django.db.backends.sqlite3', 'NAME':
BASE_DIR / 'db.sqlite3', } }

AUTH_PASSWORD_VALIDATORS = [ { 'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator', }, { 'NAME':
'django.contrib.auth.password_validation.MinimumLengthValidator', }, { 'NAME':
'django.contrib.auth.password_validation.CommonPasswordValidator', }, { 'NAME':
'django.contrib.auth.password_validation.NumericPasswordValidator', }, ]

LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True

STATIC_URL = 'static/'

CORS_ALLOWED_ORIGINS =
[ "http://localhost:3000", "http://127.0.0.1:3000", "http://localhost:5173", "http://
127.0.0.1:5173", ]

REST_FRAMEWORK = { 'DEFAULT_AUTHENTICATION_CLASSES':
( 'rest_framework_simplejwt.authentication.JWTAuthentication', ), 'DEFAULT_PERMIS
SION_CLASSES': ( 'rest_framework.permissions.AllowAny', ), }

SIMPLE_JWT = { 'ACCESS_TOKEN_LIFETIME':
timedelta(minutes=30), 'REFRESH_TOKEN_LIFETIME':
timedelta(days=30), 'ROTATE_REFRESH_TOKENS':
True, 'BLACKLIST_AFTER_ROTATION': True, 'AUTH_HEADER_TYPES':
('Bearer',), }

AUTH_USER_MODEL = 'authentication.User'

MEDIA_URL = '/media/' MEDIA_ROOT = os.path.join(BASE_DIR,
'media') ===== test_connection.py ===== import psycopg2 import sys

```

Параметры подключения

```
params = { 'dbname': 'diplom_db', 'user': 'diplom_users', 'password': '123456', 'host': '127.0.0.1', 'port': '5432', }
```

Выводим параметры для проверки

```
print("Параметры подключения:") for key, value in params.items(): print(f" {key}: {value}") # Выводим каждый символ, чтобы найти скрытые if key == 'password': print(f"  Длина: {len(value)}") for i, ch in enumerate(value): print(f" Символ {i}: '{ch}' (ord: {ord(ch)})")
```

```
print("\nПопытка подключения...")
```

```
try: conn = psycopg2.connect(**params) print("✅ Подключение успешно!") conn.close() except Exception as e: print(f"❌ Ошибка: {e}") print(f"Тип ошибки: {type(e).name}")
```

```
# Проверяем, есть ли в пароле скрытые символы
if 'password' in str(e):
    print("\nПроблема в пароле!")
    password = '123456'
    print(f"Пароль в коде: '{password}'")
    print(f"Длина: {len(password)}")
    for i, ch in enumerate(password):
        print(f"Символ {i}: '{ch}' (код: {ord(ch)})")
```

===== \urls.py ===== """ URL configuration for backend project.

The `urlpatterns` list routes URLs to views. For more information please see: <https://docs.djangoproject.com/en/6.0/topics/http/urls/> Examples: Function views 1. Add an import: `from my_app import views` 2. Add a URL to `urlpatterns`: `path('views/home', name='home')` Class-based views 1. Add an import: `from other_app.views import Home` 2. Add a URL to `urlpatterns`: `path('Home.as_view()', name='home')` Including another URLconf 1. Import the `include()` function: `from django.urls import include, path` 2. Add a URL to `urlpatterns`: `path('blog/', include('blog.urls'))` """ `from django.contrib import admin` `from django.urls import path` `from django.urls import path, include` `from django.conf import settings` `from django.conf.urls.static import static`

```
urlpatterns = [ path('admin/', admin.site.urls), path('api/', include('authentication.urls')), path('api/', include('tests.urls')), ]
```

```
if settings.DEBUG: urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

===== \wsgi.py ===== """ WSGI config for backend project.

It exposes the WSGI callable as a module-level variable named `application`.

For more information on this file,
see <https://docs.djangoproject.com/en/6.0/howto/deployment/wsgi/> """

```
import os
```

```
from django.core.wsgi import get_wsgi_application
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'backend.settings')
```

```
application = get_wsgi_application()
```

```
===== _init_.py =====
```

frontend:

```
===== \api\auth.ts =====
```

```
import api from './axios';
```

```
import {
```

```
    LoginResponse,
```

```
    RegisterResponse,
```

```
    User
```

```
} from '../types/user';
```

```
// ===== АУТЕНТИФИКАЦИЯ =====
```

```
export const register = async (
```

```
    login: string,
```

```
    email: string,
```

```
    password: string
```

```
): Promise<RegisterResponse> => {
```

```
    const response = await api.post<RegisterResponse>('/auth/register/', {
```

```
        login,
```

```

    email,

    password,

    password2: password
  });

  return response.data;
};

export const login = async (
  login: string,
  password: string
): Promise<LoginResponse> => {
  const response = await api.post<LoginResponse>('/auth/login/', {
    login,
    password
  });

  if (response.data.access) {
    localStorage.setItem('access_token', response.data.access);
    localStorage.setItem('refresh_token', response.data.refresh);
    if (response.data.user) {
      localStorage.setItem('user_id', response.data.user.id.toString());
    }
  }

  return response.data;
}

```

```
};
```

```
export const logout = async (refresh?: string): Promise<void> => {  
  const refreshToken = refresh || localStorage.getItem('refresh_token');  
  
  try {  
    if (refreshToken) {  
      await api.post('/auth/logout/', { refresh: refreshToken });  
    }  
  } catch (error) {  
    console.error('Logout error:', error);  
  } finally {  
    localStorage.removeItem('access_token');  
    localStorage.removeItem('refresh_token');  
    localStorage.removeItem('user_id');  
  }  
};
```

```
export const isAuthenticated = (): boolean => {  
  return localStorage.getItem('access_token') !== null;  
};
```

```
export const getProfile = async (): Promise<{ data: User }> => {  
  const response = await api.get('/auth/profile/');  
  return response;  
};
```

```
};
```

```
export const updateProfile = async (data: Partial<User>): Promise<{ data: User }> => {  
    const response = await api.patch('/auth/profile/', data);  
    return response;  
};
```

```
export const getUserProfile = async (userId: string): Promise<{ data: User }> => {  
    const response = await api.get(`/auth/profile/${userId}/`);  
    return response;  
};
```

```
// ===== ДРУЗЬЯ =====
```

```
export const getFriends = async (): Promise<{ data: User[] }> => {  
    const response = await api.get('/friends/');  
    return response;  
};
```

```
export const getFriendRequests = async (): Promise<{ data: any[] }> => {  
    const response = await api.get('/friends/requests/');  
    return response;  
};
```

```
export const sendFriendRequest = async (userId: number): Promise<{ data: any }> => {
```

```

    const response = await api.post(`/friends/send/${userId}/`);

    return response;

};

export const acceptFriendRequest = async (requestId: number): Promise<{ data: any }>
=> {

    const response = await api.post(`/friends/accept/${requestId}/`);

    return response;

};

export const rejectFriendRequest = async (requestId: number): Promise<{ data: any }> =>
{

    const response = await api.delete(`/friends/reject/${requestId}/`);

    return response;

};

export const checkFriendStatus = async (userId: number): Promise<{ data: { is_friend:
boolean; is_friend_request_sent: boolean } }> => {

    const response = await api.get(`/friends/check/${userId}/`);

    return response;

};

export const reportUser = async (data: { target_id: number; reason: string }):
Promise<{ data: any }> => {

    const response = await api.post(`/reports/user/`, data);

    return response;

```

```
};
```

```
export const uploadAvatar = async (file: File): Promise<string> => {  
  const formData = new FormData();  
  formData.append('avatar', file);  
  
  const response = await api.post('/auth/upload-avatar/', formData, {  
    headers: {  
      'Content-Type': 'multipart/form-data',  
    },  
  });  
  
  return response.data.url;  
};
```

```
export const getUserStatus = async (): Promise<string> => {  
  try {  
    const response = await api.get('/auth/profile/');  
    return response.data.status || 'user';  
  } catch (error) {  
    console.error('Ошибка получения статуса:', error);  
    return 'user';  
  }  
};
```



```

export const isAdmin = async (): Promise<boolean> => {

    const status = await getUserStatus();

    return status === 'admin';

};

===== \api\axios.ts =====

import axios, { AxiosInstance, InternalAxiosRequestConfig, AxiosError } from 'axios';
import { RefreshResponse } from '../types/user';

let API_URL = 'http://localhost:8000/api';

const api: AxiosInstance = axios.create({

    baseURL: API_URL,

    headers: {

        'Content-Type': 'application/json',

    },

});

// Публичные эндпоинты, на которые не нужно редиректить при 401
const publicEndpoints = [

    '/tests/',

    '/auth/login/',

    '/auth/register/',

    '/users/top/',

    '/users/search/',

    '/reports/list/',

```

```

    '/auth/profile/',
    '/friends/',
    '/profile/'
  ];

  // Перехватчик запросов
  api.interceptors.request.use(
    (config: InternalAxiosRequestConfig): InternalAxiosRequestConfig => {
      const token = localStorage.getItem('access_token');
      if (token) {
        config.headers.Authorization = `Bearer ${token}`;
      }
      return config;
    },
    (error: AxiosError) => Promise.reject(error)
  );

  // Перехватчик ответов
  api.interceptors.response.use(
    (response) => response,
    async (error: AxiosError) => {
      const originalRequest = error.config as InternalAxiosRequestConfig & { _retry?: boolean };
      // Проверяем, является ли эндпоинт публичным

```

```

const isPublicEndpoint = publicEndpoints.some(endpoint =>
    originalRequest.url?.includes(endpoint)
);

// Если ошибка 401, это не публичный эндпоинт, и это не запрос на обновление
// токена
if (error.response?.status === 401 && !originalRequest._retry && !
isPublicEndpoint) {
    originalRequest._retry = true;

    try {
        const refreshToken = localStorage.getItem('refresh_token');

        const response = await
axios.post<RefreshResponse>(`${API_URL}/auth/refresh/`, {
            refresh: refreshToken
        });

        const newAccessToken = response.data.access;

        localStorage.setItem('access_token', newAccessToken);

        if (originalRequest.headers) {
            originalRequest.headers.Authorization = `Bearer ${newAccessToken}`;
        }

        return api(originalRequest);

    } catch (refreshError) {

```

```

        // Если refresh токен не работает — выходим
        localStorage.removeItem('access_token');
        localStorage.removeItem('refresh_token');
        window.location.href = '/login';
        return Promise.reject(refreshError);
    }
}

// Для публичных эндпоинтов просто возвращаем ошибку без редиректа
return Promise.reject(error);
}

);

export default api;

===== \api\index.ts =====

// frontend/src/services/api/index.ts

export * from './auth';
export * from './tests';

===== \api\tests.ts =====

import api from './axios';

// ===== ЗАГРУЗКА КАРТИНОК НА СЕРВЕР =====

export const uploadImage = async (file: File, type: 'question' | 'test' = 'question'):
Promise<string> => {

    const formData = new FormData();

```

```

formData.append('image', file);

formData.append('type', type);


const response = await api.post('/upload/', formData, {
  headers: {
    'Content-Type': 'multipart/form-data'
  }
});

return response.data.url;
};


export const convertToBase64 = (file: File): Promise<string> => {
  return new Promise((resolve, reject) => {
    const reader = new FileReader();
    reader.readAsDataURL(file);
    reader.onload = () => resolve(reader.result as string);
    reader.onerror = error => reject(error);
  });
};


// ===== ПУЛЫ =====

export const getPools = async () => {
  const response = await api.get('/pools/');
  return response.data;
};

```

```
export const getPool = async (id: number) => {  
    const response = await api.get(`/pools/${id}/`);  
    return response.data;  
};
```

```
export const createPool = async (data: { name: string; description: string }) => {  
    const response = await api.post('/pools/', data);  
    return response.data;  
};
```

```
export const addQuestionToPool = async (poolId: number, questionId: number) => {  
    const response = await api.post(`/pools/${poolId}/questions/`, { question_id: questionId  
});  
    return response.data;  
};
```

```
// ===== ВОПРОСЫ =====
```

```
export const createQuestion = async (questionData: any) => {  
    const response = await api.post('/questions/', questionData);  
    return response.data;  
};
```

```
export const createTest = async (testData: any) => {  
    const response = await api.post('/tests/', testData);
```

```

    return response.data;
};

export const createQuestionsBatch = async (questionsData: any[]) => {
    const response = await api.post('/questions/batch/', { questions: questionsData });
    return response.data;
};

// ===== ДЛЯ ВЫВОДА =====

export const getAllTests = async () => {
    const response = await api.get('/tests/');
    return response.data;
};

export const getTests = async () => {
    const all = await getAllTests();
    return all.filter((item: any) => !item.is_survey);
};

export const getSurveys = async () => {
    const all = await getAllTests();
    return all.filter((item: any) => item.is_survey);
};

export const getTestDetails = async (testId: number) => {

```

```

    const response = await api.get(`/tests/${testId}/`);

    return response.data;
  };

export const getRecentTests = async (limit: number = 8) => {
  const tests = await getTests();

  return tests.sort((a: any, b: any) =>
    new Date(b.created_at).getTime() - new Date(a.created_at).getTime()
  ).slice(0, limit);
};

export const getPopularTests = async (limit: number = 8) => {
  const tests = await getTests();

  return tests.sort((a: any, b: any) =>
    (b.attempts_count || 0) - (a.attempts_count || 0)
  ).slice(0, limit);
};

export const getRecentSurveys = async (limit: number = 8) => {
  const surveys = await getSurveys();

  return surveys.sort((a: any, b: any) =>
    new Date(b.created_at).getTime() - new Date(a.created_at).getTime()
  ).slice(0, limit);
};

```



```

export const getTopUsers = async (limit: number = 10) => {

  const response = await api.get('/users/top/');

  return response.data.slice(0, limit);

};


// ===== ДЛЯ ПРОХОЖДЕНИЯ ТЕСТА =====

export const startTest = async (testId: number) => {

  const response = await api.post(`/tests/${testId}/start/`);

  return response.data;

};


export const getQuestion = async (testId: number, questionIndex: number) => {

  const response = await api.get(`/tests/${testId}/question/${questionIndex}/`);

  return response.data;

};


export const submitAnswer = async (attemptId: number, questionId: number, answer: any)
=> {

  const response = await api.post(`/attempts/${attemptId}/answer/${questionId}/`,
{ answer });

  return response.data;

};


export const finishTest = async (attemptId: number) => {

  const response = await api.post(`/attempts/${attemptId}/finish/`);

```

```

    return response.data;
};

export const rateTest = async (testId: number, rating: number) => {
    const response = await api.post(`/tests/${testId}/rate/`, { rating });
    return response.data;
};

export const getAttemptResults = async (attemptId: number) => {
    const response = await api.get(`/attempts/${attemptId}/`);
    return response.data;
};

// ===== ДЛЯ КОММЕНТАРИЕВ =====

export const getComments = async (testId: number) => {
    const response = await api.get(`/tests/${testId}/comments/`);
    return response.data;
};

export const addComment = async (testId: number, text: string) => {
    const response = await api.post(`/tests/${testId}/comments/`, { text });
    return response.data;
};

export const submitReport = async (data: {

```

```

    target_type: 'test' | 'comment';

    target_id: number;

    test_id?: number; // ДОБАВИТЬ ДЛЯ КОММЕНТАРИЕВ

    reason: string;

    comment?: string;
  }) => {

    const response = await api.post('/reports/', data);

    return response.data;
  };

export const getTestRating = async (testId: number) => {

  const response = await api.get(`/tests/${testId}/rate/`);

  return response.data;
};

===== \components\common\Admin\AdminPanel.module.css =====

@import "../../style/variables.css";

/* СТИЛИ ДЛЯ СТРАНИЦЫ */

.page {

  width: 100%;

  min-height: 100vh;

  background-color: var(--color-background);

  display: flex;

  justify-content: center;

  padding: 30px 20px;

  box-sizing: border-box;

```

```
}
```

```
.container {  
    width: 1700px;  
    max-width: 100%;  
    background-color: var(--color-blockColor);  
    padding: 20px;  
    box-sizing: border-box;  
    border-radius: 8px;  
}
```

```
/* стили для хедера */
```

```
.header {  
    width: 100%;  
    display: flex;  
    justify-content: space-between;  
    align-items: flex-end;  
    padding-bottom: 10px;  
    border-bottom: 1px solid var(--color-border);  
}
```

```
.title {  
    color: var(--color-text);  
    font-size: 32px;  
    font-weight: bold;
```

```
margin: 0;

}
```

```
/* стили для кнопок */
```

```
.adminButton {

    background-color: var(--color-buttonColor);

    color: var(--color-text);

    font-weight: bold;

    padding: 8px 24px;

    border: none;

    border-radius: 8px;

    cursor: pointer;

    transition: all 0.3s ease;

    font-size: 16px;

    outline: none;

}
```

```
.adminButton:hover {

    opacity: 0.8;

    transform: translateY(-2px);

}
```

```
.adminButton:focus,

.adminButton:active {

    outline: none;
```

```
    box-shadow: none;
}

/* стили для фильтров */

.filterBar {
    display: flex;
    gap: 15px;
    margin: 20px 0;
    flex-wrap: wrap;
}

.filterBtn {
    background-color: var(--color-blockColor);
    color: var(--color-text);
    border: none;
    padding: 8px 20px;
    border-radius: 8px;
    font-size: 14px;
    font-weight: bold;
    cursor: pointer;
    transition: all 0.3s ease;
    outline: none;
}

.filterBtn:hover {
```

```
background-color: var(--color-buttonColor);  
  
transform: translateY(-2px);  
  
}
```

```
.filterBtn:focus,  
.filterBtn:active {  
  
    outline: none;  
  
    box-shadow: none;  
  
}
```

```
.activeFilter {  
  
    background-color: var(--color-buttonColor);  
  
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.2);  
  
}
```

```
/* СТИЛИ ДЛЯ ОСНОВНОГО КОНТЕНТА */
```

```
.adminMain {  
  
    width: 100%;  
  
    margin-top: 30px;  
  
}
```

```
.reportList {  
  
    display: flex;  
  
    flex-direction: column;  
  
    gap: 20px;
```

```
}
```

```
/* стили для карточек жалоб */
```

```
.reportCard {  
    width: 100%;  
    background-color: rgba(255, 215, 157, 0.5);  
    padding: 20px;  
    box-sizing: border-box;  
    display: flex;  
    flex-direction: row;  
    align-items: center;  
    justify-content: space-between;  
    gap: 20px;  
    border-radius: 8px;  
    transition: all 0.3s ease;  
}
```

```
.openReport {  
    border-left: 4px solid var(--color-danger);  
}
```

```
.reviewedReport {  
    border-left: 4px solid var(--color-warning);  
}
```



```
.closedReport {  
    border-left: 4px solid var(--color-success);  
}
```

```
/* стили для секции пользователя */
```

```
.userSection {  
    min-width: 250px;  
    display: flex;  
    flex-direction: column;  
    gap: 8px;  
}
```

```
.userLabel {  
    font-size: 12px;  
    color: var(--color-text);  
    text-transform: uppercase;  
    font-weight: bold;  
}
```

```
.userCardLink {  
    text-decoration: none;  
    cursor: pointer;  
}
```

```
.userCardLink:hover .userCard {
```

```
transform: translateY(-2px);  
  
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);  
  
}
```

```
.userCard {  
  
  background-color: var(--color-buttonColor);  
  
  padding: 10px 15px;  
  
  border-radius: 8px;  
  
  display: flex;  
  
  align-items: center;  
  
  gap: 12px;  
  
  transition: all 0.3s ease;  
  
}
```

```
.userAvatar {  
  
  width: 40px;  
  
  height: 40px;  
  
  background-color: var(--color-blockColor);  
  
  border-radius: 50%;  
  
  display: flex;  
  
  align-items: center;  
  
  justify-content: center;  
  
  font-size: 18px;  
  
  font-weight: bold;  
  
  color: var(--color-text);  
  
}
```

```
}
```

```
.avatarPlaceholder {  
    display: none;  
}
```

```
.avatarPlaceholder.show {  
    display: flex;  
}
```

```
.userInfo {  
    flex: 1;  
}
```

```
.username {  
    font-weight: bold;  
    color: var(--color-text);  
    font-size: 15px;  
}
```

```
.reportDate {  
    font-size: 12px;  
    color: var(--color-text);  
}
```

```
/* стили для секций тип/причина/статус */
```

```
.reportType, .reportReason, .reportStatus {  
    min-width: 120px;  
    text-align: center;  
}
```

```
.label {  
    font-weight: bold;  
    color: var(--color-text);  
    text-transform: uppercase;  
    font-size: 11px;  
    margin-bottom: 8px;  
}
```

```
.value {  
    color: var(--color-text);  
    font-size: 14px;  
    word-break: break-word;  
}
```

```
/* стили для комментария */
```

```
.reportCommentInline {  
    margin-top: 8px;  
    padding-top: 8px;  
    border-top: 1px dashed var(--color-border-dark);
```

```
}
```

```
.commentLabel {  
    font-size: 11px;  
    color: var(--color-text);  
    text-transform: uppercase;  
    font-weight: bold;  
    margin-bottom: 4px;  
}
```

```
.commentValue {  
    color: var(--color-text);  
    font-size: 13px;  
    word-break: break-word;  
    background-color: rgba(0, 0, 0, 0.08);  
    padding: 6px 8px;  
    border-radius: 6px;  
    margin-top: 4px;  
}
```

```
/* стили для селекта статуса */
```

```
.statusSelect {  
    background-color: var(--color-buttonColor);  
    color: var(--color-text);  
    border: none;
```

```
padding: 8px 12px;
border-radius: 8px;
font-size: 13px;
cursor: pointer;
margin-top: 8px;
width: 100%;
outline: none;
}
```

```
.statusSelect:hover {
    opacity: 0.8;
}
```

```
.statusSelect:focus,
.statusSelect:active {
    outline: none;
    box-shadow: none;
}
```

```
/* стили для кнопки перехода */
```

```
.linkButton {
    background-color: var(--color-buttonColor);
    color: var(--color-text);
    border: none;
    padding: 10px 30px;
```

```
border-radius: 8px;

cursor: pointer;

font-size: 16px;

transition: all 0.3s ease;

white-space: nowrap;

outline: none;

}
```

```
.linkButton:hover {

    opacity: 0.8;

    transform: translateY(-2px);

}
```

```
.linkButton:focus,

.linkButton:active {

    outline: none;

    box-shadow: none;

}
```

```
/* стили для пустого состояния */
```

```
.noReports {

    text-align: center;

    padding: 50px;

    color: var(--color-text);

    font-size: 18px;
```

```
}
```

```
.loading {  
    text-align: center;  
    padding: 50px;  
    font-size: 18px;  
    color: var(--color-text);  
}
```

```
/* ===== АДАПТИВ ===== */
```

```
@media (max-width: 1200px) {  
    .reportCard {  
        flex-wrap: wrap;  
    }  
}
```

```
.userSection {  
    min-width: 100%;  
}
```

```
.reportType, .reportReason, .reportStatus {  
    flex: 1;  
}
```

```
.linkButton {  
    width: 100%;
```



```
    }  
  }  
}
```

```
@media (max-width: 768px) {
```

```
  .page {  
    padding: 15px;  
  }
```

```
  .container {  
    padding: 15px;  
  }
```

```
  .title {  
    font-size: 24px;  
  }
```

```
  .header {  
    flex-direction: column;  
    align-items: center;  
    gap: 15px;  
  }
```

```
  .filterBar {  
    justify-content: center;  
  }
```

```
.filterBtn {  
    padding: 6px 15px;  
    font-size: 12px;  
}
```

```
.reportCard {  
    flex-direction: column;  
    align-items: stretch;  
}
```

```
.reportType, .reportReason, .reportStatus {  
    text-align: left;  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
}
```

```
.label {  
    margin-bottom: 0;  
}
```

```
.statusSelect {  
    width: auto;  
    min-width: 150px;
```

```
    }  
  }  
}
```

```
@media (max-width: 480px) {
```

```
  .filterBar {  
    gap: 10px;  
  }
```

```
  .filterBtn {  
    padding: 5px 12px;  
    font-size: 11px;  
  }
```

```
  .userCard {  
    padding: 8px 12px;  
  }
```

```
  .userAvatar {  
    width: 35px;  
    height: 35px;  
    font-size: 16px;  
  }
```

```
  .username {  
    font-size: 13px;
```

```
}
```

```
.reportType, .reportReason, .reportStatus {  
    flex-direction: column;  
    align-items: flex-start;  
    gap: 5px;  
}
```

```
.statusSelect {  
    width: 100%;  
}
```

```
.linkButton {  
    padding: 8px 20px;  
    font-size: 14px;  
}  
}
```

```
===== \components\common\Admin\AdminPanelMain.tsx =====
```

```
import React, { useState, useEffect } from 'react';  
import { useNavigate, Link } from 'react-router-dom';  
import api from '../..../api/axios';  
import styles from "../AdminPanel.module.css";  
import { isAdmin } from '../..../api/auth';
```

```
interface Report {
```

```

id: number;

target_type: string;

target_id: number;

user: number;

user_info: {
  id: number;

  login: string;

  full_name: string;

  avatar: string | null;
};

reason: string;

comment: string;

status: 'pending' | 'reviewed' | 'resolved';

created_at: string;
}

```

```

const AdminMain = () => {

  const navigate = useNavigate();

  const [loading, setLoading] = useState(true);

  const [reports, setReports] = useState<Report[]>([]);

  const [filteredReports, setFilteredReports] = useState<Report[]>([]);

  const [activeFilter, setActiveFilter] = useState<string>('all');

  useEffect(() => {

    const checkAdmin = async () => {

```

```

    try {
        const adminStatus = await isAdmin();
        if (!adminStatus) {
            navigate('/');
        }
        await loadReports();
        setLoading(false);
    } catch (error) {
        console.error('Ошибка проверки прав:', error);
        navigate('/');
    }
};

checkAdmin();
}, [navigate]);

const loadReports = async () => {
    try {
        const response = await api.get('/reports/list/');
        console.log('Загружены репорты:', response.data);
        setReports(response.data);
        setFilteredReports(response.data);
    } catch (error) {
        console.error('Ошибка загрузки репортов:', error);
    }
};

```

```

const updateReportStatus = async (reportId: number, status: string) => {
  try {
    await api.patch(`/reports/${reportId}/`, { status });
    await loadReports();
    alert('Статус обновлен');
  } catch (error) {
    console.error('Ошибка обновления статуса:', error);
    alert('Ошибка при обновлении статуса');
  }
};

```

```

const filterByStatus = (status: string) => {
  setActiveFilter(status);
  if (status === 'all') {
    setFilteredReports(reports);
  } else {
    setFilteredReports(reports.filter(report => report.status === status));
  }
};

```

```

const getStatusClass = (status: string) => {
  switch (status) {
    case 'pending': return styles.openReport;
    case 'reviewed': return styles.reviewedReport;
  }
};

```

```

        case 'resolved': return styles.closedReport;

        default: return "";
    }
};

```

```

const getTypeText = (type: string) => {
    switch (type) {
        case 'test': return 'Тест';
        case 'comment': return 'Комментарий';
        case 'user': return 'Пользователь';
        default: return type;
    }
};

```

```

const formatDate = (dateString: string) => {
    return new Date(dateString).toLocaleString('ru-RU');
};

```

```

const getAvatarUrl = (avatar: string | null) => {
    if (!avatar) return null;
    if (avatar.startsWith('http')) return avatar;
    return `http://localhost:8000${avatar}`;
};

```

```

const getStatusCount = (status: string) => {

```



```

    if (status === 'all') return reports.length;

    return reports.filter(r => r.status === status).length;
};

const getTargetLink = (report: Report) => {
    if (report.target_type === 'user') {
        return `/profile/${report.target_id}`;
    }

    if (report.target_type === 'comment') {
        // Для комментариев используем test_id, если есть, иначе target_id
        return `/test/${report.test_id || report.target_id}`;
    }

    return `/test/${report.target_id}`;
};

const getButtonText = (report: Report) => {
    if (report.target_type === 'user') {
        return 'Перейти к профилю';
    }

    return 'Перейти к тесту';
};

if (loading) {
    return <div className={styles.loading}>Проверка прав...</div>;
}

```

```

return (
  <div className={styles.page}>
    <div className={styles.container}>
      <div className={styles.header}>
        <h1 className={styles.title}>РЕПОРТЫ</h1>
      </div>

      <div className={styles.filterBar}>
        <button
          className={` ${styles.filterBtn} ${activeFilter === 'all' ?
styles.activeFilter : ''}
          onClick={() => filterByStatus('all')}
        >
          Все ({getStatusCount('all')})
        </button>
        <button
          className={` ${styles.filterBtn} ${activeFilter === 'pending' ?
styles.activeFilter : ''}
          onClick={() => filterByStatus('pending')}
        >
          На рассмотрении ({getStatusCount('pending')})
        </button>
        <button
          className={` ${styles.filterBtn} ${activeFilter === 'reviewed' ?
styles.activeFilter : ''}

```

```

        onClick={() => filterByStatus('reviewed')}}
    >
        Проверено ({getStatusCount('reviewed')})
    </button>
    <button
        className={` ${styles.filterBtn} ${activeFilter === 'resolved' ?
styles.activeFilter : ''}
        onClick={() => filterByStatus('resolved')}}
    >
        Решено ({getStatusCount('resolved')})
    </button>
</div>

<div className={styles.adminMain}>
    <div className={styles.reportList}>
        {filteredReports.length === 0 ? (
            <div className={styles.noReports}>Нет жалоб</div>
        ) : (
            filteredReports.map((report) => {
                const user = report.user_info;
                const userName = user?.full_name || user?.login || 'Пользователь';
                const userAvatar = user?.avatar;

                return (
                    <div

```

```

        key={report.id}

        className={` ${styles.reportCard} ${
{getStatusClass(report.status)}} `}

    >

    <div className={styles.userSection}>

        <div className={styles.userLabel}>Пользователь</div>

        <Link to={` /profile/${user?.id} `}
className={styles.userCardLink}>

            <div className={styles.userCard}>

                {userAvatar ? (

                    <img

                        src={getAvatarUrl(userAvatar)}

                        alt="avatar"

                        className={styles.userAvatar}

                        onError={(e) => {

                            e.currentTarget.style.display = 'none';

e.currentTarget.parentElement?.querySelector('.avatarPlaceholder')?.classList.add(styles.s
how);

                        }}

                    />

                ) : null}

                <div className={` ${styles.userAvatar} ${
{styles.avatarPlaceholder} ${!userAvatar ? styles.show : ""}} `>

                    {userName.charAt(0).toUpperCase()}

                </div>

                <div className={styles.userInfo}>

```

```

        <div className={styles.username}>
            {userName}
        </div>
    </div>
</div>
</Link>
<div className={styles.reportDate}>
    {formatDate(report.created_at)}
</div>
</div>

<div className={styles.reportType}>
    <div className={styles.label}>Тип</div>
    <div className={styles.value}
>{getTypeText(report.target_type)}</div>
</div>

<div className={styles.reportReason}>
    <div className={styles.label}>Причина</div>
    <div className={styles.value}>{report.reason}</div>
    {report.comment && (
        <div className={styles.reportCommentInline}>
            <div className={styles.commentLabel}
>Комментарий:</div>
            <div className={styles.commentValue}
>{report.comment}</div>

```

```

        </div>

    )}
</div>

<div className={styles.reportStatus}>
    <div className={styles.label}>Статус</div>
    <select
        className={styles.statusSelect}
        value={report.status}
        onChange={(e) => updateReportStatus(report.id,
e.target.value)}
    >
        <option value="pending">На рассмотрении</option>
        <option value="reviewed">Проверено</option>
        <option value="resolved">Решено</option>
    </select>
</div>

<button
    className={styles.linkButton}
    onClick={() => {
        navigate(getTargetLink(report));
    }}
>
    {getButtonText(report)}

```

```

        </button>

    </div>

    );

    })

    })

</div>

</div>

</div>

</div>

);

};

export default AdminMain;

===== \components\common\Auth\BaseAuthLayout.module.css =====

@import "../style/variables.css";

/* стили для основного контейнера */

.main {

    min-height: 100vh;

    display: flex;

    flex-direction: row;

}

/* стили для приветственной секции */

.welcomeSection {

```

```
display: flex;

align-items: center;

justify-content: center;

color: black;

background-color: var(--color-background);

width: 60%;

}
```

```
.welcomeTitle {

color: black;

font-size: 60px;

font-weight: 700;

text-align: center;

padding: 20px;

letter-spacing: 4px;

text-transform: uppercase;

}
```

```
/* стили для формы */
```

```
.formSection {

display: flex;

align-items: center;

justify-content: center;

padding: 40px 5%;

flex-direction: column;
```



```
width: 40%;  
  
background-color: var(--color-buttonColor);  
}
```

```
.formTitle {  
  color: black;  
  font-weight: 700;  
  text-align: center;  
  letter-spacing: 4px;  
  text-transform: uppercase;  
  margin-bottom: 30px;  
}
```

```
.link {  
  color: black;  
  text-decoration: underline;  
  font-weight: 600;  
  letter-spacing: 2px;  
  margin-top: 20px;  
  margin-bottom: 0;  
}
```

```
.form {  
  display: flex;  
  align-items: center;
```

```

justify-content: center;

flex-direction: column;

width: 100%;

max-width: 400px;
}

/* ===== АДАПТИВ ===== */

@media (max-width: 1200px) {

    .welcomeTitle {

        font-size: 50px;

    }


    .formTitle {

        font-size: 45px;

    }


    .link {

        font-size: 24px;

    }

}

@media (max-width: 1000px) {

    .welcomeTitle {

        font-size: 40px;

    }

}

```

```
.formTitle {  
    font-size: 38px;  
    letter-spacing: 3px;  
}
```

```
.link {  
    font-size: 20px;  
}
```

```
.form {  
    max-width: 350px;  
}  
}
```

```
@media (max-width: 900px) {  
    .welcomeTitle {  
        font-size: 35px;  
    }  
}
```

```
.formTitle {  
    font-size: 32px;  
    letter-spacing: 2px;  
}
```

```
.link {  
    font-size: 18px;  
}  
  
.formSection {  
    padding: 30px 4%;  
}  
}  
  
@media (max-width: 768px) {  
    .welcomeSection {  
        display: none;  
    }  
  
    .formSection {  
        width: 100%;  
        padding: 20px 20px 0 20px;  
        min-height: auto;  
    }  
  
    .formTitle {  
        font-size: 36px;  
        margin-bottom: 20px;  
    }  
}
```

```
.link {  
    font-size: 18px;  
    margin-top: 15px;  
    margin-bottom: 0;  
}
```

```
.form {  
    max-width: 100%;  
    width: 100%;  
}  
}
```

```
@media (max-width: 600px) {  
    .formSection {  
        padding: 16px 16px 0 16px;  
        justify-content: none;  
    }  
}
```

```
.formTitle {  
    font-size: 32px;  
    margin-bottom: 15px;  
    letter-spacing: 2px;  
}
```

```
.link {
```

```

    font-size: 16px;

    margin-top: 12px;

    margin-bottom: 0;
  }
}

```

```

@media (max-width: 480px) {

  .formSection {

    padding: 12px 16px 0 16px;

  }
}

```

```

.formTitle {

  font-size: 28px;

  margin-bottom: 12px;

  letter-spacing: 1.5px;

}

```

```

.link {

  font-size: 14px;

  margin-top: 10px;

  margin-bottom: 0;

}

}

```

===== \components\common\Auth\BaseAuthLayout.tsx =====

```
import React from "react";
```

```

import { Link } from 'react-router-dom';

import styles from './BaseAuthLayout.module.css";

interface BaseAuthLayoutProps {

    title: string;

    linkTo: string;

    linkText: string;

    children: React.ReactNode;

}

const BaseAuthLayout = ({ title, children, linkTo, linkText }: BaseAuthLayoutProps) =>
{

    return (

        <main className={styles.main}>

            {/* ПРИВЕТСТВИЕ */}

            <div className={styles.welcomeSection}>

                <h2 className={styles.welcomeTitle}>

                    ДОБРО ПОЖАЛОВАТЬ

                </h2>

            </div>

            {/* ФОРМА */}

            <div className={styles.formSection}>

                <h2 className={styles.formTitle}>

                    {title}


```

</h2>

{children}

<Link

className={styles.link}

to={linkTo}

>

{linkText}

</Link>

</div>

</main>

);

};

export default BaseAuthLayout;

===== \components\common\Auth\LoginMain.module.css =====

.form {

display: flex;

color: black;

align-items: center;

justify-content: center;

flex-direction: column;

width: 100%;

max-width: 400px;


```
}
```

```
/* стили для полей ввода */
```

```
.form input {  
    width: 100%;  
    padding: 12px 16px;  
    background-color: #EAE7DC;  
    border: none;  
    border-radius: 8px;  
    font-size: 16px;  
    color: black;  
    box-shadow: 0 2px 4px rgba(0, 0, 0, 0.1);  
    outline: none;  
}
```

```
.form input::placeholder {  
    color: #555;  
    opacity: 0.7;  
}
```

```
.form input:focus {  
    outline: none;  
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);  
}
```

```

.form input:-webkit-autofill,
.form input:-webkit-autofill:hover,
.form input:-webkit-autofill:focus,
.form input:-webkit-autofill:active {
    -webkit-box-shadow: 0 0 0 30px #EAE7DC inset !important;
    -webkit-text-fill-color: black !important;
    background-color: #EAE7DC !important;
    color: black !important;
}

```

```

@media (max-width: 768px) {
    .form {
        max-width: 100%;
    }
}

```

===== \components\common\Auth\LoginMain.tsx =====

// src/components/LoginMain.tsx

```

import React from 'react';

import InputForm from '../Form/RegisterAuthForm/InputForm';

import BaseAuthLayout from './BaseAuthLayout';

import AuthButton from '../Button/AuthButton/Auth/AuthButton';

import { useLoginForm } from '../../hooks/useAuthForm';

import styles from './LoginMain.module.css';

```

```

const LoginMain = () => {

  const { formData, error, isSubmitting, handleChange, handleSubmit } =
  useLoginForm();

  return (

    <BaseAuthLayout

      title="ВХОД"

      linkTo="/register"

      linkText="ЗАРЕГИСТРИРОВАТЬСЯ"

    >

      {error && (

        <div style={{

          color: 'red',

          textAlign: 'center',

          marginBottom: '15px',

          fontSize: '14px'

        }}>

          {error}

        </div>

      )}

    <form className={styles.form} onSubmit={handleSubmit}>

      <InputForm

        names='login'

        pl='ЛОГИН'

```

```

        value={formData.login}

        onChange={handleChange}
    />

    <InputForm
        names='password'
        pl='ПАРОЛІЬ'
        type="password"
        value={formData.password}
        onChange={handleChange}
    />

    <AuthButton type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'ВХОД...' : 'ВОЙТИ'}
    </AuthButton>

</form>

</BaseAuthLayout>

);

};

export default LoginMain;

===== \components\common\Auth\RegisterMain.module.css =====

/* RegisterMain.module.css */

@import "../style/variables.css";

.form {

```

```
display: flex;

align-items: center;

justify-content: flex-start;

color: black;

flex-direction: column;

width: 100%;

max-width: 400px;

}
```

```
.form input {

    background-color: white;

    color: black;

}
```

```
.checkboxLabel {

    display: flex;

    align-items: center;

    justify-content: center;

    gap: 10px;

    cursor: pointer;

    margin: 20px 0;

}
```

```
.checkbox {

    width: 24px;
```

```
height: 24px;

accent-color: white;

margin: 0;

cursor: pointer;

}
```

```
.checkboxText {

  color: black;

  font-size: 16px;

  line-height: 1;

}
```

```
@media (max-width: 768px) {

  .form {

    max-width: 100%;

    justify-content: flex-start;

  }

}
```

```
.checkboxText {

  font-size: 14px;

}

}
```

```
@media (max-width: 480px) {

  .checkboxText {
```

```

        font-size: 12px;
    }

.checkbox {
    width: 18px;
    height: 18px;
}
}

===== \components\common\Auth\RegisterMain.tsx =====

// src/components/RegisterMain.tsx

import React from 'react';
import InputForm from '../Form/RegisterAuthForm/InputForm';
import BaseAuthLayout from './BaseAuthLayout';
import AuthButton from '../Button/AuthButton/Auth/AuthButton';
import { useRegisterForm } from '../../hooks/useAuthForm';
import styles from './RegisterMain.module.css';

const RegisterMain = () => {

    const { formData, error, isSubmitting, handleChange, handleSubmit } =
    useRegisterForm();

    return (

        <BaseAuthLayout

            title="РЕГИСТРАЦИЯ"

```

```
linkTo="/login"
```

```
linkText="ВОЙТИ"
```

```
>
```

```
{error && (
```

```
  <div style={{
```

```
    color: 'red',
```

```
    textAlign: 'center',
```

```
    marginBottom: '15px',
```

```
    fontSize: '14px'
```

```
  }}>
```

```
    {error}
```

```
  </div>
```

```
)}
```

```
<form className={styles.form} onSubmit={handleSubmit}>
```

```
  <InputForm
```

```
    names='login'
```

```
    pl='ЛОГИН'
```

```
    value={formData.login}
```

```
    onChange={handleChange}
```

```
  />
```

```
  <InputForm
```

```
    names='email'
```

```
    pl='EMAIL'
```

```
    value={formData.email}
```



```

        onChange={handleChange}
      />

      <InputForm
        names='password'
        pl='ПАРОЛІЬ'
        type="password"
        value={formData.password}
        onChange={handleChange}
      />

      <InputForm
        names='confirmPassword'
        pl='ПОВТОР ПАРОЛЯ'
        type="password"
        value={formData.confirmPassword}
        onChange={handleChange}
      />

      <label className={styles.checkboxLabel}>

        <input

          type="checkbox"

          name="subscribe"

          checked={formData.subscribe}

          onChange={handleChange}

          className={styles.checkbox}

        />

```

```

        <span className={styles.checkboxText}>
            Я согласен на обработку персональных данных
        </span>
    </label>

    <AuthButton type="submit" disabled={isSubmitting}>
        {isSubmitting ? 'РЕГИСТРАЦИЯ...' : 'ЗАРЕГИСТРИРОВАТЬСЯ'}
    </AuthButton>
</form>
</BaseAuthLayout>

);

};

export default RegisterMain;

===== \components\common\BanMute\MuteBanModal.module.css =====

@import "../style/variables.css";

.modalOverlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: var(--color-overlay);
    display: flex;

```

```
    justify-content: center;

    align-items: center;

    z-index: 1000;
}
```

```
.modal {

    background-color: var(--color-blockColor);

    border-radius: 10px;

    padding: 30px;

    width: 450px;

    max-width: 90%;

    box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);
}
```

```
.modalTitle {

    font-size: 24px;

    color: var(--color-text);

    margin-bottom: 20px;

    text-align: center;
}
```

```
.formGroup {

    margin-bottom: 20px;
}
```

```
.label {  
    display: block;  
    color: var(--color-text);  
    font-size: 14px;  
    margin-bottom: 8px;  
    font-weight: bold;  
}
```

```
.textarea {  
    width: 100%;  
    padding: 10px;  
    border: 1px solid var(--color-buttonColor);  
    border-radius: 8px;  
    background-color: var(--color-background);  
    color: var(--color-text);  
    font-size: 14px;  
    resize: vertical;  
    box-sizing: border-box;  
}
```

```
.textarea:focus {  
    outline: none;  
    border-color: var(--color-hover);  
}
```

```
.durationButtons {  
    display: flex;  
    flex-wrap: wrap;  
    gap: 10px;  
}
```

```
.durationBtn {  
    background-color: var(--color-buttonColor);  
    color: var(--color-text);  
    border: none;  
    padding: 8px 12px;  
    border-radius: 6px;  
    cursor: pointer;  
    font-size: 13px;  
}
```

```
.durationBtn:hover {  
    background-color: var(--color-hover);  
}
```

```
.durationBtn.active {  
    background-color: #8B6B3A;  
    box-shadow: inset 0 2px 4px rgba(0, 0, 0, 0.2);  
}
```

```
.customDuration {  
  display: flex;  
  gap: 10px;  
  margin-top: 10px;  
}
```

```
.customInput {  
  flex: 1;  
  padding: 8px;  
  border: 1px solid var(--color-buttonColor);  
  border-radius: 6px;  
  background-color: var(--color-background);  
  color: var(--color-text);  
}
```

```
.customSelect {  
  padding: 8px;  
  border: 1px solid var(--color-buttonColor);  
  border-radius: 6px;  
  background-color: var(--color-background);  
  color: var(--color-text);  
  cursor: pointer;  
}
```

```
.modalButtons {
```

```

display: flex;

justify-content: flex-end;

gap: 15px;

margin-top: 20px;
}

```

```

.cancelButton,

.submitButton {

background-color: var(--color-buttonColor);

border: none;

border-radius: 8px;

padding: 10px 20px;

font-size: 14px;

color: var(--color-text);

cursor: pointer;

font-weight: bold;
}

```

```

.cancelButton:hover,

.submitButton:hover {

background-color: var(--color-hover);
}

```

===== \components\common\BanMute\MuteBanModal.tsx =====

```

import React, { useState } from 'react';

import styles from './MuteBanModal.module.css';

```

```
interface MuteBanModalProps {

  isOpen: boolean;

  onClose: () => void;

  onSubmit: (reason: string, duration: string, durationValue: number | null, durationUnit:
string | null) => void;

  title: string;

  actionType: 'mute' | 'ban';

}
```

```
const MuteBanModal: React.FC<MuteBanModalProps> = ({

  isOpen,

  onClose,

  onSubmit,

  title,

  actionType

}) => {

  const [reason, setReason] = useState("");

  const [duration, setDuration] = useState('1d');

  const [customDuration, setCustomDuration] = useState("");

  const [customUnit, setCustomUnit] = useState('days');

  const durationOptions = [

    { label: '1 час', value: '1h', duration: 'temporary', value_num: 1, unit: 'hours' },

    { label: '1 день', value: '1d', duration: 'temporary', value_num: 1, unit: 'days' },
```



```

    { label: '3 дня', value: '3d', duration: 'temporary', value_num: 3, unit: 'days' },
    { label: '1 неделя', value: '1w', duration: 'temporary', value_num: 1, unit: 'weeks' },
    { label: '1 месяц', value: '1m', duration: 'temporary', value_num: 1, unit: 'months' },
    { label: 'Перманентно', value: 'permanent', duration: 'permanent', value_num: null,
unit: null },
    { label: 'Своя длительность', value: 'custom', duration: 'temporary', value_num: null,
unit: null },
];

```

```

const handleSubmit = () => {

```

```

    if (!reason.trim()) {
        alert('Введите причину');
        return;
    }

```

```

let durationType = 'temporary';
let durationValue: number | null = null;
let durationUnit: string | null = null;

```

```

const selected = durationOptions.find(opt => opt.value === duration);

```

```

if (selected?.value === 'permanent') {
    durationType = 'permanent';
} else if (selected?.value === 'custom') {
    if (!customDuration || parseInt(customDuration) <= 0) {

```

```

        alert('Введите корректную длительность');

        return;
    }

    durationType = 'temporary';

    durationValue = parseInt(customDuration);

    durationUnit = customUnit;
} else if (selected) {
    durationType = selected.duration;

    durationValue = selected.value_num;

    durationUnit = selected.unit;
}

onSubmit(reason, durationType, durationValue, durationUnit);

setReason("");

setDuration('1d');

setCustomDuration("");

setCustomUnit('days');

onClose();

};

const handleClose = () => {
    setReason("");

    setDuration('1d');

    setCustomDuration("");

    onClose();

```

```
};
```

```
if (!isOpen) return null;
```

```
return (
```

```
  <div className={styles.modalOverlay} onClick={handleClose}>
```

```
    <div className={styles.modal} onClick={(e) => e.stopPropagation()}>
```

```
      <h3 className={styles.modalTitle}>{title}</h3>
```

```
      <div className={styles.formGroup}>
```

```
        <label className={styles.label}>Причина</label>
```

```
        <textarea
```

```
          className={styles.textarea}
```

```
          value={reason}
```

```
          onChange={(e) => setReason(e.target.value)}
```

```
          placeholder="Укажите причину..."
```

```
          rows={3}
```

```
        />
```

```
      </div>
```

```
    <div className={styles.formGroup}>
```

```
      <label className={styles.label}>Длительность</label>
```

```
      <div className={styles.durationButtons}>
```

```
        {durationOptions.map((opt) => (
```

```
          <button
```

```

        key={opt.value}

        type="button"

        className={` ${styles.durationBtn} ${duration === opt.value ?
styles.active : ''}

        onClick={() => setDuration(opt.value)}

      >

        {opt.label}

      </button>

    )}

  </div>

</div>

```

```

{duration === 'custom' && (

  <div className={styles.customDuration}>

    <input

      type="number"

      className={styles.customInput}

      value={customDuration}

      onChange={(e) => setCustomDuration(e.target.value)}

      placeholder="Число"

      min="1"

    />

    <select

      className={styles.customSelect}

      value={customUnit}

```

```

        onChange={(e) => setCustomUnit(e.target.value)}
      >
      <option value="hours">Часы</option>
      <option value="days">Дни</option>
      <option value="weeks">Недели</option>
      <option value="months">Месяцы</option>
    </select>
  </div>
)}

<div className={styles.modalButtons}>
  <button className={styles.cancelButton} onClick={handleClose}>
    Отмена
  </button>
  <button className={styles.submitButton} onClick={handleSubmit}>
    {actionType === 'mute' ? 'Замутить' : 'Забанить'}
  </button>
</div>
</div>
</div>
);
};

export default MuteBanModal;

===== \components\common\Button\AuthButton\Auth\AuthButton.module.css

```

=====

```
@import "../../../style/variables.css";
```

```
.button {  
    border-radius: 10px;  
    padding: 12px 24px;  
    font-size: 20px;  
    margin: 15px 0 5px 0;  
    width: 100%;  
    max-width: 440px;  
    height: 50px;  
    background-color: var(--color-buttonColor);  
    border: none;  
    outline: none;  
    color: var(--color-text);  
    cursor: pointer;  
    font-weight: 600;  
    letter-spacing: 1px;  
    transition: background-color 0.2s;  
    box-sizing: border-box;  
}
```

```
.button:hover {  
    background-color: var(--color-hover);  
}
```

```
.button:focus,  
.button:active {  
    outline: none;  
    border: none;  
}
```

```
.button:disabled {  
    opacity: 0.6;  
    cursor: not-allowed;  
}
```

```
/* ===== АДАПТИВ ===== */
```

```
@media (max-width: 1000px) {  
    .button {  
        font-size: 18px;  
        height: 48px;  
        padding: 10px 20px;  
    }  
}
```

```
@media (max-width: 900px) {  
    .button {  
        font-size: 16px;  
        height: 46px;
```

```
padding: 10px 20px;
margin: 12px 0 4px 0;
}
}
```

```
@media (max-width: 768px) {
  .button {
    max-width: 100%;
    font-size: 16px;
    height: 48px;
    padding: 10px 20px;
    margin: 15px 0 5px 0;
  }
}
```

```
@media (max-width: 600px) {
  .button {
    font-size: 15px;
    height: 45px;
    padding: 10px 16px;
    margin: 12px 0 4px 0;
  }
}
```

```
@media (max-width: 480px) {
```



```

.button {
    font-size: 14px;
    height: 44px;
    padding: 8px 16px;
    margin: 10px 0 4px 0;
    border-radius: 8px;
}
}

===== \components\common\Button\AuthButton\Auth\AuthButton.tsx
=====

import React from "react";
import styles from "../AuthButton.module.css";

interface AuthButtonProps {
    children: React.ReactNode;
    type?: 'button' | 'submit' | 'reset';
    onClick?: () => void;
    disabled?: boolean;
}

const AuthButton = ({
    children,
    type = 'button',
    onClick,
    disabled = false

```

```
}: AuthButtonProps) => {
```

```
  return (
```

```
    <button
```

```
      className={styles.button}
```

```
      type={type}
```

```
      onClick={onClick}
```

```
      disabled={disabled}
```

```
    >
```

```
      {children}
```

```
    </button>
```

```
  );
```

```
};
```

```
export default AuthButton;
```

```
===== \components\common\Button\AuthButton\Report\ReportButton.module.css  
=====
```

```
@import "../../../style/variables.css";
```

```
.reportButton {
```

```
  position: absolute;
```

```
  top: 15px;
```

```
  right: 15px;
```

```
  width: 45px;
```

```
  height: 45px;
```

```
  background-color: var(--color-buttonColor);
```

```

border: none;

border-radius: 10px;

color: var(--color-text);

font-size: 20px;

font-weight: bold;

cursor: pointer;

display: flex;

align-items: center;

justify-content: center;

z-index: 10;

transition: background-color 0.3s;
}

```

```

.reportButton:focus,
.reportButton:active {
    outline: none;
}

```

```

.reportButton:hover {
    background-color: var(--color-hover);
}

```

```

===== \components\common\Button\AuthButton\Report\ReportButton.tsx
=====

```

```

import React from 'react';

import styles from './ReportButton.module.css';

```

```
interface ReportButtonProps {
  onClick: () => void;
}
```

```
const ReportButton: React.FC<ReportButtonProps> = ({ onClick }) => {
  return (
    <button
      className={styles.reportButton}
      onClick={onClick}
    >
      !
    </button>
  );
};
```

```
export default ReportButton;
```

```
===== \components\common\Card\TestCard\Card.module.css =====
```

```
@import "../style/variables.css";
```

```
.card {
  background-color: var(--color-blockColor);
  width: 340px;
  height: 500px;
  border-radius: 10px;
```

```
    color: var(--color-text);
}

/* Основная ссылка карточки */
.testLink {
    text-decoration: none;
    color: var(--color-text);
    display: block;
    height: 100%;
}

.testLink:hover {
    color: var(--color-text);
}

.imageContainer {
    width: 100%;
    height: 180px;
    overflow: hidden;
    border-radius: 10px 10px 0 0;
    background-color: var(--color-avatar);
}

.image {
    width: 100%;
```

```
height: 100%;  
object-fit: cover;  
}
```

```
.imagePlaceholder {  
width: 100%;  
height: 100%;  
background-color: var(--color-avatar);  
}
```

```
.authorBlock {  
height: 45px;  
margin-bottom: 15px;  
padding: 7px;  
}
```

```
.authorChip {  
background-color: var(--color-buttonColor);  
height: 40px;  
width: 160px;  
border-radius: 10px;  
display: flex;  
align-items: center;  
cursor: pointer;  
}
```

```
.authorAvatar {  
  background-color: var(--color-avatar);  
  height: 32px;  
  width: 32px;  
  border-radius: 50%;  
  margin-left: 4px;  
  overflow: hidden;  
}
```

```
.avatarImage {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

```
.avatarPlaceholder {  
  width: 100%;  
  height: 100%;  
  background-color: var(--color-avatar);  
}
```

```
.authorName {  
  margin-left: 8px;  
  color: var(--color-text);
```

```
font-size: 15px;  
font-weight: 600;  
}
```

```
.content {  
  display: flex;  
  flex-direction: column;  
  height: 45%;  
  padding: 0 7px 0 7px;  
}
```

```
.title {  
  margin: 0;  
  font-size: 1.5rem;  
  color: var(--color-text);  
}
```

```
.description {  
  margin: 5px 0 0 0;  
  color: var(--color-text);  
}
```

```
.surveyBadge {  
  color: var(--color-text-muted);  
  font-size: 14px;
```



```
margin: 5px 0 0 0;  
}
```

```
.stats {  
padding: 0 12px 5px 12px;  
font-size: 15px;  
}
```

```
.divider {  
height: 1px;  
background-color: var(--color-text);  
border: none;  
margin: 0;  
}
```

```
.statsRow {  
margin-top: 1px;  
display: flex;  
justify-content: space-between;  
}
```

```
.statItem {  
margin: 0;  
color: var(--color-text);  
}
```

```
===== \components\common\Card\TestCard\Card.tsx =====
```

```
import React from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
import styles from './Card.module.css';
```

```
interface CardProps {
```

```
  title: string;
```

```
  description?: string;
```

```
  author?: string;
```

```
  rating?: string;
```

```
  users?: string;
```

```
  questions?: string;
```

```
  imageUrl?: string;
```

```
  type?: 'test' | 'survey';
```

```
  testId?: string;
```

```
  authorId?: string;
```

```
  authorAvatar?: string;
```

```
}
```

```
const Card = ({
```

```
  title,
```

```
  description,
```

```
  author,
```

```
  rating,
```

```
  users,
```

```

questions,

imageUrl,

type,

testId = '#',

authorId = '#',

authorAvatar

}: CardProps) => {

  const testLink = type === 'test' ? `/test/${testId}` : `/survey/${testId}`;

  const profileLink = `/profile/${authorId}`;

  // Обработчик клика по автору - навигация через JS, не через ссылку

  const handleAuthorClick = (e: React.MouseEvent) => {

    e.preventDefault();

    e.stopPropagation();

    window.location.href = profileLink;

  };

  return (

    <div className={styles.card}>

      <Link to={testLink} className={styles.testLink}>

        <div className={styles.imageContainer}>

          {imageUrl ? (

            <img

              src={imageUrl}

              alt={title}


```

```

        className={styles.image}

        onError={(e) => {

            console.error('❌ Ошибка загрузки картинки:', imageUrl);

            e.currentTarget.style.display = 'none';

        }}

    />

): (

    <div className={styles.imagePlaceholder} />

)}

</div>

<div className={styles.authorBlock}>

    <div

        className={styles.authorChip}

        onClick={handleAuthorClick}

    >

        <div className={styles.authorAvatar}>

            {authorAvatar ? (

                <img

                    src={authorAvatar}

                    alt={author}

                    className={styles.avatarImage}

                    onError={(e) => {

                        console.error('❌ Ошибка загрузки аватара:', authorAvatar);

                        e.currentTarget.style.display = 'none';

```

```

    }}

  />

): (

  <div className={styles.avatarPlaceholder} />

)}

</div>

<p className={styles.authorName}>{author}</p>

</div>

</div>

<div className={styles.content}>

  <h3 className={styles.title}>{title}</h3>

  {description && <p className={styles.description}>{description}</p>}

  {type === 'survey' && <p className={styles.surveyBadge}>опросник</p>}}

</div>

<div className={styles.stats}>

  <hr className={styles.divider} />

  <div className={styles.statsRow}>

    <p className={styles.statItem}>★ {rating}</p>

    <p className={styles.statItem}>👤 {users}</p>

    <p className={styles.statItem}>📊 {questions}</p>

  </div>

</div>

</Link>

```

```

    </div>

);

};

export default Card;

===== \components\common\Card\UserCard\Card.module.css =====

@import "../style/variables.css";

.cardLink {
    text-decoration: none;
    display: block;
}

.card {
    background-color: var(--color-buttonColor);
    padding: 6px 10px;
    width: 180px;
    border-radius: 8px;
    color: var(--color-text);
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    cursor: pointer;
}

.card:hover {
    transform: translateY(-2px);

```

```
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
}

/* Блок автора */

.authorBlock {
    margin-bottom: 4px;
}

.authorChip {
    background-color: var(--color-buttonColor);
    border-radius: 6px;
    display: flex;
    align-items: center;
    gap: 8px;
    width: 100%;
}

/* Аватар */

.authorAvatar {
    width: 32px;
    height: 32px;
    border-radius: 50%;
    overflow: hidden;
    flex-shrink: 0;
    background-color: var(--color-avatar);
```

```
}
```

```
.avatarImage {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  display: block;  
}
```

```
.avatarPlaceholder {  
  width: 100%;  
  height: 100%;  
  background-color: var(--color-avatar);  
}
```

```
/* Имя автора */
```

```
.authorName {  
  margin: 0;  
  color: var(--color-text);  
  font-weight: 700;  
  font-size: 12px;  
  overflow: hidden;  
  text-overflow: ellipsis;  
  white-space: nowrap;  
  flex: 1;
```



```
}
```

```
/* Статистика */
```

```
.stats {
```

```
padding-top: 4px;
```

```
font-size: 14px;
```

```
}
```

```
.statsRow {
```

```
display: flex;
```

```
justify-content: space-between;
```

```
gap: 8px;
```

```
}
```

```
.statItem {
```

```
margin: 0;
```

```
color: var(--color-text);
```

```
font-weight: 700;
```

```
font-size: 14px;
```

```
display: flex;
```

```
align-items: center;
```

```
gap: 4px;
```

```
}
```

```
===== \components\common\Card\UserCard\Card.tsx =====
```

```
import React from 'react';
```

```
import styles from './Card.module.css';
```

```
interface CardProps {  
  name: string;  
  completed: string;  
  created: string;  
  userId: string;  
  avatar?: string;  
}
```

```
const Card = ({  
  name,  
  completed,  
  created,  
  userId,  
  avatar  
}: CardProps) => {  
  const profileLink = `/profile/${userId}`;  
  
  return (  
    <a href={profileLink} className={styles.cardLink} style={{ textDecoration: 'none' }}  
    >  
      <div className={styles.card}>  
        <div className={styles.authorBlock}>  
          <div className={styles.authorChip}>
```

```

<div className={styles.authorAvatar}>
  {avatar ? (
    <img
      src={avatar}
      alt={name}
      className={styles.avatarImage}
      onError={(e) => {
        e.currentTarget.style.display = 'none';
        e.currentTarget.parentElement?.classList.add(styles.avatarPlaceholder);
      }}
    />
  ) : (
    <div className={styles.avatarPlaceholder} />
  )}
</div>

<p className={styles.authorName}>{name}</p>
</div>
</div>

<div className={styles.stats}>
  <div className={styles.statsRow}>
    <p className={styles.statItem}>✅ {completed}</p>
    <p className={styles.statItem}>🔧 {created}</p>
  </div>
</div>

```

```

        </div>

    </a>

);

};

export default Card;

===== \components\common\Chip\userChip\AuthorChip.module.css
=====

@import "../../../style/variables.css";

.authorLink {
    text-decoration: none;
}

.authorChip {
    display: flex;
    align-items: center;
    gap: 10px;
    background-color: var(--color-buttonColor);
    padding: 5px 15px 5px 8px;
    border-radius: 30px;
    transition: background-color 0.3s ease;
    cursor: pointer;
    width: fit-content;
}

```

```
.authorChip:hover {  
    background-color: var(--color-hover);  
}
```

```
.authorAvatar {  
    width: 30px;  
    height: 30px;  
    border-radius: 50%;  
    overflow: hidden;  
    flex-shrink: 0;  
    background-color: var(--color-avatar);  
}
```

```
.avatarImage {  
    width: 100%;  
    height: 100%;  
    object-fit: cover;  
}
```

```
.avatarPlaceholder {  
    width: 100%;  
    height: 100%;  
    background-color: var(--color-avatar);  
}
```

```

.authorName {
  color: var(--color-text);

  font-weight: 700;

  font-size: 14px;
}

===== \components\common\Chip\userChip\AuthorChip.tsx =====

// src/components/common/AuthorChip/AuthorChip.tsx

import React from 'react';

import { Link } from 'react-router-dom';

import styles from './AuthorChip.module.css';

interface AuthorChipProps {
  authorId: number;

  authorName: string;

  authorAvatar?: string | null;
}

const AuthorChip: React.FC<AuthorChipProps> = ({ authorId, authorName, authorAvatar
}) => {

  const BASE_URL = 'http://localhost:8000';

  const getAvatarUrl = (avatar: string | null | undefined) => {

    if (!avatar) return null;

    if (avatar.startsWith('http://') || avatar.startsWith('https://')) {

```

```

    return avatar;
  }

  return `${BASE_URL}${avatar}`;
};

const avatarUrl = getAvatarUrl(authorAvatar);

return (
  <Link to={`/profile/${authorId}`} className={styles.authorLink}>
    <div className={styles.authorChip}>
      <div className={styles.authorAvatar}>
        {avatarUrl ? (
          <img
            src={avatarUrl}
            alt={authorName}
            className={styles.avatarImage}
            onError={(e) => {
              e.currentTarget.style.display = 'none';
            }}
          />
        ) : (
          <div className={styles.avatarPlaceholder} />
        )}
      </div>
      <span className={styles.authorName}>{authorName}</span>
    </div>
  </div>

```

```

        </div>

    </Link>

    );

};

export default AuthorChip;

===== \components\common\Chip\Chip.module.css =====

@import "../style/variables.css";

.chip {
    height: 50px;
    width: 155px;
    padding: 0 20px;
    background-color: var(--color-blockColor);
    border-radius: 10px;
    color: var(--color-text);
    font-size: 30px;
    border: none;
    cursor: pointer;
    display: inline-flex;
    align-items: center;
    justify-content: center;
    transition: opacity 0.2s;
    white-space: nowrap;
}

```



```
.chip:hover {  
    opacity: 0.8;  
}
```

```
/* Тип: тег (слева) */
```

```
.tag {  
    margin-right: 10px;  
}
```

```
/* Тип: фильтр (справа) */
```

```
.filter {  
    height: 50px;  
    width: 60px;  
    background-color: var(--color-blockColor);  
}
```

```
.active {  
    background-color: #7E7E7E;  
    color: white;  
}
```

```
===== \components\common\Chip\Chip.tsx =====
```

```
import React from 'react';
```

```
import styles from './Chip.module.css';
```

```

interface ChipProps {
  label: string;
  type?: 'tag' | 'filter';
  onClick?: () => void;
  isActive?: boolean;
}

const Chip = ({
  label,
  type = 'tag',
  onClick,
  isActive = false
}: ChipProps) => {
  return (
    <button
      className={` ${styles.chip} ${type === 'filter' ? styles.filter : styles.tag} ${
        isActive ? styles.active : ''
      }`
      onClick={onClick}
    >
      {label}
    </button>
  );
};

export default Chip;

```

```
===== \components\common\Create\Questions\AddToPoolModal.tsx
=====
```

```
import React from 'react';
```

```
import { QuestionPool } from '../hooks/usePools';
```

```
import styles from './QuestionBuilder.module.css';
```

```
interface AddToPoolModalProps {
```

```
  isOpen: boolean;
```

```
  pools: QuestionPool[];
```

```
  selectedPoolId: number | null;
```

```
  onClose: () => void;
```

```
  onSelectPool: (poolId: number) => void;
```

```
  onConfirm: () => void;
```

```
  onCreatePool: () => void; // Добавляем пропс для создания пула
```

```
}
```

```
const AddToPoolModal: React.FC<AddToPoolModalProps> = ({
```

```
  isOpen,
```

```
  pools,
```

```
  selectedPoolId,
```

```
  onClose,
```

```
  onSelectPool,
```

```
  onConfirm,
```

```
  onCreatePool // Добавляем
```

```
  }) => {
```

```
if (!isOpen) return null;
```

```
return (
```

```
<div className={styles.modalOverlay} onClick={onClose}>
```

```
<div className={styles.poolModal} onClick={(e) => e.stopPropagation()}>
```

```
<div className={styles.modalHeader}>
```

```
<h3>выберите пул для сохранения вопроса</h3>
```

```
<button className={styles.closeModalBtn} onClick={onClose}>✕</button>
```

```
</div>
```

```
<div className={styles.poolsList}>
```

```
{pools.map(pool => (
```

```
<div
```

```
key={pool.id}
```

```
className={` ${styles.poolCard} ${selectedPoolId === pool.id ? styles.selected :  
"}`}
```

```
onClick={() => onSelectPool(pool.id)}
```

```
>
```

```
<div className={styles.poolCardHeader}>
```

```
<h4>{pool.title}</h4>
```

```
<span className={styles.questionsCount}>{pool.questions.length}  
вопросов</span>
```

```
</div>
```

```
<p className={styles.poolDescription}>{pool.description}</p>
```

```
</div>
```

```
)))
```

```

</div>

<button className={styles.createPoolBtn} onClick={onCreatePool}>
  + создать новый пул вопросов
</button>

<div className={styles.modalButtons}>
  <button type="button" className={styles.cancelBtn} onClick={onClose}>отмена</button>

  <button type="button" className={styles.confirmBtn} onClick={onConfirm}>добавить</button>
</div>

</div>

</div>

</div>

);

};

```

```
export default AddToPoolModal;
```

```
===== \components\common\Create\Questions\CreatePoolModal.tsx
=====
```

```
import React, { useState } from 'react';
```

```
import styles from './QuestionBuilder.module.css';
```

```
interface CreatePoolModalProps {
```

```
  isOpen: boolean;
```

```
  onClose: () => void;
```

```
  onCreate: (title: string, description: string) => Promise<void>;
```

```
}
```

```

const CreatePoolModal: React.FC<CreatePoolModalProps> = ({ isOpen, onClose,
onCreate }) => {

  const [title, setTitle] = useState("");

  const [description, setDescription] = useState("");


  if (!isOpen) return null;


  const handleSubmit = async () => {

    if (!title.trim()) {

      alert('Введите название пула');

      return;

    }

    await onCreate(title, description);

    setTitle("");

    setDescription("");

    onClose();

  };


  return (

    <div className={styles.createPoolModalOverlay} onClick={onClose}>

      <div className={styles.createPoolModal} onClick={(e) => e.stopPropagation()}>

        <h3>создать новый пул вопросов</h3>

        <input

          type="text"

```

```

        maxLength={50}

        value={title}

        onChange={(e) => setTitle(e.target.value)}

        placeholder="название пула"
    />

    <textarea

        maxLength={200}

        value={description}

        onChange={(e) => setDescription(e.target.value)}

        placeholder="описание пула"

        rows={3}

    />

    <div className={styles.modalButtons}>

        <button type="button" className={styles.cancelBtn} onClick={onClose}
>отмена</button>

        <button type="button" className={styles.confirmBtn} onClick={handleSubmit}
>создать</button>

    </div>

</div>

</div>

);

};

export default CreatePoolModal;

===== \components\common\Create\Questions\PoolModal.tsx =====

```

```
import React from 'react';

import { QuestionPool, PoolQuestion } from '../../../../hooks/usePools';

import styles from './QuestionBuilder.module.css';
```

```
interface PoolModalProps {

  isOpen: boolean;

  pools: QuestionPool[];

  selectedPoolId: number | null;

  onClose: () => void;

  onSelectPool: (poolId: number) => void;

  onBack: () => void;

  onSelectQuestion: (question: PoolQuestion) => void;

  onCreatePool: () => void;

}
```

```
const PoolModal: React.FC<PoolModalProps> = ({

  isOpen,

  pools,

  selectedPoolId,

  onClose,

  onSelectPool,

  onBack,

  onSelectQuestion,

  onCreatePool

}) => {
```



```
if (!isOpen) return null;
```

```
const selectedPool = pools.find(p => p.id === selectedPoolId);
```

```
return (
```

```
  <div className={styles.modalOverlay} onClick={onClose}>
```

```
    <div className={styles.poolModal} onClick={(e) => e.stopPropagation()}>
```

```
      {selectedPoolId === null ? (
```

```
        <
```

```
          <div className={styles.modalHeader}>
```

```
            <h3>выберите пул вопросов</h3>
```

```
            <button className={styles.closeModalBtn} onClick={onClose}>✕</button>
```

```
          </div>
```

```
          <div className={styles.poolsList}>
```

```
            {pools.map(pool => (
```

```
              <div key={pool.id} className={styles.poolCard} onClick={() =>
onSelectPool(pool.id)}>
```

```
                <div className={styles.poolCardHeader}>
```

```
                  <h4>{pool.title}</h4>
```

```
                  <span className={styles.questionsCount}>{pool.questions.length}
вопросов</span>
```

```
                </div>
```

```
                <p className={styles.poolDescription}>{pool.description}</p>
```

```
              </div>
```

```
            )))
```

```

</div>

<button className={styles.createPoolBtn} onClick={onCreatePool}>

  + создать новый пул вопросов

</button>

</>

):(

<div className={styles.modalHeader}>

  <button className={styles.backBtn} onClick={onBack}>←</button>

  <h3>{selectedPool?.title}</h3>

  <button className={styles.closeModalBtn} onClick={onClose}>✕</button>

</div>

<p className={styles.poolDescriptionFull}>{selectedPool?.description}</p>

<div className={styles.questionsList}>

  {selectedPool?.questions.map((question, idx) => (

    <div key={question.id} className={styles.poolQuestionCard} onClick={() =>
onSelectQuestion(question)}>

      <div className={styles.questionCardHeader}>

        <span className={styles.questionNumber}>Вопрос {idx + 1}</span>

        <button className={styles.selectQuestionBtn}>выбрать</button>

      </div>

      <p className={styles.questionCardText}>{question.questionText}</p>

      <div className={styles.answersPreview}>

        <div className={` ${styles.previewAnswer} ${styles.correct}`}>

          ✓ {question.rightAnswer}


```

```

        </div>

        {question.wrongAnswers.map((answer, i) => (

            answer && (

                <div key={i} className={styles.previewAnswer}>

                    ✖ {answer}

                </div>

            )

        )})

    </div>

</div>

    )})

</div>

</>

    })

</div>

</div>

);

};

```

```
export default PoolModal;
```

```

===== \components\common\Create\Questions\QuestionBuilder.module.css
=====

```

```
@import "../../style/variables.css";
```

```
.questionBuilder {
```

```
    margin-top: 20px;
}
```

```
.questionBlock {
    background: var(--color-blockColor);
    border-radius: 15px;
    padding: 20px;
    margin-top: 20px;
}
```

```
.questionHead {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 15px;
}
```

```
.questionHead h3 {
    font-size: 20px;
    font-weight: bold;
    color: var(--color-text);
    margin: 0;
    text-transform: lowercase;
}
```

```
.deleteBtn {  
  width: 32px;  
  height: 32px;  
  background: transparent;  
  border: none;  
  font-size: 24px;  
  font-weight: bold;  
  color: var(--color-text);  
  cursor: pointer;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  transition: transform 0.2s ease;  
}
```

```
.deleteBtn:hover {  
  transform: scale(1.2);  
}
```

```
.questionInner {  
  display: flex;  
  gap: 20px;  
  flex-wrap: wrap;  
}
```

```
.questionLeft {  
    flex: 1;  
    min-width: 180px;  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    gap: 10px;  
}
```

```
.questionLeft span {  
    font-weight: bold;  
    font-size: 14px;  
    color: var(--color-text);  
}
```

```
.smallUpload {  
    width: 120px;  
    height: 120px;  
    background: var(--color-buttonColor);  
    border-radius: 10px;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
    cursor: pointer;  
    overflow: hidden;
```

```
    transition: all 0.3s ease;
}
```

```
.smallUpload:hover {
    opacity: 0.8;
}
```

```
.smallUpload img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
```

```
.smallPlus {
    font-size: 40px;
    color: var(--color-text);
    font-weight: bold;
}
```

```
.poolBtn {
    width: 100%;
    padding: 10px;
    background: var(--color-buttonColor) !important;
    color: var(--color-text) !important;
    font-weight: bold !important;
}
```

```
    font-size: 14px !important;

    border: none;

    border-radius: 8px;

    cursor: pointer;

    transition: all 0.3s ease;
}
```

```
.poolBtn:hover {

    opacity: 0.8;
}
```

```
.questionRight {

    flex: 2;

    min-width: 260px;

    display: flex;

    flex-direction: column;

    gap: 12px;
}
```

```
.questionRight span {

    font-weight: bold;

    font-size: 14px;

    color: var(--color-text);
}
```



```
.questionRight input {  
    background: #FFD79D;  
    padding: 10px;  
    border: none;  
    border-radius: 8px;  
    box-shadow: 0 4px 6px rgba(0,0,0,0.15);  
    color: var(--color-text);  
}
```

```
.questionRight input:disabled {  
    opacity: 0.6;  
    background: #e8c88a;  
}
```

```
.answersBox {  
    display: flex;  
    flex-direction: column;  
    gap: 10px;  
}
```

```
.checkboxLabel {  
    display: flex;  
    align-items: center;  
    gap: 8px;  
    cursor: pointer;
```

```
    margin-top: 10px;
}
```

```
.checkboxLabel input {
    width: 18px;
    height: 18px;
    cursor: pointer;
}
```

```
.checkboxLabel span {
    font-weight: normal;
    font-size: 14px;
    color: var(--color-text);
}
```

```
.imagePreview {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
```

```
.addQuestionBtn {
    width: 100%;
    padding: 14px;
    background: transparent !important;
```

```
color: var(--color-text) !important;

border: none;

font-size: 40px;

font-weight: bold;

cursor: pointer;

margin-top: 10px;

transition: all 0.3s ease;

}

.addQuestionBtn:hover {

    color: var(--color-buttonColor) !important;

}
```

```
/* Модальные окна */
```

```
.modalOverlay {

    position: fixed;

    top: 0;

    left: 0;

    right: 0;

    bottom: 0;

    background-color: var(--color-overlay);

    display: flex;

    justify-content: center;

    align-items: center;

    z-index: 1000;
```

```
}
```

```
.createPoolModalOverlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  right: 0;  
  bottom: 0;  
  background-color: rgba(0, 0, 0, 0.8);  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  z-index: 2000;  
}
```

```
.poolModal {  
  background-color: var(--color-blockColor);  
  border-radius: 15px;  
  width: 90%;  
  max-width: 800px;  
  max-height: 80vh;  
  display: flex;  
  flex-direction: column;  
  overflow: hidden;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
```

```
}
```

```
.modalHeader {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;  
  padding: 20px 25px;  
  border-bottom: 1px solid var(--color-border);  
}
```

```
.modalHeader h3 {  
  margin: 0;  
  font-size: 22px;  
  color: var(--color-text);  
}
```

```
.closeModalBtn {  
  background: none;  
  border: none;  
  font-size: 24px;  
  cursor: pointer;  
  color: var(--color-text);  
  padding: 5px;  
  transition: opacity 0.3s ease;  
}
```

```
.closeModalBtn:hover {  
    opacity: 0.7;  
}
```

```
.backBtn {  
    background: none;  
    border: none;  
    font-size: 24px;  
    cursor: pointer;  
    color: var(--color-text);  
    padding: 5px;  
    margin-right: 10px;  
    transition: opacity 0.3s ease;  
}
```

```
.backBtn:hover {  
    opacity: 0.7;  
}
```

```
.poolsList {  
    padding: 20px;  
    display: flex;  
    flex-direction: column;  
    gap: 15px;
```

```

    overflow-y: auto;

    padding-bottom: 10px;
}

.poolCard {
    background-color: rgba(255, 215, 157, 0.5);
    border-radius: 10px;
    padding: 15px 20px;
    cursor: pointer;
    transition: all 0.3s ease;
    border: 1px solid transparent;
}

.poolCard:hover {
    background-color: rgba(255, 215, 157, 0.8);
    transform: translateX(5px);
    border-color: var(--color-buttonColor);
}

.poolCard.selected {
    background-color: rgba(190, 143, 76, 0.3);
    border-color: var(--color-buttonColor);
}

.poolCardHeader {

```

```
display: flex;

justify-content: space-between;

align-items: center;

margin-bottom: 8px;

}
```

```
.poolCardHeader h4 {

margin: 0;

font-size: 18px;

color: var(--color-text);

}
```

```
.questionsCount {

font-size: 12px;

color: var(--color-text-muted);

}
```

```
.poolDescription {

margin: 0;

font-size: 14px;

color: var(--color-text);

}
```

```
.createPoolBtn {

margin: 10px 20px 20px 20px;
```



```
padding: 12px;

background-color: var(--color-buttonColor);

border: none;

border-radius: 8px;

color: var(--color-text);

font-size: 16px;

font-weight: bold;

cursor: pointer;

transition: all 0.3s ease;

}
```

```
.createPoolBtn:hover {

    background-color: var(--color-hover);

    transform: translateY(-2px);

}
```

```
.questionsList {

    padding: 20px;

    display: flex;

    flex-direction: column;

    gap: 15px;

    overflow-y: auto;

}
```

```
.poolQuestionCard {
```

```
background-color: rgba(255, 215, 157, 0.5);  
border-radius: 10px;  
padding: 15px;  
cursor: pointer;  
transition: all 0.3s ease;  
}
```

```
.poolQuestionCard:hover {  
    background-color: rgba(255, 215, 157, 0.8);  
    transform: translateX(5px);  
}
```

```
.questionCardHeader {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    margin-bottom: 10px;  
}
```

```
.questionNumber {  
    font-size: 14px;  
    color: var(--color-text-muted);  
    font-weight: bold;  
}
```

```
.selectQuestionBtn {  
    background-color: var(--color-buttonColor);  
    border: none;  
    border-radius: 6px;  
    padding: 5px 12px;  
    font-size: 12px;  
    cursor: pointer;  
    transition: all 0.3s ease;  
    color: var(--color-text);  
    font-weight: bold;  
}
```

```
.selectQuestionBtn:hover {  
    background-color: var(--color-hover);  
}
```

```
.questionCardText {  
    margin: 0 0 10px 0;  
    font-size: 16px;  
    color: var(--color-text);  
    font-weight: bold;  
}
```

```
.answersPreview {  
    display: flex;
```

```
flex-wrap: wrap;

gap: 8px;

margin-top: 8px;
}
```

```
.previewAnswer {

  font-size: 13px;

  padding: 4px 8px;

  background-color: rgba(0, 0, 0, 0.05);

  border-radius: 5px;

  color: var(--color-text);
}
```

```
.previewAnswer.correct {

  background-color: rgba(76, 175, 80, 0.3);

  color: #2e7d32;

  font-weight: bold;
}
```

```
.poolDescriptionFull {

  padding: 0 20px;

  margin: 0 0 5px 0;

  font-size: 14px;

  color: var(--color-text);
}
```

```
.createPoolModal {  
    background-color: var(--color-blockColor);  
    border-radius: 15px;  
    width: 90%;  
    max-width: 500px;  
    padding: 20px;  
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
}
```

```
.createPoolModal h3 {  
    margin: 0 0 20px 0;  
    font-size: 24px;  
    color: var(--color-text);  
    text-align: center;  
}
```

```
.createPoolModal input,  
.createPoolModal textarea {  
    width: 100%;  
    padding: 10px;  
    margin-bottom: 15px;  
    border: 1px solid var(--color-buttonColor);  
    border-radius: 8px;  
    background-color: var(--color-background);  
}
```

```
    box-sizing: border-box;

    color: var(--color-text);
}
```

```
.createPoolModal input::placeholder,
.createPoolModal textarea::placeholder {
    color: var(--color-text-muted);
}
```

```
.modalButtons {
    display: flex;
    justify-content: flex-end;
    gap: 15px;
    margin-top: 20px;
    padding: 0 20px 25px 20px;
}
```

```
.cancelBtn {
    padding: 8px 20px;
    background-color: #999;
    border: none;
    border-radius: 6px;
    cursor: pointer;
    color: white;
    transition: opacity 0.3s ease;
```

```
}
```

```
.confirmBtn {  
  padding: 8px 20px;  
  background-color: var(--color-buttonColor);  
  border: none;  
  border-radius: 6px;  
  cursor: pointer;  
  font-weight: bold;  
  color: var(--color-text);  
  transition: opacity 0.3s ease;  
}
```

```
.cancelBtn:hover,  
.confirmBtn:hover {  
  opacity: 0.8;  
}
```

```
@media (max-width: 768px) {  
  .questionInner {  
    flex-direction: column;  
  }  
}
```

```
.addQuestionBtn {  
  font-size: 32px;
```

```

    }

    .poolModal {
        width: 95%;
        max-height: 85vh;
    }

    .modalHeader h3 {
        font-size: 18px;
    }

    .poolCardHeader h4 {
        font-size: 16px;
    }

    .questionCardText {
        font-size: 14px;
    }

    .createPoolModal {
        width: 95%;
    }
}

/* Для модального окна добавления в пул */

```



```
.poolModal .modalButtons {
  margin-top: 10px;
  padding: 0 20px 25px 20px;
}
```

===== \components\common\Create\Questions\QuestionBuilder.tsx =====

```
import React, { useState } from 'react';
import QuestionItem from './QuestionItem';
import PoolModal from './PoolModal';
import CreatePoolModal from './CreatePoolModal';
import AddToPoolModal from './AddToPoolModal';
import { usePools, Question, PoolQuestion } from '../../../hooks/usePools';
import styles from './QuestionBuilder.module.css';
```

```
interface QuestionBuilderProps {
  questions: Question[];
  onQuestionsChange: (questions: Question[]) => void;
  isSurvey: boolean;
  onModalOpenChange?: (isOpen: boolean) => void;
}
```

```
const QuestionBuilder: React.FC<QuestionBuilderProps> = ({
  questions,
  onQuestionsChange,
  isSurvey,
  onModalOpenChange
```

```

  }) => {

    const [nextId, setNextId] = useState(questions.length + 1);

    const [questionImagePreviews, setQuestionImagePreviews] = useState<{ [key: number]:
string }>({});

    // Модалки

    const [isPoolModalOpen, setIsPoolModalOpen] = useState(false);

    const [selectedPoolId, setSelectedPoolId] = useState<number | null>(null);

    const [currentQuestionId, setCurrentQuestionId] = useState<number | null>(null);

    const [isCreatePoolModalOpen, setIsCreatePoolModalOpen] = useState(false);

    const [isAddToPoolModalOpen, setIsAddToPoolModalOpen] = useState(false);

    const [selectedPoolForQuestion, setSelectedPoolForQuestion] = useState<number |
null>(null);

    const [currentQuestionForPool, setCurrentQuestionForPool] = useState<number |
null>(null);

    const { questionPools, createNewPool, addQuestionToPoolById } = usePools();

    const updateQuestion = (id: number, field: string, value: any) => {

      if (field === 'questionImage' && value instanceof File) {

        const preview = URL.createObjectURL(value);

        setQuestionImagePreviews(prev => ({ ...prev, [id]: preview }));

      }

      onQuestionsChange(questions.map(q => q.id === id ? { ...q, [field]: value } : q));

    };

```

```

const updateWrongAnswer = (qId: number, idx: number, value: string) => {
  onQuestionsChange(questions.map(q => {
    if (q.id === qId) {
      const newWrong = [...q.wrongAnswers];
      newWrong[idx] = value;
      return { ...q, wrongAnswers: newWrong };
    }
    return q;
  }));
};

```

```

const addQuestion = () => {
  if (questions.length >= 50) {
    alert('Максимум 50 вопросов');
    return;
  }
  onQuestionsChange([...questions, {
    id: nextId,
    questionText: "",
    questionImage: null,
    rightAnswer: "",
    wrongAnswers: ["", "", ""],
    isTextInput: false,
    textAnswer: ""
  }]);
};

```

```

    setNextId(nextId + 1);
};

const removeQuestion = (id: number) => {
  if (questions.length > 1) {
    if (questionImagePreviews[id]) {
      URL.revokeObjectURL(questionImagePreviews[id]);
    }
    onQuestionsChange(questions.filter(q => q.id !== id));
  }
};

```

```

const openPoolModal = (questionId: number) => {
  onModalOpenChange?.(true);
  setCurrentQuestionId(questionId);
  setIsPoolModalOpen(true);
};

```

```

const closePoolModal = () => {
  onModalOpenChange?.(false);
  setIsPoolModalOpen(false);
  setSelectedPoolId(null);
  setCurrentQuestionId(null);
};

```

```

const selectQuestionFromPool = (question: PoolQuestion) => {
  if (currentQuestionId !== null) {
    onQuestionsChange(questions.map(q => {
      if (q.id === currentQuestionId) {
        return {
          ...q,
          questionText: question.questionText,
          rightAnswer: question.rightAnswer,
          isTextInput: question.isTextInput,
          textAnswer: question.textAnswer,
          wrongAnswers: [...question.wrongAnswers]
        };
      }
    }));
    return q;
  }
  closePoolModal();
};

```

```

const openAddToPoolModal = (questionId: number) => {
  onModalOpenChange?.(true);
  setCurrentQuestionForPool(questionId);
  setIsAddToPoolModalOpen(true);
};

```

```
const closeAddToPoolModal = () => {  
  onModalOpenChange?.(false);  
  setIsAddToPoolModalOpen(false);  
  setSelectedPoolForQuestion(null);  
  setCurrentQuestionForPool(null);  
};
```

```
const handleAddToPool = async () => {  
  if (!selectedPoolForQuestion || !currentQuestionForPool) {  
    alert('Выберите пул');  
    return;  
  }  
}
```

```
const currentQuestion = questions.find(q => q.id === currentQuestionForPool);  
if (!currentQuestion) return;
```

```
if (!currentQuestion.questionText.trim()) {  
  alert('Сначала заполните текст вопроса');  
  return;  
}
```

```
const success = await addQuestionToPoolById(selectedPoolForQuestion,  
currentQuestion);
```

```
if (success) {  
  alert('Вопрос успешно добавлен в пул!');
```

```

    closeAddToPoolModal();

    } else {

        alert('Ошибка при добавлении вопроса в пул');

    }

};

const handleCreatePool = async (title: string, description: string) => {

    const newPool = await createNewPool(title, description);

    if (newPool) {

        alert('Пул вопросов создан!');

        setIsCreatePoolModalOpen(false);

    }

};

return (

    <div>

        {questions.map((q, idx) => (

            <QuestionItem

                key={q.id}

                question={q}

                index={idx}

                isLast={questions.length === 1}

                imagePreview={questionImagePreviews[q.id]}

                isSurvey={isSurvey}

                onUpdate={updateQuestion}

```

```

    onUpdateWrongAnswer={updateWrongAnswer}
    onRemove={removeQuestion}
    onOpenPool={openPoolModal}
    onAddToPool={openAddToPoolModal}
  />
  )))

```

```

<button type="button" className={styles.addQuestionBtn} onClick={addQuestion}>
  + ДОБАВИТЬ ВОПРОС
</button>

```

```

<PoolModal
  isOpen={isPoolModalOpen}
  pools={questionPools}
  selectedPoolId={selectedPoolId}
  onClose={closePoolModal}
  onSelectPool={setSelectedPoolId}
  onBack={() => setSelectedPoolId(null)}
  onSelectQuestion={selectQuestionFromPool}
  onCreatePool={() => setIsCreatePoolModalOpen(true)}
/>

```

```

<CreatePoolModal
  isOpen={isCreatePoolModalOpen}
  onClose={() => setIsCreatePoolModalOpen(false)}

```



```

        onCreate={handleCreatePool}

    />

    <AddToPoolModal
        isOpen={isAddToPoolModalOpen}
        pools={questionPools}
        selectedPoolId={selectedPoolForQuestion}
        onClose={closeAddToPoolModal}
        onSelectPool={setSelectedPoolForQuestion}
        onConfirm={handleAddToPool}
        onCreatePool={() => setIsCreatePoolModalOpen(true)}
    />

</div>

);

};

export default QuestionBuilder;

===== \components\common\Create\Questions\QuestionItem.tsx =====

import React from 'react';

import styles from './QuestionBuilder.module.css';

interface Question {
    id: number;
    questionText: string;
    questionImage: File | null;

```

```
rightAnswer: string;

wrongAnswers: string[];

isTextInput: boolean;

textAnswer: string;

}
```

```
interface QuestionItemProps {

  question: Question;

  index: number;

  isLast: boolean;

  imagePreview?: string;

  isSurvey: boolean;

  onUpdate: (id: number, field: string, value: any) => void;

  onUpdateWrongAnswer: (qId: number, idx: number, value: string) => void;

  onRemove: (id: number) => void;

  onOpenPool: (id: number) => void;

  onAddToPool: (id: number) => void;

}
```

```
const QuestionItem: React.FC<QuestionItemProps> = ({

  question,

  index,

  isLast,

  imagePreview,

  isSurvey,
```

```

onUpdate,
onUpdateWrongAnswer,
onRemove,
onOpenPool,
onAddToPool
}))=> {

const handleOpenPool = (e: React.MouseEvent, id: number) => {

    e.preventDefault();

    e.stopPropagation();

    onOpenPool(id);

};

const handleAddToPool = (e: React.MouseEvent, id: number) => {

    e.preventDefault();

    e.stopPropagation();

    onAddToPool(id);

};

const handleRemove = (e: React.MouseEvent, id: number) => {

    e.preventDefault();

    e.stopPropagation();

    onRemove(id);

};

return (

```

```
<div className={styles.questionBlock}>
```

```
<div className={styles.questionHead}>
```

```
<h3>вопрос {index + 1}</h3>
```

```
{!isLast && (
```

```
<button
```

```
type="button"
```

```
className={styles.deleteBtn}
```

```
onClick={(e) => handleRemove(e, question.id)}
```

```
>
```

```
×
```

```
</button>
```

```
)}
```

```
</div>
```

```
<div className={styles.questionInner}>
```

```
<div className={styles.questionLeft}>
```

```
<span>картинка вопроса {!isSurvey && '*'}</span>
```

```
<div
```

```
className={styles.smallUpload}
```

```
onClick={() => document.getElementById(`qImg_${question.id}`)?.click()}
```

```
>
```

```
{(imagePreview || question.questionImage) ? (
```

```
<img
```

```
src={imagePreview || URL.createObjectURL(question.questionImage!)}
```

```
alt="question"
```

```

        className={styles.imagePreview}

      />

    ) : (

      <div className={styles.smallPlus}>+</div>

    )}

    <input

      id={`qImg_${question.id}`}

      type="file"

      accept="image/*"

      style={{ display: 'none' }}

      onChange={e => onUpdate(question.id, 'questionImage', e.target.files?.[0] ||
null)}

    />

  </div>

  <button

    type="button"

    className={styles.poolBtn}

    onClick={(e) => handleOpenPool(e, question.id)}

  >

    выбрать из пула

  </button>

  <button

    type="button"

    className={styles.poolBtn}

    onClick={(e) => handleAddToPool(e, question.id)}

```

```

>

    сохранить в пул

</button>

</div>

<div className={styles.questionRight}>

    <input

        maxLength={100}

        value={question.questionText}

        onChange={e => onUpdate(question.id, 'questionText', e.target.value)}

        placeholder={`текст вопроса${!isSurvey ? ' (мин 10 символов)' : ''}}

    />

<div className={styles.answersBox}>

    {!isSurvey ? (

        <div>

            <span>правильный ответ *</span>

            <input

                value={question.rightAnswer}

                onChange={e => onUpdate(question.id, 'rightAnswer', e.target.value)}

                placeholder="введите правильный ответ"

                disabled={question.isTextInput}

            />

            <span>неправильные ответы (хотя бы один)</span>

```

```

{question.wrongAnswers.map((ans, i) => (
  <input
    key={i}
    value={ans}
    onChange={e => onUpdateWrongAnswer(question.id, i, e.target.value)}
    placeholder={`вариант ${i + 1}`}
    disabled={question.isTextInput}
  />
))}

```

```

<label className={styles.checkboxLabel}>
  <input
    type="checkbox"
    checked={question.isTextInput}
    onChange={e => onUpdate(question.id, 'isTextInput', e.target.checked)}
  />
  <span>ТЕКСТОВЫЙ ВВОД ОТВЕТА</span>
</label>

```

```

{question.isTextInput && (
  <input
    value={question.textAnswer}
    onChange={e => onUpdate(question.id, 'textAnswer', e.target.value)}
    placeholder="ПРАВИЛЬНЫЙ ТЕКСТОВЫЙ ОТВЕТ"
  />

```

```

    })
  </>
): (
  <
  <span>варианты ответов</span>
  <input
    value={question.rightAnswer}
    onChange={e => onUpdate(question.id, 'rightAnswer', e.target.value)}
    placeholder="вариант 1"
    disabled={question.isTextInput}
  />
  {question.wrongAnswers.map((ans, i) => (
    <input
      key={i}
      value={ans}
      onChange={e => onUpdateWrongAnswer(question.id, i, e.target.value)}
      placeholder={`вариант ${i + 2}`}
      disabled={question.isTextInput}
    />
  ))}

  <label className={styles.checkboxLabel}>
    <input
      type="checkbox"
      checked={question.isTextInput}

```



```

        onChange={e => onUpdate(question.id, 'isTextInput', e.target.checked)}

      />

      <span>ТЕКСТОВЫЙ ВВОД ОТВЕТА</span>

    </label>

  </>

  )}

</div>

</div>

</div>

</div>

);

};

```

```
export default QuestionItem;
```

```
===== \components\common\Create\CreateMain.module.css =====
```

```
@import "../style/variables.css";
```

```

.createPage {

  min-height: 100vh;

  background-color: var(--color-background);

}

```

```

.createContainer {

  max-width: 900px;

  margin: 0 auto;

```

```
padding: 20px;  
}
```

```
h2 {  
  font-size: 40px;  
  color: var(--color-text);  
  text-transform: lowercase;  
  text-align: center;  
  margin: 0 0 20px 0;  
  font-weight: bold;  
}
```

```
.createForm {  
  width: 100%;  
}
```

```
.formGroup {  
  width: 100%;  
  margin-bottom: 25px;  
}
```

```
.labelRow {  
  display: flex;  
  justify-content: space-between;  
  margin-bottom: 8px;
```

```
}
```

```
.labelRow span:first-child,
```

```
.groupLabel {
```

```
    font-size: 30px;
```

```
    font-weight: bold;
```

```
    color: var(--color-text);
```

```
    text-transform: lowercase;
```

```
}
```

```
.labelRow span:last-child {
```

```
    font-size: 12px;
```

```
    color: var(--color-text-muted);
```

```
}
```

```
.formGroup input,
```

```
.formGroup textarea {
```

```
    width: 100%;
```

```
    padding: 12px;
```

```
    background: var(--color-blockColor);
```

```
    border: none;
```

```
    border-radius: 8px;
```

```
    box-shadow: 0 4px 6px rgba(0,0,0,0.15);
```

```
    font-size: 14px;
```

```
    color: var(--color-text);
```

```
}
```

```
.formGroup input:focus,
```

```
.formGroup textarea:focus {
```

```
  outline: none;
```

```
  border: none;
```

```
  box-shadow: 0 4px 6px rgba(0,0,0,0.15);
```

```
}
```

```
.formGroup textarea {
```

```
  resize: vertical;
```

```
}
```

```
.formGroup input::placeholder,
```

```
.formGroup textarea::placeholder {
```

```
  color: var(--color-text-muted);
```

```
}
```

```
.coverUpload {
```

```
  width: 240px;
```

```
  height: 240px;
```

```
  background: var(--color-blockColor);
```

```
  border-radius: 15px;
```

```
  display: flex;
```

```
  align-items: center;
```

```
justify-content: center;

cursor: pointer;

margin: 10px auto 0 auto;

box-shadow: 0 4px 6px rgba(0,0,0,0.15);

overflow: hidden;

}
```

```
.coverUpload:focus,

.coverUpload:active {

    outline: none;

    border: none;

    box-shadow: 0 4px 6px rgba(0,0,0,0.15);

}
```

```
.coverUpload img {

    width: 100%;

    height: 100%;

    object-fit: cover;

}
```

```
.plusIcon {

    font-size: 60px;

    color: var(--color-text);

    font-weight: bold;

}
```

```
.rowButtons {  
    display: grid;  
    grid-template-columns: 1fr 1fr;  
    gap: 15px;  
}
```

```
.rowButtons button,  
.topicsGrid button {  
    padding: 14px;  
    background: var(--color-blockColor);  
    border: none;  
    border-radius: 8px;  
    font-size: 16px;  
    font-weight: bold;  
    color: var(--color-text);  
    cursor: pointer;  
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);  
    transition: all 0.3s ease;  
}
```

```
.rowButtons button:focus,  
.rowButtons button:active,  
.topicsGrid button:focus,  
.topicsGrid button:active {
```

```
outline: none;

border: none;

box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}
```

```
.rowButtons button.active,
.topicsGrid button.active {
  background: var(--color-buttonColor);
  color: var(--color-text);
}
```

```
.topicsGrid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(100px, 1fr));
  gap: 12px;
}
```

```
.submitBtn {
  width: 100%;
  padding: 14px;
  background: transparent !important;
  color: var(--color-text) !important;
  border: none !important;
  outline: none !important;
  font-size: 40px;
```

```
font-weight: bold;

text-decoration: none;

cursor: pointer;

margin-top: 10px;

transition: all 0.3s ease;

box-shadow: none !important;

-webkit-appearance: none;

-moz-appearance: none;

appearance: none;

}
```

```
.submitBtn:hover {

    color: var(--color-buttonColor) !important;

    background: transparent !important;

    border: none !important;

    outline: none !important;

}
```

```
.submitBtn:focus,

.submitBtn:active {

    outline: none !important;

    border: none !important;

    box-shadow: none !important;

    background: transparent !important;

}
```



```
.submitBtn:disabled {  
    opacity: 0.5;  
    cursor: not-allowed;  
}
```

```
.imagePreview {  
    width: 100%;  
    height: 100%;  
    object-fit: cover;  
}
```

```
.checkboxLabel {  
    display: flex;  
    align-items: center;  
    gap: 8px;  
    cursor: pointer;  
    margin-top: 10px;  
}
```

```
.checkboxLabel input {  
    width: 18px;  
    height: 18px;  
    cursor: pointer;  
}
```

```
.checkboxLabel input:focus,  
.checkboxLabel input:active {  
    outline: none;  
    border: none;  
}
```

```
.checkboxLabel span {  
    font-weight: normal;  
    font-size: 14px;  
    color: var(--color-text);  
}
```

```
@media (max-width: 768px) {  
    .createContainer {  
        padding: 15px;  
    }  
}
```

```
h2 {  
    font-size: 28px;  
}
```

```
.labelRow span:first-child,  
.groupLabel {  
    font-size: 22px;
```

```

    }

    .submitBtn {

        font-size: 32px;

    }

}

===== \components\common\Create\CreateTestOrSurvey.tsx =====

import React, { useState, useEffect } from 'react';

import { useNavigate } from 'react-router-dom';

import api from '../../api/axios';

import { createTest, uploadImage, createQuestionsBatch, createQuestion } from
'../../api/tests';

import styles from './CreateMain.module.css';

import QuestionBuilder from './Questions/QuestionBuilder';

interface CreateTestOrSurveyProps {

    type: 'test' | 'survey';

}

interface Question {

    id: number;

    questionText: string;

    questionImage: File | null;

    rightAnswer: string;

    wrongAnswers: string[];

```

```
isTextInput: boolean;

textAnswer: string;

}
```

```
const CreateTestOrSurvey: React.FC<CreateTestOrSurveyProps> = ({ type }) => {

  const navigate = useNavigate();

  const isSurvey = type === 'survey';
```

```
// Проверка бана
```

```
const [isBanned, setIsBanned] = useState(false);

const [banReason, setBanReason] = useState("");

const [banUntil, setBanUntil] = useState<string | null>(null);

const [banPermanent, setBanPermanent] = useState(false);

const [checkingBan, setCheckingBan] = useState(true);
```

```
// Базовые поля
```

```
const [title, setTitle] = useState("");

const [description, setDescription] = useState("");

const [visibility, setVisibility] = useState<'public' | 'private'>('public');

const [topics, setTopics] = useState<string[]>([]);

const [coverImage, setCoverImage] = useState<File | null>(null);

const [coverImagePreview, setCoverImagePreview] = useState<string | null>(null);
```

```
// Только для теста
```

```
const [timeLimit, setTimeLimit] = useState("");
```

```

const [noTimeLimit, setNoTimeLimit] = useState(false);

const [passingScore, setPassingScore] = useState('70');

const [attemptsLimit, setAttemptsLimit] = useState(""); // НОВОЕ ПОЛЕ

const [noAttemptsLimit, setNoAttemptsLimit] = useState(false); // Без ограничения
попыток


// Вопросы

const [questions, setQuestions] = useState<Question[]>([

  { id: 1, questionText: "", questionImage: null, rightAnswer: "", wrongAnswers: ["", "", ""],
    isTextInput: false, textAnswer: "" }

]);


const [isSubmitting, setIsSubmitting] = useState(false);

const [isModalOpen, setIsModalOpen] = useState(false);


const topicsList = ['наука', 'спорт', 'сериалы', 'анимации', 'игры'];


useEffect(() => {

  checkBanStatus();

}, []);


const checkBanStatus = async () => {

  try {

    const profile = await api.get('/auth/profile/');

    if (profile.data.is_banned) {

```

```

    setIsBanned(true);

    setBanReason(profile.data.ban_reason || "");

    setBanUntil(profile.data.ban_until);

    setBanPermanent(profile.data.ban_permanent || false);

  }

  } catch (error) {

    console.error('Ошибка проверки бана:', error);

  } finally {

    setCheckingBan(false);

  }

};

```

```

const getBanUntilText = () => {

  if (!banUntil) return "";

  const date = new Date(banUntil);

  return date.toLocaleString('ru-RU');

};

```

```

const toggleTopic = (topic: string) => {

  setTopics(prev => prev.includes(topic) ? prev.filter(t => t !== topic) : [...prev, topic]);

};

```

// Валидация

```

const validateForm = (): boolean => {

  if (!title.trim()) {

```

```

    alert('Введите название ${isSurvey ? 'опроса' : 'теста'}');

    return false;
}

if (!isSurvey) {

    if (title.trim().length < 6) {

        alert('Название должно содержать минимум 6 символов');

        return false;

    }

    if (description.trim().length < 10) {

        alert('Описание должно содержать минимум 10 символов');

        return false;

    }

    if (!coverImage) {

        alert('Добавьте обложку для теста');

        return false;

    }

} else {

    if (description.trim().length < 10) {

        alert('Описание должно содержать минимум 10 символов');

        return false;

    }

}

```

// Валидация вопросов

```

if (questions.length < 4) {

    alert(`Минимум 4 вопроса необходимо для создания ${isSurvey ? 'опроса' :
'теста'}`);

    return false;

}


for (let i = 0; i < questions.length; i++) {

    const q = questions[i];

    if (!q.questionText.trim()) {

        alert(`Вопрос ${i + 1}: Заполните текст вопроса`);

        return false;

    }


    if (!isSurvey) {

        if (q.questionText.trim().length < 10) {

            alert(`Вопрос ${i + 1}: Текст вопроса должен содержать минимум 10
символов`);

            return false;

        }

    }


    if (!isSurvey) {

        if (q.isTextInput) {

            if (!q.textAnswer.trim()) {

```



```

    alert(`Вопрос ${i + 1}: Введите правильный текстовый ответ`);

    return false;

}

} else {

    if (!q.rightAnswer.trim()) {

        alert(`Вопрос ${i + 1}: Введите правильный ответ`);

        return false;

    }

    const hasWrongAnswer = q.wrongAnswers.some(a => a.trim().length > 0);

    if (!hasWrongAnswer) {

        alert(`Вопрос ${i + 1}: Добавьте хотя бы один неправильный вариант
ответа`);

        return false;

    }

}

} else {

    // Для опроса - проверяем что заполнен хотя бы один вариант ответа ИЛИ
    // включен текстовый ввод

    const hasRightAnswer = q.rightAnswer.trim().length > 0;

    const hasWrongAnswer = q.wrongAnswers.some(a => a.trim().length > 0);

    if (!hasRightAnswer && !hasWrongAnswer && !q.isTextInput) {

        alert(`Вопрос ${i + 1}: Заполните хотя бы один вариант ответа или включите
"текстовый ввод ответа`);

        return false;

```

```
    }  
  }  
}
```

```
if (topics.length === 0) {  
  alert('Выберите хотя бы одну тему');  
  return false;  
}
```

```
if (!isSurvey) {  
  if (!noTimeLimit && (!timeLimit || parseInt(timeLimit) <= 0)) {  
    alert('Укажите ограничение времени или отметьте "без ограничения");  
    return false;  
  }  
}
```

```
if (!passingScore || parseInt(passingScore) < 0 || parseInt(passingScore) > 100) {  
  alert('Укажите проходной балл (от 0 до 100)');  
  return false;  
}
```

```
// Валидация лимита попыток
```

```
if (!noAttemptsLimit && (!attemptsLimit || parseInt(attemptsLimit) <= 0)) {  
  alert('Укажите ограничение попыток или отметьте "без ограничения");  
  return false;  
}
```

```

    }

    return true;
  };

const handleSubmit = async (e: React.FormEvent) => {
  e.preventDefault();
  if (isModalOpen) return;
  if (!validateForm()) return;

  setIsSubmitting(true);

  try {
    // 1. Загружаем обложку
    let coverImageUrl = null;
    if (coverImage) {
      coverImageUrl = await uploadImage(coverImage, 'test');
    }

    // 2. Создаем вопросы
    const questionsData = [];

    for (const q of questions) {
      let answers: any[] = [];
      let textAnswer = "";

```

```

let isTextInput = false;

if (isSurvey) {
    // Для опроса
    isTextInput = q.isTextInput;

    if (q.isTextInput) {
        // Текстовый ввод - no answers

        answers = [];

        textAnswer = "";
    } else {
        // Обычные варианты ответов

        if (q.rightAnswer.trim()) {
            answers.push({ text: q.rightAnswer, is_correct: true, order_index: 0 });
        }

        q.wrongAnswers.forEach((answer, idx) => {
            if (answer.trim()) {
                answers.push({ text: answer, is_correct: false, order_index: idx + 1 });
            }
        });

        textAnswer = "";
    }
} else {
    // Для теста

    isTextInput = q.isTextInput;

```

```

if (q.isTextInput) {
    textAnswer = q.textAnswer;
    answers = [{ text: q.textAnswer, is_correct: true, order_index: 0 }];
} else {
    answers.push({ text: q.rightAnswer, is_correct: true, order_index: 0 });
    q.wrongAnswers.forEach((answer, idx) => {
        if (answer.trim()) {
            answers.push({ text: answer, is_correct: false, order_index: idx + 1 });
        }
    });
}
}

// Картинку вопроса загружаем через uploadImage
let questionImageUrl = null;
if (q.questionImage) {
    questionImageUrl = await uploadImage(q.questionImage, 'question');
}

questionsData.push({
    text: q.questionText,
    image: questionImageUrl,
    difficulty: 1,
    explanation: "",

```

```

    is_text_input: isTextInput,

    text_answer: textAnswer,

    answers: answers

  });
}

// 3. Создаем вопросы в БД
let questionIds: number[] = [];

if (isSurvey) {
  for (const qData of questionsData) {
    const createdQuestion = await createQuestion(qData as any);
    questionIds.push(createdQuestion.id);
  }
} else {
  const createdQuestions = await createQuestionsBatch(questionsData);
  if (!createdQuestions?.created?.length) {
    throw new Error('Ошибка при создании вопросов');
  }
  questionIds = createdQuestions.created.map((q: any) => q.id);
}

// 4. Создаем тест/опрос
const testData = {
  title: title,

```

```

        description: description,
        is_open: visibility === 'public',
        is_survey: isSurvey,
        time_limit: isSurvey ? 0 : (noTimeLimit ? 0 : (parseInt(timeLimit) || 0)),
        passing_score: isSurvey ? 0 : parseInt(passingScore),
        attempts_limit: isSurvey ? 0 : (noAttemptsLimit ? 0 : (parseInt(attemptsLimit) ||
0)), // ДОБАВЛЕНО
        image: coverImageUrl,
        topics: topics,
        question_ids: questionIds
    };

    await createTest(testData);

    alert(`${isSurvey ? 'Опрос' : 'Тест'} успешно создан!`);

    navigate('/testlist');

} catch (error: any) {
    console.error('Ошибка:', error);

    alert(`Ошибка: ${error.response?.data?.error || error.message}`);
} finally {
    setIsSubmitting(false);
}
};

// Если проверка бана ещё идёт

```

```

if (checkingBan) {

  return (

    <div className={styles.container}>

      <div className={styles.loading}>Загрузка...</div>

    </div>

  );

}

// Если пользователь забанен - показываем сообщение

if (isBanned) {

  return (

    <div className={styles.container}>

      <div className={styles.bannedMessage}>

        <div className={styles.bannedText}>Ваш аккаунт забанен</div>

        {banReason && <div className={styles.bannedReason}>Причина: {banReason}</div>}

        {banUntil && !banPermanent && (

          <div className={styles.bannedUntil}>До: {getBanUntilText()}</div>

        )}

        {banPermanent && <div
className={styles.bannedPermanent}>Перманентно</div>}

        <button

          className={styles.backButton}

          onClick={() => navigate('/')}>


```


Вернуться на главную

</button>

</div>

</div>

);

}

return (

<div className={styles.container}>

<h2>создание {isSurvey ? 'опроса' : 'теста'}</h2>

<form onSubmit={handleSubmit} className={styles.createForm}>

{/* Название */}

<div className={styles.formGroup}>

<div className={styles.labelRow}>

введите название {!isSurvey && '(мин 6)'}

максимум 50 символов

</div>

<input

maxLength={50}

value={title}

onChange={e => setTitle(e.target.value)}

placeholder={`пример: \${isSurvey ? 'опрос' : 'тест'} iq`}

/>

</div>

```

    { /* Описание */ }

    <div className={styles.formGroup}>

        <div className={styles.labelRow}>

            <span>введите описание (мин 10)</span>

            <span>максимум 200 символов</span>

        </div>

        <textarea

            maxLength={200}

            value={description}

            onChange={e => setDescription(e.target.value)}

            placeholder={`пример: ${isSurvey ? 'опрос' : 'тест'} iq`}

            rows={4}

        />

    </div>

    { /* Обложка */ }

    <div className={styles.formGroup}>

        <span className={styles.groupLabel}>картинка для обложки {!isSurvey && '*'}</span>

        <div className={styles.coverUpload} onClick={() =>
document.getElementById('coverInput')?.click()}>

            {coverImagePreview ? (

                <img src={coverImagePreview} alt="cover"
className={styles.imagePreview} />

            ) : (

```

```

    <div className={styles.plusIcon}>+</div>
  )}
  <input
    id="coverInput"
    type="file"
    accept="image/*"
    style={{ display: 'none' }}
    onChange={e => {
      const file = e.target.files?.[0] || null;
      if (file) {
        setCoverImage(file);
        setCoverImagePreview(URL.createObjectURL(file));
      }
    }}
  />
</div>
</div>

{/* ВИДИМОСТЬ */}
<div className={styles.formGroup}>
  <span className={styles.groupLabel}>ВИДИМОСТЬ</span>
  <div className={styles.rowButtons}>
    <button
      type="button"
      className={visibility === 'public' ? styles.active : ''}

```

```

        onClick={() => setVisibility('public')}
      >
        публичный
      </button>
    <button
      type="button"
      className={visibility === 'private' ? styles.active : ""}
      onClick={() => setVisibility('private')}
    >
      приватный
    </button>
  </div>
</div>

{/* Только для теста */}
{!isSurvey && (
  <div className={styles.formGroup}>
    <span className={styles.groupLabel}>ограничение времени (минуты)</span>
    <input
      type="number"
      value={timeLimit}
      onChange={e => setTimeLimit(e.target.value)}
      placeholder="например: 30"
      disabled={noTimeLimit}

```

```

/>

<label className={styles.checkboxLabel}>

  <input type="checkbox" checked={noTimeLimit} onChange={e =>
setNoTimeLimit(e.target.checked)} />

  <span>без ограничения по времени</span>

</label>

</div>


<div className={styles.formGroup}>

  <span className={styles.groupLabel}>проходной балл (%)</span>

  <input

    type="number"

    value={passingScore}

    onChange={e => setPassingScore(e.target.value)}

    placeholder="например: 70"

    min="0"

    max="100"

  />

</div>


{/* НОВОЕ ПОЛЕ - ограничение попыток */}

<div className={styles.formGroup}>

  <span className={styles.groupLabel}>ограничение попыток</span>

  <input

    type="number"

```

```

    value={attemptsLimit}

    onChange={e => setAttemptsLimit(e.target.value)}

    placeholder="например: 3"

    disabled={noAttemptsLimit}

    min="1"

  />

  <label className={styles.checkboxLabel}>

    <input type="checkbox" checked={noAttemptsLimit} onChange={e =>
setNoAttemptsLimit(e.target.checked)} />

    <span>без ограничения попыток</span>

  </label>

</div>

</>

)}

{/* Темы */}

<div className={styles.formGroup}>

  <span className={styles.groupLabel}>выберите темы</span>

  <div className={styles.topicsGrid}>

    {topicsList.map(t => (

      <button

        key={t}

        type="button"

        className={topics.includes(t) ? styles.active : ""}

        onClick={() => toggleTopic(t)}

```

```

    >

    {t}

  </button>

  )})

</div>

</div>

{ /* Вопросы */ }

<div className={styles.formGroup}>

  <span className={styles.groupLabel}>

    вопросы * (минимум 4 {!isSurvey && ' , каждый с картинкой'})

  </span>

  <QueryBuilder

    questions={questions}

    onQuestionsChange={setQuestions}

    isSurvey={isSurvey}

    onModalOpenChange={setIsModalOpen}

  />

</div>

  <button type="submit" className={styles.submitBtn} disabled={isSubmitting}>

    {isSubmitting ? `СОЗДАНИЕ ${isSurvey ? 'ОПРОСА' : 'ТЕСТА'}...` : `СОЗДАТЬ
    ${isSurvey ? 'ОПРОС' : 'ТЕСТ'}`}

  </button>

</form>

```

```

    </div>

);

};

export default CreateTestOrSurvey;

===== \components\common\Footer\Footer.module.css =====

.footer {
  background-color: #D8C3A5;
  color: black;
  padding: 20px 30px;
  display: flex;
  justify-content: space-between;
}

.info {
  display: flex;
  flex-direction: column;
  align-items: flex-start;
  text-align: left;
}

.info h3 {
  font-size: 32px;
  margin-bottom: 20px;
}

```



```
.info h3:nth-child(2) {  
    font-size: 18px;  
    margin-bottom: 15px;  
    font-weight: normal;  
}
```

```
/* Строка для СОЦ СЕТИ: и иконок справа */
```

```
.socialRow {  
    display: flex;  
    align-items: center;  
    gap: 15px;  
}
```

```
.info .socialRow h3 {  
    font-size: 18px;  
    margin-bottom: 0;  
    font-weight: normal;  
}
```

```
.socialIcons {  
    display: flex;  
    gap: 10px;  
}
```

```
.socialIcon {  
  width: 35px;  
  height: 35px;  
  border-radius: 8px;  
  display: block;  
  transition: opacity 0.2s;  
}
```

```
.socialIcon:hover {  
  opacity: 0.8;  
}
```

```
.nav {  
  display: flex;  
  gap: 60px;  
}
```

```
.nav h3 {  
  margin-bottom: 20px;  
  font-size: 24px;  
}
```

```
.nav ul {  
  list-style: none;  
  padding: 0;
```

```
}
```

```
.nav li {  
  margin-bottom: 10px;  
}
```

```
.socialIcons {  
  display: flex;  
  gap: 15px;  
  align-items: center;  
}
```

```
.socialIcon {  
  display: inline-flex;  
  transition: transform 0.2s ease;  
}
```

```
.socialIcon:hover {  
  transform: scale(1.1);  
}
```

```
.socialIcon img {  
  width: 30px;  
  height: 30px;  
  display: block;
```

```
}
```

```
/* ===== АДАПТИВ ДЛЯ ТЕЛЕФОНОВ (до 768px) ===== */
```

```
@media (max-width: 768px) {
```

```
  .footer {
```

```
    padding: 20px;
```

```
  }
```

```
/* Скрываем блок с навигацией (тесты, опросы, создать) */
```

```
.nav {
```

```
  display: none;
```

```
}
```

```
/* Информация занимает всю ширину */
```

```
.info {
```

```
  width: 100%;
```

```
}
```

```
.info h3 {
```

```
  font-size: 24px;
```

```
  margin-bottom: 15px;
```

```
}
```

```
.info h3:nth-child(2) {
```

```
    font-size: 16px;
  }
}
```

```
.info .socialRow h3 {
  font-size: 16px;
}
```

```
.socialIcon {
  width: 30px;
  height: 30px;
}
}
```

```
===== \components\common\Footer\Footer.tsx =====
```

```
import React from 'react';
import { useNavigate } from 'react-router-dom';
import styles from './Footer.module.css';
import tgLogo from '../assets/footer/tg_logo.png';
import vkLogo from '../assets/footer/vk_logo.png';
import ytLogo from '../assets/footer/yt_logo.png';
```

```
const Footer = () => {
  const navigate = useNavigate();

  // Обработчики навигации с параметрами
  const handleTestsNew = () => {
```

```
    navigate('/testlist?sort=new&type=test');  
  };
```

```
const handleTestsPopular = () => {  
  navigate('/testlist?sort=popular&type=test');  
};
```

```
const handleTestsTopics = () => {  
  navigate('/testlist?type=test');  
};
```

```
const handleSurveysNew = () => {  
  navigate('/testlist?sort=new&type=survey');  
};
```

```
const handleSurveysPopular = () => {  
  navigate('/testlist?sort=popular&type=survey');  
};
```

```
const handleSurveysTopics = () => {  
  navigate('/testlist?type=survey');  
};
```

```
const handleCreateTest = () => {  
  navigate('/create/test');
```

```
};
```

```
const handleCreateSurvey = () => {  
  navigate('/create/survey');  
};
```

```
return (  
  <footer className={styles.footer}>  
    { /* Левый блок - информация */ }  
    <div className={styles.info}>  
      <h3>ОНЛАЙН ТЕСТЫ</h3>  
      <h3>ПОЧТА: example@gmail.com</h3>  
      <div className={styles.socialRow}>  
        <h3>СОЦ СЕТИ:</h3>  
        <div className={styles.socialIcons}>  
          <a  
            href="https://t.me"  
            target="_blank"  
            rel="noopener noreferrer"  
            className={styles.socialIcon}  
          >  
            <img src={tgLogo} alt="Telegram" />  
          </a>  
          <a  
            href="https://vk.com"
```

```

        target="_blank"

        rel="noopener noreferrer"

        className={styles.socialIcon}
    >

    <img src={vkLogo} alt="VK" />

</a>

<a

    href="https://youtube.com"

    target="_blank"

    rel="noopener noreferrer"

    className={styles.socialIcon}

    >

    <img src={ytLogo} alt="YouTube" />

</a>

</div>

</div>

</div>

{ /* Правый блок - навигация в 3 колонки */ }

<div className={styles.nav}>

    { /* Первая колонка - ТЕСТЫ */ }

    <div>

        <h3>ТЕСТЫ</h3>

        <ul>

            <li onClick={handleTestsNew}>ХОББИЕ</li>

```



```
<li onClick={handleTestsPopular}>ЛУЧШИЕ</li>
<li onClick={handleTestsTopics}>ТЕМЫ</li>
</ul>
</div>
```

```
{/* Вторая колонка - ОПРОСЫ */}
```

```
<div>
  <h3>ОПРОСЫ</h3>
  <ul>
    <li onClick={handleSurveysNew}>НОВЫЕ</li>
    <li onClick={handleSurveysPopular}>ЛУЧШИЕ</li>
    <li onClick={handleSurveysTopics}>ТЕМЫ</li>
  </ul>
</div>
```

```
{/* Третья колонка - СОЗДАТЬ */}
```

```
<div>
  <h3>СОЗДАТЬ</h3>
  <ul>
    <li onClick={handleCreateTest}>ТЕСТЫ</li>
    <li onClick={handleCreateSurvey}>ОПРОСЫ</li>
  </ul>
</div>
</div>
</footer>
```

```

);

};

export default Footer;

===== \components\common\Form\RegisterAuthForm\InputForm.module.css
=====

.inputWrapper {
    width: 100%;
    margin-bottom: 15px;
}

.input {
    border-radius: 10px;
    padding: 12px 16px;
    font-size: 20px;
    width: 100%;
    height: 50px;
    background-color: #D8C3A5;
    border: none;
    box-sizing: border-box;
}

.input:focus {
    outline: 2px solid #BE8F4C;
}

```

```
/* ===== АДАПТИВ ===== */
```

```
@media (max-width: 1000px) {
```

```
  .input {
```

```
    font-size: 18px;
```

```
    height: 48px;
```

```
    padding: 10px 14px;
```

```
  }
```

```
  .inputWrapper {
```

```
    margin-bottom: 12px;
```

```
  }
```

```
}
```

```
@media (max-width: 900px) {
```

```
  .input {
```

```
    font-size: 16px;
```

```
    height: 46px;
```

```
    padding: 10px 14px;
```

```
  }
```

```
}
```

```
@media (max-width: 768px) {
```

```
  .input {
```

```
    font-size: 16px;
```

```

        height: 44px;

        padding: 10px 14px;
    }

    .inputWrapper {
        margin-bottom: 12px;
    }
}

@media (max-width: 600px) {
    .input {
        font-size: 15px;

        height: 42px;

        padding: 9px 12px;
    }

    .inputWrapper {
        margin-bottom: 10px;
    }
}

@media (max-width: 480px) {
    .input {
        font-size: 14px;

        height: 40px;

```

```

padding: 8px 12px;
}

.inputWrapper {
margin-bottom: 8px;
}
}

===== \components\common\Form\RegisterAuthForm\InputForm.tsx
=====

import React from 'react';
import styles from './InputForm.module.css';

interface InputFormProps {
names: string;
pl: string;
type?: string;
value?: string;
onChange?: (e: React.ChangeEvent<HTMLInputElement>) => void;
required?: boolean;
}

const InputForm: React.FC<InputFormProps> = ({
names,
pl,
type = 'text',

```

```

    value,
    onChange,
    required = true
  )) => {
    return (
      <div className={styles.inputWrapper}>
        <input
          type={type}
          name={names}
          placeholder={pl}
          className={styles.input}
          value={value}
          onChange={onChange}
          required={required}
        />
      </div>
    );
  };

```

```
export default InputForm;
```

```
===== \components\common\Header\Header.module.css =====
```

```
@import "../style/variables.css";
```

```
.header {
```

```
  height: 80px;
```

```
background-color: var(--color-blockColor);

font-size: 20px;

color: var(--color-text);

padding: 1rem;

position: sticky;

display: flex;

top: 0;

z-index: 1000;

box-shadow: 0 10px 10px -5px rgba(0, 0, 0, 0.3);
}

.container {

width: 100%;

margin: 0;

display: flex;

justify-content: space-between;

align-items: center;

position: relative;

font-family: 'Inter', -apple-system, BlinkMacSystemFont, sans-serif;

gap: 20px;
}

.logo {

color: var(--color-text);

text-decoration: none;
```

```
font-size: 32px;  
white-space: nowrap;  
}
```

```
.logo:hover {  
  text-decoration: none;  
}
```

```
.nav {  
  position: absolute;  
  left: 50%;  
  transform: translateX(-50%);  
  display: flex;  
  gap: 4rem;  
}
```

```
.navLink {  
  color: var(--color-text);  
  text-decoration: none;  
  white-space: nowrap;  
  transition: opacity 0.2s;  
}
```

```
.navLink:hover {  
  text-decoration: none;
```



```
    opacity: 0.7;
}
```

```
.loginButton {
    height: 40px;
    background-color: var(--color-buttonColor);
    font-size: 20px;
    border-radius: 10px;
    display: flex;
    align-items: center;
    justify-content: center;
    color: white;
    text-decoration: none;
    gap: 8px;
    transition: opacity 0.2s;
    flex-shrink: 0;
}
```

```
.loginButton:not(:has(span)) {
    width: 40px;
}
```

```
.loginButton:has(span) {
    width: 120px;
}
```

```
.loginButton:hover {  
  opacity: 0.85;  
  text-decoration: none;  
}
```

```
.loginButton img {  
  height: 25px;  
  width: 30px;  
}
```

```
.mobileButtons {  
  display: none;  
  gap: 12px;  
  flex-shrink: 0;  
}
```

```
.searchButton {  
  width: 60px;  
  height: 60px;  
  background-color: var(--color-buttonColor);  
  border: none;  
  border-radius: 12px;  
  display: flex;  
  align-items: center;
```

```
justify-content: center;

cursor: pointer;

transition: opacity 0.2s, transform 0.1s;

flex-shrink: 0;
}

.profileButton {

width: 60px;

height: 60px;

background-color: var(--color-buttonColor);

border: none;

border-radius: 12px;

display: flex;

align-items: center;

justify-content: center;

cursor: pointer;

transition: opacity 0.2s, transform 0.1s;

text-decoration: none;

flex-shrink: 0;
}
```

```
.burgerButton {

width: 60px;

height: 60px;

background-color: var(--color-buttonColor);
```

```
border: none;

border-radius: 12px;

display: flex;

align-items: center;

justify-content: center;

cursor: pointer;

transition: opacity 0.2s, transform 0.1s;

flex-shrink: 0;

}
```

```
.searchButton:active,

.profileButton:active,

.burgerButton:active {

  transform: scale(0.96);

}
```

```
.searchButton svg,

.profileButton svg,

.burgerButton svg {

  width: 28px;

  height: 28px;

  stroke: white;

}
```

```
.mobileMenu {
```

```
position: fixed;

top: 0;

right: -320px;

width: 280px;

height: 100vh;

background-color: var(--color-blockColor);

z-index: 2000;

transition: right 0.3s ease;

box-shadow: -5px 0 15px rgba(0, 0, 0, 0.2);

overflow-y: auto;

}
```

```
.mobileMenuOpen {

  right: 0;

}
```

```
.mobileMenuHeader {

  display: flex;

  justify-content: flex-end;

  padding: 20px;

}
```

```
.closeMenuButton {

  background: none;

  border: none;
```

```
font-size: 32px;

cursor: pointer;

color: var(--color-text);

width: 50px;

height: 50px;

display: flex;

align-items: center;

justify-content: center;

}
```

```
.mobileNav {

display: flex;

flex-direction: column;

padding: 20px;

gap: 10px;

}
```

```
.mobileNavLink {

color: var(--color-text);

text-decoration: none;

font-size: 24px;

padding: 20px 0;

border-bottom: 1px solid var(--color-border);

text-align: center;

transition: opacity 0.2s;
```

```
}
```

```
.mobileNavLink:hover {  
  text-decoration: none;  
  opacity: 0.7;  
}
```

```
.mobileNavLink:active {  
  background-color: rgba(190, 143, 76, 0.3);  
}
```

```
.searchOverlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  width: 100%;  
  height: 100%;  
  background-color: var(--color-overlay);  
  z-index: 2000;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
}
```

```
.searchMenu {
```

```
width: 100%;  
max-height: 100%;  
background-color: var(--color-background);  
animation: slideDown 0.3s ease;  
overflow-y: auto;  
}
```

```
@keyframes slideDown {  
  from {  
    transform: translateY(-100%);  
  }  
  to {  
    transform: translateY(0);  
  }  
}
```

```
.searchMenuHeader {  
  display: flex;  
  justify-content: flex-end;  
  padding: 16px 20px;  
  background-color: var(--color-background);  
  border-bottom: 1px solid var(--color-border);  
}
```

```
.closeSearchButton {
```



```
background: none;

border: none;

font-size: 28px;

cursor: pointer;

color: var(--color-text);

width: 44px;

height: 44px;

display: flex;

align-items: center;

justify-content: center;

}
```

```
.overlay {

position: fixed;

top: 0;

left: 0;

width: 100%;

height: 100%;

background-color: var(--color-overlay);

z-index: 1500;

}
```

```
@media (max-width: 1100px) {

.nav {

gap: 2rem;
```

```
}
```

```
.logo {  
  font-size: 26px;  
}
```

```
.loginButton:has(span) {  
  width: 100px;  
  font-size: 18px;  
}  
}
```

```
@media (max-width: 950px) {  
  .container {  
    gap: 15px;  
  }  
}
```

```
.logo {  
  font-size: 22px;  
  white-space: nowrap;  
}
```

```
.nav {  
  gap: 1.5rem;  
  position: relative;
```

```
left: auto;

transform: none;

margin: 0 auto;

}
```

```
.loginButton:has(span) {

width: 90px;

font-size: 16px;

height: 36px;

}
```

```
.loginButton:not(:has(span)) {

width: 36px;

height: 36px;

}
```

```
.loginButton img {

height: 20px;

width: 24px;

}

}
```

```
@media (max-width: 850px) {

.logo {

font-size: 18px;
```

```
}
```

```
.nav {  
  gap: 1rem;  
}
```

```
.navLink {  
  font-size: 16px;  
}
```

```
.loginButton:has(span) {  
  width: 80px;  
  font-size: 14px;  
  height: 34px;  
}
```

```
.loginButton:not(:has(span)) {  
  width: 34px;  
  height: 34px;  
}
```

```
.loginButton img {  
  height: 18px;  
  width: 22px;  
}
```

```
}
```

```
@media (max-width: 768px) {
```

```
  .header {
```

```
    height: auto;
```

```
    min-height: 80px;
```

```
    padding: 10px 16px;
```

```
  }
```

```
  .logo {
```

```
    display: none;
```

```
  }
```

```
  .nav {
```

```
    display: none;
```

```
  }
```

```
  .loginButton {
```

```
    display: none;
```

```
  }
```

```
  .mobileButtons {
```

```
    display: flex;
```

```
    margin-left: auto;
```

```
  }
```

```
.searchButton {  
  display: flex;  
}  
}
```

```
@media (min-width: 769px) {  
  .searchButton {  
    display: none;  
  }  
}
```

```
@media (max-width: 480px) {  
  .header {  
    padding: 8px 12px;  
  }  
}
```

```
.searchButton,  
.profileButton,  
.burgerButton {  
  width: 50px;  
  height: 50px;  
  border-radius: 10px;  
}
```

```

.searchButton svg,

.profileButton svg,

.burgerButton svg {

  width: 24px;

  height: 24px;

}

}

===== \components\common\Header\Header.tsx =====

import React, { useState, useEffect, useRef } from 'react';

import { Link, useLocation } from 'react-router-dom';

import styles from './Header.module.css';

import loginIcon from '../../assets/header/login.png';

import SearchSection from '../Search/SearchSection/SearchSection';

import { getUserStatus } from '../../api/auth';

const Header = () => {

  const [isMenuOpen, setIsMenuOpen] = useState(false);

  const [isSearchOpen, setIsSearchOpen] = useState(false);

  const location = useLocation();

  const searchRef = useRef(null);

  const [isAuthenticated, setIsAuthenticated] = useState(false);

  const [userId, setUserId] = useState<string | null>(null);

  const [isAdmin, setIsAdmin] = useState(false);

  const showSearch = true;

```

```

useEffect(() => {

  const checkAuth = async () => {

    const token = localStorage.getItem('access_token');

    const userStr = localStorage.getItem('user');

    if (token && userStr) {

      try {

        const user = JSON.parse(userStr);

        setIsAuthenticated(true);

        setUserId(user.id?.toString() || null);

        // Проверяем статус администратора

        const status = await getUserStatus();

        setIsAdmin(status === 'admin');

      } catch (e) {

        setIsAuthenticated(false);

        setUserId(null);

        setIsAdmin(false);

      }

    } else {

      setIsAuthenticated(false);

      setUserId(null);

      setIsAdmin(false);

    }
  }
}

```



```
};
```

```
checkAuth();
```

```
window.addEventListener('storage', checkAuth);
```

```
return () => {
```

```
  window.removeEventListener('storage', checkAuth);
```

```
};
```

```
}, []);
```

```
const toggleMenu = () => {
```

```
  setIsMenuOpen(!isMenuOpen);
```

```
};
```

```
const closeMenu = () => {
```

```
  setIsMenuOpen(false);
```

```
};
```

```
const toggleSearch = () => {
```

```
  setIsSearchOpen(!isSearchOpen);
```

```
  if (isMenuOpen) setIsMenuOpen(false);
```

```
};
```

```
const closeSearch = () => {
```

```
  setIsSearchOpen(false);
```

```
};
```

```
useEffect(() => {
```

```
  const handleClickOutside = (event) => {
```

```
    if (searchRef.current && !searchRef.current.contains(event.target)) {
```

```
      closeSearch();
```

```
    }
```

```
  };
```

```
  if (isSearchOpen) {
```

```
    document.addEventListener('mousedown', handleClickOutside);
```

```
    document.body.style.overflow = 'hidden';
```

```
  } else {
```

```
    document.body.style.overflow = '';
```

```
  }
```

```
  return () => {
```

```
    document.removeEventListener('mousedown', handleClickOutside);
```

```
    document.body.style.overflow = '';
```

```
  };
```

```
}, [isSearchOpen]);
```

```
return (
```

```
  <
```

```
    <header className={styles.header}>
```

```

<div className={styles.container}>

  {showSearch && (

    <button className={styles.searchButton} onClick={toggleSearch} aria-
label="Поиск">

      <svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="white"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">

        <path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path>

        <circle cx="12" cy="7" r="4"></circle>

      </svg>

    </button>

  )}

  <Link to="/" className={styles.logo}>

    ОНЛАЙН-ТЕСТЫ

  </Link>

  <nav className={styles.nav}>

    <Link to="/" className={styles.navLink}>ГЛАВНАЯ</Link>

    <Link to="/testlist" className={styles.navLink}>ТЕСТЫ</Link>

    <Link to="/create" className={styles.navLink}>СОЗДАТЬ</Link>

    { /* Ссылка на админ панель только для администратора */ }

    {isAdmin && (

      <Link to="/admin" className={styles.navLink}>АДМИН</Link>

    )}

  </nav>

```

{/* Одна и та же кнопка: если авторизован - ведёт на профиль, если нет - на регистрацию */}

<Link to={isAuthenticated ? `/profile/\${userId}` : "/register"}
className={styles.loginButton}>

{!isAuthenticated && войти}

</Link>

<div className={styles.mobileButtons}>

<Link to={isAuthenticated ? `/profile/\${userId}` : "/register"}
className={styles.profileButton} aria-label="Профиль">

<svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="white"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">

<path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path>

<circle cx="12" cy="7" r="4"></circle>

</svg>

</Link>

<button className={styles.burgerButton} onClick={toggleMenu} aria-
label="Меню">

<svg width="28" height="28" viewBox="0 0 24 24" fill="none" stroke="white"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">

<path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4 4v2"></path>

<circle cx="12" cy="7" r="4"></circle>

</svg>

</button>

```

    </div>

</div>

</header>

<div className={` ${styles.mobileMenu} ${isMenuOpen ? styles.mobileMenuOpen :
"}}`}>

    <div className={styles.mobileMenuHeader}>

        <button className={styles.closeMenuButton} onClick={closeMenu}> ✕ </button>

    </div>

    <nav className={styles.mobileNav}>

        <Link to="/" className={styles.mobileNavLink} onClick={closeMenu}
>ГЛАВНАЯ</Link>

        <Link to="/testlist" className={styles.mobileNavLink} onClick={closeMenu}
>ТЕСТЫ</Link>

        <Link to="/create" className={styles.mobileNavLink} onClick={closeMenu}
>СОЗДАТЬ</Link>

        {isAdmin && (

            <Link to="/admin" className={styles.mobileNavLink} onClick={closeMenu}
>АДМИН</Link>

        )}

        <Link to={isAuthenticated ? `/profile/${userId}` : "/register"}
className={styles.mobileNavLink} onClick={closeMenu}>

            {isAuthenticated ? "ПРОФИЛЬ" : "ВОЙТИ"}

        </Link>

    </nav>

</div>

```

```

    {isSearchOpen && (
      <div className={styles.searchOverlay}>
        <div className={styles.searchMenu} ref={searchRef}>
          <div className={styles.searchMenuHeader}>
            <button className={styles.closeSearchButton}
onClick={closeSearch}>✕</button>
          </div>
          <SearchSection isMobile={true} showTypeSelector={false} />
        </div>
      </div>
    )}

    {isMenuOpen && <div className={styles.overlay} onClick={closeMenu}></div>}
  </>
);
};

```

export default Header;

```

===== \components\common\Home\ActiveUsers\UsersSection.module.css
=====

```

```
@import "../style/variables.css";
```

```

.section {
  background-color: var(--color-background);
  padding: 0px 133px 27px 133px;
}

```

```
}
```

```
.title {
```

```
  color: var(--color-text);
```

```
  font-size: 20px;
```

```
  margin-bottom: 25px;
```

```
  margin-top: 0;
```

```
  font-weight: 700;
```

```
  text-align: left;
```

```
  font-family: 'Arial', sans-serif;
```

```
  text-transform: uppercase;
```

```
}
```

```
.wrapper {
```

```
  position: relative;
```

```
  overflow: visible;
```

```
}
```

```
.grid {
```

```
  display: flex;
```

```
  gap: 32px;
```

```
  overflow-x: auto;
```

```
  scroll-behavior: smooth;
```

```
  scroll-snap-type: x mandatory;
```

```
  padding-bottom: 10px;
```

```

}

/* переопределяем ширину карточки */
.grid > * {
    flex: 0 0 auto;
    scroll-snap-align: center;
}

.grid::-webkit-scrollbar {
    display: none;
}

.grid {
    -ms-overflow-style: none;
    scrollbar-width: none;
}

.scrollButtonLeft,
.scrollButtonRight {
    position: absolute;
    top: 50%;
    transform: translateY(-50%);
    background: none;
    border: none;
    font-size: 48px;

```



```
    cursor: pointer;

    color: var(--color-text);

    z-index: 10;

    padding: 10px;

    transition: opacity 0.2s;
}

.scrollButtonLeft:active,
.scrollButtonRight:active {

    outline: none;

    border: none;

    background: none;

    box-shadow: none;

    opacity: 1;
}

.scrollButtonLeft:focus,
.scrollButtonRight:focus {

    outline: none;
}

.scrollButtonLeft {

    left: -60px;
}
```

```
.scrollButtonRight {  
  right: -60px;  
}
```

```
.scrollButtonLeft:hover,  
.scrollButtonRight:hover {  
  opacity: 0.7;  
}
```

```
/* Стили карточки */
```

```
.card {  
  background-color: var(--color-blockColor);  
  border-radius: 10px;  
  padding: 20px;  
  text-decoration: none;  
  transition: box-shadow 0.2s;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  width: 200px;  
  flex-shrink: 0;  
  scroll-snap-align: start;  
  box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);  
}
```

```
.card:hover {  
  background-color: var(--color-blockColor);  
}
```

```
.avatar {  
  width: 100px;  
  height: 100px;  
  border-radius: 50%;  
  overflow: hidden;  
  margin-bottom: 15px;  
  background-color: var(--color-avatar);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  position: relative;  
}
```

```
.avatar img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

```
.avatarPlaceholder {  
  width: 100%;
```

```
height: 100%;  
  
background-color: var(--color-avatar);  
  
display: none;  
  
align-items: center;  
  
justify-content: center;  
  
color: white;  
  
font-size: 48px;  
  
font-weight: bold;  
  
}
```

```
.avatarPlaceholder.show {  
  
  display: flex;  
  
}
```

```
.name {  
  
  font-size: 18px;  
  
  font-weight: bold;  
  
  color: var(--color-text);  
  
  margin-bottom: 12px;  
  
  text-align: center;  
  
  word-break: break-word;  
  
}
```

```
.stats {  
  
  width: 100%;
```

```
display: flex;

flex-direction: column;

gap: 8px;
}


.stat {

display: flex;

justify-content: space-between;

align-items: baseline;

gap: 10px;

background-color: var(--color-background);

padding: 8px 12px;

border-radius: 8px;
}


.statValue {

font-size: 20px;

font-weight: bold;

color: var(--color-text);
}


.statLabel {

font-size: 12px;

color: var(--color-text-muted);
}
```

```
/* Загрузка и пусто */

.loading, .empty {

    text-align: center;

    padding: 40px;

    color: var(--color-text-muted);

    font-size: 16px;

}


/* Планшет */

@media (max-width: 1024px) {

    .section {

        padding: 0px 60px 27px 60px;

    }


    .grid {

        gap: 24px;

    }


    .scrollButtonLeft {

        left: -50px;

    }


    .scrollButtonRight {

        right: -50px;

    }

}
```

```
}  
  
}  
  
/* Мобильный */  
  
@media (max-width: 768px) {  
  .section {  
    padding: 20px 0;  
  }  
  
  .title {  
    font-size: 18px;  
    margin-bottom: 20px;  
    margin-left: 16px;  
  }  
  
  .grid {  
    gap: 20px;  
    padding: 0 16px;  
  }  
  
  .scrollButtonLeft,  
  .scrollButtonRight {  
    font-size: 32px;  
    width: 36px;  
    height: 36px;
```

```
display: flex;

align-items: center;

justify-content: center;

border-radius: 50%;

left: 8px;

padding: 0;

}
```

```
.scrollButtonRight {

    left: auto;

    right: 8px;

}

}
```

```
/* Маленький телефон */

@media (max-width: 480px) {

    .section {

        padding: 16px 0;

    }

    .title {

        font-size: 16px;

        margin-bottom: 16px;

        margin-left: 12px;

    }

}
```



```
.grid {  
    gap: 16px;  
    padding: 0 12px;  
}
```

```
.scrollButtonLeft,  
.scrollButtonRight {  
    font-size: 24px;  
    width: 32px;  
    height: 32px;  
    left: 4px;  
}
```

```
.scrollButtonRight {  
    left: auto;  
    right: 4px;  
}  
}
```

```
/* Для 390px */
```

```
@media (max-width: 390px) {  
    .grid {  
        gap: 14px;  
    }  
}
```

```

}

===== \components\common\Home\ActiveUsers\UsersSection.tsx =====

import React, { useRef, useState, useEffect } from 'react';

import Card from '../Card/UserCard/Card';

import api from '../../api/axios';

import styles from './UsersSection.module.css';

interface ActiveUsersProps {
  title: string;
}

const ActiveUsers = ({ title }: ActiveUsersProps) => {
  const scrollContainerRef = useRef<HTMLDivElement>(null);

  const [showLeftArrow, setShowLeftArrow] = useState(false);
  const [showRightArrow, setShowRightArrow] = useState(true);

  const [users, setUsers] = useState<any[]>([]);

  const [loading, setLoading] = useState(true);

  const BASE_URL = 'http://localhost:8000';

  useEffect(() => {
    loadUsers();
  }, []);

  const loadUsers = async () => {
    setLoading(true);

```

```

try {

  const response = await api.get('/users/top/');

  console.log('API /users/top/ response:', response.data); // Отладка


  const formattedUsers = response.data.map((user: any) => {

    // Формируем правильный URL для аватара

    let avatarUrl = null;

    if (user.avatar) {

      if (user.avatar.startsWith('http://') || user.avatar.startsWith('https://')) {

        avatarUrl = user.avatar;

      } else if (user.avatar.startsWith('data:image')) {

        avatarUrl = user.avatar;

      } else {

        avatarUrl = `${BASE_URL}${user.avatar}`;

      }

    }

  })


  // ВАЖНО: проверяем какие поля приходят с сервера

  console.log('User data:', {

    id: user.id,

    name: user.full_name || user.login,

    avatar: avatarUrl,

    completed_tests: user.completed_tests_count || user.completed_tests || 0,

    created_tests: user.created_tests || user.tests_count || 0

  });

```

```

return {
  id: user.id,
  name: user.full_name || user.login,
  avatar: avatarUrl,
  completed_tests: user.completed_tests_count || user.completed_tests || 0,
  created_tests: user.created_tests || user.tests_count || 0
};
});

setUsers(formattedUsers.slice(0, 10));
} catch (error) {
  console.error('Ошибка загрузки пользователей:', error);
  setUsers([]);
} finally {
  setLoading(false);
}
};

const checkScroll = () => {
  if (scrollContainerRef.current) {
    const { scrollLeft, scrollWidth, clientWidth } = scrollContainerRef.current;
    setShowLeftArrow(scrollLeft > 0);
    setShowRightArrow(scrollLeft + clientWidth < scrollWidth - 10);
  }
};

```

```

useEffect(() => {
  const container = scrollContainerRef.current;
  if (container) {
    container.addEventListener('scroll', checkScroll);
    checkScroll();
    return () => container.removeEventListener('scroll', checkScroll);
  }
}, [users]);

```

```

const scroll = (direction: 'left' | 'right') => {
  if (scrollContainerRef.current) {
    const scrollAmount = 400;
    scrollContainerRef.current.scrollBy({
      left: direction === 'right' ? scrollAmount : -scrollAmount,
      behavior: 'smooth'
    });
  }
};

```

```

if (loading) {
  return (
    <section className={styles.section}>
      <div className={styles.container}>
        <h2 className={styles.title}>{title}</h2>

```

```

        <div className={styles.loading}>Загрузка...</div>

    </div>

</section>

);

}

if (users.length === 0) {

    return (

        <section className={styles.section}>

            <div className={styles.container}>

                <h2 className={styles.title}>{title}</h2>

                <div className={styles.empty}>Нет пользователей</div>

            </div>

        </section>

    );

}

return (

    <section className={styles.section}>

        <div className={styles.container}>

            <h2 className={styles.title}>{title}</h2>

            <div className={styles.wrapper}>

                {showLeftArrow && (

                    <button

```

```

        className={styles.scrollButtonLeft}

        onClick={() => scroll('left')}

    <button>

        &lt;

    </button>

    )}

<div className={styles.grid} ref={scrollContainerRef}>

    {users.map((user) => (

        <Card

            key={user.id}

            name={user.name}

            avatar={user.avatar}

            completed={user.completed_tests.toString()}

            created={user.created_tests.toString()}

            userId={user.id.toString()}

        />

    ))}

</div>

{showRightArrow && (

    <button

        className={styles.scrollButtonRight}

        onClick={() => scroll('right')}

    >

```

```

        &gt;

    </button>

    })

</div>

</div>

</section>

);

};

export default ActiveUsers;

===== \components\common\Home\TestSection\TestSection.module.css
=====

@import "../../../style/variables.css";

.section {

    background-color: var(--color-background);

    padding: 0px 133px 27px 133px;

}

.title {

    color: var(--color-text);

    font-size: 40px;

    margin-bottom: 25px;

    margin-top: 0;

    font-weight: 700;

```



```
text-align: left;

letter-spacing: normal;

font-family: 'Arial', sans-serif;

}
```

```
.wrapper {

  position: relative;

  overflow: visible;

}
```

```
.grid {

  display: flex;

  gap: 30px;

  overflow-x: auto;

  scroll-behavior: smooth;

  scroll-snap-type: x mandatory;

}
```

```
.grid > * {

  flex: 0 0 auto;

  width: calc(25% - 22.5px);

  min-width: 250px;

  scroll-snap-align: start;

  box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);

  border-radius: 10px;
```

```
    transition: box-shadow 0.2s;
}
```

```
.grid > *:hover {
    box-shadow: 0 8px 20px rgba(0, 0, 0, 0.15);
}
```

```
.grid::-webkit-scrollbar {
    display: none;
}
```

```
.grid {
    -ms-overflow-style: none;
    scrollbar-width: none;
}
```

```
.scrollButtonLeft,
.scrollButtonRight {
    position: absolute;
    top: 50%;
    transform: translateY(-50%);
    background: none;
    border: none;
    font-size: 48px;
    cursor: pointer;
```

```
color: var(--color-text);  
z-index: 10;  
padding: 10px;  
outline: none;  
transition: opacity 0.2s;  
}
```

```
.scrollButtonLeft:focus,  
.scrollButtonRight:focus {  
  outline: none;  
}
```

```
.scrollButtonLeft:active,  
.scrollButtonRight:active {  
  outline: none;  
}
```

```
.scrollButtonLeft {  
  left: -60px;  
}
```

```
.scrollButtonRight {  
  right: -60px;  
}
```

```
.scrollButtonLeft:hover,  
.scrollButtonRight:hover {  
  opacity: 0.7;  
}
```

```
@media (max-width: 1024px) {  
  .section {  
    padding: 0px 60px 27px 60px;  
  }
```

```
.title {  
  font-size: 40px;  
}
```

```
.grid {  
  gap: 20px;  
}
```

```
.grid > * {  
  width: calc(33.333% - 14px);  
}
```

```
.scrollButtonLeft {  
  left: -50px;  
}
```

```
.scrollButtonRight {  
  right: -50px;  
}  
}
```

```
@media (max-width: 768px) {  
  .section {  
    padding: 20px 0 20px 0;  
  }  
}
```

```
.title {  
  font-size: 40px;  
  margin-bottom: 20px;  
  margin-left: 16px;  
}
```

```
.wrapper {  
  overflow: hidden;  
}
```

```
.grid {  
  gap: 60px;  
  padding: 0;  
}
```

```
.grid > * {  
  width: 350px;  
  min-width: 350px;  
  scroll-snap-align: center;  
  margin: 0;  
}
```

```
.grid > *:first-child {  
  margin-left: calc(50% - 175px);  
}
```

```
.grid > *:last-child {  
  margin-right: calc(50% - 175px);  
}
```

```
.scrollButtonLeft,  
.scrollButtonRight {  
  display: flex !important;  
  font-size: 32px;  
  color: var(--color-text);  
  border-radius: 50%;  
  width: 36px;  
  height: 36px;  
  left: 8px;
```

```
}
```

```
.scrollButtonRight {
```

```
  left: auto;
```

```
  right: 8px;
```

```
}
```

```
}
```

```
@media (max-width: 480px) {
```

```
  .grid {
```

```
    gap: 50px;
```

```
}
```

```
.grid > * {
```

```
  width: 310px;
```

```
  min-width: 310px;
```

```
}
```

```
.grid > *:first-child {
```

```
  margin-left: calc(50% - 155px);
```

```
}
```

```
.grid > *:last-child {
```

```
  margin-right: calc(50% - 155px);
```

```
}
```

```
.scrollButtonLeft,  
.scrollButtonRight {  
  font-size: 28px;  
  width: 32px;  
  height: 32px;  
  left: 4px;  
}
```

```
.scrollButtonRight {  
  left: auto;  
  right: 4px;  
}  
}
```

```
.loadingContainer {  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  justify-content: center;  
  padding: 40px;  
  gap: 20px;  
}
```

```
.loadingImage {
```



```

width: 200px;

height: auto;

object-fit: contain;

}

```

```

.loadingText {

  font-size: 24px;

  font-weight: bold;

  color: var(--color-text);

  text-align: center;

  letter-spacing: 2px;

}

```

===== \components\common\Home\TestSection\TestSection.tsx =====

```

import React, { useRef, useState, useEffect } from 'react';

import Card from '../Card/TestCard/Card';

import { getRecentTests, getPopularTests, getRecentSurveys, getTestRating } from
'../../../../../api/tests';

import styles from './TestSection.module.css';

```

```

interface TestSectionProps {

  title: string;

  type: 'recent' | 'popular' | 'surveys';

}

```

```

const TestSection = ({ title, type }: TestSectionProps) => {

```

```

const scrollContainerRef = useRef<HTMLDivElement>(null);

const [showLeftArrow, setShowLeftArrow] = useState(false);

const [showRightArrow, setShowRightArrow] = useState(true);

const [tests, setTests] = useState<any[]>([]);

const [loading, setLoading] = useState(true);

const [refreshKey, setRefreshKey] = useState(0);


const BASE_URL = 'http://localhost:8000';


const loadTests = async () => {

  setLoading(true);

  try {

    let data;

    if (type === 'recent') {

      data = await getRecentTests(8);

    } else if (type === 'popular') {

      data = await getPopularTests(8);

    } else {

      data = await getRecentSurveys(8);

    }

    const testsWithRating = await Promise.all(data.map(async (test: any) => {

      try {

        const ratingData = await getTestRating(test.id);

```

```
// Формируем полный URL для картинки теста
```

```
let imageFullUrl = null;
```

```
if (test.image) {
```

```
  if (test.image.startsWith('http')) {
```

```
    imageFullUrl = test.image;
```

```
  } else {
```

```
    imageFullUrl = `${BASE_URL}${test.image}`;
```

```
  }
```

```
} else if (test.image_url) {
```

```
  if (test.image_url.startsWith('http')) {
```

```
    imageFullUrl = test.image_url;
```

```
  } else {
```

```
    imageFullUrl = `${BASE_URL}${test.image_url}`;
```

```
  }
```

```
}
```

```
// Формируем полный URL для аватара автора
```

```
let authorAvatarUrl = null;
```

```
if (test.author_info?.avatar) {
```

```
  if (test.author_info.avatar.startsWith('http')) {
```

```
    authorAvatarUrl = test.author_info.avatar;
```

```
  } else {
```

```
    authorAvatarUrl = `${BASE_URL}${test.author_info.avatar}`;
```

```
  }
```

```
}
```

```

return {
    ...test,
    average_rating: ratingData.average_rating || 0,
    ratings_count: ratingData.ratings_count || 0,
    full_image_url: imageFullUrl,
    full_avatar_url: authorAvatarUrl
};
} catch (error) {
    // Формируем полный URL для картинки теста при ошибке
    let imageFullUrl = null;
    if (test.image) {
        if (test.image.startsWith('http')) {
            imageFullUrl = test.image;
        } else {
            imageFullUrl = `${BASE_URL}${test.image}`;
        }
    } else if (test.image_url) {
        if (test.image_url.startsWith('http')) {
            imageFullUrl = test.image_url;
        } else {
            imageFullUrl = `${BASE_URL}${test.image_url}`;
        }
    }
}

```

```

let authorAvatarUrl = null;

if (test.author_info?.avatar) {

  if (test.author_info.avatar.startsWith('http')) {

    authorAvatarUrl = test.author_info.avatar;

  } else {

    authorAvatarUrl = `${BASE_URL}${test.author_info.avatar}`;

  }

}

return {

  ...test,

  average_rating: 0,

  ratings_count: 0,

  full_image_url: imageFullUrl,

  full_avatar_url: authorAvatarUrl

};

}

));

setTests(testsWithRating);

} catch (error) {

  console.error('Ошибка загрузки:', error);

} finally {

  setLoading(false);

}

```

```
};
```

```
useEffect(() => {  
  loadTests();  
}, [type, refreshKey]);
```

```
useEffect(() => {  
  const handleProfileUpdate = () => {  
    setRefreshKey(prev => prev + 1);  
  };  
};
```

```
window.addEventListener('profileUpdated', handleProfileUpdate);  
return () => window.removeEventListener('profileUpdated', handleProfileUpdate);  
}, []);
```

```
useEffect(() => {  
  const handleFocus = () => {  
    loadTests();  
  };  
  window.addEventListener('focus', handleFocus);  
  return () => window.removeEventListener('focus', handleFocus);  
}, [type]);
```

```
const checkScroll = () => {  
  if (scrollContainerRef.current) {
```

```

const { scrollLeft, scrollWidth, clientWidth } = scrollContainerRef.current;

setShowLeftArrow(scrollLeft > 0);

setShowRightArrow(scrollLeft + clientWidth < scrollWidth - 10);

}

};

```

```

useEffect(() => {

  const container = scrollContainerRef.current;

  if (container) {

    container.addEventListener('scroll', checkScroll);

    checkScroll();

    return () => container.removeEventListener('scroll', checkScroll);

  }

}, [tests]);

```

```

const scroll = (direction: 'left' | 'right') => {

  if (scrollContainerRef.current) {

    const scrollAmount = 400;

    scrollContainerRef.current.scrollBy({

      left: direction === 'right' ? scrollAmount : -scrollAmount,

      behavior: 'smooth'

    });

  }

};

```

```

if (loading) {
  return (
    <section className={styles.section}>
      <div className={styles.container}>
        <h2 className={styles.title}>{title}</h2>
        <div className={styles.loadingContainer}>
           {
              e.currentTarget.style.display = 'none';
            }}
          />
          <div className={styles.loadingText}>ЗАГРУЗКА</div>
        </div>
      </div>
    </section>
  );
}

```

```

if (tests.length === 0) {
  return (
    <section className={styles.section}>
      <div className={styles.container}>

```



```

    <h2 className={styles.title}>{title}</h2>

    <div className={styles.empty}>Нет тестов</div>

  </div>

</section>

);
}

return (
  <section className={styles.section}>
    <div className={styles.container}>
      <h2 className={styles.title}>{title}</h2>

      <div className={styles.wrapper}>
        {showLeftArrow && (
          <button
            className={styles.scrollButtonLeft}
            onClick={() => scroll('left')}
          >
            &lt;
          </button>
        )}

        <div className={styles.grid} ref={scrollContainerRef}>
          {tests.map((test) => (
            <Card

```

```

        key={test.id}

        title={test.title}

        description={test.description}

        author={test.author_info?.full_name || test.author_info?.login || 'Неизвестный
автор'}

        rating={`$${test.average_rating || 0}/5`}

        users={test.ratings_count?.toString() || '0'}

        questions={test.questions_count?.toString() || '0'}

        type={test.is_survey ? 'survey' : 'test'}

        testId={test.id.toString()}

        authorId={test.author_info?.id?.toString() || ''}

        imageUrl={test.full_image_url || undefined}

        authorAvatar={test.full_avatar_url || undefined}

      />

    )))}

  </div>

```

```

    {showRightArrow && (

      <button

        className={styles.scrollButtonRight}

        onClick={() => scroll('right')}

      >

        &gt;

      </button>

    )}

```

```

        </div>

    </div>

</section>

);

};

export default TestSection;

===== \components\common\Profile\ProfileHeader\ProfileHeader.module.css
=====

@import "../../style/variables.css";

.profileInfoBlockMain {
    display: flex;
    flex-direction: row;
    flex-wrap: wrap;
}

.profileImage {
    display: flex;
    padding: 2%;
    width: 20%;
    height: 100%;
}

.profileImage img {

```

```
width: 280px;

height: 290px;

background-color: gray;

border-radius: 10px;

}
```

```
.profileInfoBlockMainMain {

color: black;

font-family: Arial, Helvetica, sans-serif;

font-size: 30px;

display: flex;

flex-direction: column;

flex: 1;

padding: 2%;

}
```

```
.profileName span,

.profileBio span {

font-family: Arial, Helvetica, sans-serif;

color: black;

}
```

```
.profileName {

margin-bottom: 10px;

}
```

```
.profileBio {  
    display: flex;  
    justify-content: space-between;  
    width: 100%;  
    height: 100%;  
    word-wrap: break-word;  
    word-break: break-word;  
    white-space: normal;  
    overflow-wrap: break-word;  
    font-family: Arial, Helvetica, sans-serif;  
    color: black;  
}
```

```
.profileStats {  
    display: flex;  
    flex-direction: row;  
    gap: 30px;  
    margin-top: 20px;  
    flex-wrap: wrap;  
    font-size: 20px;  
}
```

```
.profileStats p {  
    margin: 0;
```

```
font-size: 20px;

font-family: Arial, Helvetica, sans-serif;

color: black;

}
```

```
.profileInfoBlockMainEnd {

    display: flex;

    flex-direction: column;

    align-items: flex-end;

    justify-content: flex-start;

    gap: 10px;

    padding: 2%;

}
```

/* Общие стили для всех кнопок - убираем обводку и делаем текст черным */

```
.logoutButton,

.editButton,

.friendButton,

.muteButton,

.banButton,

.reportButton {

    background-color: var(--color-buttonColor);

    color: black;

    border: none;

    padding: 10px 20px;
```

```
border-radius: 5px;

cursor: pointer;

font-size: 14px;

font-family: Arial, Helvetica, sans-serif;

transition: background-color 0.2s;

width: 180px;

}
```

```
.logoutButton:focus,

.logoutButton:active,

.editButton:focus,

.editButton:active,

.friendButton:focus,

.friendButton:active,

.muteButton:focus,

.muteButton:active,

.banButton:focus,

.banButton:active,

.reportButton:focus,

.reportButton:active,

button:focus,

button:active {

    outline: none;

    border: none;

    box-shadow: none;
```

```
}
```

```
.logoutButton:hover,
```

```
.editButton:hover,
```

```
.friendButton:hover,
```

```
.muteButton:hover,
```

```
.banButton:hover {
```

```
    background-color: #8B6B3A;
```

```
}
```

```
.reportButton {
```

```
    width: 32px;
```

```
    height: 32px;
```

```
    background-color: var(--color-buttonColor, #BE8F4C);
```

```
    color: black;
```

```
    border: none;
```

```
    font-size: 18px;
```

```
    font-weight: bold;
```

```
    cursor: pointer;
```

```
    display: flex;
```

```
    align-items: center;
```

```
    justify-content: center;
```

```
    transition: all 0.3s ease;
```

```
    padding: 0;
```

```
}
```



```
.reportButton:hover {  
    background-color: #a47a3f;  
    transform: scale(1.05);  
}
```

```
.loading,  
.error {  
    text-align: center;  
    padding: 40px;  
    color: black;  
    font-family: Arial, Helvetica, sans-serif;  
}
```

```
.modalOverlay {  
    position: fixed;  
    top: 0;  
    left: 0;  
    right: 0;  
    bottom: 0;  
    background-color: rgba(0, 0, 0, 0.7);  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    z-index: 1000;
```

```
}
```

```
.modalContent {  
    background-color: #D8C3A5;  
    border-radius: 15px;  
    padding: 30px;  
    width: 90%;  
    max-width: 500px;  
    max-height: 90vh;  
    overflow-y: auto;  
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
}
```

```
.modalTitle {  
    font-size: 28px;  
    color: black;  
    margin-bottom: 25px;  
    text-align: center;  
    font-family: Arial, sans-serif;  
    font-weight: bold;  
}
```

```
.formGroup {  
    margin-bottom: 20px;  
}
```

```
.formGroup label {  
    display: block;  
    margin-bottom: 8px;  
    color: black;  
    font-size: 16px;  
    font-weight: bold;  
    font-family: Arial, sans-serif;  
}
```

```
.formGroup input,  
.formGroup textarea {  
    width: 100%;  
    padding: 12px;  
    border: 1px solid #BE8F4C;  
    border-radius: 8px;  
    background-color: #EAE7DC;  
    color: black;  
    font-size: 14px;  
    font-family: Arial, sans-serif;  
    box-sizing: border-box;  
}
```

```
.formGroup input:focus,  
.formGroup textarea:focus {
```

```
outline: none;

border-color: #8B6B3A;

box-shadow: 0 0 5px rgba(139, 107, 58, 0.3);

}
```

```
.formGroup textarea {

    resize: vertical;

    min-height: 80px;

}
```

```
.avatarPreview {

    display: flex;

    flex-direction: column;

    align-items: center;

    margin-bottom: 20px;

}
```

```
.avatarPreview img {

    width: 120px;

    height: 120px;

    border-radius: 10px;

    background-color: #4C4C4C;

    object-fit: cover;

    margin-bottom: 10px;

}
```

```
.avatarPreview button {  
    background-color: #BE8F4C;  
    color: black;  
    border: none;  
    border-radius: 8px;  
    padding: 8px 16px;  
    font-size: 14px;  
    cursor: pointer;  
    font-family: Arial, sans-serif;  
    transition: background-color 0.3s;  
}
```

```
.avatarPreview button:focus,  
.avatarPreview button:active {  
    outline: none;  
    border: none;  
    box-shadow: none;  
}
```

```
.avatarPreview button:hover {  
    background-color: #8B6B3A;  
}
```

```
.fileInput {
```

```
    display: none;
}
```

```
.modalButtons {
    display: flex;
    justify-content: flex-end;
    gap: 15px;
    margin-top: 25px;
}
```

```
.cancelButton {
    background-color: #999;
    color: white;
    padding: 10px 20px;
    border: none;
    border-radius: 8px;
    font-size: 16px;
    cursor: pointer;
    font-family: Arial, sans-serif;
    transition: background-color 0.3s;
}
```

```
.cancelButton:focus,
.cancelButton:active {
    outline: none;
```

```
border: none;

box-shadow: none;

}
```

```
.cancelButton:hover {

    background-color: #777;

}
```

```
.saveButton {

    background-color: #BE8F4C;

    color: white;

    font-weight: bold;

    padding: 10px 20px;

    border: none;

    border-radius: 8px;

    font-size: 16px;

    cursor: pointer;

    font-family: Arial, sans-serif;

    transition: background-color 0.3s;

}
```

```
.saveButton:focus,

.saveButton:active {

    outline: none;

    border: none;
```

```
    box-shadow: none;
}

.saveButton:hover {
    background-color: #8B6B3A;
}
```

```
/* ПЛАНШЕТ */
```

```
@media (max-width: 1024px) {
    .profileInfoBlockMainMain {
        font-size: 24px;
    }
}
```

```
.profileStats {
    gap: 20px;
}
```

```
.profileStats p {
    font-size: 18px;
}
```

```
.logoutButton,
.editButton,
.friendButton,
.muteButton,
```



```
.banButton {  
    width: 160px;  
    padding: 8px 16px;  
}
```

```
.profileImage img {  
    width: 100%;  
    max-width: 200px;  
    height: auto;  
}
```

```
.profileImage {  
    width: 25%;  
}  
}
```

```
/* мобильные */
```

```
@media (max-width: 768px) {  
    .profileInfoBlockMain {  
        flex-direction: column;  
        align-items: center;  
    }  
}
```

```
.profileImage {  
    width: 100%;
```

```
    justify-content: center;

    padding: 15px;
}
```

```
.profileImage img {

    width: 150px;

    height: 150px;
}
```

```
.profileInfoBlockMainMain {

    width: 100%;

    text-align: center;

    padding: 10px 15px;

    font-size: 22px;

    align-items: center;
}
```

```
.profileName {

    font-size: 28px;

    margin-bottom: 12px;

    text-align: center;
}
```

```
.profileBio {

    text-align: center;
}
```

```
    justify-content: center;

    margin-bottom: 18px;

    font-size: 18px;
}
```

```
.profileStats {

    justify-content: center;

    text-align: center;
}
```

```
.profileStats p {

    font-size: 16px;

    margin: 0;
}
```

```
.profileInfoBlockMainEnd {

    align-items: center;

    justify-content: center;

    width: 100%;

    padding: 15px;
}
```

```
.logoutButton,

.editButton,

.friendButton,
```

```

.muteButton,

.banButton {

    width: 200px;

    padding: 12px 24px;

    font-size: 16px;

}

}

/* маленькие телефоны */

@media (max-width: 480px) {

    .profileImage img {

        width: 120px;

        height: 120px;

    }

    .profileName {

        font-size: 24px;

    }

    .profileBio {

        font-size: 16px;

    }

    .profileStats {

        flex-direction: column;

```

```

        gap: 8px;

        align-items: center;
    }

    .profileStats p {
        font-size: 14px;
    }

    .logoutButton,
    .editButton,
    .friendButton,
    .muteButton,
    .banButton {
        width: 180px;
        padding: 10px 20px;
        font-size: 14px;
    }
}

/* очень маленькие телефоны */
@media (max-width: 360px) {
    .profileImage img {
        width: 110px;
        height: 110px;
    }
}

```

```

.profileName {
    font-size: 24px;
}

.profileBio {
    font-size: 15px;
}

.profileStats p {
    font-size: 14px;
}
}

/* альбомная ориентация */
@media (max-width: 768px) and (orientation: landscape) {
    .profileInfoBlockMain {
        flex-direction: row;
    }

    .profileImage {
        width: 30%;
    }

    .profileImage img {

```

```

        width: 100px;

        height: 100px;
    }

    .profileInfoBlockMainMain {

        width: 70%;

        text-align: left;
    }

    .profileBio {

        text-align: left;

        justify-content: flex-start;
    }

    .profileStats {

        text-align: left;

        justify-content: flex-start;
    }

    .profileInfoBlockMainEnd {

        width: 100%;
    }
}

```

```

/* адаптив модального окна */

```

```
@media (max-width: 480px) {
```

```
  .modalContent {
```

```
    padding: 20px;
```

```
    width: 95%;
```

```
  }
```

```
  .modalTitle {
```

```
    font-size: 24px;
```

```
    margin-bottom: 20px;
```

```
  }
```

```
  .formGroup label {
```

```
    font-size: 14px;
```

```
  }
```

```
  .formGroup input,
```

```
  .formGroup textarea {
```

```
    padding: 10px;
```

```
    font-size: 13px;
```

```
  }
```

```
  .avatarPreview img {
```

```
    width: 100px;
```

```
    height: 100px;
```

```
  }
```



```
.cancelButton,  
  
.saveButton {  
    padding: 8px 16px;  
    font-size: 14px;  
}  
}  
  
.unmuteButton,  
  
.unbanButton {  
    background-color: var(--color-buttonColor);  
    color: black;  
    border: none;  
    padding: 10px 20px;  
    border-radius: 5px;  
    cursor: pointer;  
    font-size: 14px;  
    font-family: Arial, Helvetica, sans-serif;  
    transition: background-color 0.2s;  
    width: 180px;  
}  
  
.unmuteButton:hover,  
  
.unbanButton:hover {  
    background-color: #8B6B3A;
```

```
}
```

```
===== \components\common\Profile\ProfileHeader\ProfileHeader.tsx  
=====
```

```
import React, { useState, useEffect } from 'react';  
  
import { useNavigate } from 'react-router-dom';  
  
import api from '../../api/axios';  
  
import { uploadAvatar } from '../../api/auth';  
  
import ReportModal from '../ReportModal/ReportModal';  
  
import MuteBanModal from '../BanMute/MuteBanModal';  
  
import styles from './ProfileHeader.module.css';
```

```
interface ProfileHeaderProps {  
  
  isOwnProfile: boolean;  
  
  userId: number | null;  
  
}
```

```
interface UserData {  
  
  id: number;  
  
  login: string;  
  
  full_name: string;  
  
  bio: string;  
  
  avatar: string | null;  
  
  created_at: string;  
  
  last_seen: string;  
  
  friends_count: number;
```

```

status?: string;

is_muted?: boolean;

is_banned?: boolean;

mute_reason?: string;

ban_reason?: string;

mute_until?: string;

ban_until?: string;

mute_permanent?: boolean;

ban_permanent?: boolean;
}

```

```

const ProfileHeader: React.FC<ProfileHeaderProps> = ({ isOwnProfile, userId }) => {

  const [userData, setUserData] = useState<UserData | null>(null);

  const [isModalOpen, setIsModalOpen] = useState(false);

  const [fullName, setFullName] = useState("");

  const [bio, setBio] = useState("");

  const [avatarFile, setAvatarFile] = useState<File | null>(null);

  const [previewUrl, setPreviewUrl] = useState<string | null>(null);

  const [loading, setLoading] = useState(true);

  const [isReportModalOpen, setIsReportModalOpen] = useState(false);

  const [friendStatus, setFriendStatus] = useState<'none' | 'request_sent' | 'friend'>('none');

  const [isAdmin, setIsAdmin] = useState(false);

  const navigate = useNavigate();

  const [isMuteModalOpen, setIsMuteModalOpen] = useState(false);

  const [isBanModalOpen, setIsBanModalOpen] = useState(false);

```

```
const BASE_URL = 'http://localhost:8000';
```

```
const reportReasons = [  
  "Оскорбительное поведение",  
  "Спам",  
  "Другое"  
];
```

```
useEffect(() => {  
  loadUserData();  
  checkAdminStatus();  
  if (!isOwnProfile && userId) {  
    checkFriendStatus();  
  }  
}, [userId, isOwnProfile]);
```

```
const loadUserData = async () => {  
  try {  
    setLoading(true);  
    let response;  
  
    if (isOwnProfile) {  
      response = await api.get('/auth/profile/');  
    } else if (userId) {
```

```

    response = await api.get(`/auth/profile/${userId}/`);
  }

  if (response && response.data) {
    setData(response.data);
    setFullName(response.data.full_name || response.data.login);
    setBio(response.data.bio || "");
  }
} catch (error) {
  console.error('Ошибка загрузки:', error);
} finally {
  setLoading(false);
}
};

```

```

const checkAdminStatus = async () => {
  try {
    const profile = await api.get('/auth/profile/');
    setIsAdmin(profile.data.status === 'admin');
  } catch (error) {
    console.error('Ошибка проверки админа:', error);
  }
};

```

```

const checkFriendStatus = async () => {

```

```

try {
  const response = await api.get(`/friends/check/${userId!}/`);
  if (response.data.is_friend) {
    setFriendStatus('friend');
  } else if (response.data.is_friend_request_sent) {
    setFriendStatus('request_sent');
  } else {
    setFriendStatus('none');
  }
} catch (error) {
  console.error('Ошибка проверки статуса дружбы:', error);
}
};

```

```

const handleLogout = async () => {
  try {
    const refresh = localStorage.getItem('refresh_token');
    if (refresh) {
      await api.post('/auth/logout/', { refresh });
    }
  } catch (error) {
    console.error('Ошибка при выходе:', error);
  } finally {
    localStorage.clear();
    navigate('/login');
  }
}

```

```

    }
  };

const handleAddFriend = async () => {
  if (!userId) {
    alert('Ошибка: ID пользователя не найден');
    return;
  }

  try {
    await api.post(`/friends/send/${userId}/`);
    setFriendStatus('request_sent');
    alert('Запрос на добавление в друзья отправлен!');
  } catch (error: any) {
    console.error('Ошибка:', error);
    alert(`Ошибка: ${error.response?.data?.error || error.message || 'Неизвестная ошибка'}`);
  }
};

const handleRemoveFriend = async () => {
  if (!userId) {
    alert('Ошибка: ID пользователя не найден');
    return;
  }
}

```

```

try {
    await api.delete(`/friends/remove/${userId}/`);
    setFriendStatus('none');
    alert('Пользователь удалён из друзей');
} catch (error: any) {
    console.error('Ошибка:', error);
    alert(`Ошибка: ${error.response?.data?.error || error.message || 'Неизвестная
ошибка'}`);
}
};

```

```

const handleCancelRequest = async () => {

```

```

    if (!userId) {
        alert('Ошибка: ID пользователя не найден');
        return;
    }

```

```

try {
    await api.delete(`/friends/cancel-request/${userId}/`);
    setFriendStatus('none');
    alert('Запрос отменён');
} catch (error: any) {
    console.error('Ошибка:', error);
    alert(`Ошибка: ${error.response?.data?.error || error.message || 'Неизвестная
ошибка'}`);
}
};

```



```

ошибка'}`);

    }

};

```

```

const handleMuteUser = async (reason: string, duration: string, durationValue: number |
null, durationUnit: string | null) => {

    try {

        await api.post(`/admin/users/${userId}/mute/`, {

            reason,

            duration,

            duration_value: durationValue,

            duration_unit: durationUnit

        });

        alert(`Пользователь замучен ${duration === 'permanent' ? 'перманентно' : ''}`);

        setIsMuteModalOpen(false);

        await loadUserData();

    } catch (error: any) {

        console.error('Ошибка мута:', error);

        alert(`Ошибка: ${error.response?.data?.error || 'Не удалось замутить
пользователя'}`);

    }

};

```

```

const handleBanUser = async (reason: string, duration: string, durationValue: number |
null, durationUnit: string | null) => {

    if (window.confirm('Вы уверены, что хотите забанить этого пользователя?')) {

```

```

    try {
      await api.post(`/admin/users/${userId}/ban/`, {
        reason,
        duration,
        duration_value: durationValue,
        duration_unit: durationUnit
      });
      alert(`Пользователь забанен ${duration === 'permanent' ? 'перманентно' : ''}`);
    };

    setIsBanModalOpen(false);
    await loadUserData();
  } catch (error: any) {
    console.error('Ошибка бана:', error);
    alert(`Ошибка: ${error.response?.data?.error || 'Не удалось забанить пользователя'}`);
  }
}
};

```

```

const handleUnmuteUser = async () => {
  if (window.confirm('Снять мут с пользователя?')) {
    try {
      await api.post(`/admin/users/${userId}/unmute/`);
      alert('Мут снят');
      await loadUserData();
    }
  }
};

```

```

    } catch (error: any) {
        console.error('Ошибка снятия мута:', error);
        alert('Не удалось снять мут');
    }
}
};

```

```

const handleUnbanUser = async () => {
    if (window.confirm('Снять бан с пользователя?')) {
        try {
            await api.post(`/admin/users/${userId}/unban/`);
            alert('Бан снят');
            await loadUserData();
        } catch (error: any) {
            console.error('Ошибка снятия бана:', error);
            alert('Не удалось снять бан');
        }
    }
};

```

```

const handleAvatarChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0];
    if (file) {
        setAvatarFile(file);
        const url = URL.createObjectURL(file);
    }
};

```

```

        setPreviewUrl(url);
    }
};

const handleSave = async () => {
    try {
        let avatarUrl = userData?.avatar;

        if (avatarFile) {
            avatarUrl = await uploadAvatar(avatarFile);
            console.log('Аватар загружен:', avatarUrl);
        }

        const updateData: any = {};

        if (fullName !== (userData?.full_name || userData?.login)) {
            updateData.full_name = fullName;
        }

        if (bio !== userData?.bio) {
            updateData.bio = bio;
        }

        if (avatarUrl && avatarFile) {
            updateData.avatar = avatarUrl;
        }

        if (Object.keys(updateData).length > 0) {

```

```

        await api.patch('/auth/profile/', updateData);
    }

    await loadUserData();

    setIsModalOpen(false);

    if (previewUrl) {
        URL.revokeObjectURL(previewUrl);
        setPreviewUrl(null);
    }

    setAvatarFile(null);

    alert('Профиль успешно обновлен!');

} catch (error: any) {
    console.error('Ошибка обновления:', error);

    alert(`Ошибка при обновлении профиля: ${error.response?.data?.error ||
'Попробуйте другое фото'}`);
}

};

const handleCloseModal = () => {
    if (previewUrl) {
        URL.revokeObjectURL(previewUrl);
    }
}

```

```

        setPreviewUrl(null);
    }

    setAvatarFile(null);

    setIsModalOpen(false);
};

const openReportModal = () => {
    setIsReportModalOpen(true);
};

const closeReportModal = () => {
    setIsReportModalOpen(false);
};

const handleSubmitReport = async (reason: string, comment: string) => {
    const token = localStorage.getItem('access_token');

    if (!token) {
        const confirm = window.confirm('Вы не авторизованы. Хотите перейти на
страницу входа?');

        if (confirm) {
            navigate('/login');
        }

        return;
    }
};

```

```

try {
  await api.post('/reports/user/', {
    target_id: userId!,
    reason: reason,
    comment: comment
  });
  alert('Жалоба отправлена');
  closeReportModal();
} catch (error: any) {
  console.error('Ошибка отправки жалобы:', error);

  if (error.response?.status === 401) {
    const confirm = window.confirm('Сессия истекла. Хотите перейти на
    страницу входа?');
    if (confirm) {
      navigate('/login');
    }
  } else {
    alert(`Ошибка при отправке жалобы: ${error.response?.data?.error ||
    error.message}`);
  }
}
};

const getAvatarUrl = (avatar: string | null | undefined) => {

```

```

    if (!avatar) {

        return 'https://via.placeholder.com/150/4C4C4C/FFFFFF?text=Avatar';

    }

    if (avatar.startsWith('http://') || avatar.startsWith('https://')) {

        return avatar;

    }

    if (avatar.startsWith('data:image')) {

        return avatar;

    }

    return `${BASE_URL}${avatar}`;

};

```

```

const renderFriendButton = () => {

    if (friendStatus === 'friend') {

        return (

            <button

                className={styles.friendButton}

                onClick={handleRemoveFriend}

            >

                Удалить из друзей

            </button>

        );

    } else if (friendStatus === 'request_sent') {

        return (

            <button

```



```

        className={styles.friendButton}

        onClick={handleCancelRequest}

    >

        Отменить запрос

    </button>

    );
} else {
    return (
        <button

            className={styles.friendButton}

            onClick={handleAddFriend}

        >

            Добавить в друзья

        </button>

    );
}
};

if (loading) {
    return <div className={styles.loading}>Загрузка...</div>;
}

if (!userData) {
    return <div className={styles.error}>Ошибка загрузки профиля</div>;
}

```

```

const displayName = userData.full_name || userData.login;

const displayBio = userData.bio || 'Нет описания';

const avatarUrl = getAvatarUrl(userData.avatar);

const formattedDate = new Date(userData.created_at).toLocaleDateString('ru-RU');

const formattedLastSeen = new Date(userData.last_seen).toLocaleString('ru-RU');

return (
  <div className={styles.profileInfoBlockMain}>
    <div className={styles.profileImage}>
      <img
        alt="аватарка пользователя"
        src={avatarUrl}
        onError={(e) => {
          console.error('Ошибка загрузки аватара:', avatarUrl);
          e.currentTarget.src = 'https://via.placeholder.com/150/4C4C4C/FFFFFF?text=Avatar';
        }}
      />
    </div>
    <div className={styles.profileInfoBlockMainMain}>
      <div className={styles.profileName}><span>{displayName}</span></div>
      <div className={styles.profileBio}><span>{displayBio}</span></div>
      <div className={styles.profileStats}>

```

<p>Дата регистрации: {formattedDate}</p>

<p>Последний заход: {formattedLastSeen}</p>

<p>Друзей: {userData.friends_count}</p>

</div>

</div>

{isOwnProfile && (

<div className={styles.profileInfoBlockMainEnd}>

<button className={styles.logoutButton} onClick={handleLogout}>

Выйти

</button>

<button className={styles.editButton} onClick={() =>
setIsModalOpen(true)}>

Редактировать профиль

</button>

</div>

)}

{!isOwnProfile && (

<div className={styles.profileInfoBlockMainEnd}>

<button

className={styles.reportButton}

onClick={openReportModal}

>

!

```

</button>

{renderFriendButton()}

{isAdmin && (
  <div>
    {userData.is_muted ? (
      <button
        className={styles.unmuteButton}
        onClick={handleUnmuteUser}
      >
        СНЯТЬ МУТ
      </button>
    ) : (
      <button
        className={styles.muteButton}
        onClick={() => setIsMuteModalOpen(true)}
      >
        ЗАМУТИТЬ
      </button>
    )}
  </div>
  {userData.is_banned ? (
    <button
      className={styles.unbanButton}
      onClick={handleUnbanUser}
    >

```

```

        Снять бан

    </button>

    ) : (

    <button

        className={styles.banButton}

        onClick={() => setIsBanModalOpen(true)}

    >

        Забанить

    </button>

    )}

</>

)}

</div>

)}

</div>

```

```

    { /* Модальное окно редактирования профиля */ }

    { isOwnProfile && isModalOpen && (

        <div className={styles.modalOverlay} onClick={handleCloseModal}>

            <div className={styles.modalContent} onClick={(e) =>
e.stopPropagation()}>

                <h2 className={styles.modalTitle}>Редактировать профиль</h2>

                <div className={styles.avatarPreview}>

                    <img

```

```

src={previewUrl || avatarUrl}

alt="Аватарка"

onError={(e) => {

    e.currentTarget.src =
'https://via.placeholder.com/150/4C4C4C/FFFFFF?text=Avatar';

}}

/>

<input

    type="file"

    id="avatarInput"

    accept="image/*"

    className={styles.fileInput}

    onChange={handleAvatarChange}

/>

<button onClick={() =>
document.getElementById('avatarInput')?.click()}>

    Выбрать аватарку

</button>

</div>

<div className={styles.formGroup}>

    <label>Имя пользователя</label>

    <input

        type="text"

        value={fullName}

```

```
        onChange={(e) => setFullName(e.target.value)}  
        placeholder="Введите имя"  
        maxLength={50}  
      />  
    </div>
```

```
    <div className={styles.formGroup}>  
      <label>Описание (био)</label>  
      <textarea  
        value={bio}  
        onChange={(e) => setBio(e.target.value)}  
        placeholder="Расскажите о себе"  
        maxLength={200}  
        rows={3}  
      />  
    </div>
```

```
    <div className={styles.modalButtons}>  
      <button className={styles.cancelButton}  
onClick={handleCloseModal}>  
        Отмена  
      </button>  
      <button className={styles.saveButton} onClick={handleSave}>  
        Сохранить  
      </button>
```

```
        </div>

    </div>

</div>

}}
```

```
{/* Модальное окно жалобы */}

<ReportModal

    isOpen={isReportModalOpen}

    onClose={closeReportModal}

    onSubmit={handleSubmitReport}

    title="Пожаловаться на пользователя"

    reasons={reportReasons}

/>
```

```
{/* Модальные окна мута/бана */}

<MuteBanModal

    isOpen={isMuteModalOpen}

    onClose={() => setIsMuteModalOpen(false)}

    onSubmit={handleMuteUser}

    title="Замутить пользователя"

    actionType="mute"

/>
```

```
<MuteBanModal

    isOpen={isBanModalOpen}

    onClose={() => setIsBanModalOpen(false)}
```



```

        onSubmit={handleBanUser}

        title="Забанить пользователя"

        actionType="ban"

    />

</>

);

};

export default ProfileHeader;

===== \components\common\Profile\ProfileHeader\ProfileTabs.module.css
=====

@import "../../../style/variables.css";

.profileInfoBlockBottom {
    display: flex;
    flex-direction: row;
    justify-content: center;
}

.profileInfoBlockBottom button {
    background-color: var(--color-blockColor);
    color: var(--color-text);
    font-size: 20px;
    margin: 1.5%;
    border: none;

```

```

    cursor: pointer;

    padding: 10px 20px;

    transition: all 0.3s ease;

    font-family: Arial, Helvetica, sans-serif;

    outline: none;

    box-shadow: none;
}

.profileInfoBlockBottom button:focus,
.profileInfoBlockBottom button:active,
.profileInfoBlockBottom button:focus-visible {

    outline: none;

    box-shadow: none;

    border: none;
}

.profileInfoBlockBottom button.active {

    color: #7E7E7E;

    font-weight: bold;
}

/* ПЛАНШЕТ */

@media (max-width: 1024px) {

    .profileInfoBlockBottom button {

        font-size: 20px;
    }
}

```

```

        margin: 1%;

        padding: 10px 18px;
    }
}

/* мобильные */

@media (max-width: 768px) {
    .profileInfoBlockBottom {
        flex-wrap: wrap;

        gap: 12px;

        padding: 12px;
    }

    .profileInfoBlockBottom button {
        font-size: 18px;

        margin: 0;

        padding: 10px 20px;

        flex: 1;

        min-width: 110px;
    }
}

/* маленькие телефоны */

@media (max-width: 480px) {
    .profileInfoBlockBottom button {

```

```

    font-size: 16px;

    padding: 8px 14px;

    min-width: 90px;

  }
}

```

```

/* очень маленькие телефоны */

```

```

@media (max-width: 360px) {

  .profileInfoBlockBottom button {

    font-size: 15px;

    padding: 6px 12px;

    min-width: 80px;

  }

}

```

```

===== \components\common\Profile\ProfileHeader\ProfileTabs.tsx =====

```

```

import React from 'react';

import styles from './ProfileTabs.module.css';

```

```

interface ProfileTabsProps {

  activeTab: string;

  setActiveTab: (tab: string) => void;

}

```

```

const ProfileTabs: React.FC<ProfileTabsProps> = ({ activeTab, setActiveTab }) => {

  const tabs = [

```

```

    { id: 'statistics', label: 'Статистика' },
    { id: 'tests', label: 'Мои тесты' },
    { id: 'friends', label: 'Друзья' }
  ];

  return (
    <div className={styles.profileInfoBlockBottom}>
      {tabs.map(tab => (
        <button
          key={tab.id}
          className={activeTab === tab.id ? styles.active : ""}
          onClick={() => setActiveTab(tab.id)}
        >
          {tab.label}
        </button>
      ))}
    </div>
  );
};

export default ProfileTabs;

===== \components\common\Profile\StatsBlock\FriendsTab.module.css
=====

@import "../../../style/variables.css";

```

```
.friendsContent {  
    padding: 20px;  
    width: 100%;  
    height: 100%;  
    overflow: auto;  
    display: flex;  
    flex-direction: column;  
    font-family: Arial, Helvetica, sans-serif;  
    color: black;  
}
```

```
/* фиксированный заголовок */
```

```
.stickyHeader {  
    position: sticky;  
    top: 0;  
    background-color: var(--color-blockColor, #D8C3A5);  
    z-index: 10;  
    padding-bottom: 10px;  
    margin-bottom: 15px;  
}
```

```
/* табы */
```

```
.tabsHeader {  
    display: flex;  
    gap: 20px;
```

```
border-bottom: 1px solid #e0e0e0;  
padding-bottom: 10px;  
margin-bottom: 20px;  
}
```

```
.tabButton {  
    background: none;  
    border: none;  
    padding: 8px 0;  
    font-size: 16px;  
    font-weight: 500;  
    cursor: pointer;  
    color: #000000;  
    transition: color 0.3s;  
    outline: none;  
    text-decoration: none !important;  
}
```

```
.tabButton:focus,  
.tabButton:active,  
.tabButton:focus-visible {  
    outline: none;  
    box-shadow: none;  
}
```

```
.tabButton:hover {  
    color: #000000;  
}
```

```
.tabButton.activeTab {  
    color: #000000;  
}
```

```
/* список друзей */
```

```
.friendsList {  
    display: flex;  
    flex-direction: column;  
    gap: 10px;  
    max-height: 330px;  
    overflow-y: auto;  
    padding-right: 10px;  
    flex: 1;  
}
```

```
/* скроллбар */
```

```
.friendsList::-webkit-scrollbar {  
    width: 8px;  
    height: 8px;  
}
```



```
.friendsList::-webkit-scrollbar-track {  
    background: rgba(0, 0, 0, 0.1);  
    border-radius: 10px;  
}
```

```
.friendsList::-webkit-scrollbar-thumb {  
    background: rgba(0, 0, 0, 0.3);  
    border-radius: 10px;  
}
```

```
.friendsList::-webkit-scrollbar-thumb:hover {  
    background: rgba(0, 0, 0, 0.5);  
}
```

```
/* карточка друга */
```

```
.friendItem {  
    display: flex;  
    align-items: center;  
    gap: 15px;  
    padding: 12px;  
    background-color: var(--color-buttonColor);  
    border-radius: 10px;  
    transition: background-color 0.2s;  
    width: 100%;  
}
```

```
.friendItem:hover {  
    background-color: #c4985b;  
}
```

```
/* карточка заявки в друзья */
```

```
.friendRequestItem {  
    display: flex;  
    align-items: center;  
    gap: 15px;  
    padding: 12px;  
    background-color: var(--color-buttonColor);  
    border-radius: 10px;  
    margin-bottom: 10px;  
    width: 100%;  
    transition: background-color 0.2s;  
}
```

```
.friendRequestItem:hover {  
    background-color: #c4985b;  
}
```

```
/* ссылка на профиль друга */
```

```
.friendLink {  
    display: flex;
```

```
    align-items: center;

    gap: 15px;

    flex: 1;

    text-decoration: none;

    color: inherit;
}
```

```
.friendLink:hover {

    text-decoration: none;
}
```

```
/* aBarap */

.friendAvatar {

    width: 48px;

    height: 48px;

    border-radius: 50%;

    overflow: hidden;

    flex-shrink: 0;

    background-color: #4C4C4C;
}
```

```
.avatarImage {

    width: 100%;

    height: 100%;

    object-fit: cover;
```

```
    display: block;
}

/* информация о друге */
.friendInfo {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 4px;
}

.friendName {
    font-size: 16px;
    font-weight: 600;
    color: black;
    text-decoration: none;
}

.friendLink:hover .friendName {
    text-decoration: underline;
    color: black;
}

.friendLastSeen {
    font-size: 12px;
```

```
    color: #000000;
}
```

```
.requestDate {
    font-size: 12px;
    color: #000000;
}
```

/ кнопка удаления */*

```
.removeFriendButton {
    background-color: transparent;
    color: black;
    border: none;
    border-radius: 6px;
    padding: 6px 16px;
    font-size: 13px;
    cursor: pointer;
    transition: text-decoration 0.2s;
    flex-shrink: 0;
}
```

```
.removeFriendButton:focus,
.removeFriendButton:active,
.removeFriendButton:focus-visible {
    outline: none;
```

```
    box-shadow: none;
}

.removeFriendButton:hover {
    text-decoration: underline;
    background-color: transparent;
}

/* кнопки принятия/отклонения */
.requestButtons {
    display: flex;
    gap: 10px;
    flex-shrink: 0;
}

.acceptButton {
    background-color: transparent;
    color: black;
    border: none;
    border-radius: 6px;
    padding: 6px 16px;
    font-size: 13px;
    cursor: pointer;
    transition: text-decoration 0.2s;
}
```

```
.acceptButton:focus,  
.acceptButton:active,  
.acceptButton:focus-visible {  
    outline: none;  
    box-shadow: none;  
}  
  
.acceptButton:hover {  
    text-decoration: underline;  
    background-color: transparent;  
}  
  
.rejectButton {  
    background-color: transparent;  
    color: black;  
    border: none;  
    border-radius: 6px;  
    padding: 6px 16px;  
    font-size: 13px;  
    cursor: pointer;  
    transition: text-decoration 0.2s;  
}  
  
.rejectButton:focus,
```

```
.rejectButton:active,
```

```
.rejectButton:focus-visible {
```

```
    outline: none;
```

```
    box-shadow: none;
```

```
}
```

```
.rejectButton:hover {
```

```
    text-decoration: underline;
```

```
    background-color: transparent;
```

```
}
```

```
/* пустое состояние - НЕТ ДРУЗЕЙ / НЕТ ЗАЯВОК */
```

```
.empty {
```

```
    text-align: center;
```

```
    padding: 60px 40px;
```

```
    color: #000000;
```

```
    font-family: Arial, Helvetica, sans-serif;
```

```
    font-size: 18px;
```

```
    font-weight: bold;
```

```
}
```

```
.loading {
```

```
    text-align: center;
```

```
    padding: 40px;
```

```
    color: #888;
```



```
}
```

```
.error {
```

```
    text-align: center;
```

```
    padding: 40px;
```

```
    color: #dc3545;
```

```
}
```

```
/* ПЛАНШЕТ */
```

```
@media (max-width: 1024px) {
```

```
    .friendsContent {
```

```
        padding: 15px;
```

```
    }
```

```
    .friendItem, .friendRequestItem {
```

```
        padding: 12px;
```

```
    }
```

```
    .friendAvatar {
```

```
        width: 50px;
```

```
        height: 50px;
```

```
    }
```

```
    .friendName {
```

```
        font-size: 17px;
```

```

}

.friendLastSeen, .requestDate {
    font-size: 13px;
}

.empty {
    padding: 50px 30px;
    font-size: 17px;
}
}

/* мобильные */

@media (max-width: 768px) {
    .friendsContent {
        padding: 15px;
    }

    .friendsList {
        max-height: 400px;
        gap: 10px;
    }

    .friendItem, .friendRequestItem {
        padding: 12px;
    }
}

```

```

}

.friendAvatar {
    width: 55px;
    height: 55px;
}

.friendName {
    font-size: 16px;
}

.friendLastSeen, .requestDate {
    font-size: 13px;
}

.empty {
    padding: 45px 25px;
    font-size: 16px;
}
}

/* маленькие телефоны */
@media (max-width: 480px) {
    .friendsContent {
        padding: 12px;
    }
}

```

```
}
```

```
.friendAvatar {  
  width: 50px;  
  height: 50px;  
  margin-right: 12px;  
}
```

```
.friendName {  
  font-size: 15px;  
}
```

```
.friendLastSeen, .requestDate {  
  font-size: 12px;  
}
```

```
.friendsList {  
  max-height: 350px;  
  gap: 8px;  
}
```

```
.acceptButton,  
.rejectButton,  
.removeFriendButton {  
  padding: 4px 12px;
```

```

        font-size: 12px;
    }

    .empty {
        padding: 35px 20px;
        font-size: 15px;
    }
}

/* очень маленькие телефоны */
@media (max-width: 360px) {
    .friendAvatar {
        width: 45px;
        height: 45px;
    }

    .friendName {
        font-size: 14px;
    }

    .friendLastSeen, .requestDate {
        font-size: 11px;
    }

    .empty {

```

```
padding: 30px 15px;

font-size: 14px;

}

}
```

```
/* альбомная ориентация */
```

```
@media (max-width: 768px) and (orientation: landscape) {

.friendsList {

    max-height: 250px;

}
```

```
.friendAvatar {

    width: 45px;

    height: 45px;

}

}
```

```
===== \components\common\Profile\StatsBlock\FriendsTab.tsx =====
```

```
import React, { useState, useEffect } from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
import api from '../.../api/axios';
```

```
import styles from './FriendsTab.module.css';
```

```
interface Friend {
```

```
    id: number;
```

```
    login: string;
```

```

    full_name: string;
    avatar: string | null;
    last_seen: string;
  }

```

```

interface FriendRequest {
  id: number;
  from_user: {
    id: number;
    login: string;
    full_name: string;
    avatar: string | null;
  };
  created_at: string;
}

```

```

interface FriendsTabProps {
  isOwnProfile?: boolean;
  userId?: number | null;
}

```

```

const FriendsTab: React.FC<FriendsTabProps> = ({ isOwnProfile = true, userId = null })
=> {
  const [friends, setFriends] = useState<Friend[]>([]);
  const [friendRequests, setFriendRequests] = useState<FriendRequest[]>([]);

```

```

const [loading, setLoading] = useState<boolean>(true);

const [activeTab, setActiveTab] = useState<'friends' | 'requests'>('friends');

const [error, setError] = useState<string | null>(null);


const BASE_URL = 'http://localhost:8000';


useEffect(() => {

  console.log('=== FriendsTab useEffect START ===');

  console.log('isOwnProfile:', isOwnProfile);

  console.log('userId:', userId);


  if (isOwnProfile) {

    console.log('Загружаем своих друзей');

    loadMyFriends();

    loadFriendRequests();

  } else if (userId) {

    console.log('Загружаем друзей пользователя ID:', userId);

    loadUserFriends(userId);

  } else {

    console.error('Нет userId и не свой профиль!');

    setLoading(false);

  }

}, [isOwnProfile, userId]);


const loadMyFriends = async () => {

```



```

try {
  console.log('Запрос GET /friends/');
  const response = await api.get('/friends/');
  console.log('Ответ:', response.data);
  setFriends(response.data);
} catch (err) {
  console.error('Ошибка:', err);
  setError('Не удалось загрузить друзей');
} finally {
  setLoading(false);
}
};

```

```

const loadUserFriends = async (targetUserId: number) => {
  try {
    console.log(`Запрос GET /friends/${targetUserId}/`);
    const response = await api.get(`/friends/${targetUserId}/`);
    console.log('Ответ:', response.data);
    setFriends(response.data);
  } catch (err: any) {
    console.error('Ошибка:', err);
    console.error('Статус:', err.response?.status);
    console.error('Данные:', err.response?.data);

    setError(`Не удалось загрузить друзей: ${err.response?.data?.error ||
err.message}`);
  }
};

```

```

    } finally {
        setLoading(false);
    }
};

const loadFriendRequests = async () => {
    try {
        const response = await api.get('/friends/requests/');
        setFriendRequests(response.data);
    } catch (err) {
        console.error('Ошибка загрузки заявок:', err);
    }
};

const getAvatarUrl = (avatar: string | null) => {
    if (!avatar) return 'https://via.placeholder.com/48/4C4C4C/FFFFFF?text=User';
    if (avatar.startsWith('http') || avatar.startsWith('data:image')) {
        return avatar;
    }
    return `${BASE_URL}${avatar}`;
};

// ===== ИСПРАВЛЕННЫЕ ФУНКЦИИ =====

const handleRemoveFriend = async (friendId: number) => {

```

```

try {
  console.log('Удаляем друга с ID:', friendId);

  const response = await api.delete(`/friends/remove/${friendId}/`);

  console.log('Ответ сервера:', response.data);

  setFriends(prevFriends => prevFriends.filter(f => f.id !== friendId));

  console.log('Друг успешно удален');
} catch (err: any) {
  console.error('Ошибка при удалении друга:', err);

  let errorMessage = 'Не удалось удалить друга';

  if (err.response?.data?.error) {
    errorMessage = err.response.data.error;
  } else if (err.response?.data?.detail) {
    errorMessage = err.response.data.detail;
  }

  setError(errorMessage);

  setTimeout(() => setError(null), 3000);
}
};

```

```

const handleAcceptRequest = async (requestId: number) => {
  try {
    console.log('Принимаем заявку с ID:', requestId);

    const response = await api.post(`/friends/accept/${requestId}/`);

    console.log('Ответ сервера:', response.data);

```

```

    setFriendRequests(prevRequests => prevRequests.filter(r => r.id !== requestId));

    await loadMyFriends();

    console.log('Заявка принята');
  } catch (err: any) {

    console.error('Ошибка при принятии заявки:', err);

    let errorMessage = 'Не удалось принять заявку';

    if (err.response?.data?.error) {

      errorMessage = err.response.data.error;

    } else if (err.response?.data?.detail) {

      errorMessage = err.response.data.detail;

    }

    setError(errorMessage);

    setTimeout(() => setError(null), 3000);

  }
};

```

```

const handleRejectRequest = async (requestId: number) => {

  try {

    console.log('Отклоняем заявку с ID:', requestId);

    const response = await api.delete(`/friends/reject/${requestId}/`);

    console.log('Ответ сервера:', response.data);

    setFriendRequests(prevRequests => prevRequests.filter(r => r.id !== requestId));

    console.log('Заявка отклонена');
  }
};

```

```

    } catch (err: any) {

        console.error('Ошибка при отклонении заявки:', err);

        let errorMessage = 'Не удалось отклонить заявку';

        if (err.response?.data?.error) {

            errorMessage = err.response.data.error;

        } else if (err.response?.data?.detail) {

            errorMessage = err.response.data.detail;

        }

        setError(errorMessage);

        setTimeout(() => setError(null), 3000);

    }

};

if (loading) {

    return <div className={styles.loading}>Загрузка...</div>;

}

if (error) {

    return <div className={styles.error}>Ошибка: {error}</div>;

}

return (

    <div className={styles.friendsContent}>

        <div className={styles.stickyHeader}>

            <div className={styles.tabsHeader}>

```

```

    <button
      className={` ${styles.tabButton} ${activeTab === 'friends' ?
styles.activeTab : ''}
      onClick={() => setActiveTab('friends')}
    >
      Друзья ({friends.length})
    </button>

    {isOwnProfile && (
      <button
        className={` ${styles.tabButton} ${activeTab === 'requests' ?
styles.activeTab : ''}
        onClick={() => setActiveTab('requests')}
      >
        Заявки ({friendRequests.length})
      </button>
    )}
  </div>
</div>

```

```

{activeTab === 'friends' && (
  <div className={styles.friendsList}>
    {friends.length === 0 ? (
      <div className={styles.empty}>Нет друзей</div>
    ) : (
      friends.map((friend) => (

```

```

<div className={styles.friendItem} key={friend.id}>
  <Link to={` /profile/${friend.id}`} className={styles.friendLink}>
    <div className={styles.friendAvatar}>
      <img
        src={getAvatarUrl(friend.avatar)}
        alt={friend.full_name || friend.login}
        className={styles.avatarImage}
        onError={(e) => {
          (e.target as HTMLImageElement).src =
'https://via.placeholder.com/48/4C4C4C/FFFFFF?text=User';
        }}
      />
    </div>
    <div className={styles.friendInfo}>
      <span className={styles.friendName}>{friend.full_name ||
friend.login}</span>
      <span className={styles.friendLastSeen}>
        Последний заход: {new
Date(friend.last_seen).toLocaleString('ru-RU')}
      </span>
    </div>
  </Link>
  {isOwnProfile && (
    <button
      className={styles.removeFriendButton}
      onClick={() => handleRemoveFriend(friend.id)}

```

```

        >
        Удалить
    </button>
  })
</div>
))
}}
</div>
}}
```

```

{isOwnProfile && activeTab === 'requests' && (
  <div className={styles.friendsList}>
    {friendRequests.length === 0 ? (
      <div className={styles.empty}>Нет входящих заявок</div>
    ) : (
      friendRequests.map((request) => (
        <div className={styles.friendRequestItem} key={request.id}>
          <Link to={` /profile/${request.from_user.id}`}
className={styles.friendLink}>
            <div className={styles.friendAvatar}>
              <img
                src={getAvatarUrl(request.from_user.avatar)}
                alt={request.from_user.full_name || request.from_user.login}
                className={styles.avatarImage}
                onError={(e) => {
```



```

        (e.target as HTMLImageElement).src =
'https://via.placeholder.com/48/4C4C4C/FFFFFF?text=User';

    }}

/>

</div>

<div className={styles.friendInfo}>

    <span className={styles.friendName}>

        {request.from_user.full_name || request.from_user.login}

    </span>

    <span className={styles.requestDate}>

        Заявка от: {new
Date(request.created_at).toLocaleDateString('ru-RU')}

    </span>

</div>

</Link>

<div className={styles.requestButtons}>

    <button

        className={styles.acceptButton}

        onClick={() => handleAcceptRequest(request.id)}

    >

        Принять

    </button>

    <button

        className={styles.rejectButton}

        onClick={() => handleRejectRequest(request.id)}

```

```

        >
        Отклонить
    </button>
</div>
</div>
))
})
</div>
})
</div>
);
};

```

```
export default FriendsTab;
```

```

===== \components\common\Profile\StatsBlock\MyTestsTab.module.css
=====

```

```
@import "../../../style/variables.css";
```

```

.testsContent {
    padding: 20px;
    width: 100%;
    height: 100%;
    overflow: auto;
    display: flex;
    flex-direction: column;

```

```
font-family: Arial, Helvetica, sans-serif;

color: var(--color-text);

}


/* фиксированный заголовок */

.stickyHeader {

    position: sticky;

    top: 0;

    background-color: var(--color-blockColor);

    z-index: 10;

    padding-bottom: 10px;

    margin-bottom: 15px;

}


.stickyHeader h3 {

    color: var(--color-text);

    font-family: Arial, Helvetica, sans-serif;

    margin: 0;

    font-size: 24px;

    font-weight: bold;

}


/* обёртка для таблицы */

.tableWrapper {

    overflow-x: auto;
```

```
width: 100%;

flex: 1;

}

/* скроллбар */

.tableWrapper::-webkit-scrollbar {

width: 8px;

height: 8px;

}

.tableWrapper::-webkit-scrollbar-track {

background: var(--color-scrollbar-track);

border-radius: 10px;

}

.tableWrapper::-webkit-scrollbar-thumb {

background: var(--color-scrollbar-thumb);

border-radius: 10px;

}

.tableWrapper::-webkit-scrollbar-thumb:hover {

background: var(--color-scrollbar-thumb-hover);

}

/* таблица */
```

```

.testsTable {
    width: 100%;

    border-collapse: collapse;

    font-size: 14px;

    font-family: Arial, Helvetica, sans-serif;

    color: var(--color-text);
}

/* шапка таблицы */
.testsTable th {
    background-color: var(--color-buttonColor);

    padding: 12px;

    text-align: left;

    font-weight: bold;

    position: sticky;

    top: 0;

    border: none;

    font-family: Arial, Helvetica, sans-serif;

    color: var(--color-text);
}

/* ячейки */
.testsTable td {
    padding: 10px 12px;

    border-bottom: 1px solid var(--color-border);
}

```

```
font-family: Arial, Helvetica, sans-serif;

color: var(--color-text);

}
```

```
/* hover на строку - убираем */

.testsTable tr:hover {

    background-color: transparent;

}
```

```
/* рейтинг */

.rating {

    color: var(--color-text);

    font-weight: bold;

}
```

```
/* публичный / приватный */

.public {

    color: var(--color-text);

    font-weight: bold;

}
```

```
.private {

    color: var(--color-text);

    font-weight: bold;

}
```

```
/* информация о тесте */

.testInfo {

    display: flex;

    flex-direction: column;

}


.testDescription {

    font-size: 12px;

    color: var(--color-text-muted);

    margin-top: 4px;

}


/* ССЫЛКИ */

.testLink a {

    text-decoration: none !important;

    font-weight: 700 !important;

    color: var(--color-text) !important;

    transition: opacity 0.2s ease;

}


.testLink a:hover {

    opacity: 0.7;

    text-decoration: none !important;

}
```

```
/* загрузка, ошибка */
```

```
.loading,
```

```
.error {
```

```
    text-align: center;
```

```
    padding: 40px;
```

```
    font-family: Arial, Helvetica, sans-serif;
```

```
    font-size: 14px;
```

```
}
```

```
.loading {
```

```
    color: var(--color-text-muted);
```

```
}
```

```
.error {
```

```
    color: var(--color-danger);
```

```
}
```

```
/* пустое состояние - НЕТ СОЗДАННЫХ ТЕСТОВ */
```

```
.empty {
```

```
    text-align: center;
```

```
    padding: 60px 40px;
```

```
    font-family: Arial, Helvetica, sans-serif;
```

```
    font-size: 18px;
```

```
    font-weight: bold;
```



```
    color: var(--color-text);
}

/* планшет */

@media (max-width: 1024px) {

    .testsContent {

        padding: 15px;

    }

    .stickyHeader h3 {

        font-size: 22px;

    }

    .testsTable th,

    .testsTable td {

        padding: 12px 15px;

        font-size: 15px;

    }

    .testDescription {

        font-size: 14px;

        margin-top: 6px;

    }

    .testInfo strong {
```

```
        font-size: 16px;
    }

.empty {
    padding: 50px 30px;
    font-size: 17px;
}

}

/* МОБИЛЬНЫЕ */

@media (max-width: 768px) {

    .testsContent {
        padding: 15px;
    }

    .stickyHeader h3 {
        font-size: 20px;
    }

    .testsTable {
        min-width: 700px;
        font-size: 14px;
    }

    .testsTable th,
```

```
.testsTable td {  
    padding: 12px 15px;  
    font-size: 14px;  
}  
  
.testInfo strong {  
    font-size: 16px;  
}  
  
.testDescription {  
    font-size: 14px;  
    margin-top: 6px;  
    max-width: 250px;  
}  
  
.rating {  
    font-size: 14px;  
}  
  
.empty {  
    padding: 45px 25px;  
    font-size: 16px;  
}  
}
```

```
/* маленькие телефоны */
```

```
@media (max-width: 480px) {
```

```
  .testsContent {
```

```
    padding: 12px;
```

```
}
```

```
  .stickyHeader h3 {
```

```
    font-size: 18px;
```

```
}
```

```
  .testsTable {
```

```
    min-width: 650px;
```

```
    font-size: 13px;
```

```
}
```

```
  .testsTable th,
```

```
  .testsTable td {
```

```
    padding: 10px 12px;
```

```
    font-size: 13px;
```

```
}
```

```
  .testInfo strong {
```

```
    font-size: 15px;
```

```
}
```

```
.testDescription {  
    font-size: 13px;  
    margin-top: 5px;  
    max-width: 220px;  
}  
  
.loading,  
.error {  
    padding: 30px;  
    font-size: 13px;  
}  
  
.empty {  
    padding: 35px 20px;  
    font-size: 15px;  
}  
}  
  
/* очень маленькие телефоны */  
@media (max-width: 360px) {  
    .testsTable {  
        min-width: 580px;  
        font-size: 12px;  
    }  
}
```

```

.testsTable th,

.testsTable td {

    padding: 8px 10px;

    font-size: 12px;

}


.testInfo strong {

    font-size: 14px;

}


.testDescription {

    font-size: 12px;

    max-width: 200px;

}


.empty {

    padding: 30px 15px;

    font-size: 14px;

}

}


/* альбомная ориентация */

@media (max-width: 768px) and (orientation: landscape) {

    .testDescription {

        max-width: 300px;
    }
}

```

```

        font-size: 13px;

    }

}

/* кнопка создать первый тест */

.createButton {

    display: inline-block;

    margin-top: 15px;

    padding: 12px 24px;

    background-color: var(--color-buttonColor);

    color: var(--color-text);

    text-decoration: none;

    border-radius: 8px;

    font-size: 16px;

    font-weight: bold;

    font-family: Arial, Helvetica, sans-serif;

    transition: opacity 0.2s;

    border: none;

    cursor: pointer;

}

.createButton:focus,

.createButton:active,

.createButton:focus-visible {

    outline: none;

```

```

border: none;

box-shadow: none;

}

```

```

.createButton: hover {

  opacity: 0.7;

  text-decoration: none;

}

```

```

===== \components\common\Profile\StatsBlock\MyTestsTab.tsx =====

```

```

import React, { useState, useEffect } from 'react';

import { Link } from 'react-router-dom';

import api from '../.../api/axios';

import styles from './MyTestsTab.module.css';

```

```

interface MyTestsTabProps {

  userId?: number | null;

  isOwnProfile?: boolean;

}

```

```

interface Test {

  id: number;

  title: string;

  description?: string;

  is_survey?: boolean;

  questions_count: number;

```



```

    attempts_count: number;

    average_rating: number | null;

    is_open: boolean;

    access_token?: string;
  }

```

```

type SortField = 'title' | 'type' | 'questions_count' | 'attempts_count' | 'average_rating' |
'is_open' | null;

```

```

type SortOrder = 'asc' | 'desc' | null;

```

```

const MyTestsTab: React.FC<MyTestsTabProps> = ({ userId, isOwnProfile = true }) =>
{

```

```

    const [tests, setTests] = useState<Test[]>([]);

```

```

    const [loading, setLoading] = useState(true);

```

```

    const [error, setError] = useState<string | null>(null);

```

```

    const [sortField, setSortField] = useState<SortField>(null);

```

```

    const [sortOrder, setSortOrder] = useState<SortOrder>(null);

```

```

    useEffect(() => {

```

```

        loadCreatedTests();

```

```

    }, [userId, isOwnProfile]);

```

```

const loadCreatedTests = async () => {

```

```

    try {

```

```

        setLoading(true);

```

```
setError(null);
```

```
let targetUserId = userId;
```

```
if (isOwnProfile && !targetUserId) {
```

```
  try {
```

```
    const profile = await api.get('/auth/profile/');
```

```
    targetUserId = profile.data.id;
```

```
  } catch (err) {
```

```
    console.error('Ошибка получения профиля:', err);
```

```
    setError('Не удалось получить данные пользователя');
```

```
    return;
```

```
  }
```

```
}
```

```
if (!targetUserId) {
```

```
  setError('ID пользователя не найден');
```

```
  return;
```

```
}
```

```
// ВАЖНО: для своего профиля используем my-tests, для чужого - created-  
tests
```

```
let url;
```

```
if (isOwnProfile) {
```

```
  url = `/profile/${targetUserId}/my-tests/`;
```

```

    } else {

        url = `/profile/${targetUserId}/created-tests/`;

    }

    const response = await api.get(url);

    console.log("Загружены тесты:", response.data);

    setTests(response.data);

} catch (err: any) {

    console.error('Ошибка загрузки тестов:', err);

    const errorMessage = err.response?.data?.detail || err.message || 'Не удалось
загрузить созданные тесты';

    setError(errorMessage);

} finally {

    setLoading(false);

}

};

const handleSort = (field: SortField) => {

    console.log('Сортировка по полю:', field);

    if (sortField === field) {

        if (sortOrder === 'desc') {

            setSortField(null);

            setSortOrder(null);

        } else {

            setSortOrder('desc');

```

```

    }
  } else {
    setSortField(field);
    setSortOrder('asc');
  }
};

const getSortedTests = () => {
  if (!sortField || !sortOrder) {
    return tests;
  }

  const sorted = [...tests];

  sorted.sort((a, b) => {
    let aValue: any;
    let bValue: any;

    switch (sortField) {
      case 'title':
        aValue = a.title.toLowerCase();
        bValue = b.title.toLowerCase();
        break;
      case 'type':
        aValue = a.is_survey ? 'опрос' : 'тест';

```

```

        bValue = b.is_survey ? 'опрос' : 'тест';

        break;

    case 'questions_count':

        aValue = a.questions_count || 0;

        bValue = b.questions_count || 0;

        break;

    case 'attempts_count':

        aValue = a.attempts_count || 0;

        bValue = b.attempts_count || 0;

        break;

    case 'average_rating':

        aValue = a.average_rating || 0;

        bValue = b.average_rating || 0;

        break;

    case 'is_open':

        aValue = a.is_open;

        bValue = b.is_open;

        break;

    default:

        return 0;

}

if (aValue < bValue) return sortOrder === 'asc' ? -1 : 1;

if (aValue > bValue) return sortOrder === 'asc' ? 1 : -1;

return 0;

```

```

    });

    return sorted;
  };

  // Генерация ссылки в зависимости от того, свой это профиль или чужой
  const getTestLink = (test: Test) => {
    // Если это свои тесты (владелец профиля) - ссылка на статистику
    if (isOwnProfile) {
      return `/test/${test.id}/stats`;
    }

    // Если это чужие тесты - ссылка на прохождение
    if (!test.is_open && test.access_token) {
      return `/test/${test.id}?token=${test.access_token}`;
    }

    return `/test/${test.id}`;
  };

  const getRating = (rating: number | null) => {
    if (rating === null || rating === undefined) return '0';

    return rating.toFixed(1);
  };

  const sortedTests = getSortedTests();

```

```

if (loading) {

    return <div className={styles.loading}>Загрузка тестов...</div>;

}

if (error) {

    return (

        <div className={styles.error}>

            <p>Ошибка: {error}</p>

            <button onClick={loadCreatedTests} className={styles.retryButton}>

                Повторить попытку

            </button>

        </div>

    );

}

if (tests.length === 0) {

    return (

        <div className={styles.empty}>

            <p>Нет созданных тестов</p>

            {isOwnProfile && (

                <Link to="/create" className={styles.createButton}>

                    Создать первый тест

                </Link>

            )}

        </div>

```

```

    );
}

return (
  <div className={styles.testsContent}>
    <div className={styles.stickyHeader}>
      <h3>Мои тесты</h3>
    </div>
    <div className={styles.tableWrapper}>
      <table className={styles.testsTable}>
        <thead>
          <tr>
            <th onClick={() => handleSort('title')} className={styles.sortable}>
              Название теста
            </th>
            <th onClick={() => handleSort('type')} className={styles.sortable}>
              Тип
            </th>
            <th onClick={() => handleSort('questions_count')}
className={styles.sortable}>
              Вопросов
            </th>
            <th onClick={() => handleSort('attempts_count')}
className={styles.sortable}>
              Прошло человек

```



```

        </th>

        <th onClick={() => handleSort('average_rating')}
className={styles.sortable}>

            Рейтинг

        </th>

        <th onClick={() => handleSort('is_open')} className={styles.sortable}
>

            Доступ

        </th>

    </tr>

</thead>

<tbody>

    {sortedTests.map((test) => (

        <tr key={test.id}>

            <td className={styles.testLink}>

                <Link to={getTestLink(test)}>

                    <div className={styles.testInfo}>

                        <strong>{test.title}</strong>

                        {test.description && (

                            <span className={styles.testDescription}
>{test.description}</span>

                            )}

                        </div>

                    </Link>

                </td>

                <td>{test.is_survey ? 'опрос' : 'тест'}</td>

```

```

        <td>{test.questions_count || 0}</td>

        <td>{test.attempts_count || 0}</td>

        <td className={styles.rating}>★
{getRating(test.average_rating)}</td>

        <td className={test.is_open ? styles.public : styles.private}>
            {test.is_open ? 'Публичный' : 'Приватный'}
        </td>

    </tr>

    )})
</tbody>

</table>

</div>

</div>

);

};

```

```
export default MyTestsTab;
```

```

===== \components\common\Profile\StatsBlock\ProfileStatsBlock.module.css
=====

```

```

.bottomProfile {
    width: 100%;

    height: 50%;

    display: flex;

    align-items: center;

    justify-content: center;

```

```
padding: 0 0 2% 0;
}

.profileStatsBlock {
    width: 90%;
    height: 440px;
    display: flex;
    flex-direction: column;
    background-color: var(--color-blockColor);
    border-radius: 10px;
    overflow: auto;
    padding: 0;
    font-family: Arial, Helvetica, sans-serif;
    color: black;
}
```

```
/* ПЛАНШЕТ */
```

```
@media (max-width: 1024px) {
    .profileStatsBlock {
        width: 95%;
        height: auto;
        min-height: 400px;
    }
}
```

```
/* мобильные */
```

```
@media (max-width: 768px) {
```

```
  .bottomProfile {
```

```
    padding: 0 0 3% 0;
```

```
    height: auto;
```

```
  }
```

```
  .profileStatsBlock {
```

```
    width: 95%;
```

```
    height: auto;
```

```
    border-radius: 8px;
```

```
  }
```

```
}
```

```
/* маленькие телефоны */
```

```
@media (max-width: 480px) {
```

```
  .profileStatsBlock {
```

```
    width: 98%;
```

```
  }
```

```
}
```

```
===== \components\common\Profile\StatsBlock\ProfileStatsBlock.tsx  
=====
```

```
import React from 'react';
```

```
import StatisticsTab from './StatisticsTab';
```

```
import MyTestsTab from './MyTestsTab';
```

```

import FriendsTab from './FriendsTab';

import styles from './ProfileStatsBlock.module.css';

interface ProfileStatsBlockProps {

  activeTab: string;

  isOwnProfile?: boolean;

  userId?: number | null;

}

const ProfileStatsBlock: React.FC<ProfileStatsBlockProps> = ({ activeTab, isOwnProfile
= true, userId = null }) => {

  const renderContent = () => {

    switch(activeTab) {

      case 'statistics':

        return <StatisticsTab userId={userId} isOwnProfile={isOwnProfile} />;

      case 'tests':

        return <MyTestsTab userId={userId} isOwnProfile={isOwnProfile} />;

      case 'friends':

        return <FriendsTab isOwnProfile={isOwnProfile} userId={userId} />;

      default:

        return null;

    }

  };

  return (

```

```

    <div className={styles.bottomProfile}>

      <div className={styles.profileStatsBlock}>

        {renderContent()}

      </div>

    </div>

  );

};

export default ProfileStatsBlock;

===== \components\common\Profile\StatsBlock\StatisticsTab.module.css
=====

@import "../../../style/variables.css";

.statisticsContent {
  padding: 20px;
  width: 100%;
  height: 100%;
  overflow: auto;
  display: flex;
  flex-direction: column;
  font-family: Arial, Helvetica, sans-serif;
  color: var(--color-text);
}

/* фиксированный заголовок */

```

```
.stickyHeader {  
    position: sticky;  
    top: 0;  
    background-color: var(--color-blockColor);  
    z-index: 10;  
    padding-bottom: 10px;  
    margin-bottom: 15px;  
}
```

```
.stickyHeader h3 {  
    color: var(--color-text);  
    font-family: Arial, Helvetica, sans-serif;  
    margin: 0;  
    font-size: 24px;  
    font-weight: bold;  
}
```

/* обёртка для таблицы */

```
.tableWrapper {  
    overflow-x: auto;  
    width: 100%;  
    flex: 1;  
}
```

/* скроллбар */

```
.tableWrapper::-webkit-scrollbar {  
    width: 8px;  
    height: 8px;  
}
```

```
.tableWrapper::-webkit-scrollbar-track {  
    background: var(--color-scrollbar-track);  
    border-radius: 10px;  
}
```

```
.tableWrapper::-webkit-scrollbar-thumb {  
    background: var(--color-scrollbar-thumb);  
    border-radius: 10px;  
}
```

```
.tableWrapper::-webkit-scrollbar-thumb:hover {  
    background: var(--color-scrollbar-thumb-hover);  
}
```

```
/* таблица */
```

```
.statsTable {  
    width: 100%;  
    border-collapse: collapse;  
    font-size: 14px;  
    font-family: Arial, Helvetica, sans-serif;
```



```

    color: var(--color-text);
}

/* шапка таблицы */

.statsTable th {
    background-color: var(--color-buttonColor);
    padding: 12px;
    text-align: left;
    font-weight: bold;
    position: sticky;
    top: 0;
    border: none;
    font-family: Arial, Helvetica, sans-serif;
    color: var(--color-text);
}

/* ячейки */

.statsTable td {
    padding: 10px 12px;
    border-bottom: 1px solid var(--color-border);
    font-family: Arial, Helvetica, sans-serif;
    color: var(--color-text);
}

/* hover на строку - убираем */

```

```
.statsTable tr:hover {  
    background-color: transparent;  
}
```

```
/* результат теста */
```

```
.statsTable .result {  
    font-weight: bold;  
    color: var(--color-text);  
}
```

```
/* ССЫЛКИ */
```

```
.testLink a,  
.authorLink a {  
    text-decoration: none !important;  
    font-weight: 600 !important;  
    transition: opacity 0.2s ease;  
}
```

```
.testLink a {  
    font-weight: 700 !important;  
    color: var(--color-text) !important;  
}
```

```
.authorLink a {  
    font-weight: 600 !important;
```

```
    color: var(--color-text) !important;
}
```

```
.testLink a:hover,
.authorLink a:hover {
    opacity: 0.7;
    text-decoration: none !important;
}
```

```
/* загрузка, ошибка */
.loading,
.error {
    text-align: center;
    padding: 40px;
    font-family: Arial, Helvetica, sans-serif;
    font-size: 14px;
}
```

```
.loading {
    color: var(--color-text-muted);
}
```

```
.error {
    color: var(--color-danger);
}
```

```
/* пустое состояние - НЕТ ПРОЙДЕННЫХ ТЕСТОВ */
```

```
.empty {  
    text-align: center;  
    padding: 60px 40px;  
    color: var(--color-text);  
    font-family: Arial, Helvetica, sans-serif;  
    font-size: 18px;  
    font-weight: bold;  
}
```

```
/* ПЛАНШЕТ */
```

```
@media (max-width: 1024px) {
```

```
    .statisticsContent {  
        padding: 15px;  
    }
```

```
    .stickyHeader h3 {  
        font-size: 22px;  
    }
```

```
    .statsTable th,  
    .statsTable td {  
        padding: 10px 12px;  
        font-size: 13px;
```

```

    }

.empty {
    padding: 50px 30px;
    font-size: 17px;
}
}

/* мобильные */

@media (max-width: 768px) {
    .statisticsContent {
        padding: 12px;
    }

    .stickyHeader h3 {
        font-size: 20px;
    }

    .statsTable {
        min-width: 600px;
        font-size: 13px;
    }

    .statsTable th,
    .statsTable td {

```

```
padding: 10px 12px;

font-size: 13px;
}

.empty {

padding: 45px 25px;

font-size: 16px;
}
}

/* маленькие телефоны */

@media (max-width: 480px) {

.statisticsContent {

padding: 10px;
}

.stickyHeader h3 {

font-size: 18px;
}

.statsTable {

min-width: 500px;

font-size: 12px;
}
```

```

.statsTable th,

.statsTable td {

    padding: 8px 10px;

    font-size: 12px;

}


.loading,

.error {

    padding: 30px;

    font-size: 13px;

}


.empty {

    padding: 35px 20px;

    font-size: 15px;

}

}


/* альбомная ориентация на мобильных */

@media (max-width: 768px) and (orientation: landscape) {

    .statisticsContent {

        padding: 10px;

    }


    .statsTable {

```

```
        min-width: 550px;
    }

    .statsTable td {
        padding: 8px 10px;
    }
}

/* очень маленькие телефоны */
@media (max-width: 360px) {
    .statsTable {
        min-width: 450px;
        font-size: 11px;
    }

    .statsTable th,
    .statsTable td {
        padding: 6px 8px;
        font-size: 11px;
    }

    .empty {
        padding: 30px 15px;
        font-size: 14px;
    }
}
```



```
}
```

```
===== \components\common\Profile\StatsBlock\StatisticsTab.tsx =====
```

```
import React, { useState, useEffect } from 'react';
```

```
import { Link } from 'react-router-dom';
```

```
import api from '../.../api/axios';
```

```
import styles from './StatisticsTab.module.css';
```

```
interface StatisticsTabProps {
```

```
  userId?: number | null;
```

```
  isOwnProfile?: boolean;
```

```
}
```

```
const StatisticsTab: React.FC<StatisticsTabProps> = ({ userId, isOwnProfile = true }) =>  
{
```

```
  const [attempts, setAttempts] = useState<any[]>([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState<string | null>(null);
```

```
  useEffect(() => {
```

```
    loadCompletedTests();
```

```
  }, [userId]);
```

```
  const loadCompletedTests = async () => {
```

```
    try {
```

```
      setLoading(true);
```

```

let targetUserId = userId;

if (isOwnProfile && !targetUserId) {
    const profile = await api.get('/auth/profile/');
    targetUserId = profile.data.id;
}

if (!targetUserId) {
    setError('ID пользователя не найден');
    return;
}

const response = await api.get(`/profile/${targetUserId}/completed-tests/`);
console.log('API response:', response.data);
setAttempts(response.data);
} catch (err) {
    console.error('Ошибка загрузки статистики:', err);
    setError('Не удалось загрузить статистику');
} finally {
    setLoading(false);
}
};

if (loading) {

```

```

    return <div className={styles.loading}>Загрузка...</div>;
}

if (error) {
    return <div className={styles.error}>{error}</div>;
}

if (attempts.length === 0) {
    return <div className={styles.empty}>Нет пройденных тестов</div>;
}

return (
    <div className={styles.statisticsContent}>
        <div className={styles.stickyHeader}>
            <h3>Статистика по тестам</h3>
        </div>
        <div className={styles.tableWrapper}>
            <table className={styles.statsTable}>
                <thead>
                    <tr>
                        <th>Тест</th>
                        <th>Тип теста</th>
                        <th>Автор</th>
                        <th>Вопросов</th>
                        <th>Результат</th>

```

```

        <th>Дара</th>

    </tr>

</thead>

<tbody>

    {attempts.map((attempt: any, index: number) => {

        const isSurvey = attempt.type === 'опрос';

        return (

            <tr key={attempt.id || index}>

                <td className={styles.testLink}>

                    <Link to={`/test/${attempt.id}`}>{attempt.name}</Link>

                </td>

                <td>{attempt.type}</td>

                <td className={styles.authorLink}>

                    <Link
to={`/profile/${attempt.author_id}`}>{attempt.author}</Link>

                </td>

                <td>{attempt.questions}</td>

                <td className={styles.result}>

                    {isSurvey ? '—' : attempt.result}

                </td>

                <td>{attempt.date}</td>

            </tr>

        );

    })}

</tbody>

```

```

        </table>

    </div>

</div>

);

};

export default StatisticsTab;

===== \components\common\Profile\profile.module.css =====

@import '../..../style/variables.css';

* {

    margin: 0;

    padding: 0;

    box-sizing: border-box;

}

/* основные блоки */

.profileInfoBlock,

.profileStatsBlock {

    width: 90%;

    height: 440px;

    display: flex;

    background-color: var(--color-blockColor);

    border-radius: 10px;

}

```

```
.topProfile,  
.bottomProfile {  
    width: 100%;  
    height: 50%;  
    display: flex;  
    align-items: center;  
    justify-content: center;  
}
```

```
.topProfile {  
    padding: 2% 0 2% 0;  
}
```

```
.bottomProfile {  
    padding: 0 0 2% 0;  
}
```

```
/* верхний блок */
```

```
.profileInfoBlock {  
    display: flex;  
    flex-direction: column;  
}
```

```
/* разделитель */
```

```
.hr {  
    padding: 0 2% 0 2%;  
}
```

```
.hr hr {  
    height: 1px;  
    border: none;  
    background-color: black;  
}
```

```
/* НИЖНИЙ БЛОК */
```

```
.profileStatsBlock {  
    overflow: auto;  
    display: flex;  
    flex-direction: column;  
    padding: 0;  
    font-family: Arial, Helvetica, sans-serif;  
    color: black;  
}
```

```
/* ПЛАНШЕТ */
```

```
@media (max-width: 1024px) {  
    .profileInfoBlock,  
    .profileStatsBlock {  
        width: 95%;
```

```

        height: auto;

        min-height: 400px;
    }
}

/* мобильные */

@media (max-width: 768px) {

    .topProfile {

        padding: 2% 0;

        height: auto;
    }

    .bottomProfile {

        padding: 0 0 3% 0;

        height: auto;
    }

    .profileInfoBlock,
    .profileStatsBlock {

        width: 95%;

        height: auto;

        border-radius: 8px;
    }

    .profileInfoBlock {

```



```

        flex-direction: column;
    }

    .hr {
        padding: 0 15px;
    }
}

/* маленькие телефоны */
@media (max-width: 480px) {
    .profileInfoBlock,
    .profileStatsBlock {
        width: 98%;
    }
}

/* очень маленькие телефоны */
@media (max-width: 360px) {
    .profileInfoBlockBottom button {
        font-size: 15px;
        padding: 6px 12px;
        min-width: 80px;
    }
}

```

```

/* альбомная ориентация */

@media (max-width: 768px) and (orientation: landscape) {

  .profileInfoBlock {

    flex-direction: row;

  }

}

===== \components\common\Profile\ProfileMain.tsx =====

import React, { useState } from 'react';

import ProfileHeader from './ProfileHeader/ProfileHeader';

import ProfileTabs from './ProfileHeader/ProfileTabs';

import ProfileStatsBlock from './StatsBlock/ProfileStatsBlock';

import styles from './profile.module.css';

interface ProfileMainProps {

  isOwnProfile: boolean;

  userId: number | null;

}

const ProfileMain: React.FC<ProfileMainProps> = ({ isOwnProfile, userId }) => {

  const [activeTab, setActiveTab] = useState('statistics');

  console.log('ProfileMain - userId:', userId); // Добавь отладку

  console.log('ProfileMain - isOwnProfile:', isOwnProfile);

  return (

```

<

```
<div className={styles.topProfile}>

  <div className={styles.profileInfoBlock}>

    <ProfileHeader isOwnProfile={isOwnProfile} userId={userId} />

    <div className={styles.hr}><hr /></div>

    <ProfileTabs activeTab={activeTab} setActiveTab={setActiveTab} />

  </div>

</div>

<ProfileStatsBlock activeTab={activeTab} isOwnProfile={isOwnProfile}
userId={userId} />

</>

);

};
```

```
export default ProfileMain;
```

```
===== \components\common\Question\Question.module.css =====
```

```
* {

  font-family: Arial, sans-serif;

}
```

```
.page {

  background-color: #EAE7DC;

  min-height: 100vh;

  display: flex;

  flex-direction: column;
```

```
align-items: center;

justify-content: flex-start;

position: relative;

}
```

```
/* Хедер */
```

```
.header {

position: fixed;

top: 0;

left: 0;

right: 0;

width: 100%;

height: 35px;

background-color: #D8C3A5;

display: flex;

align-items: center;

justify-content: space-between;

padding: 0 40px;

box-sizing: border-box;

z-index: 100;

box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);

}
```

```
.questionCounter {

font-size: 18px;
```

```
color: black;  
  
font-weight: bold;  
}
```

```
.title {  
  
font-size: 20px;  
  
color: black;  
  
font-weight: bold;  
}
```

```
.exitButton {  
  
text-decoration: none !important;  
  
color: black;  
  
font-size: 18px;  
  
font-weight: bold;  
  
padding: 8px 16px;  
  
border-radius: 8px;  
  
transition: background-color 0.3s;  
}
```

```
.exitButton:hover {  
  
background-color: rgba(0, 0, 0, 0.1);  
  
text-decoration: none !important;  
}
```

/* Таймер - десктоп версия */

```
.timer {  
    position: fixed;  
    top: 45px;  
    left: 20px;  
    font-size: 20px;  
    font-weight: bold;  
    color: black;  
    background-color: rgba(216, 195, 165, 0.95);  
    padding: 8px 16px;  
    border-radius: 8px;  
    z-index: 100;  
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
}
```

/* Основной блок */

```
.mainContainer {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    width: 100%;  
    padding: 90px 20px 40px 20px;  
    box-sizing: border-box;  
}
```

```
.questionBlock {  
    width: 95%;  
    max-width: 2200px;  
    background-color: #D8C3A5;  
    border-radius: 10px;  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    padding: 60px 80px 60px 80px;  
    box-sizing: border-box;  
    box-shadow: 0 8px 25px rgba(0, 0, 0, 0.15);  
    position: relative;  
    margin-top: 20px;  
}
```

/* Кнопка репорта */

```
.reportButton {  
    position: absolute;  
    top: 20px;  
    right: 20px;  
    width: 35px;  
    height: 35px;  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 8px;
```

```
color: black;

font-size: 18px;

font-weight: bold;

cursor: pointer;

display: flex;

align-items: center;

justify-content: center;

transition: all 0.3s ease;

box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);

}
```

```
.reportButton:hover {

background-color: #a47a3f;

transform: scale(1.05);

}
```

```
.reportButton:focus {

outline: none;

}
```

```
/* Картинка - УВЕЛИЧЕННАЯ */
```

```
.imageContainer {

width: 100%;

max-width: 1000px;

height: 400px;
```



```
background-color: #4C4C4C;

border-radius: 10px;

margin-bottom: 40px;

margin-top: 20px;

box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);

overflow: hidden;

display: flex;

align-items: center;

justify-content: center;

}
```

```
.questionImage {

width: 100%;

height: 100%;

object-fit: cover;

border-radius: 10px;

}
```

```
.imagePlaceholder {

width: 100%;

height: 100%;

background-color: #4C4C4C;

border-radius: 10px;

}
```

```
/* Текст вопроса */
```

```
.questionText {  
    font-size: 36px;  
    color: black;  
    text-align: center;  
    margin-bottom: 50px;  
    line-height: 1.4;  
    max-width: 1200px;  
    margin-top: 10px;  
}
```

```
/* Контейнер с ответами (для QuestionPage) */
```

```
.answersContainer {  
    display: flex;  
    justify-content: center;  
    gap: 80px;  
    width: 100%;  
    max-width: 1600px;  
}
```

```
.answersColumn {  
    flex: 1;  
    display: flex;  
    flex-direction: column;  
    gap: 20px;
```

```
}
```

```
/* Кнопки ответов */
```

```
.answerButton {  
    background-color: rgba(255, 215, 157, 0.5);  
    border: none;  
    border-radius: 10px;  
    padding: 12px 18px;  
    font-size: 16px;  
    color: black;  
    cursor: pointer;  
    transition: all 0.3s ease;  
    text-align: center;  
    width: 100%;  
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
}
```

```
.answerButton:hover {  
    background-color: rgba(255, 215, 157, 0.8);  
    transform: translateY(-2px);  
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);  
}
```

```
.answerButton.selected {  
    background-color: rgba(255, 215, 157, 0.95);
```

```
border: 2px solid #BE8F4C;

box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);

}


/* Контейнер с инпутом (для QuestionInputPage) */

.inputContainer {

display: flex;

flex-direction: column;

align-items: center;

gap: 30px;

width: 100%;

max-width: 800px;

}


/* Инпут для ответа */

.answerInput {

width: 100%;

padding: 15px 20px;

font-size: 18px;

color: black;

background-color: rgba(255, 215, 157, 0.5);

border: 2px solid #BE8F4C;

border-radius: 10px;

outline: none;

transition: all 0.3s ease;
```

```
    box-sizing: border-box;
}
```

```
.answerInput:focus {
    background-color: rgba(255, 215, 157, 0.8);
    border-color: #a47a3f;
    box-shadow: 0 0 10px rgba(190, 143, 76, 0.3);
}
```

```
.answerInput::placeholder {
    color: rgba(0, 0, 0, 0.5);
}
```

```
/* Кнопка отправки */
```

```
.submitButton {
    background-color: #BE8F4C;
    border: none;
    border-radius: 10px;
    padding: 12px 40px;
    font-size: 18px;
    font-weight: bold;
    color: black;
    cursor: pointer;
    transition: all 0.3s ease;
    width: auto;
```

```
    min-width: 200px;
}
```

```
.submitButton:hover {
    background-color: #a47a3f;
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);
}
```

```
.submitButton:focus {
    outline: none;
}
```

```
/* Модальное окно */
```

```
.modalOverlay {
    position: fixed;
    top: 0;
    left: 0;
    right: 0;
    bottom: 0;
    background-color: rgba(0, 0, 0, 0.5);
    display: flex;
    justify-content: center;
    align-items: center;
    z-index: 1000;
```

```
}
```

```
.modal {  
  background-color: #D8C3A5;  
  border-radius: 10px;  
  padding: 30px;  
  width: 400px;  
  max-width: 90%;  
  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);  
}
```

```
.modalTitle {  
  font-size: 24px;  
  color: black;  
  margin-bottom: 20px;  
  text-align: center;  
}
```

```
.modalReasons {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
  margin-bottom: 30px;  
}
```

```
.radioLabel {  
    display: flex;  
    align-items: center;  
    gap: 10px;  
    cursor: pointer;  
    color: black;  
    font-size: 16px;  
}
```

```
.radioLabel input {  
    width: 18px;  
    height: 18px;  
    cursor: pointer;  
}
```

```
.radioLabel input:focus {  
    outline: none;  
}
```

```
.modalButtons {  
    display: flex;  
    justify-content: flex-end;  
    gap: 15px;  
}
```



```
.cancelButton {  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 8px;  
    padding: 10px 20px;  
    font-size: 14px;  
    color: black;  
    cursor: pointer;  
    font-weight: bold;  
}
```

```
.cancelButton:focus {  
    outline: none;  
}
```

```
.cancelButton:hover {  
    background-color: #a47a3f;  
}
```

```
.submitButtonModal {  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 8px;  
    padding: 10px 20px;  
    font-size: 14px;
```

```
color: black;

cursor: pointer;

font-weight: bold;

}
```

```
.submitButtonModal:focus {

    outline: none;

}
```

```
.submitButtonModal:hover {

    background-color: #a47a3f;

}
```

```
/* ===== АДАПТИВНОСТЬ ===== */
```

```
/* Планшет и меньше */
```

```
@media (max-width: 1920px) {

    .questionBlock {

        width: 90%;

        max-width: 2000px;

    }

}
```

```
.answersContainer {

    gap: 60px;

}
```

```
}
```

```
@media (max-width: 1600px) {  
  .questionBlock {  
    padding: 50px 50px 50px 50px;  
  }  
}
```

```
.answersContainer {  
  gap: 50px;  
}
```

```
.questionText {  
  font-size: 32px;  
}
```

```
.imageContainer {  
  height: 350px;  
  max-width: 900px;  
}  
}
```

```
@media (max-width: 1200px) {  
  .questionBlock {  
    width: 95%;  
    padding: 40px 30px 40px 30px;  
  }  
}
```

```
}
```

```
.questionText {  
  font-size: 28px;  
}
```

```
.answerButton {  
  padding: 10px 16px;  
  font-size: 15px;  
}
```

```
.answersContainer {  
  gap: 40px;  
}
```

```
.imageContainer {  
  max-width: 800px;  
  height: 300px;  
}  
}
```

```
@media (max-width: 1024px) {  
  .questionBlock {  
    padding: 35px 20px 35px 20px;  
  }  
}
```

```
.questionText {  
  font-size: 24px;  
}
```

```
.answersContainer {  
  flex-direction: column;  
  gap: 15px;  
}
```

```
.answersColumn {  
  width: 100%;  
}
```

```
.timer {  
  top: 45px;  
  font-size: 18px;  
  padding: 6px 14px;  
}
```

```
.reportButton {  
  width: 30px;  
  height: 30px;  
  font-size: 16px;  
}
```

```
.imageContainer {  
    height: 250px;  
}  
}
```

```
/* Мобильный (до 768px) */  
@media (max-width: 768px) {  
    .page {  
        justify-content: flex-start;  
    }  
}
```

```
.header {  
    padding: 0 15px;  
    height: 45px;  
}
```

```
.questionCounter {  
    font-size: 14px;  
}
```

```
.title {  
    font-size: 16px;  
    max-width: 150px;  
    overflow: hidden;
```

```
text-overflow: ellipsis;  
white-space: nowrap;  
}
```

```
.exitButton {  
    font-size: 14px;  
    padding: 5px 10px;  
}
```

```
/* Таймер на мобилке - прижат под хедер */
```

```
.timer {  
    position: fixed;  
    top: 55px;  
    left: 15px;  
    right: auto;  
    width: auto;  
    margin: 0;  
    font-size: 18px;  
    font-weight: bold;  
    background-color: rgba(216, 195, 165, 0.95);  
    padding: 6px 14px;  
    border-radius: 8px;  
    z-index: 99;  
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
}
```

```
.mainContainer {  
    padding: 100px 15px 30px 15px;  
}
```

```
.questionBlock {  
    width: 95%;  
    padding: 40px 15px 40px 15px;  
    margin-top: 0;  
    min-height: 550px;  
}
```

```
.reportButton {  
    top: 10px;  
    right: 10px;  
    width: 32px;  
    height: 32px;  
    font-size: 16px;  
}
```

```
.imageContainer {  
    height: 220px;  
    margin-bottom: 35px;  
    margin-top: 35px;  
    max-width: 100%;
```



```
}
```

```
.questionText {  
  font-size: 22px;  
  margin-bottom: 35px;  
  text-align: center;  
  padding: 0 10px;  
}
```

```
.answersContainer {  
  gap: 15px;  
  width: 100%;  
}
```

```
.answersColumn {  
  gap: 15px;  
}
```

```
.answerButton {  
  padding: 12px 16px;  
  font-size: 16px;  
}
```

```
.inputContainer {  
  gap: 25px;
```

```
width: 100%;  
max-width: 100%;  
}
```

```
.answerInput {  
padding: 14px 18px;  
font-size: 16px;  
}
```

```
.submitButton {  
padding: 12px 30px;  
font-size: 18px;  
min-width: 180px;  
}
```

```
.modal {  
padding: 20px;  
width: 90%;  
}
```

```
.modalTitle {  
font-size: 20px;  
}
```

```
.radioLabel {
```

```
    font-size: 14px;
}

.cancelButton,
.submitButtonModal {
    padding: 8px 16px;
    font-size: 13px;
}
}

/* Маленький телефон (до 480px) */
@media (max-width: 480px) {
    .header {
        height: 42px;
    }

    .questionCounter {
        font-size: 12px;
    }

    .title {
        font-size: 14px;
        max-width: 120px;
    }
}
```

```
.exitButton {  
    font-size: 12px;  
    padding: 4px 8px;  
}
```

```
.timer {  
    top: 52px;  
    left: 12px;  
    font-size: 16px;  
    padding: 5px 12px;  
}
```

```
.mainContainer {  
    padding: 95px 12px 20px 12px;  
}
```

```
.questionBlock {  
    width: 100%;  
    padding: 35px 12px 35px 12px;  
    min-height: 500px;  
}
```

```
.imageContainer {  
    height: 180px;  
    margin-bottom: 30px;
```

```
margin-top: 30px;  
}
```

```
.questionText {  
    font-size: 20px;  
    margin-bottom: 30px;  
}
```

```
.answerButton {  
    padding: 10px 14px;  
    font-size: 15px;  
}
```

```
.answerInput {  
    padding: 12px 15px;  
    font-size: 15px;  
}
```

```
.submitButton {  
    padding: 10px 25px;  
    font-size: 16px;  
    min-width: 160px;  
}
```

```
.reportButton {
```

```
width: 28px;

height: 28px;

font-size: 14px;

}

}

/* Очень маленький телефон (до 360px) */

@media (max-width: 360px) {

.header {

padding: 0 10px;

height: 38px;

}

.questionCounter {

font-size: 10px;

}

.title {

font-size: 12px;

max-width: 100px;

}

.exitButton {

font-size: 10px;

padding: 3px 6px;
```

```
}
```

```
.timer {
```

```
  top: 48px;
```

```
  left: 10px;
```

```
  font-size: 14px;
```

```
  padding: 4px 10px;
```

```
}
```

```
.mainContainer {
```

```
  padding: 85px 10px 20px 10px;
```

```
}
```

```
.questionBlock {
```

```
  min-height: 450px;
```

```
  padding: 30px 10px 30px 10px;
```

```
}
```

```
.imageContainer {
```

```
  height: 150px;
```

```
}
```

```
.questionText {
```

```
  font-size: 18px;
```

```
}
```

```
.answerButton {  
    font-size: 14px;  
    padding: 8px 12px;  
}
```

```
.answerInput {  
    font-size: 14px;  
    padding: 10px 12px;  
}
```

```
.submitButton {  
    font-size: 15px;  
    padding: 8px 20px;  
    min-width: 140px;  
}  
}
```

```
/* Альбомная ориентация на телефонах */
```

```
@media (max-width: 768px) and (orientation: landscape) {  
    .timer {  
        position: fixed;  
        top: 55px;  
        left: 15px;  
        width: auto;
```



```
font-size: 16px;  
padding: 5px 12px;  
margin: 0;  
}
```

```
.mainContainer {  
padding: 90px 15px 20px 15px;  
}
```

```
.questionBlock {  
min-height: 380px;  
padding: 30px 15px 30px 15px;  
}
```

```
.imageContainer {  
height: 160px;  
margin-bottom: 25px;  
}
```

```
.questionText {  
font-size: 18px;  
margin-bottom: 25px;  
}
```

```
.answerButton {
```

```
padding: 8px 12px;

font-size: 14px;

}
```

```
.answerInput {

padding: 10px 15px;

font-size: 14px;

}
```

```
.submitButton {

padding: 8px 20px;

font-size: 15px;

}

}
```

```
===== \components\common\Question\QuestionInputPage.tsx =====
```

```
import React, { useState, useEffect } from 'react';

import { Link, useParams, useNavigate, useLocation } from 'react-router-dom';

import { getQuestion, submitAnswer, finishTest } from '../api/tests';

import styles from './Question.module.css';
```

```
interface QuestionInputPageProps {

  initialTimeLeft?: number | null;

}
```

```
const QuestionInputPage = ({ initialTimeLeft }: QuestionInputPageProps) => {
```

```

const { testId, questionIndex } = useParams();

const navigate = useNavigate();

const location = useLocation();

const [attemptId, setAttemptId] = useState<number | null>(null);

const [question, setQuestion] = useState<any>(null);

const [answer, setAnswer] = useState("");

const [loading, setLoading] = useState(true);

const [timeLeft, setTimeLeft] = useState<number | null>(initialTimeLeft || null);


const BASE_URL = 'http://localhost:8000';


useEffect(() => {

    const state = location.state as any;

    if (state?.attemptId) {

        setAttemptId(state.attemptId);

    }

    if (state?.timeLeft && timeLeft === null) {

        setTimeLeft(state.timeLeft);

    }

}, [location]);


useEffect(() => {

    if (testId && questionIndex && attemptId) {

        loadQuestion();

    }

}

```

```

    }, [testId, questionIndex, attemptId]);

    useEffect(() => {
      if (question && timeLeft === null && question.time_limit && question.time_limit >
0) {
        if (Number(questionIndex) === 1) {
          setTimeLeft(question.time_limit * 60);
        }
      }
    }, [question]);

    useEffect(() => {
      if (timeLeft !== null && timeLeft > 0) {
        const timer = setInterval(() => {
          setTimeLeft(prev => {
            if (prev === null || prev <= 1) {
              clearInterval(timer);
              handleTimeOut();
              return 0;
            }
            return prev - 1;
          });
        }, 1000);
        return () => clearInterval(timer);
      }
    }

```

```
}, [timeLeft]);
```

```
const loadQuestion = async () => {  
  setLoading(true);  
  try {  
    const data = await getQuestion(Number(testId), Number(questionIndex));  
    setQuestion(data);  
  } catch (error) {  
    console.error('Ошибка загрузки вопроса:', error);  
  } finally {  
    setLoading(false);  
  }  
};
```

```
const handleTimeOut = async () => {  
  if (attemptId) {  
    await finishTest(attemptId);  
    navigate(`/test/${testId}/result`, { state: { attemptId } });  
  }  
};
```

```
const handleSubmitAnswer = async () => {  
  if (!answer.trim()) {  
    alert('Введите ответ');  
    return;  
  }  
};
```

```

    }

    if (attemptId && question?.question_id) {
      try {
        await submitAnswer(attemptId, question.question_id, answer);

        const nextIndex = Number(questionIndex) + 1;
        if (nextIndex <= question.total) {
          navigate(`/test/${testId}/question/${nextIndex}`, {
            state: { attemptId, timeLeft }
          });
        } else {
          const result = await finishTest(attemptId);
          navigate(`/test/${testId}/result`, { state: { attemptId, score: result.score } });
        }
      } catch (error) {
        console.error('Ошибка отправки ответа:', error);
        alert('Ошибка при отправке ответа');
      }
    }
  };

  const formatTime = (seconds: number) => {
    const mins = Math.floor(seconds / 60);
    const secs = seconds % 60;

```

```

    return `${mins}:${secs < 10 ? '0' : ''}${secs}`;
};

if (loading) {
    return <div className={styles.loading}>Загрузка...</div>;
}

if (!question) {
    return <div className={styles.error}>Вопрос не найден</div>;
}

const imageFullUrl = question.question_image ? `${BASE_URL}${question.question_image}` : null;

return (
    <div className={styles.page}>
        <header className={styles.header}>
            <div className={styles.questionCounter}>
                Вопрос {question.current}/{question.total}
            </div>
            <div className={styles.title}>
                {question.test_title}
            </div>
            <Link to="/" className={styles.exitButton}>
                Выход

```

```

    </Link>

</header>

{timeLeft !== null && (
  <div className={styles.timer}>
    ⌚ {formatTime(timeLeft)}
  </div>
)}

<div className={styles.mainContainer}>
  <div className={styles.questionBlock}>
    {imageFullUrl && (
      <div className={styles.imageContainer}>
        <img
          src={imageFullUrl}
          alt="question"
          className={styles.questionImage}
          onError={(e) => {
            console.error('Ошибка загрузки картинки вопроса:',
imageFullUrl);
            e.currentTarget.style.display = 'none';
          }}
        />
      </div>
    )}
  </div>
)}

```



```

    <div className={styles.questionText}>
        {question.question_text}
    </div>

    <div className={styles.inputContainer}>
        <input
            type="text"
            className={styles.answerInput}
            placeholder="Введите ответ"
            value={answer}
            onChange={(e) => setAnswer(e.target.value)}
            onKeyDown={(e) => e.key === 'Enter' && handleSubmitAnswer()}
        />

        <button className={styles.submitButton}
onClick={handleSubmitAnswer}>

            Отправить

        </button>

    </div>

</div>

</div>

</div>

);

};

```

```

export default QuestionInputPage;

===== \components\common\Question\QuestionPage.tsx =====

import React, { useState, useEffect } from 'react';

import { Link, useParams, useNavigate, useLocation } from 'react-router-dom';

import { getQuestion, submitAnswer, finishTest } from '../../api/tests';

import styles from './Question.module.css';

interface QuestionPageProps {
  initialTimeLeft?: number | null;
}

const QuestionPage = ({ initialTimeLeft }: QuestionPageProps) => {
  const { testId, questionIndex } = useParams();

  const navigate = useNavigate();

  const location = useLocation();

  const [attemptId, setAttemptId] = useState<number | null>(null);

  const [question, setQuestion] = useState<any>(null);

  const [shuffledAnswers, setShuffledAnswers] = useState<any[]>([]);

  const [selectedAnswer, setSelectedAnswer] = useState<string | null>(null);

  const [loading, setLoading] = useState(true);

  const [timeLeft, setTimeLeft] = useState<number | null>(initialTimeLeft || null);

  const BASE_URL = 'http://localhost:8000';

  useEffect(() => {

```

```

const state = location.state as any;

if (state?.attemptId) {

    setAttemptId(state.attemptId);

}

// Восстанавливаем таймер из location.state если есть

if (state?.timeLeft && timeLeft === null) {

    setTimeLeft(state.timeLeft);

}

}, [location]);

useEffect(() => {

    if (testId && questionIndex && attemptId) {

        loadQuestion();

    }

}, [testId, questionIndex, attemptId]);

// Получаем общее время теста только один раз при загрузке первого вопроса

useEffect(() => {

    if (question && timeLeft === null && question.time_limit && question.time_limit >
0) {

        // Только для первого вопроса устанавливаем таймер

        if (Number(questionIndex) === 1) {

            setTimeLeft(question.time_limit * 60);

        }

    }

}

```

```
}, [question]);
```

```
useEffect(() => {  
  if (timeLeft !== null && timeLeft > 0) {  
    const timer = setInterval(() => {  
      setTimeLeft(prev => {  
        if (prev === null || prev <= 1) {  
          clearInterval(timer);  
          handleTimeOut();  
          return 0;  
        }  
        return prev - 1;  
      });  
    }, 1000);  
    return () => clearInterval(timer);  
  }  
}, [timeLeft]);
```

```
const shuffleArray = (array: any[]) => {  
  const shuffled = [...array];  
  for (let i = shuffled.length - 1; i > 0; i--) {  
    const j = Math.floor(Math.random() * (i + 1));  
    [shuffled[i], shuffled[j]] = [shuffled[j], shuffled[i]];  
  }  
  return shuffled;  
}
```

```
};
```

```
const loadQuestion = async () => {  
  setLoading(true);  
  try {  
    const data = await getQuestion(Number(testId), Number(questionIndex));  
    setQuestion(data);  
  
    if (data.answers && data.answers.length > 0) {  
      const shuffled = shuffleArray(data.answers);  
      setShuffledAnswers(shuffled);  
    }  
  } catch (error) {  
    console.error('Ошибка загрузки вопроса:', error);  
  } finally {  
    setLoading(false);  
  }  
};
```

```
const handleTimeOut = async () => {  
  if (attemptId) {  
    await finishTest(attemptId);  
    navigate(`/test/${testId}/result`, { state: { attemptId } });  
  }  
};
```

```

const handleAnswerClick = async (answerId: string) => {

    setSelectedAnswer(answerId);

    if (attemptId && question?.question_id) {

        try {

            await submitAnswer(attemptId, question.question_id, answerId);

            const nextIndex = Number(questionIndex) + 1;

            if (nextIndex <= question.total) {

                // Передаём оставшееся время на следующий вопрос

                navigate(`/test/${testId}/question/${nextIndex}`, {

                    state: { attemptId, timeLeft }

                });

            } else {

                const result = await finishTest(attemptId);

                navigate(`/test/${testId}/result`, { state: { attemptId, score: result.score } });

            }

        } catch (error) {

            console.error('Ошибка отправки ответа:', error);

            alert('Ошибка при отправке ответа');

        }

    }

};

```

```

const formatTime = (seconds: number) => {

  const mins = Math.floor(seconds / 60);

  const secs = seconds % 60;

  return `${mins}:${secs < 10 ? '0' : ''}${secs}`;

};

if (loading) {

  return <div className={styles.loading}>Загрузка...</div>;

}

if (!question) {

  return <div className={styles.error}>Вопрос не найден</div>;

}

const midIndex = Math.ceil(shuffledAnswers.length / 2);

const leftAnswers = shuffledAnswers.slice(0, midIndex);

const rightAnswers = shuffledAnswers.slice(midIndex);

const imageFullUrl = question.question_image ? `${BASE_URL}${question.question_image}` : null;

return (

  <div className={styles.page}>

    <header className={styles.header}>

      <div className={styles.questionCounter}>

        Вопрос {question.current}/{question.total}

```

```

</div>

<div className={styles.title}>

  {question.test_title}

</div>

<Link to="/" className={styles.exitButton}>

  Выход

</Link>

</header>

```

```

{timeLeft !== null && (
  <div className={styles.timer}>

    🕒 {formatTime(timeLeft)}

  </div>

)}

```

```

<div className={styles.mainContainer}>

  <div className={styles.questionBlock}>

    {imageFullUrl && (
      <div className={styles.imageContainer}>

        <img

          src={imageFullUrl}

          alt="question"

          className={styles.questionImage}

          onError={(e) => {

            console.error('Ошибка загрузки картинки вопроса:',

```



```
imageFullUrl);
```

```
        e.currentTarget.style.display = 'none';
```

```
    }}
```

```
  />
```

```
</div>
```

```
}}
```

```
<div className={styles.questionText}>
```

```
  {question.question_text}
```

```
</div>
```

```
<div className={styles.answersContainer}>
```

```
  <div className={styles.answersColumn}>
```

```
    {leftAnswers.map((answer: any) => (
```

```
      <button
```

```
        key={answer.id}
```

```
        className={` ${styles.answerButton} ${selectedAnswer ===  
answer.id.toString() ? styles.selected : ""}`}
```

```
        onClick={() => handleAnswerClick(answer.id.toString())}
```

```
      >
```

```
        {answer.text}
```

```
      </button>
```

```
    )}}
```

```
  </div>
```

```
<div className={styles.answersColumn}>
```

```

        {rightAnswers.map((answer: any) => (
            <button
                key={answer.id}
                className={` ${styles.answerButton} ${selectedAnswer ===
answer.id.toString() ? styles.selected : ""}`}
                onClick={() => handleAnswerClick(answer.id.toString())}
            >
                {answer.text}
            </button>
        ))}
    </div>
</div>
</div>
</div>
</div>
);
};

```

```
export default QuestionPage;
```

```

===== \components\common\ReportModal\ReportModal.module.css
=====

```

```

.modalOverlay {
    position: fixed;
    top: 0;
    left: 0;

```

```
right: 0;

bottom: 0;

background-color: rgba(0, 0, 0, 0.5);

display: flex;

justify-content: center;

align-items: center;

z-index: 1000;

}


.modal {

background-color: #D8C3A5;

border-radius: 10px;

padding: 30px;

width: 400px;

max-width: 90%;

box-shadow: 0 4px 20px rgba(0, 0, 0, 0.3);

}


.modalTitle {

font-size: 24px;

color: black;

margin-bottom: 20px;

text-align: center;

}
```

```
.modalReasons {  
  display: flex;  
  flex-direction: column;  
  gap: 15px;  
  margin-bottom: 20px;  
}
```

```
.radioLabel {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  cursor: pointer;  
  color: black;  
  font-size: 16px;  
}
```

```
.radioLabel input {  
  width: 18px;  
  height: 18px;  
  cursor: pointer;  
}
```

```
.radioLabel input:focus {  
  outline: none;  
}
```

```
.modalComment {  
    margin-bottom: 20px;  
}
```

```
.commentLabel {  
    display: block;  
    color: black;  
    font-size: 14px;  
    margin-bottom: 8px;  
    font-weight: bold;  
}
```

```
.commentTextarea {  
    width: 100%;  
    padding: 10px;  
    border: 1px solid #BE8F4C;  
    border-radius: 8px;  
    background-color: #EAE7DC;  
    color: black;  
    font-size: 14px;  
    font-family: Arial, sans-serif;  
    resize: vertical;  
    box-sizing: border-box;  
}
```

```
.commentTextarea:focus {  
    outline: none;  
    border-color: #8B6B3A;  
    box-shadow: 0 0 5px rgba(139, 107, 58, 0.3);  
}
```

```
.modalButtons {  
    display: flex;  
    justify-content: flex-end;  
    gap: 15px;  
}
```

```
.cancelButton,  
.submitButton {  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 8px;  
    padding: 10px 20px;  
    font-size: 14px;  
    color: black;  
    cursor: pointer;  
    font-weight: bold;  
    transition: background-color 0.3s;  
}
```

```
.cancelButton:focus,  
.submitButton:focus {  
    outline: none;  
}
```

```
.cancelButton:hover,  
.submitButton:hover {  
    background-color: #a47a3f;  
}
```

```
@media (max-width: 480px) {  
    .modal {  
        padding: 20px;  
        width: 90%;  
    }
```

```
.modalTitle {  
    font-size: 20px;  
}
```

```
.radioLabel {  
    font-size: 14px;  
}
```

```

.cancelButton,

.submitButton {
  padding: 8px 16px;
  font-size: 13px;
}
}

===== \components\common\ReportModal\ReportModal.tsx =====

import React, { useState } from 'react';
import styles from './ReportModal.module.css';

interface ReportModalProps {
  isOpen: boolean;
  onClose: () => void;
  onSubmit: (reason: string, comment: string) => void;
  title?: string;
  reasons?: string[];
}

const ReportModal: React.FC<ReportModalProps> = ({
  isOpen,
  onClose,
  onSubmit,
  title = "Пожаловаться",
  reasons = [
    "Оскорбительное поведение",

```



```

    "Спам",
    "Другое"
  ]
}) => {
  const [selectedReason, setSelectedReason] = useState("");
  const [comment, setComment] = useState("");

  const handleSubmit = () => {
    if (!selectedReason) {
      alert('Выберите причину жалобы');
      return;
    }

    if (selectedReason === 'Другое' && !comment.trim()) {
      alert('Пожалуйста, опишите причину жалобы подробнее');
      return;
    }

    onSubmit(selectedReason, comment);

    setSelectedReason("");
    setComment("");
  };

  const handleClose = () => {
    setSelectedReason("");
  };

```

```

    setComment("");

    onClose();

};

const isOtherSelected = selectedReason === 'Другое';

if (!isOpen) return null;

return (
  <div className={styles.modalOverlay} onClick={handleClose}>
    <div className={styles.modal} onClick={(e) => e.stopPropagation()}>
      <h3 className={styles.modalTitle}>{title}</h3>
      <div className={styles.modalReasons}>
        {reasons.map((reason) => (
          <label key={reason} className={styles.radioLabel}>
            <input
              type="radio"
              name="reportReason"
              value={reason}
              checked={selectedReason === reason}
              onChange={(e) => setSelectedReason(e.target.value)}
            />
            <span>{reason}</span>
          </label>
        ))}
      </div>
    </div>
  </div>
);

```

```
</div>
```

```
{isOtherSelected && (
```

```
  <div className={styles.modalComment}>
```

```
    <label className={styles.commentLabel}>
```

```
      Укажите причину подробнее <span  
      className={styles.required}>*</span>
```

```
    </label>
```

```
    <textarea
```

```
      className={styles.commentTextarea}
```

```
      value={comment}
```

```
      onChange={(e) => setComment(e.target.value)}
```

```
      placeholder="Опишите проблему..."
```

```
      rows={3}
```

```
    />
```

```
  </div>
```

```
)}
```

```
<div className={styles.modalButtons}>
```

```
  <button className={styles.cancelButton} onClick={handleClose}>
```

```
    Отмена
```

```
  </button>
```

```
  <button className={styles.submitButton} onClick={handleSubmit}>
```

```
    Отправить
```

```
  </button>
```

```

        </div>

    </div>

</div>

);

};

export default ReportModal;

===== \components\common\Result\TestResult.module.css =====

.testResult {

    max-width: 1200px;

    margin: 0 auto;

    padding: 40px 20px;

    background-color: #EAE7DC;

    min-height: 100vh;

}

.mainResultBlock {

    background-color: #D8C3A5;

    border-radius: 20px;

    padding: 30px;

    margin-bottom: 30px;

}

.testName {

    text-align: center;

```

```
}
```

```
.testName h3 {  
    font-size: 32px;  
    font-weight: bold;  
    color: #000;  
    margin: 0 0 20px 0;  
    text-align: center;  
    text-transform: uppercase;  
}
```

```
.statsResult {  
    display: flex;  
    justify-content: space-between;  
    align-items: center;  
    flex-wrap: wrap;  
    gap: 20px;  
    margin-bottom: 30px;  
    padding-bottom: 20px;  
    border-bottom: 1px solid rgba(0,0,0,0.1);  
    position: relative;  
}
```

```
.statsResult > span {  
    font-size: 25px;
```

```
font-weight: 700;

color: #000;

flex-shrink: 0;
}
```

```
.typeBadge {

font-size: 25px;

font-weight: 700;

color: #000;

padding: 6px 20px;

text-align: center;

display: block;

width: fit-content;

margin: 0;

position: absolute;

left: 50%;

transform: translateX(-50%);
}
```

```
.resultResult {

text-align: center;

padding: 30px;
}
```

```
.resultPercent {
```

```
font-size: 48px;  
  
font-weight: 700;  
  
margin: 0 0 10px 0;  
  
color: #000;  
  
}
```

```
.resultText {  
  
font-size: 24px;  
  
font-weight: 700;  
  
margin: 0;  
  
color: #000;  
  
}
```

```
.ratingSection {  
  
display: flex;  
  
align-items: center;  
  
justify-content: center;  
  
gap: 15px;  
  
padding-top: 20px;  
  
border-top: 1px solid rgba(0,0,0,0.1);  
  
}
```

```
.ratingSection span {  
  
color: #000;  
  
font-size: 16px;
```

```
}
```

```
.stars {  
    display: flex;  
    gap: 8px;  
}
```

```
.star {  
    font-size: 32px;  
    cursor: pointer;  
    color: #ccc;  
    transition: color 0.2s;  
    background: transparent;  
    border: none;  
    padding: 0;  
    margin: 0;  
    line-height: 1;  
    -webkit-text-stroke: 2px #000;  
    text-stroke: 2px #000;  
    paint-order: stroke fill;  
}
```

```
.star.active {  
    color: #ffc107;  
    -webkit-text-stroke: 2px #000;
```



```
}
```

```
.star:hover {  
    color: #ffc107;  
    -webkit-text-stroke: 2px #000;  
}
```

```
.star:focus {  
    outline: none;  
}
```

```
.commentSections {  
    background-color: #D8C3A5;  
    border-radius: 20px;  
    padding: 30px;  
}
```

```
.commentTitle {  
    font-size: 24px;  
    font-weight: bold;  
    display: block;  
    margin-bottom: 20px;  
    color: #000;  
}
```

```
.commentForm {  
    margin-bottom: 30px;  
}
```

```
.formTitle {  
    display: block;  
    font-size: 14px;  
    color: #000;  
    margin-bottom: 10px;  
}
```

```
.commentForm form {  
    display: flex;  
    gap: 10px;  
}
```

```
.commentInput {  
    flex: 1;  
    padding: 12px 16px;  
    border: none;  
    border-radius: 10px;  
    font-size: 14px;  
    outline: none;  
    background-color: #FFD79D;  
    color: #000;
```

```
    box-shadow: 0 2px 6px rgba(0, 0, 0, 0.1);  
}
```

```
.commentInput:focus {  
    outline: none;  
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.15);  
}
```

```
.commentInput::placeholder {  
    color: #666;  
}
```

```
.submitButton {  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 10px;  
    padding: 12px 24px;  
    font-size: 14px;  
    font-weight: bold;  
    cursor: pointer;  
    color: #000;  
    outline: none;  
}
```

```
.submitButton:focus {
```

```
outline: none;

box-shadow: none;

}
```

```
.submitButton:hover {

    background-color: #a47a3f;

}
```

```
.commentsList {

    display: flex;

    flex-direction: column;

    gap: 15px;

}
```

```
.commentItem {

    background-color: #FFD79D;

    border-radius: 15px;

    padding: 15px;

}
```

```
.commentHeader {

    display: flex;

    justify-content: space-between;

    align-items: center;

    margin-bottom: 8px;
```

```
}
```

```
.commentAuthor {  
    font-weight: bold;  
    color: #000;  
    font-size: 14px;  
}
```

```
.commentDate {  
    font-size: 11px;  
    color: #666;  
    margin-bottom: 8px;  
}
```

```
.commentText {  
    font-size: 14px;  
    color: #000;  
    line-height: 1.4;  
}
```

```
.noComments {  
    text-align: center;  
    padding: 40px;  
    color: #666;  
}
```

```
.reportCommentButton {  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 6px;  
    padding: 4px 12px;  
    font-size: 14px;  
    font-weight: bold;  
    cursor: pointer;  
    color: #000;  
    outline: none;  
}
```

```
.reportCommentButton:focus {  
    outline: none;  
    box-shadow: none;  
}
```

```
.reportCommentButton:hover {  
    background-color: #a47a3f;  
}
```

```
.loading, .error {  
    text-align: center;  
    padding: 50px;
```

```
font-size: 18px;

color: #000;
}


/* Адаптив для мобильных устройств */

@media (max-width: 768px) {

    .testResult {

        padding: 20px;
    }


    .mainResultBlock,

    .commentSections {

        padding: 20px;
    }


    .resultPercent {

        font-size: 32px;
    }


    .commentForm form {

        flex-direction: column;
    }


    .testName h3 {

        font-size: 24px;
    }
}
```

```
}
```

```
.resultText {  
    font-size: 18px;  
}
```

```
/* Адаптив для statsResult - перестраиваем в колонку */
```

```
.statsResult {  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    gap: 15px;  
    position: static;  
}
```

```
.statsResult > span {  
    font-size: 20px;  
    order: 1;  
}
```

```
.typeBadge {  
    position: static;  
    left: auto;  
    transform: none;  
    order: 2;
```



```

    font-size: 20px;

    padding: 4px 16px;
}

/* Обертка для автора - если есть класс authorWrapper */
.statsResult :global(.authorChip),
.statsResult > div:last-child {
    order: 3;
}

/* Звезды на мобильных */
.star {
    font-size: 28px;
    -webkit-text-stroke: 1.5px #000;
}

.ratingSection {
    gap: 10px;
    flex-wrap: wrap;
}

.resultResult {
    padding: 20px;
}
}

```

```
/* Для совсем маленьких экранов */
```

```
@media (max-width: 480px) {
```

```
  .testResult {
```

```
    padding: 15px;
```

```
  }
```

```
  .mainResultBlock,
```

```
  .commentSections {
```

```
    padding: 15px;
```

```
  }
```

```
  .testName h3 {
```

```
    font-size: 20px;
```

```
  }
```

```
  .statsResult > span,
```

```
  .typeBadge {
```

```
    font-size: 18px;
```

```
  }
```

```
  .resultPercent {
```

```
    font-size: 28px;
```

```
  }
```

```
.resultText {  
    font-size: 16px;  
}  
  
.star {  
    font-size: 24px;  
    -webkit-text-stroke: 1.5px #000;  
}  
  
.commentTitle {  
    font-size: 20px;  
}  
  
.submitButton {  
    padding: 10px 16px;  
    font-size: 12px;  
}  
  
.commentInput {  
    padding: 10px 12px;  
    font-size: 12px;  
}  
}  
  
.mutedMessage {
```

```
background-color: #a47a3f;

border-radius: 10px;

padding: 15px 20px;

text-align: center;

margin-bottom: 20px;

border: 1px solid rgba(0, 0, 0, 0.1);
}
```

```
.mutedIcon {

    display: none;
}
```

```
.mutedText {

    font-size: 16px;

    font-weight: bold;

    color: #000000;

    margin-bottom: 8px;
}
```

```
.mutedReason {

    font-size: 13px;

    color: #000000;

    margin-bottom: 5px;
}
```

```
.mutedUntil {  
    font-size: 12px;  
    color: #000000;  
    opacity: 0.7;  
}  
  
/* Контейнер загрузки */  
.loadingContainer {  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    min-height: 100vh;  
    padding: 40px;  
    gap: 20px;  
    background-color: #EAE7DC;  
}  
  
/* Картинка загрузки */  
.loadingImage {  
    width: 200px;  
    height: auto;  
    object-fit: contain;  
}
```

```
/* Текст загрузки */
```

```
.loadingText {  
  font-size: 24px;  
  font-weight: bold;  
  color: #000000;  
  text-align: center;  
  letter-spacing: 2px;  
}
```

```
===== \components\common\Result\TestResult.tsx =====
```

```
import React, { useState, useEffect } from 'react';  
import { useParams, useLocation, useNavigate } from 'react-router-dom';  
import api from '../api/axios';  
  
import { getAttemptResults, rateTest, getComments, addComment, getTestDetails,  
submitReport } from '../api/tests';  
  
import ReportModal from '../components/common/ReportModal/ReportModal';  
import AuthorChip from '../components/common/Chip/userChip/AuthorChip';  
import styles from './TestResult.module.css';
```

```
const TestResult = () => {  
  const { testId } = useParams();  
  const location = useLocation();  
  const navigate = useNavigate();  
  const [attempt, setAttempt] = useState<any>(null);  
  const [test, setTest] = useState<any>(null);  
  const [loading, setLoading] = useState(true);
```

```

const [userRating, setUserRating] = useState<number | null>(null);

const [comment, setComment] = useState("");

const [comments, setComments] = useState<any[]>([]);

const [isAuthenticated, setIsAuthenticated] = useState(false);

const [isReportModalOpen, setIsReportModalOpen] = useState(false);

const [currentReportType, setCurrentReportType] = useState<'test' | 'comment' |
null>(null);

const [currentCommentId, setCurrentCommentId] = useState<number | null>(null);

const [isMuted, setIsMuted] = useState(false);

const [muteReason, setMuteReason] = useState("");

const [muteUntil, setMuteUntil] = useState<string | null>(null);


const commentReportReasons = [
  "Оскорбительное поведение",
  "Спам",
  "Другое"
];


useEffect(() => {
  const token = localStorage.getItem('access_token');

  setIsAuthenticated(!!token);

  if (token) {
    checkMuteStatus();
  }
}

```

```

const state = location.state as any;

if (state?.attemptId) {

    loadResults(state.attemptId);

} else {

    navigate(`/test/${testId}`);

}

}, [location]);

useEffect(() => {

    if (testId) {

        loadComments();

        loadTestDetails();

    }

}, [testId]);

const checkMuteStatus = async () => {

    try {

        const profile = await api.get('/auth/profile/');

        setIsMuted(profile.data.is_muted || false);

        setMuteReason(profile.data.mute_reason || "");

        setMuteUntil(profile.data.mute_until);

    } catch (error) {

        console.error('Ошибка проверки мута:', error);

    }

}

```



```
};
```

```
const loadComments = async () => {  
  try {  
    const data = await getComments(Number(testId));  
    setComments(data);  
  } catch (error) {  
    console.error('Ошибка загрузки комментариев:', error);  
  }  
};
```

```
const loadTestDetails = async () => {  
  try {  
    const data = await getTestDetails(Number(testId));  
    setTest(data);  
  } catch (error) {  
    console.error('Ошибка загрузки теста:', error);  
  }  
};
```

```
const loadResults = async (attemptId: number) => {  
  try {  
    const data = await getAttemptResults(attemptId);  
    setAttempt(data);  
  } catch (error) {
```

```

        console.error('Ошибка загрузки результатов:', error);
    } finally {
        setLoading(false);
    }
};

```

```

const handleRate = async (rating: number) => {
    if (!isAuthenticated) {
        alert('Оценивать тест могут только авторизованные пользователи');
        return;
    }

```

```

    setUserRating(rating);

    try {
        await rateTest(Number(testId), rating);
        alert('Спасибо за оценку!');
    } catch (error: any) {
        alert(error.response?.data?.error || 'Ошибка при оценке');
    }
};

```

```

const handleSubmitComment = async (e: React.FormEvent) => {
    e.preventDefault();

    if (!isAuthenticated) {

```

```

    alert('Комментарии могут оставлять только авторизованные пользователи');

    return;
}

if (isMuted) {
    let message = 'Вы замучены.';

    if (muteReason) message += ` Причина: ${muteReason}`;

    if (muteUntil) message += ` До: ${new Date(muteUntil).toLocaleString()}`;

    alert(message);

    return;
}

if (!comment.trim()) return;

try {
    await addComment(Number(testId), comment);

    setComment("");

    await loadComments();

    alert('Комментарий добавлен');
} catch (error: any) {
    alert(error.response?.data?.error || 'Ошибка при добавлении комментария');
}

};

const openReportModal = (type: 'test' | 'comment', commentId?: number) => {

```

```

    setCurrentReportType(type);

    if (commentId) setCurrentCommentId(commentId);

    setIsReportModalOpen(true);
};

const closeReportModal = () => {
    setIsReportModalOpen(false);

    setCurrentReportType(null);

    setCurrentCommentId(null);
};

const handleSubmitReport = async (reason: string, comment: string) => {
    const token = localStorage.getItem('access_token');

    if (!token) {
        const confirm = window.confirm('Вы не авторизованы. Хотите перейти на
страницу входа?');

        if (confirm) {
            navigate('/login');
        }

        return;
    }

    try {
        await submitReport({
            target_type: currentReportType === 'test' ? 'test' : 'comment',

```

```

    target_id: currentReportType === 'test' ? Number(testId) : currentCommentId!,
    test_id: currentReportType === 'comment' ? Number(testId) : undefined,
    reason: reason,
    comment: comment
  });

  alert('Жалоба отправлена');

  closeReportModal();
} catch (error: any) {
  console.error('Ошибка отправки жалобы:', error);
  alert(`Ошибка: ${error.response?.data?.error || error.message}`);
}
};

```

```

if (loading) {
  return (
    <div className={styles.testResult}>
      <div className={styles.loadingContainer}>
         {
            e.currentTarget.style.display = 'none';
          }}
        />

```

```

        <div className={styles.loadingText}>ЗАГРУЗКА</div>

    </div>

</div>

);
}

if (!attempt || !test) {
    return <div className={styles.error}>Результаты не найдены</div>;
}

const authorInfo = test.author_info || {};
const authorId = authorInfo.id;
const authorName = authorInfo.full_name || authorInfo.login || 'автор';
const authorAvatar = authorInfo.avatar;

const scorePercent = attempt.score;
const passingScore = test.passing_score || 70;
const isSurvey = test.is_survey;

const scoreText = isSurvey ? 'ОПРОС ПРОЙДЕН' : (scorePercent >= passingScore ?
'ПРОЙДЕН' : 'НЕ ПРОЙДЕН');
const typeDisplay = isSurvey ? 'ОПРОС' : 'ТЕСТ';

const getMuteUntilText = () => {
    if (!muteUntil) return '';

```

```

const date = new Date(muteUntil);

return date.toLocaleString('ru-RU');

};

return (
  <div className={styles.testResult}>
    <div className={styles.mainResultBlock}>
      <div className={styles.testName}>
        <h3>{test.title}</h3>
      </div>

      <div className={styles.statsResult}>
        <span>Пройдено: {new Date(attempt.finished_at ||
attempt.started_at).toLocaleDateString()}</span>
        <div className={styles.typeBadge}>{typeDisplay}</div>

        <AuthorChip
          authorId={authorId}
          authorName={authorName}
          authorAvatar={authorAvatar}
        />
      </div>

      <div className={styles.resultResult}>
        {!isSurvey ? (

```

```

    </>

    <h2 className={styles.resultPercent}>РЕЗУЛТАТ {scorePercent}
%</h2>

    <p className={styles.resultText}>{scoreText}</p>

  </>

) : (

  </>

    <h2 className={styles.resultText} style={{ fontSize: '48px',
fontWeight: 'bold' }}>{scoreText}</h2>

  </>

)}

</div>

{isAuthenticated && (

  <div className={styles.ratingSection}>

    <span>Оцените тест:</span>

    <div className={styles.stars}>

      {[1, 2, 3, 4, 5].map((star) => (

        <button

          key={star}

          className={` ${styles.star} ${userRating && star <= userRating ?
styles.active : ''}`

          onClick={() => handleRate(star)}

        >

          ★

        </button>

```



```

    }}}
</div>

</div>

)}
</div>

<div className={styles.commentSections}>

  <span className={styles.commentTitle}>КОММЕНТАРИИ</span>

  {isAuthenticated && (
    <div className={styles.mutedMessage}>
      <div className={styles.mutedText}>Вы замучены</div>
      {muteReason && <div className={styles.mutedReason}>Причина:
{muteReason}</div>}
      {muteUntil && <div className={styles.mutedUntil}>До:
{getMuteUntilText()}</div>}
    </div>
  ) : (
    <div className={styles.commentForm}>
      <span className={styles.formTitle}>оставить комментарий</span>
      <form onSubmit={handleSubmitComment}>
        <input
          name="comment"

```

```

        placeholder="ПРИМЕР: КЛАССНЫЙ ТЕСТ"
        value={comment}
        onChange={(e) => setComment(e.target.value)}
        className={styles.commentInput}
    />
    <button type="submit" className={styles.submitButton}
>ОТПРАВИТЬ</button>
</form>
</div>
)}
</>
)}

<div className={styles.commentsList}>
    {comments.length === 0 ? (
        <div className={styles.noComments}>Нет комментариев. Будьте
первым!</div>
    ) : (
        comments.map((c) => (
            <div key={c.id} className={styles.commentItem}>
                <div className={styles.commentHeader}>
                    <div className={styles.commentAuthor}>
                        <span>{c.user_info?.full_name || c.user_info?.login ||
'Пользователь'}</span>
                    </div>
                </div>
            <button

```

```

        className={styles.reportCommentButton}
        onClick={() => openReportModal('comment', c.id)}
      >
        !
      </button>
    </div>
    <div className={styles.commentDate}>
      {new Date(c.created_at).toLocaleDateString()}
    </div>
    <div className={styles.commentText}>{c.text}</div>
  </div>
))
})
</div>
</div>

```

```

<ReportModal
  isOpen={isReportModalOpen}
  onClose={closeReportModal}
  onSubmit={handleSubmitReport}
  title="Пожаловаться на комментарий"
  reasons={commentReportReasons}
/>
</div>

```

```
);
```

```
};
```

```
export default TestResult;
```

```
===== \components\common\Search\SearchSection\SearchSection.module.css  
=====
```

```
.section {
```

```
  background-color: #EAE7DC;
```

```
  padding: 54px 133px 27px 133px;
```

```
  text-align: center;
```

```
}
```

```
/* Новый wrapper для строки поиска */
```

```
.searchWrapper {
```

```
  display: flex;
```

```
  gap: 15px;
```

```
  align-items: center;
```

```
  max-width: 1648px;
```

```
  width: 100%;
```

```
  margin: 0 auto;
```

```
}
```

```
.searchInput {
```

```
  flex: 1;
```

```
  height: 60px;
```

```
  background-color: #D8C3A5;
```

```
border-radius: 20px;

outline: none;

border: none;

color: #000000;

padding-left: 20px;

font-size: 30px;

}
```

```
.searchInput::placeholder {

font-size: 30px;

color: rgba(0, 0, 0, 0.5);

}
```

```
.searchInput:focus {

outline: 2px solid #BE8F4C;

}
```

```
/* Новая кнопка НАЙТИ */

.searchButton {

height: 60px;

padding: 0 30px;

background-color: #BE8F4C;

border: none;

border-radius: 20px;

color: black;
```

```
font-size: 24px;  
font-weight: bold;  
cursor: pointer;  
transition: opacity 0.2s;  
white-space: nowrap;  
}
```

```
.searchButton:hover {  
  opacity: 0.85;  
}
```

```
.chipContainer {  
  width: 100%;  
  margin-top: 15px;  
  max-width: 1648px;  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  gap: 15px;  
  position: relative;  
}
```

```
.tags {  
  display: flex;  
  gap: 10px;
```

```
flex-wrap: wrap;

}

/* Стили для селектора типа */

.typeSelector {
    position: relative;
}

.typeButton {
    padding: 8px 24px;
    font-size: 30px;
    font-family: Arial, sans-serif;
    background-color: #D8C3A5;
    height: 50px;
    text-align: center;
    color: #000000;
    border: none;
    border-radius: 10px;
    cursor: pointer;
    transition: all 0.3s ease;
    white-space: nowrap;
}

.typeButton:hover {
    background-color: #c9b48c;
```

```
}
```

```
/* Выпадающий список */
```

```
.dropdown {  
    position: absolute;  
    top: 65px;  
    left: 0;  
    background-color: #D8C3A5;  
    border: 1px solid rgba(0, 0, 0, 0.2);  
    border-radius: 15px;  
    overflow: hidden;  
    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);  
    z-index: 100;  
    min-width: 120px;  
}
```

```
.dropdownItem {  
    padding: 10px 24px;  
    font-size: 16px;  
    font-family: Arial, sans-serif;  
    font-weight: bold;  
    background-color: transparent;  
    color: #000000;  
    border: none;  
    cursor: pointer;
```



```
transition: all 0.3s ease;

width: 100%;

text-align: center;
}

.dropdownItem:hover {
  background-color: #BE8F4C;
}

.dropdownItem.active {
  background-color: #BE8F4C;
}

.filters {
  display: flex;
  gap: 10px;
  margin-left: 0;
  flex-wrap: wrap;
  justify-content: flex-end;
}

.rightGroup {
  display: flex;
  align-items: center;
}
```

```
/* ===== АДАПТИВ ДЛЯ ПЛАНШЕТОВ (768px - 1024px) ===== */
```

```
@media (max-width: 1024px) {
```

```
  .section {
```

```
    padding: 40px 60px 27px 60px;
```

```
  }
```

```
  .searchInput {
```

```
    font-size: 24px;
```

```
    height: 55px;
```

```
  }
```

```
  .searchInput::placeholder {
```

```
    font-size: 24px;
```

```
  }
```

```
  .searchButton {
```

```
    height: 55px;
```

```
    padding: 0 25px;
```

```
    font-size: 20px;
```

```
  }
```

```
  .typeButton {
```

```
    font-size: 24px;
```

```
    height: 45px;
```

```
padding: 6px 20px;

}

}

/* ===== АДАПТИВ ДЛЯ МОБИЛЬНЫХ (до 768px) ===== */

@media (max-width: 768px) {

  .section {

    padding: 20px 16px 20px 16px;

  }


  .searchWrapper {

    gap: 10px;

  }


  .searchInput {

    height: 50px;

    font-size: 18px;

    padding-left: 16px;

    border-radius: 15px;

  }


  .searchInput::placeholder {

    font-size: 18px;

  }

}
```

```
.searchButton {  
  height: 50px;  
  padding: 0 20px;  
  font-size: 16px;  
  border-radius: 15px;  
}
```

```
.chipContainer {  
  flex-direction: column;  
  height: auto;  
  gap: 12px;  
  margin-top: 20px;  
}
```

```
.tags {  
  justify-content: center;  
  width: 100%;  
  gap: 8px;  
}
```

```
.rightGroup {  
  display: none;  
}
```

```
.typeSelector {
```

```

    display: none;

}

}

/* ===== ДЛЯ ЭКРАНОВ 900px (фикс сломанных кнопок) ===== */

@media (max-width: 900px) {

    .section {

        padding: 30px 40px 27px 40px;

    }


    .chipContainer {

        flex-wrap: wrap;

    }


    .tags {

        flex-wrap: wrap;

        justify-content: center;

    }


    .rightGroup {

        width: 100%;

        justify-content: center;

    }


    .filters {

```

```

    justify-content: center;
}

.typeButton {
    font-size: 20px;
    height: 40px;
    padding: 4px 16px;
    white-space: nowrap;
}
}

/* ===== ДЛЯ ОЧЕНЬ МАЛЕНЬКИХ ЭКРАНОВ (до 480px) ===== */
@media (max-width: 480px) {
    .section {
        padding: 16px 12px 20px 12px;
    }

    .searchInput {
        height: 44px;
        font-size: 16px;
        border-radius: 12px;
    }

    .searchInput::placeholder {
        font-size: 16px;
    }
}

```

```
}
```

```
.searchButton {  
  height: 44px;  
  padding: 0 15px;  
  font-size: 14px;  
  border-radius: 12px;  
}
```

```
.tags {  
  gap: 6px;  
}  
}
```

```
/* ===== ДЛЯ БОЛЬШИХ ЭКРАНОВ (от 1400px) ===== */
```

```
@media (min-width: 1400px) {
```

```
  .searchInput {  
    font-size: 36px;  
    height: 70px;  
  }
```

```
  .searchInput::placeholder {  
    font-size: 36px;  
  }
```

```
.searchButton {  
    height: 70px;  
    padding: 0 40px;  
    font-size: 28px;  
}
```

```
.typeButton {  
    font-size: 36px;  
    height: 60px;  
}  
}
```

/ Стили для поиска пользователей - не влияют на существующий дизайн */*

```
.userSearchContainer {  
    position: relative;  
}
```

```
.userSearchDropdown {  
    position: absolute;  
    top: 70px;  
    right: 0;  
    width: 300px;  
    background-color: black;  
    border-radius: 10px;
```



```
box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);  
z-index: 1000;  
overflow: hidden;  
}
```

```
.userSearchInput {  
width: 100%;  
padding: 12px 15px;  
border: none;  
border-bottom: 1px solid #e0e0e0;  
outline: none;  
font-size: 16px;  
}
```

```
.userSearchResults {  
max-height: 350px;  
overflow-y: auto;  
}
```

```
.userResult {  
display: flex;  
align-items: center;  
gap: 12px;  
padding: 12px 15px;  
cursor: pointer;
```

```
    transition: background-color 0.2s;
}
```

```
.userResult:hover {
    background-color: black;
}
```

```
.userAvatar {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    overflow: hidden;
    background-color: #4C4C4C;
    flex-shrink: 0;
}
```

```
.userAvatar img {
    width: 100%;
    height: 100%;
    object-fit: cover;
}
```

```
.avatarPlaceholder {
    width: 100%;
    height: 100%;
}
```

```
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
}
```

```
.userInfo {  
  flex: 1;  
}
```

```
.userName {  
  font-weight: 600;  
  color: #333;  
  font-size: 14px;  
}
```

```
.userLogin {  
  font-size: 12px;  
  color: #888;  
  margin-top: 2px;  
}
```

```
.noUsers {  
  padding: 20px;  
  text-align: center;  
  color: #888;  
  font-size: 14px;  
}
```

```
@media (max-width: 768px) {
```

```
  .userSearchDropdown {
```

```
    position: fixed;
```

```
    top: auto;
```

```
    left: 50%;
```

```
    transform: translateX(-50%);
```

```
    width: 90%;
```

```
    max-width: 300px;
```

```
  }
```

```
}
```

```
/* Кнопка поиска пользователей - такой же стиль как у кнопки НАЙТИ */
```

```
.userSearchButton {
```

```
  height: 60px;
```

```
  padding: 0 30px;
```

```
  background-color: #BE8F4C;
```

```
  border: none;
```

```
  border-radius: 20px;
```

```
  color: black;
```

```
  font-size: 24px;
```

```
  font-weight: bold;
```

```
  cursor: pointer;
```

```
  transition: opacity 0.2s;
```

```
  white-space: nowrap;
```

```
}
```

```
.userSearchButton:hover {  
    opacity: 0.85;  
}
```

```
/* Оверлей для модального окна */
```

```
.userResultsOverlay {  
    position: fixed;  
    top: 0;  
    left: 0;  
    right: 0;  
    bottom: 0;  
    background-color: rgba(0, 0, 0, 0.5);  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    z-index: 1000;  
}
```

```
/* Модальное окно с результатами */
```

```
.userResultsModal {  
    background-color: #EAE7DC;  
    border-radius: 20px;  
    width: 90%;  
    max-width: 600px;
```

```
max-height: 80vh;

overflow: hidden;

box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);
}
```

```
.userResultsHeader {

display: flex;

justify-content: space-between;

align-items: center;

padding: 20px;

border-bottom: 1px solid #D8C3A5;

background-color: #D8C3A5;
}
```

```
.userResultsHeader h3 {

margin: 0;

font-size: 20px;

color: #000;
}
```

```
.closeButton {

background: none;

border: none;

font-size: 24px;

cursor: pointer;
```

```
color: #000;  
padding: 0 10px;  
}
```

```
.closeButton:hover {  
    color: #BE8F4C;  
}
```

```
.userResultsList {  
    padding: 15px;  
    max-height: 60vh;  
    overflow-y: auto;  
}
```

```
.userResultCard {  
    display: flex;  
    align-items: center;  
    gap: 15px;  
    padding: 15px;  
    background-color: #D8C3A5;  
    border-radius: 15px;  
    margin-bottom: 10px;  
    cursor: pointer;  
    transition: transform 0.2s, background-color 0.2s;  
}
```

```
.userResultCard:hover {  
    background-color: #c9b48c;  
    transform: translateX(5px);  
}
```

```
.userResultAvatar {  
    width: 60px;  
    height: 60px;  
    border-radius: 50%;  
    overflow: hidden;  
    background-color: #4C4C4C;  
    flex-shrink: 0;  
}
```

```
.userResultAvatar img {  
    width: 100%;  
    height: 100%;  
    object-fit: cover;  
}
```

```
.avatarImage {  
    width: 100%;  
    height: 100%;  
    object-fit: cover;
```



```
}
```

```
.avatarPlaceholder {  
  width: 100%;  
  height: 100%;  
  background: #555555;  
}
```

```
.userResultInfo {  
  flex: 1;  
}
```

```
.userResultName {  
  font-size: 18px;  
  font-weight: bold;  
  color: #000;  
  margin-bottom: 5px;  
}
```

```
.userResultLogin {  
  font-size: 14px;  
  color: #555;  
  margin-bottom: 5px;  
}
```

```
.userResultStats {  
    font-size: 12px;  
    color: #666;  
}
```

```
.loadingUsers {  
    text-align: center;  
    padding: 40px;  
    color: #888;  
}
```

```
.noUsers {  
    text-align: center;  
    padding: 40px;  
    color: #888;  
}
```

/ Адаптив для мобильных */*

```
@media (max-width: 768px) {  
    .userSearchButton {  
        height: 50px;  
        padding: 0 20px;  
        font-size: 18px;  
    }  
}
```

```
.userResultsModal {  
    width: 95%;  
}
```

```
.userResultAvatar {  
    width: 50px;  
    height: 50px;  
}
```

```
.userResultName {  
    font-size: 16px;  
}  
}
```

/ Кнопка поиска пользователей - такой же стиль как у кнопки НАЙТИ */*

```
.userSearchButton {  
    height: 60px;  
    padding: 0 30px;  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 20px;  
    color: black;  
    font-size: 24px;  
    font-weight: bold;  
    cursor: pointer;
```

```
transition: opacity 0.2s;  
white-space: nowrap;  
}
```

```
.userSearchButton:hover {  
    opacity: 0.85;  
}
```

```
/* Адаптив для мобильных */
```

```
@media (max-width: 768px) {  
    .userSearchButton {  
        height: 50px;  
        padding: 0 20px;  
        font-size: 18px;  
    }  
}
```

```
@media (min-width: 1400px) {  
    .userSearchButton {  
        height: 70px;  
        padding: 0 40px;  
        font-size: 28px;  
    }  
}
```

```
.filters {  
  display: flex;  
  gap: 10px;  
  align-items: center;  
}
```

```
.filters .chip {  
  height: 50px;  
  padding: 8px 24px;  
  font-size: 30px;  
  background-color: #D8C3A5;  
  border-radius: 10px;  
  display: flex;  
  align-items: center;  
  cursor: pointer;  
}
```

```
@media (max-width: 1024px) {  
  .filters .chip {  
    height: 45px;  
    font-size: 24px;  
    padding: 6px 20px;  
  }  
}
```

```
@media (max-width: 768px) {  
  .filters .chip {  
    height: 40px;  
    font-size: 20px;  
    padding: 4px 16px;  
  }  
}
```

/* Поиск пользователей - исправленный блок */

/* Контейнер поиска пользователей */

```
.userSearchContainer {  
  position: relative;  
}
```

/* Кнопка поиска пользователей () - такой же стиль как у кнопки НАЙТИ */

```
.searchButton:first-of-type {  
  background-color: #BE8F4C;  
}
```

/* Выпадающее окно поиска пользователей */

```
.userSearchDropdown {  
  position: absolute;  
  top: 70px;  
  right: 0;
```

```
width: 320px;

background-color: #D8C3A5;

border-radius: 20px;

box-shadow: 0 4px 15px rgba(0, 0, 0, 0.2);

z-index: 1000;

overflow: hidden;

}
```

```
/* Поле ввода в выпадающем окне */
```

```
.userSearchInput {

width: 100%;

padding: 12px 16px;

border: none;

border-bottom: 1px solid #BE8F4C;

outline: none;

font-size: 16px;

background-color: #D8C3A5;

color: #000000;

}
```

```
.userSearchInput::placeholder {

color: rgba(0, 0, 0, 0.5);

}
```

```
.userSearchInput:focus {
```

```
outline: none;

}

/* Список результатов */

.userSearchResults {

    max-height: 350px;

    overflow-y: auto;

}

/* Результат поиска */

.userResult {

    display: flex;

    align-items: center;

    gap: 12px;

    padding: 12px 16px;

    cursor: pointer;

    transition: background-color 0.2s;

    background-color: #D8C3A5;

}

.userResult:hover {

    background-color: #c9b48c;

}

/* Аватар пользователя */
```



```
.userAvatar {  
  width: 48px;  
  height: 48px;  
  border-radius: 50%;  
  overflow: hidden;  
  background-color: #4C4C4C;  
  flex-shrink: 0;  
}
```

```
.userAvatar img {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

```
.avatarPlaceholder {  
  width: 100%;  
  height: 100%;  
  background-color: #4C4C4C;  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  color: white;  
  font-size: 20px;  
  font-weight: bold;
```

```
}
```

```
/* Информация о пользователе */
```

```
.userInfo {
```

```
    flex: 1;
```

```
}
```

```
.userName {
```

```
    font-weight: 600;
```

```
    color: #000000;
```

```
    font-size: 14px;
```

```
    margin-bottom: 2px;
```

```
}
```

```
.userLogin {
```

```
    font-size: 12px;
```

```
    color: #555555;
```

```
    margin-top: 2px;
```

```
}
```

```
/* Состояние "нет пользователей" */
```

```
.noUsers {
```

```
    padding: 20px;
```

```
    text-align: center;
```

```
    color: #555555;
```

```
font-size: 14px;

background-color: #D8C3A5;

}
```

```
/* Адаптив для мобильных */
```

```
@media (max-width: 768px) {

.userSearchDropdown {

    position: fixed;

    top: auto;

    left: 50%;

    transform: translateX(-50%);

    width: 90%;

    max-width: 320px;

}
```

```
.userAvatar {

    width: 40px;

    height: 40px;

}

}
```

```
===== \components\common\Search\SearchSection\SearchSection.tsx
=====
```

```
import React, { useState, useEffect, useRef } from 'react';

import { useNavigate, useLocation } from 'react-router-dom';

import Chip from '../Chip/Chip';
```

```

import api from '../.../api/axios';

import styles from './SearchSection.module.css';

interface SearchSectionProps {
  onTypeChange?: (type: 'tests' | 'surveys') => void;
  showTypeSelector?: boolean;
  isMobile?: boolean;
  onTopicsClick?: () => void;
  selectedTopicsCount?: number;
}

const SearchSection = ({
  onTypeChange,
  showTypeSelector = false,
  isMobile = false,
  onTopicsClick,
  selectedTopicsCount = 0
}: SearchSectionProps) => {
  const navigate = useNavigate();
  const location = useLocation();

  const [selectedType, setSelectedType] = useState<'tests' | 'surveys'>('tests');
  const [isDropdownOpen, setIsDropdownOpen] = useState(false);
  const [searchQuery, setSearchQuery] = useState("");
  const [searchUsers, setSearchUsers] = useState<any[]>([]);
  const [showUserSearch, setShowUserSearch] = useState(false);

```

```

const [userSearchQuery, setUserSearchQuery] = useState("");
const userSearchRef = useRef<HTMLDivElement>(null);

const BASE_URL = 'http://localhost:8000';

// Функция для получения полного URL аватара
const getAvatarUrl = (avatar: string | null) => {
  if (!avatar) return null;
  if (avatar.startsWith('http://') || avatar.startsWith('https://')) {
    return avatar;
  }
  return `${BASE_URL}${avatar}`;
};

// Чтение параметров из URL при загрузке
useEffect(() => {
  const params = new URLSearchParams(location.search);
  const search = params.get('search');
  const type = params.get('type');

  if (search) setSearchQuery(search);
  if (type === 'test') setSelectedType('tests');
  if (type === 'survey') setSelectedType('surveys');
}, []);

```

// Поиск пользователей

```
useEffect(() => {  
  
  const searchUsersDebounce = setTimeout(async () => {  
  
    if (userSearchQuery.length >= 2) {  
  
      try {  
  
        const response = await api.get(`/users/search/?search=${userSearchQuery}`);  
  
        setSearchUsers(response.data);  
  
      } catch (error) {  
  
        console.error('Ошибка поиска пользователей:', error);  
  
      }  
  
    } else {  
  
      setSearchUsers([]);  
  
    }  
  
  }, 300);  
  
  return () => clearTimeout(searchUsersDebounce);  
}, [userSearchQuery]);
```

// Заккрытие поиска пользователей при клике вне

```
useEffect(() => {  
  
  const handleClickOutside = (event: MouseEvent) => {  
  
    if (userSearchRef.current && !userSearchRef.current.contains(event.target as Node))  
  {  
  
    setShowUserSearch(false);  
  
    setUserSearchQuery("");  
  
  }  
}
```

```

    }
  };

  document.addEventListener('mousedown', handleClickOutside);

  return () => document.removeEventListener('mousedown', handleClickOutside);
}, []);

```

```

const handleTypeSelect = (type: 'tests' | 'surveys') => {
  setSelectedType(type);
  setIsDropdownOpen(false);
  if (onTypeChange) {
    onTypeChange(type);
  }
};

```

```

const handleSearch = () => {
  const params = new URLSearchParams();
  if (searchQuery) params.append('search', searchQuery);
  if (selectedType === 'tests') params.append('type', 'test');
  if (selectedType === 'surveys') params.append('type', 'survey');

  navigate(`/testlist?${params.toString()}`);
};

```

```

const handleKeyPress = (e: React.KeyboardEvent) => {
  if (e.key === 'Enter') {

```

```

    handleSearch();
  }
};

```

```

const handleClick = (userId: number) => {
  setShowUserSearch(false);
  setUserSearchQuery("");
  navigate(`/profile/${userId}`);
};

```

```

return (
  <section className={styles.section}>
    <div className={styles.searchWrapper}>
      <input
        placeholder='🔍 ПОИСК ТЕСТОВ...'
        className={styles.searchInput}
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
        onKeyDown={handleKeyPress}
      />
      <button className={styles.searchButton} onClick={handleSearch}>
        НАЙТИ
      </button>

      {/* Кнопка поиска пользователей */}
    </div>
  </section>
);

```



```

<div className={styles.userSearchContainer} ref={userSearchRef}>

  <button

    className={styles.searchButton}

    onClick={() => setShowUserSearch(!showUserSearch)}

  >

```



```

</button>

{showUserSearch && (

  <div className={styles.userSearchDropdown}>

    <input

      type="text"

      placeholder="Поиск пользователей..."

      value={userSearchQuery}

      onChange={(e) => setUserSearchQuery(e.target.value)}

      className={styles.userSearchInput}

      autoFocus

    />

    <div className={styles.userSearchResults}>

      {searchUsers.length === 0 && userSearchQuery.length >= 2 && (

        <div className={styles.noUsers}>Пользователи не найдены</div>

      )}

      {searchUsers.map(user => {

        const avatarUrl = getAvatarUrl(user.avatar);

        const userName = user.full_name || user.login;

        const firstLetter = userName.charAt(0).toUpperCase();

```

```

return (
  <div
    key={user.id}
    className={styles.userResult}
    onClick={() => handleUserClick(user.id)}
  >
    <div className={styles.userAvatar}>
      {avatarUrl ? (
        <img
          src={avatarUrl}
          alt={user.login}
          onError={(e) => {
            e.currentTarget.style.display = 'none';

e.currentTarget.parentElement?.querySelector('.avatarPlaceholder').classList.add(styles.s
how);

          }}
        />
      ) : null}
      <div className={` ${styles.avatarPlaceholder} ${!avatarUrl ?
styles.show : ""}`>
        {firstLetter}
      </div>
    </div>
  <div className={styles.userInfo}>

```

```

        <div className={styles.userName}>{userName}</div>

        <div className={styles.userLogin}>@{user.login}</div>

    </div>

</div>

);

}}

</div>

</div>

)}

</div>

</div>

```

```

<div className={styles.chipContainer}>

    <div className={styles.tags}>

        <Chip

            label="HOBBIE"

            type="tag"

            onClick={() => {

                const params = new URLSearchParams();

                if (searchQuery) params.append('search', searchQuery);

                params.append('sort', 'new');

                if (selectedType === 'tests') params.append('type', 'test');

                if (selectedType === 'surveys') params.append('type', 'survey');

                navigate(`/testlist?${params.toString()}`);

            }}

        </Chip>

    </div>

</div>

```

/>

<Chip

label="ЛУЧШИЕ"

type="tag"

onClick={() => {

const params = new URLSearchParams();

if (searchQuery) params.append('search', searchQuery);

params.append('sort', 'popular');

if (selectedType === 'tests') params.append('type', 'test');

if (selectedType === 'surveys') params.append('type', 'survey');

navigate(`/testlist?\${params.toString()}`);

}}

/>

<Chip

label="СТАРИЕ"

type="tag"

onClick={() => {

const params = new URLSearchParams();

if (searchQuery) params.append('search', searchQuery);

params.append('sort', 'old');

if (selectedType === 'tests') params.append('type', 'test');

if (selectedType === 'surveys') params.append('type', 'survey');

navigate(`/testlist?\${params.toString()}`);

}}

/>

```

<Chip
  label={`ТЕМЫ${selectedTopicsCount ? ` (${selectedTopicsCount})` : ''}`}
  type="tag"
  onClick={onTopicsClick}
/>
</div>

```

```

{!isMobile && (
  <div className={styles.rightGroup}>
    <div className={styles.filters}>
      {showTypeSelector && (
        <div className={styles.typeSelector}>
          <button
            className={styles.typeButton}
            onClick={() => setIsDropdownOpen(!isDropdownOpen)}
          >
            ТИПЫ ТЕКСТОВ: {selectedType === 'tests' ? 'ТЕСТ' : 'ОПРОС'}
          </button>

          {isDropdownOpen && (
            <div className={styles.dropdown}>
              <button
                className={` ${styles.dropdownItem} ${selectedType === 'tests' ?
styles.active : ''}`}
                onClick={() => handleTypeSelect('tests')}

```

>

ТИПЫ ТЕСТОВ: ТЕСТ

</button>

<button

className={` \${styles.dropdownItem} \${selectedType === 'surveys' ?
styles.active : ''}`}

onClick={() => handleTypeSelect('surveys')}

>

ТИПЫ ТЕСТОВ: ОПРОС

</button>

</div>

)}

</div>

)}

<Chip

label="A ↓ Я"

type="filter"

onClick={() => {

const params = new URLSearchParams();

if (searchQuery) params.append('search', searchQuery);

params.append('sort', 'az');

if (selectedType === 'tests') params.append('type', 'test');

if (selectedType === 'surveys') params.append('type', 'survey');

navigate(`/testlist?\${params.toString()}`);

```

        }}
      />
    </div>

  </div>

)}

</div>

</section>

);

};

export default SearchSection;

===== \components\common\Test\TestMain.tsx =====

import React, { useState, useEffect } from 'react';

import { Link, useParams, useNavigate } from 'react-router-dom';

import { getTestDetails, startTest, getComments, addComment, submitReport,
getTestRating } from '../../api/tests';

import ReportModal from '../../components/common/ReportModal/ReportModal';

import AuthorChip from '../../components/common/Chip/userChip/AuthorChip';

import api from '../../api/axios';

import styles from './TestPage.module.css';

const TestPage = () => {

  const { testId } = useParams<{ testId: string }>();

  const navigate = useNavigate();

  const [test, setTest] = useState<any>(null);

```

```

const [comments, setComments] = useState<any[]>([]);

const [loading, setLoading] = useState(true);

const [testRating, setTestRating] = useState<{ average_rating: number; ratings_count:
number; user_rating: number | null }>({
    average_rating: 0,
    ratings_count: 0,
    user_rating: null
});

const [isReportModalOpen, setIsReportModalOpen] = useState(false);

const [currentReportType, setCurrentReportType] = useState<'test' | 'comment' |
null>(null);

const [currentCommentId, setCurrentCommentId] = useState<number | null>(null);

const [newComment, setNewComment] = useState("");

const [isAdmin, setIsAdmin] = useState(false);

const BASE_URL = 'http://localhost:8000';

const testReportReasons = ["Несоответствие теме", "Неприемлемый контент",
"Спам", "Другое"];

const commentReportReasons = ["Оскорбительное поведение", "Спам", "Другое"];

useEffect(() => {
    if (testId) {
        loadTestData();
        loadComments();
        loadTestRating();
    }

```



```

        checkAdminStatus();
    }
}, [testId]);

// Проверка статуса админа - ТОЛЬКО ЕСЛИ ЕСТЬ ТОКЕН
const checkAdminStatus = async () => {
    const token = localStorage.getItem('access_token');
    if (!token) {
        setIsAdmin(false);
        return;
    }

    try {
        const profile = await api.get('/auth/profile/');
        setIsAdmin(profile.data.status === 'admin');
    } catch (error) {
        console.error('Ошибка проверки админа:', error);
        setIsAdmin(false);
    }
};

// Загрузка данных теста
const loadTestData = async () => {
    try {
        const data = await getTestDetails(Number(testId));
    }
};

```

```

        setTest(data);
    } catch (error) {
        console.error('Ошибка загрузки теста:', error);
    } finally {
        setLoading(false);
    }
};

```

// Загрузка комментариев

```

const loadComments = async () => {
    try {
        const data = await getComments(Number(testId));
        setComments(data);
    } catch (error) {
        console.error('Ошибка загрузки комментариев:', error);
    }
};

```

// Загрузка рейтинга

```

const loadTestRating = async () => {
    try {
        const data = await getTestRating(Number(testId));
        setTestRating(data);
    } catch (error) {
        console.error('Ошибка загрузки рейтинга:', error);
    }
};

```

```

    }
};

// Начало прохождения теста
const handleStart = async () => {
  try {
    const response = await startTest(Number(testId));
    navigate(`/test/${testId}/question/1`, { state: { attemptId: response.attempt_id } });
  } catch (error: any) {
    alert(`Не удалось начать тест: ${error.response?.data?.error || 'Неизвестная ошибка'}`);
  }
};

// Добавление комментария
const handleAddComment = async () => {
  if (!newComment.trim()) return;
  try {
    await addComment(Number(testId), newComment);
    setNewComment("");
    await loadComments();
  } catch (error) {
    alert('Не удалось добавить комментарий');
  }
};

```

```

// Удаление теста (только для админа)

const handleDeleteTest = async () => {

  if (window.confirm('Вы уверены, что хотите удалить этот тест?')) {

    try {

      await api.delete(`/tests/${testId}/delete/`);

      alert('Тест успешно удалён');

      navigate('/testlist');

    } catch (error: any) {

      alert(`Ошибка при удалении теста: ${error.response?.data?.error ||
error.message}`);

    }

  }

};

// Удаление комментария (только для админа)

const handleDeleteComment = async (commentId: number) => {

  if (window.confirm('Вы уверены, что хотите удалить этот комментарий?')) {

    try {

      await api.delete(`/comments/${commentId}/`);

      alert('Комментарий успешно удалён');

      await loadComments();

    } catch (error: any) {

      console.error('Ошибка при удалении комментария:', error);

      alert(`Ошибка при удалении комментария: ${error.response?.data?.error ||

```

```
error.message}``);
```

```
    }
```

```
  }
```

```
};
```

```
// Открытие модальной жалобы
```

```
const openReportModal = (type: 'test' | 'comment', commentId?: number) => {
```

```
  setCurrentReportType(type);
```

```
  if (commentId) setCurrentCommentId(commentId);
```

```
  setIsReportModalOpen(true);
```

```
};
```

```
const closeReportModal = () => {
```

```
  setIsReportModalOpen(false);
```

```
  setCurrentReportType(null);
```

```
  setCurrentCommentId(null);
```

```
};
```

```
// Отправка жалобы
```

```
const handleSubmitReport = async (reason: string, comment: string) => {
```

```
  const token = localStorage.getItem('access_token');
```

```
  if (!token) {
```

```
    const confirm = window.confirm('Вы не авторизованы. Хотите перейти на  
страницу входа?');
```

```
    if (confirm) navigate('/login');
```

```

    return;
}

try {
    if (currentReportType === 'comment') {
        await submitReport({
            target_type: 'comment',
            target_id: currentCommentId!,
            test_id: Number(testId),
            reason: reason,
            comment: comment
        });
    } else {
        await submitReport({
            target_type: 'test',
            target_id: Number(testId),
            reason: reason,
            comment: comment
        });
    }

    alert('Жалоба отправлена');

    closeReportModal();
} catch (error: any) {
    alert(`Ошибка: ${error.response?.data?.error || error.message}`);
}

```

```
};
```

```
const getModalProps = () => {  
  if (currentReportType === 'test') {  
    return { title: 'Пожаловаться на тест', reasons: testReportReasons };  
  }  
  return { title: 'Пожаловаться на комментарий', reasons: commentReportReasons };  
};
```

```
if (loading) {  
  return (  
    <div className={styles.page}>  
      <div className={styles.container}>  
        <div className={styles.loading}>Загрузка...</div>  
      </div>  
    </div>  
  );  
}
```

```
if (!test) {  
  return (  
    <div className={styles.page}>  
      <div className={styles.container}>  
        <div className={styles.error}>Тест не найден</div>  
      </div>  
    </div>  
  );  
}
```

```

    </div>

    );
}

const displayAuthor = test.author_info?.full_name || test.author_info?.login || 'автор';
const timeDisplay = test.time_limit ? `${test.time_limit} минут` : 'без ограничения';
const typeDisplay = test.is_survey ? 'опрос' : 'тест';
const averageRating = testRating.average_rating || 0;
const ratingsCount = testRating.ratings_count || 0;
const attemptsCount = test.attempts_count || 0;

const imageFullUrl = test.image ? `${BASE_URL}${test.image}` : null;
const modalProps = getModalProps();

return (
  <div className={styles.page}>
    <div className={styles.container}>
      <div className={styles.testInfoBlock}>
        <div className={styles.imageContainer}>
          {imageFullUrl ? (
            <img
              src={imageFullUrl}
              alt={test.title}
              className={styles.testImage}
              onError={(e) => {

```



```

        e.currentTarget.style.display = 'none';

    }}

/>

):(

    <div className={styles.imagePlaceholder} />

)}

<div className={styles.imageButtons}>

    {isAdmin ? (

        <button

            className={styles.deleteTestButton}

            onClick={handleDeleteTest}

        >

            Удалить тест

        </button>

    ) : (

        <button

            className={styles.reportButton}

            onClick={() => openReportModal('test')}

        >

            !

        </button>

    )}

</div>

</div>

```

```

<div className={styles.authorStatsRow}>

  <AuthorChip

    authorId={test.author_info?.id}

    authorName={displayAuthor}

    authorAvatar={test.author_info?.avatar}

  />

  <div className={styles.statsInfo}>

    <span> ⭐ {averageRating.toFixed(1)}/5 ({ratingsCount})</span>

    <span> 👤 {attemptsCount}</span>

  </div>

</div>

<div className={styles.titleSection}>

  <h2 className={styles.testTitle}>{test.title}</h2>

  <span className={styles.testDescription}>{test.description || 'Нет
описания'}</span>

  <button onClick={handleStart} className={styles.startButton}>

    начать

  </button>

</div>

<hr className={styles.divider} />

<div className={styles.testDetails}>

  <span> ⌚ {timeDisplay}</span>

```

```

        <span> 📋 {test.questions_count} вопросов</span>

        <span> 📊 {typeDisplay}</span>

    </div>

</div>

<div className={styles.commentsBlock}>

    <h3 className={styles.commentsTitle}>Комментарии</h3>

    <div className={styles.commentsList}>

        {comments.length === 0 ? (

            <div className={styles.noComments}>Нет комментариев. Будьте
первым!</div>

        ) : (

            comments.map((comment) => (

                <div key={comment.id} className={styles.commentItem}>

                    <div className={styles.commentTop}>

                        <Link to={` /profile/${comment.user_id ||
comment.user_info?.id}`} className={styles.commentUserLink}>

                            <div className={styles.testCommentUser}>

                                <div className={styles.avatarComment}>

                                    {comment.user_info?.avatar ? (

                                        <img

                                            src={comment.user_info.avatar.startsWith('http') ?
comment.user_info.avatar : `${BASE_URL}${comment.user_info.avatar}`}

                                            alt={comment.user_info?.full_name ||
comment.user_info?.login}

```

```

        className={styles.avatarImageSmall}

        onError={(e) => {
            e.currentTarget.style.display = 'none';
        }}
    />

    ) : (
        <div
            className={styles.avatarPlaceholderComment} />
        >
    </div>

    <span>{comment.user_info?.full_name ||
comment.user_info?.login}</span>

    </div>

</Link>

<div className={styles.commentActions}>
    {isAdmin ? (
        <button
            className={styles.deleteCommentButton}
            onClick={() => handleDeleteComment(comment.id)}
        >
            ✕
        </button>
    ) : (
        <button
            className={styles.reportCommentButton}

```

```

        onClick={() => openReportModal('comment',
comment.id)}

        >

        !

    </button>

    })

</div>

</div>

<div className={styles.commentBody}>

    <span>{comment.text}</span>

</div>

</div>

))

)}

</div>

</div>

</div>

<ReportModal

    isOpen={isReportModalOpen}

    onClose={closeReportModal}

    onSubmit={handleSubmitReport}

    title={modalProps.title}

    reasons={modalProps.reasons}

/>

```

```

        </div>

    );

};

export default TestPage;

===== \components\common\Test\testPage.module.css =====

.page {
    background-color: #EAE7DC;
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 40px;
}

.container {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 30px;
    max-width: 700px;
    width: 100%;
}

/* Блок информации о тесте */

```

```
.testInfoBlock {  
    width: 700px;  
    background-color: #D8C3A5;  
    border-radius: 10px;  
    display: flex;  
    flex-direction: column;  
    overflow: hidden;  
}  
  
/* Контейнер для изображения */  
.imageContainer {  
    width: 100%;  
    height: 330px;  
    background-color: #4C4C4C;  
    position: relative;  
    overflow: hidden;  
}  
  
.testImage {  
    width: 100%;  
    height: 100%;  
    object-fit: cover;  
    display: block;  
}
```

```
.imagePlaceholder {  
    width: 100%;  
    height: 100%;  
    background-color: #4C4C4C;  
}
```

```
/* Кнопки на картинке */
```

```
.imageButtons {  
    position: absolute;  
    top: 15px;  
    right: 15px;  
    display: flex;  
    gap: 10px;  
    z-index: 10;  
}
```

```
/* Кнопка репорта */
```

```
.reportButton {  
    width: 45px;  
    height: 45px;  
    background-color: #BE8F4C;  
    border: none;  
    border-radius: 10px;  
    color: black;  
    font-size: 20px;
```



```
font-weight: bold;

cursor: pointer;

display: flex;

align-items: center;

justify-content: center;

transition: background-color 0.3s;

}
```

```
.reportButton:focus {

    outline: none;

}
```

```
.reportButton:hover {

    background-color: #a47a3f;

}
```

```
/* Кнопка удаления теста */

.deleteTestButton {

    width: auto;

    min-width: 120px;

    height: 45px;

    padding: 0 15px;

    background-color: #BE8F4C;

    border: none;

    border-radius: 10px;
```

```
color: black;

font-size: 14px;

font-weight: bold;

cursor: pointer;

display: flex;

align-items: center;

justify-content: center;

transition: background-color 0.3s;

}
```

```
.deleteTestButton:focus {

    outline: none;

}
```

```
.deleteTestButton:hover {

    background-color: #a47a3f;

}
```

```
/* Строка с автором и статистикой */
```

```
.authorStatsRow {

    display: flex;

    justify-content: space-between;

    align-items: center;

    padding: 12px 20px;

}
```

```
.testCreator {  
  display: flex;  
  align-items: center;  
  gap: 10px;  
  background-color: #BE8F4C;  
  padding: 5px 15px 5px 8px;  
  border-radius: 10px;  
  transition: background-color 0.3s ease;  
  cursor: pointer;  
}
```

```
.testCreator:hover {  
  background-color: #8B6B3A;  
}
```

```
.avatarSmall {  
  width: 30px;  
  height: 30px;  
  border-radius: 50%;  
  overflow: hidden;  
  flex-shrink: 0;  
}
```

```
.avatarImage {
```

```
width: 100%;  
height: 100%;  
object-fit: cover;  
}
```

```
.avatarPlaceholderSmall {  
width: 100%;  
height: 100%;  
background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);  
}
```

```
.authorLink {  
text-decoration: none !important;  
color: black;  
}
```

```
.authorLink:hover {  
text-decoration: none !important;  
color: black;  
}
```

```
.statsInfo {  
display: flex;  
gap: 15px;  
}
```

```
.statsInfo span {  
    font-size: 16px;  
    color: black;  
}
```

```
/* Секция с названием и описанием */
```

```
.titleSection {  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    padding: 0 30px 15px 30px;  
}
```

```
.testTitle {  
    font-size: 40px;  
    margin: 0 0 10px 0;  
    text-align: center;  
    color: black;  
}
```

```
.testDescription {  
    font-size: 18px;  
    text-align: center;
```

```
color: black;

margin-bottom: 40px;

word-wrap: break-word;

max-width: 100%;

}


.startButton {

background-color: #BE8F4C;

border: none;

border-radius: 10px;

padding: 15px 60px;

font-size: 20px;

font-weight: bold;

color: black;

cursor: pointer;

transition: background-color 0.3s;

}


.startButton:focus {

outline: none;

}


.startButton:hover {

background-color: #a47a3f;

}
```

```
/* Разделитель */
```

```
.divider {  
    width: 90%;  
    margin: 0 auto;  
    border: none;  
    height: 1px;  
    background-color: black;  
}
```

```
/* Детали теста */
```

```
.testDetails {  
    display: flex;  
    justify-content: space-around;  
    padding: 20px 30px;  
    font-size: 16px;  
    color: black;  
}
```

```
/* Блок комментариев */
```

```
.commentsBlock {  
    width: 700px;  
    background-color: #D8C3A5;  
    border-radius: 10px;  
    display: flex;
```

```
flex-direction: column;

overflow: hidden;

}
```

```
.commentsTitle {

font-size: 30px;

margin: 20px 0 20px 30px;

color: black;

font-weight: 100;

}
```

```
.commentsList {

padding: 0 20px 20px 20px;

display: flex;

flex-direction: column;

gap: 15px;

}
```

```
/* Один комментарий */
```

```
.commentItem {

background-color: rgba(255, 215, 157, 0.5);

border-radius: 10px;

padding: 0 0 15px 0;

box-shadow: 0 4px 8px rgba(0, 0, 0, 0.15);

transition: box-shadow 0.3s ease;
```



```
}
```

```
.commentItem:hover {  
    box-shadow: 0 6px 12px rgba(0, 0, 0, 0.2);  
}
```

```
.commentTop {  
    display: flex;  
    justify-content: space-between;  
    position: relative;  
    align-items: center;  
    margin-bottom: 10px;  
}
```

```
.testCommentUser {  
    display: flex;  
    align-items: center;  
    gap: 10px;  
    background-color: #BE8F4C;  
    padding: 5px 15px 5px 8px;  
    border-radius: 0 0 10px 0;  
    transition: background-color 0.3s ease;  
    cursor: pointer;  
}
```

```
.testCommentUser:hover {  
  background-color: #8B6B3A;  
}
```

```
.avatarComment {  
  width: 30px;  
  height: 30px;  
  border-radius: 50%;  
  overflow: hidden;  
  flex-shrink: 0;  
}
```

```
.avatarImageSmall {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}
```

```
.avatarPlaceholderComment {  
  width: 100%;  
  height: 100%;  
  background: linear-gradient(135deg, #f093fb 0%, #f5576c 100%);  
}
```

```
.commentUserLink {
```

```
text-decoration: none !important;

color: black;

}
```

```
.commentUserLink:hover {

text-decoration: none !important;

color: black;

}
```

```
/* Действия с комментариями */
```

```
.commentActions {

display: flex;

gap: 8px;

margin-right: 15px;

}
```

```
/* Кнопка репорта комментария */
```

```
.reportCommentButton {

display: flex;

align-items: center;

justify-content: center;

width: 30px;

height: 30px;

background-color: #BE8F4C;

border: none;
```

```
border-radius: 8px;

color: black;

font-size: 16px;

font-weight: bold;

cursor: pointer;

transition: background-color 0.3s;

}
```

```
.reportCommentButton:focus {

    outline: none;

}
```

```
.reportCommentButton:hover {

    background-color: #a47a3f;

}
```

/ Кнопка удаления комментария */*

```
.deleteCommentButton {

    width: 30px;

    height: 30px;

    background-color: #BE8F4C;

    border: none;

    border-radius: 8px;

    color: black;

    font-size: 16px;
```

```
font-weight: bold;

cursor: pointer;

display: flex;

align-items: center;

justify-content: center;

transition: background-color 0.3s;

}
```

```
.deleteCommentButton:focus {

    outline: none;

}
```

```
.deleteCommentButton:hover {

    background-color: #a47a3f;

}
```

```
.commentBody {

    word-wrap: break-word;

    padding: 0 15px;

}
```

```
.commentBody span {

    font-size: 14px;

    color: black;

    line-height: 1.4;
```

```
}
```

```
.noComments {  
    text-align: center;  
    padding: 30px;  
    color: #666;  
    font-style: italic;  
}
```

```
/* Адаптивность */
```

```
@media (max-width: 768px) {  
    .page {  
        padding: 20px 15px;  
    }  
}
```

```
.testInfoBlock,  
.commentsBlock {  
    width: 100%;  
}
```

```
.imageContainer {  
    height: 280px;  
}
```

```
.testTitle {
```

```
    font-size: 32px;  
}
```

```
.testDescription {  
    font-size: 16px;  
    margin-bottom: 30px;  
}
```

```
.startButton {  
    padding: 12px 40px;  
    font-size: 18px;  
}
```

```
.commentsTitle {  
    font-size: 24px;  
    margin-left: 20px;  
}
```

```
.imageButtons {  
    top: 10px;  
    right: 10px;  
    gap: 8px;  
}
```

```
.deleteTestButton {
```

```
    min-width: 100px;

    height: 40px;

    font-size: 12px;
}
```

```
.reportButton {

    width: 40px;

    height: 40px;

    font-size: 18px;
}
}
```

```
@media (max-width: 480px) {

    .page {

        padding: 15px;
    }
}
```

```
.imageContainer {

    height: 220px;
}
```

```
.testTitle {

    font-size: 24px;
}
```



```
.testDescription {  
  font-size: 14px;  
}
```

```
.startButton {  
  padding: 10px 30px;  
  font-size: 16px;  
}
```

```
.testDetails {  
  font-size: 12px;  
  padding: 15px;  
}
```

```
.commentsTitle {  
  font-size: 20px;  
}
```

```
.authorStatsRow {  
  flex-direction: column;  
  gap: 10px;  
  align-items: flex-start;  
}
```

```
.statsInfo {
```

```
    align-self: flex-end;
}
```

```
.imageButtons {
    top: 8px;
    right: 8px;
    gap: 6px;
}
```

```
.deleteTestButton {
    min-width: 80px;
    height: 35px;
    font-size: 11px;
    padding: 0 10px;
}
```

```
.reportButton {
    width: 35px;
    height: 35px;
    font-size: 16px;
}
```

```
.reportCommentButton,
.deleteCommentButton {
    width: 25px;
```

```
height: 25px;

font-size: 12px;

}

}

/* Очень маленькие телефоны */

@media (max-width: 360px) {

.testTitle {

font-size: 20px;

}

.testDescription {

font-size: 12px;

}

.startButton {

padding: 8px 20px;

font-size: 14px;

}

.testDetails {

font-size: 10px;

padding: 10px;

}
```

```
.commentsTitle {  
    font-size: 18px;  
    margin-left: 15px;  
}
```

```
.deleteTestButton {  
    min-width: 70px;  
    height: 30px;  
    font-size: 10px;  
}
```

```
.reportButton {  
    width: 30px;  
    height: 30px;  
    font-size: 14px;  
}
```

```
}
```

===== \components\common\TestList\TestsList.module.css =====

```
/* src/pages/TestsList/TestsList.module.css */
```

```
.page {  
    width: 100%;  
    min-height: 100vh;  
    background-color: #EAE7DC;  
}
```

```
.main {  
    flex: 1;  
    padding-top: 30px; /* Добавляем отступ сверху */  
}
```

```
.container {  
    max-width: 1648px;  
    margin: 0 auto;  
    padding: 0 0 40px 0;  
    width: 100%;  
    box-sizing: border-box;  
}
```

```
/* Сетка карточек - 4 в ряд, как на главной странице */
```

```
.cardsGrid {  
    display: grid;  
    grid-template-columns: repeat(4, 1fr);  
    gap: 25px;  
    margin-bottom: 30px;  
}
```

```
/* Карточки растягиваются на всю ширину */
```

```
.cardsGrid > * {  
    width: 100%;
```

```
}
```

```
/* Элемент загрузки */
```

```
.loadMore {  
    text-align: center;  
    padding: 20px;  
    color: #000000;  
    font-size: 14px;  
    font-family: Arial, sans-serif;  
    opacity: 0.7;  
}
```

```
/* Статус загрузки */
```

```
.loadingStatus {  
    text-align: center;  
    padding: 15px;  
    color: #000000;  
    font-size: 14px;  
    font-family: Arial, sans-serif;  
    opacity: 0.6;  
    border-top: 1px solid #D8C3A5;  
    margin-top: 10px;  
}
```

```
/* Адаптивность */
```

```
@media (max-width: 1400px) {  
  .container {  
    padding: 0 50px 40px 50px;  
  }  
}
```

```
.cardsGrid {  
  gap: 20px;  
}  
}
```

```
@media (max-width: 1200px) {  
  .cardsGrid {  
    grid-template-columns: repeat(3, 1fr);  
    gap: 20px;  
  }  
}
```

```
@media (max-width: 900px) {  
  .cardsGrid {  
    grid-template-columns: repeat(2, 1fr);  
    gap: 20px;  
  }  
}
```

```
.container {  
  padding: 0 30px 40px 30px;
```

```

    }
}

@media (max-width: 600px) {
    .cardsGrid {
        grid-template-columns: 1 fr;
        gap: 20px;
    }
}

/* Скрываем SearchSection на мобильных экранах */
.hideOnMobile {
    display: block;
}

@media (max-width: 768px) {
    .hideOnMobile {
        display: none;
    }

    .main {
        padding-top: 20px; /* Уменьшаем отступ на мобильных */
    }
}

```



```
@media (max-width: 480px) {  
  .main {  
    padding-top: 15px; /* Еще меньше на маленьких телефонах */  
  }  
}
```

```
.modalOverlay {  
  position: fixed;  
  top: 0;  
  left: 0;  
  right: 0;  
  bottom: 0;  
  background-color: rgba(0, 0, 0, 0.5);  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  z-index: 1000;  
}
```

```
.modalContent {  
  background-color: #EAE7DC;  
  border-radius: 20px;  
  padding: 30px;  
  width: 90%;  
  max-width: 500px;
```

```
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
}
```

```
.modalTitle {  
    font-size: 24px;  
    color: #000;  
    margin-bottom: 20px;  
    text-align: center;  
}
```

```
.topicsGrid {  
    display: grid;  
    grid-template-columns: repeat(2, 1fr);  
    gap: 15px;  
    margin-bottom: 30px;  
}
```

```
.topicButton {  
    padding: 12px;  
    border: 2px solid #D8C3A5;  
    border-radius: 10px;  
    background-color: #D8C3A5;  
    font-size: 16px;  
    cursor: pointer;  
    transition: all 0.2s;
```

```
}
```

```
.topicButton:hover {  
  background-color: #c9b48c;  
}
```

```
.topicButton.active {  
  background-color: #BE8F4C;  
  border-color: #BE8F4C;  
  color: white;  
}
```

```
.modalButtons {  
  display: flex;  
  justify-content: flex-end;  
  gap: 15px;  
}
```

```
.clearButton {  
  padding: 10px 20px;  
  border: none;  
  border-radius: 8px;  
  background-color: #ccc;  
  cursor: pointer;  
}
```

```
.applyButton {  
    padding: 10px 20px;  
    border: none;  
    border-radius: 8px;  
    background-color: #BE8F4C;  
    color: white;  
    cursor: pointer;  
}
```

```
/* Контейнер загрузки */
```

```
.loadingContainer {  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    min-height: 60vh;  
    padding: 40px;  
    gap: 20px;  
}
```

```
/* Картинка загрузки */
```

```
.loadingImage {  
    width: 200px;  
    height: auto;
```

```

    object-fit: contain;

}

/* Текст загрузки */

.loadingText {
    font-size: 24px;
    font-weight: bold;
    color: #000000;
    text-align: center;
    letter-spacing: 2px;
}

===== \components\common\TestList\TestsList.tsx =====

import React, { useState, useEffect, useRef } from 'react';
import { useLocation, useNavigate } from 'react-router-dom';
import SearchSection from '../Search/SearchSection/SearchSection';
import Card from '../Card/TestCard/Card';
import { getTests, getSurveys, getTestRating } from '../../api/tests';
import styles from './TestsList.module.css';

const TestsList = () => {
    const location = useLocation();
    const navigate = useNavigate();

    const [selectedType, setSelectedType] = useState<'tests' | 'surveys'>('tests');
    const [displayedCards, setDisplayedCards] = useState(12);
    const [allCards, setAllCards] = useState<any[]>([]);

```

```

const [loading, setLoading] = useState(true);

const loadMoreRef = useRef<HTMLDivElement>(null);

const [isLoadingMore, setIsLoadingMore] = useState(false);

const [isTopicsModalOpen, setIsTopicsModalOpen] = useState(false);

const [selectedTopics, setSelectedTopics] = useState<string[]>([]);

const BASE_URL = 'http://localhost:8000';

const topicsList = ['наука', 'спорт', 'сериалы', 'анимации', 'игры'];

// Чтение параметров из URL
useEffect(() => {

  const params = new URLSearchParams(location.search);

  const type = params.get('type');

  const search = params.get('search');

  const sort = params.get('sort');

  const topics = params.get('topics');

  if (type === 'test') setSelectedType('tests');

  if (type === 'survey') setSelectedType('surveys');

  if (topics) {

    setSelectedTopics(topics.split(','));

  } else {

    setSelectedTopics([]);

  }
}

```

```

    loadCards(search, sort, topics);

}, [location.search]);

// Загружаем карточки при изменении selectedType - СБРАСЫВАЕМ ТЕМЫ
useEffect(() => {

    const params = new URLSearchParams(location.search);

    const search = params.get('search');

    const sort = params.get('sort');

    // ОЧИЩАЕМ ТЕМЫ ПРИ СМЕНЕ ТИПА

    params.delete('topics');

    setSelectedTopics([]);

    navigate(`?${params.toString()}`, { replace: true });

    loadCards(search, sort, null);

}, [selectedType]);

const loadCards = async (search?: string | null, sort?: string | null, topics?: string | null)
=> {

    setLoading(true);

    try {

        let data;

        if (selectedType === 'tests') {

            data = await getTests();

        } else {

            data = await getSurveys();

        }

    }

```

```

// Фильтрация по поиску

let filteredData = data;

if (search) {

  filteredData = data.filter((test: any) =>

    test.title.toLowerCase().includes(search.toLowerCase())

  );

}


// Фильтрация по темам

if (topics) {

  const topicArray = topics.split(',');

  filteredData = filteredData.filter((test: any) =>

    test.topics && topicArray.some(topic => test.topics.includes(topic))

  );

}


// Добавляем рейтинг для КАЖДОГО теста

const cardsWithData = await Promise.all(filteredData.map(async (test: any) => {

  try {

    const ratingData = await getTestRating(test.id);

    const imageFullUrl = test.image ?

      (test.image.startsWith('http') ? test.image : `${BASE_URL}${test.image}`) : null;

```



```

let authorAvatarUrl = null;

if (test.author_info?.avatar) {

    if (test.author_info.avatar.startsWith('http') ||
test.author_info.avatar.startsWith('data:image')) {

        authorAvatarUrl = test.author_info.avatar;

    } else {

        authorAvatarUrl = `${BASE_URL}${test.author_info.avatar}`;

    }

}

return {

    ...test,

    average_rating: ratingData.average_rating || 0,

    ratings_count: ratingData.ratings_count || 0,

    full_image_url: imageFullUrl,

    full_avatar_url: authorAvatarUrl

};

} catch (error) {

const imageFullUrl = test.image ?

    (test.image.startsWith('http') ? test.image : `${BASE_URL}${test.image}`) : null;

let authorAvatarUrl = null;

if (test.author_info?.avatar) {

    if (test.author_info.avatar.startsWith('http') ||
test.author_info.avatar.startsWith('data:image')) {

```

```

        authorAvatarUrl = test.author_info.avatar;

    } else {

        authorAvatarUrl = `${BASE_URL}${test.author_info.avatar}`;

    }

}

return {

    ...test,

    average_rating: 0,

    ratings_count: 0,

    full_image_url: imageFullUrl,

    full_avatar_url: authorAvatarUrl

};

}

}));

// СОРТИРУЕМ ПОСЛЕ ПОЛУЧЕНИЯ РЕЙТИНГА

let sortedCards = cardsWithData;

if (sort === 'new') {

    sortedCards = cardsWithData.sort((a: any, b: any) =>

        new Date(b.created_at).getTime() - new Date(a.created_at).getTime()

    );

} else if (sort === 'old') {

    sortedCards = cardsWithData.sort((a: any, b: any) =>

        new Date(a.created_at).getTime() - new Date(b.created_at).getTime()

```

```

    );
  } else if (sort === 'popular') {
    sortedCards = cardsWithData.sort((a: any, b: any) =>
      (b.average_rating || 0) - (a.average_rating || 0)
    );
  } else if (sort === 'az') {
    sortedCards = cardsWithData.sort((a: any, b: any) =>
      a.title.localeCompare(b.title)
    );
  }

  setAllCards(sortedCards);

  setDisplayedCards(12);
} catch (error) {
  console.error('Ошибка загрузки:', error);
} finally {
  setLoading(false);
}
};

const handleTypeChange = (type: 'tests' | 'surveys') => {
  setSelectedType(type);

  // Остальное делает useEffect
};

```

```

const handleTopicsApply = () => {
  const params = new URLSearchParams(location.search);
  if (selectedTopics.length > 0) {
    params.set('topics', selectedTopics.join(','));
  } else {
    params.delete('topics');
  }
  navigate(`?${params.toString()}`, { replace: true });
  setIsTopicsModalOpen(false);
};

```

```

const toggleTopic = (topic: string) => {
  if (selectedTopics.includes(topic)) {
    setSelectedTopics(selectedTopics.filter(t => t !== topic));
  } else {
    setSelectedTopics([...selectedTopics, topic]);
  }
};

```

// Бесконечный скролл

```

useEffect(() => {
  const observer = new IntersectionObserver(
    (entries) => {
      if (entries[0].isIntersecting && displayedCards < allCards.length && !
isLoadingMore) {

```

```

    setIsLoadingMore(true);

    setTimeout(() => {
        setDisplayedCards(prev => Math.min(prev + 8, allCards.length));
        setIsLoadingMore(false);
    }, 500);
}
},
{ threshold: 0.1 }
);

```

```

if (loadMoreRef.current) {
    observer.observe(loadMoreRef.current);
}

```

```

return () => observer.disconnect();
}, [displayedCards, allCards.length, isLoadingMore]);

```

```

const visibleCards = allCards.slice(0, displayedCards);

```

```

// Кастомная загрузка с картинкой

```

```

if (loading) {
    return (
        <div className={styles.page}>
            <main className={styles.main}>
                <div className={styles.container}>

```

```

    <div className={styles.loadingContainer}>

       {

          e.currentTarget.style.display = 'none';

        }}

      />

      <div className={styles.loadingText}>ЗАГРУЗКА</div>

    </div>

  </div>

</main>

</div>

);

}

return (

  <

    <div className={styles.page}>

      <main className={styles.main}>

        <div className={styles.hideOnMobile}>

          <SearchSection

            onChange={handleTypeChange}

            showTypeSelector={true}

```

```

onTopicsClick={() => setIsTopicsModalOpen(true)}

selectedTopicsCount={selectedTopics.length}

/>

</div>

<div className={styles.container}>

  <div className={styles.cardsGrid}>

    {visibleCards.map((card) => (

      <Card

        key={card.id}

        title={card.title}

        description={card.description}

        author={card.author_info?.full_name || card.author_info?.login ||
'Неизвестный автор'}

        rating={` ${card.average_rating || 0}/5`}

        users={card.ratings_count?.toString() || '0'}

        questions={card.questions_count?.toString() || '0'}

        type={card.is_survey ? 'survey' : 'test'}

        testId={card.id.toString()}

        authorId={card.author_info?.id?.toString() || ''}

        imageUrl={card.full_image_url || undefined}

        authorAvatar={card.full_avatar_url}

      />

    ))}

  </div>

```

```

    {displayedCards < allCards.length && (
      <div ref={loadMoreRef} className={styles.loadMore}>
        {isLoadingMore ? 'Загрузка...' : 'Загрузить ещё'}
      </div>
    )}

    <div className={styles.loadingStatus}>
      Показано {visibleCards.length} из {allCards.length}
    </div>
  </div>
</main>
</div>

{/* Модальное окно выбора тем */}
{isTopicsModalOpen && (
  <div className={styles.modalOverlay} onClick={() =>
    setIsTopicsModalOpen(false)}>
    <div className={styles.modalContent} onClick={(e) => e.stopPropagation()}>
      <h3 className={styles.modalTitle}>Выберите темы</h3>
      <div className={styles.topicsGrid}>
        {topicsList.map(topic => (
          <button
            key={topic}
            className={` ${styles.topicButton} ${selectedTopics.includes(topic) ?

```



```

styles.active : "`"}
      onClick={() => toggleTopic(topic)}
    >
      {topic}
    </button>
  )))
</div>

<div className={styles.modalButtons}>
  <button
    className={styles.clearButton}
    onClick={() => {
      setSelectedTopics([]);
      const params = new URLSearchParams(location.search);
      params.delete('topics');
      navigate(`?${params.toString()}`, { replace: true });
      setIsTopicsModalOpen(false);
    }}
  >
    ОЧИСТИТЬ
  </button>
  <button
    className={styles.applyButton}
    onClick={handleTopicsApply}
  >
    Применить ( {selectedTopics.length})

```

```

        </button>

    </div>

</div>

</div>

    )}

</>

);

};

export default TestsList;

===== \components\common\TestStastPage\TestStastPageMain.tsx =====

import React, { useState, useEffect } from 'react';

import { useParams, useNavigate, Link } from 'react-router-dom';

import api from '../../api/axios';

import styles from './TestStatsPageMain.module.css';

interface AnswerDetail {

    question_id: number;

    question_text: string;

    user_answer: string;

    is_correct: boolean;

    correct_answer?: string;

}

interface Attempt {

```

```
id: number;

user: {

    id: number;

    login: string;

    full_name: string;

    avatar: string | null;

};

score: number;

is_passed: boolean;

started_at: string;

finished_at: string | null;

answers: any;

answers_details?: AnswerDetail[];

}
```

```
interface TestStats {

    id: number;

    title: string;

    description: string;

    is_survey: boolean;

    questions_count: number;

    total_attempts: number;

    passed_attempts: number;

    average_score: number;

    attempts: Attempt[];
```

```
}
```

```
type SortField = 'user' | 'result' | 'date';
```

```
type SortOrder = 'asc' | 'desc';
```

```
const TestStatsPageMain = () => {
```

```
  const { testId } = useParams<{ testId: string }>();
```

```
  const navigate = useNavigate();
```

```
  const [stats, setStats] = useState<TestStats | null>(null);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [error, setError] = useState<string | null>(null);
```

```
  const [selectedAttempt, setSelectedAttempt] = useState<Attempt | null>(null);
```

```
  const [isModalOpen, setIsModalOpen] = useState(false);
```

```
  const [loadingDetails, setLoadingDetails] = useState(false);
```

```
  const [sortField, setSortField] = useState<SortField>('date');
```

```
  const [sortOrder, setSortOrder] = useState<SortOrder>('desc');
```

```
  useEffect(() => {
```

```
    if (testId) {
```

```
      loadStats();
```

```
    }
```

```
  }, [testId]);
```

```
  const loadStats = async () => {
```

```
    const token = localStorage.getItem('access_token');
```

```

if (!token) {

    setError('Для просмотра статистики необходимо авторизоваться');

    setLoading(false);

    return;
}

try {

    const response = await api.get(`/tests/${testId}/stats/`);

    setStats(response.data);

} catch (err: any) {

    console.error('Ошибка загрузки статистики:', err);

    if (err.response?.status === 401) {

        setError('Сессия истекла. Пожалуйста, войдите заново');

    } else if (err.response?.status === 403) {

        setError('У вас нет прав для просмотра статистики этого теста');

    } else {

        setError('Не удалось загрузить статистику');

    }

} finally {

    setLoading(false);

}

};

const loadAttemptDetails = async (attemptId: number) => {

    setLoadingDetails(true);

```

```

try {

    const response = await api.get(`/attempts/${attemptId}/details/`);

    const details = response.data.answers_details || [];

    if (stats && !stats.is_survey) {

        const questionsResponse = await api.get(`/tests/${testId}/`);

        const testData = questionsResponse.data;

        const questionsMap = new Map();

        testData.questions.forEach((q: any) => {

            const correctAnswer = q.answers.find((a: any) => a.is_correct);

            questionsMap.set(q.id, {

                correct_answer: correctAnswer ? correctAnswer.text : q.text_answer

            });

        });

        return details.map((d: any) => ({

            ...d,

            correct_answer: questionsMap.get(d.question_id)?.correct_answer

        }));

    }

    return details;

} catch (err) {

    console.error('Ошибка загрузки деталей:', err);

    return [];

} finally {

```

```
        setLoadingDetails(false);  
    }  
};
```

```
const openAttemptModal = async (attempt: Attempt) => {  
    setSelectedAttempt(attempt);  
    setIsModalOpen(true);  
  
    const details = await loadAttemptDetails(attempt.id);  
    setSelectedAttempt(prev => prev ? { ...prev, answers_details: details } : prev);  
};
```

```
const closeModal = () => {  
    setIsModalOpen(false);  
    setSelectedAttempt(null);  
};
```

```
const formatDate = (dateString: string) => {  
    return new Date(dateString).toLocaleString('ru-RU');  
};
```

```
const handleSort = (field: SortField) => {  
    if (sortField === field) {  
        setSortOrder(sortOrder === 'asc' ? 'desc' : 'asc');  
    } else {
```

```

        setSortField(field);

        setSortOrder('asc');
    }
};

```

```

const getSortIcon = (field: SortField) => {
    if (sortField !== field) return "";
    return sortOrder === 'asc' ? '↑' : '↓';
};

```

```

const getSortedAttempts = () => {
    if (!stats) return [];

```

```

    const sorted = [...stats.attempts];

```

```

    sorted.sort((a, b) => {

```

```

        let aVal: any;

```

```

        let bVal: any;

```

```

        switch (sortField) {

```

```

            case 'user':

```

```

                aVal = (a.user.full_name || a.user.login).toLowerCase();

```

```

                bVal = (b.user.full_name || b.user.login).toLowerCase();

```

```

                break;

```

```

            case 'result':

```



```

        aVal = a.score;

        bVal = b.score;

        break;

    case 'date':

        aVal = new Date(a.finished_at || a.started_at);

        bVal = new Date(b.finished_at || b.started_at);

        break;

    default:

        return 0;

    }

    if (aVal < bVal) return sortOrder === 'asc' ? -1 : 1;

    if (aVal > bVal) return sortOrder === 'asc' ? 1 : -1;

    return 0;

});

return sorted;

};

const sortedAttempts = getSortedAttempts();

if (loading) {

    return (

        <div className={styles.page}>

            <div className={styles.container}>

```

```

<div className={styles.loadingContainer}>
   {
      e.currentTarget.style.display = 'none';
    }}
  />
  <div className={styles.loadingText}>ЗАГРУЗКА
СТАТИСТИКИ</div>
</div>
</div>
</div>
);
}

if (error) {
  return (
    <div className={styles.page}>
      <div className={styles.container}>
        <div className={styles.error}>{error}</div>
        <button onClick={() => navigate('/profile')} className={styles.backButton}
>
  Вернуться к профилю

```

```

        </button>

    </div>

</div>

);
}

if (!stats) {
    return (
        <div className={styles.page}>
            <div className={styles.container}>
                <div className={styles.error}>Статистика не найдена</div>
            </div>
        </div>
    );
}

```

```

const passRate = stats.total_attempts > 0
    ? Math.round((stats.passed_attempts / stats.total_attempts) * 100)
    : 0;

```

```

return (
    <div className={styles.page}>
        <div className={styles.container}>
            <div className={styles.header}>
                <button onClick={() => navigate('/profile')} className={styles.backLink}>

```

Назад к профилю

</button>

<Link to={` /test/\${stats.id}`} className={styles.testLink}>

<h1 className={styles.title}>{stats.title}</h1>

</Link>

<div className={styles.testType}>

{stats.is_survey ? 'ОПРОС' : 'ТЕСТ'}

</div>

</div>

{stats.description && (

<div className={styles.description}>

{stats.description}

</div>

)}

<div className={styles.statsCards}>

<div className={styles.statCard}>

<div className={styles.statValue}>{stats.total_attempts}</div>

<div className={styles.statLabel}>Всего прохождений</div>

</div>

<div className={styles.statCard}>

<div className={styles.statValue}>{stats.passed_attempts}</div>

<div className={styles.statLabel}>Успешно пройдено</div>

</div>

```

<div className={styles.statCard}>

  <div className={styles.statValue}>{passRate}%</div>

  <div className={styles.statLabel}>Процент успеха</div>

</div>

<div className={styles.statCard}>

  <div
className={styles.statValue}>{stats.average_score.toFixed(1)}%</div>

  <div className={styles.statLabel}>Средний результат</div>

</div>

<div className={styles.statCard}>

  <div className={styles.statValue}>{stats.questions_count}</div>

  <div className={styles.statLabel}>Вопросов</div>

</div>

</div>

<div className={styles.attemptsSection}>

  <h2 className={styles.sectionTitle}>История прохождений</h2>

  {sortedAttempts.length === 0 ? (

    <div className={styles.empty}>Нет прохождений</div>

  ) : (

    <div className={styles.tableWrapper}>

      <table className={styles.attemptsTable}>

        <thead>

          <tr>

            <th onClick={() => handleSort('user')}

```

```

className={styles.sortable}>
    Пользователь {getSortIcon('user')}
</th>
<th onClick={() => handleSort('result')}
className={styles.sortable}>
    Результат {getSortIcon('result')}
</th>
<th onClick={() => handleSort('date')}
className={styles.sortable}>
    Дата прохождения {getSortIcon('date')}
</th>
<th>Детали</th>
</tr>
</thead>
<tbody>
    {sortedAttempts.map((attempt) => {
        const isAnonymous = attempt.user.id === 0;
        const userName = attempt.user.full_name || attempt.user.login;

        return (
            <tr key={attempt.id}>
                <td className={styles.userCell}>
                    {isAnonymous ? (
                        <div className={styles.userLinkDisabled}>
                            <div className={styles.userAvatar}>

```

```

        <div className={styles.avatarPlaceholder}>
            A
        </div>
    </div>
    <span>{userName}</span>
</div>
): (
    <Link to={` /profile/${attempt.user.id}`}
className={styles.userLink}>
        <div className={styles.userAvatar}>
            {attempt.user.avatar ? (
                <img
                    src={attempt.user.avatar.startsWith('http') ?
attempt.user.avatar : `http://localhost:8000${attempt.user.avatar}`}
                    alt={attempt.user.login}
                    onError={(e) => {
                        e.currentTarget.style.display = 'none';
                    }}
                />
            ) : (
                <div className={styles.avatarPlaceholder}>
                    {userName.charAt(0).toUpperCase()}
                </div>
            )}
        </div>
    )}
</div>

```

```

        <span>{userName}</span>

    </Link>

    )}

</td>

<td>

    {stats.is_survey ? (

        <span className={styles.passed}>Пройден</span>

    ) : (

        <span className={styles.score}>

            {attempt.score}%

        </span>

    )}

</td>

<td>{formatDate(attempt.finished_at || attempt.started_at)}

</td>

<td>

    <button

        className={styles.detailsButton}

        onClick={() => openAttemptModal(attempt)}

    >

        Подробнее

    </button>

</td>

</tr>

);

```



```

    }}}
  </tbody>

</table>

</div>

  )}

</div>

</div>

```

```

{isModalOpen && selectedAttempt && (
  <div className={styles.modalOverlay} onClick={closeModal}>
    <div className={styles.modal} onClick={(e) => e.stopPropagation()}>
      <div className={styles.modalHeader}>
        <h3>Детали прохождения</h3>
        <button className={styles.closeModal}
onClick={closeModal}>✕</button>
      </div>
      <div className={styles.modalBody}>
        <div className={styles.modalUserInfo}>
          <p><strong>Пользователь:</strong>
{selectedAttempt.user.full_name || selectedAttempt.user.login}</p>
          <p><strong>Дата:</strong> {formatDate(selectedAttempt.finished_at
|| selectedAttempt.started_at)}</p>
          <!--stats?.is_survey && (
            <p><strong>Результат:</strong> {selectedAttempt.score}%</p>
          --)
          <p><strong>Статус:</strong> {selectedAttempt.is_passed ?

```

'Пройден' : 'Не пройден'}</p>

</div>

{loadingDetails ? (

<div className={styles.loadingAnswers}>Загрузка ответов...</div>

) : selectedAttempt.answers_details &&
selectedAttempt.answers_details.length > 0 ? (

<div className={styles.answersSection}>

<h4>ОТВЕТЫ на вопросы:</h4>

<div className={styles.answersList}>

{selectedAttempt.answers_details.map((answer, index) => (

<div key={answer.question_id}
className={styles.answerItem}>

<div className={styles.questionNumber}>Вопрос {index
+ 1}</div>

<div className={styles.questionText}
>{answer.question_text}</div>

<div className={styles.userAnswer}>

<div className={styles.answerRow}>

ОТВЕТ
пользователя:

<span className={answer.is_correct ?
styles.correctAnswer : styles.wrongAnswer}>

{answer.user_answer || '(не указан)'}

</div>

{!stats?.is_survey && answer.correct_answer && (

```

        <div className={styles.answerRow}>
            <span className={styles.answerLabel}
>Правильный ответ:</span>
            <span className={styles.correctAnswerText}>
                {answer.correct_answer}
            </span>
        </div>
    )}
</div>

    <div className={answer.is_correct ?
styles.answerStatusCorrect : styles.answerStatusWrong}>
        {answer.is_correct ? 'Верно' : 'Неверно'}
    </div>
</div>
    )}
</div>
    ) : (
        <div className={styles.noAnswers}>Нет данных об ответах</div>
    )}
</div>
</div>
    )}
</div>

```

```

    );

};

export default TestStatsPageMain;

===== \components\common\TestStatsPage\TestStatsPageMain.module.css
=====

@import "../../style/variables.css";

.page {
    min-height: 100vh;
    background-color: var(--color-background);
    padding: 20px;
}

.container {
    max-width: 1200px;
    margin: 0 auto;
}

/* Заголовок */

.header {
    display: flex;
    align-items: center;
    justify-content: space-between;
    flex-wrap: wrap;

```

```
gap: 20px;

margin-bottom: 30px;

padding-bottom: 20px;

border-bottom: 1px solid var(--color-border);
}
```

```
.backLink {

background: none;

border: none;

font-size: 16px;

color: var(--color-text);

cursor: pointer;

padding: 8px 16px;

border-radius: 8px;
}
```

```
.backLink:hover {

background-color: var(--color-border);
}
```

```
.title {

font-size: 32px;

font-weight: bold;

color: var(--color-text);

margin: 0;
```

```
    text-align: center;
}
```

```
.testLink {
    text-decoration: none;
}
```

```
.testLink:hover {
    text-decoration: underline;
}
```

```
.testType {
    padding: 6px 16px;
    background-color: var(--color-buttonColor);
    border-radius: 20px;
    font-size: 14px;
    font-weight: bold;
    color: var(--color-text);
}
```

```
/* Описание */
```

```
.description {
    background-color: var(--color-blockColor);
    border-radius: 10px;
    padding: 20px;
```

```

margin-bottom: 30px;

color: var(--color-text);

font-size: 16px;
}

/* Карточки статистики */

.statsCards {

display: grid;

grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));

gap: 20px;

margin-bottom: 40px;
}

.statCard {

background-color: var(--color-blockColor);

border-radius: 10px;

padding: 20px;

text-align: center;
}

.statValue {

font-size: 36px;

font-weight: bold;

color: var(--color-text);

margin-bottom: 8px;

```

```
}
```

```
.statLabel {
```

```
    font-size: 14px;
```

```
    color: var(--color-text);
```

```
    opacity: 0.7;
```

```
}
```

```
/* Секция попыток */
```

```
.attemptsSection {
```

```
    background-color: var(--color-blockColor);
```

```
    border-radius: 10px;
```

```
    padding: 20px;
```

```
}
```

```
.sectionTitle {
```

```
    font-size: 24px;
```

```
    font-weight: bold;
```

```
    color: var(--color-text);
```

```
    margin-bottom: 20px;
```

```
}
```

```
.tableWrapper {
```

```
    overflow-x: auto;
```

```
}
```



```
.attemptsTable {  
    width: 100%;  
    border-collapse: collapse;  
}
```

```
.attemptsTable th,  
.attemptsTable td {  
    padding: 12px;  
    text-align: left;  
    border-bottom: 1px solid var(--color-border);  
    color: var(--color-text);  
}
```

```
.attemptsTable th {  
    background-color: var(--color-buttonColor);  
    font-weight: bold;  
}
```

```
.sortable {  
    cursor: pointer;  
    user-select: none;  
}
```

```
.attemptsTable th:hover,
```

```
.attemptsTable th.sortable:hover {  
    background-color: var(--color-buttonColor);  
    opacity: 1;  
}
```

```
.attemptsTable tbody tr:hover {  
    background-color: transparent;  
}
```

```
.attemptsTable tbody tr:hover td {  
    background-color: transparent;  
}
```

```
.userCell {  
    min-width: 180px;  
}
```

```
.userLink {  
    display: flex;  
    align-items: center;  
    gap: 10px;  
    text-decoration: none;  
    color: var(--color-text);  
}
```

```
.userLink:hover {  
    text-decoration: underline;  
}
```

```
.userLinkDisabled {  
    display: flex;  
    align-items: center;  
    gap: 10px;  
    color: var(--color-text);  
    cursor: default;  
}
```

```
.userAvatar {  
    width: 32px;  
    height: 32px;  
    border-radius: 50%;  
    overflow: hidden;  
    background-color: var(--color-avatar);  
    display: flex;  
    align-items: center;  
    justify-content: center;  
}
```

```
.userAvatar img {  
    width: 100%;
```

```
height: 100%;  
  
object-fit: cover;  
  
}  
  
.avatarPlaceholder {  
  width: 100%;  
  height: 100%;  
  background-color: var(--color-buttonColor);  
  display: flex;  
  align-items: center;  
  justify-content: center;  
  font-size: 14px;  
  font-weight: bold;  
  color: var(--color-text);  
}  
  
.score, .passed {  
  font-weight: bold;  
  color: var(--color-text);  
}  
  
.detailsButton {  
  background-color: var(--color-buttonColor);  
  border: none;  
  border-radius: 6px;
```

```
padding: 6px 12px;  
font-size: 12px;  
color: var(--color-text);  
cursor: pointer;  
}
```

```
.detailsButton:hover {  
    background-color: var(--color-hover);  
}
```

```
.modalOverlay {  
    position: fixed;  
    top: 0;  
    left: 0;  
    right: 0;  
    bottom: 0;  
    background-color: var(--color-overlay);  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    z-index: 1000;  
}
```

```
.modal {  
    background-color: var(--color-blockColor);
```

```
border-radius: 15px;

width: 90%;

max-width: 700px;

max-height: 80vh;

overflow: hidden;

display: flex;

flex-direction: column;

}
```

```
.modalHeader {

  display: flex;

  justify-content: space-between;

  align-items: center;

  padding: 20px;

  border-bottom: 1px solid var(--color-border);

}
```

```
.modalHeader h3 {

  margin: 0;

  font-size: 20px;

  color: var(--color-text);

}
```

```
.closeModal {

  background: none;
```

```
border: none;

font-size: 20px;

cursor: pointer;

color: var(--color-text);

padding: 0 8px;

}
```

```
.closeModal:hover {

  opacity: 0.7;

}
```

```
.modalBody {

  padding: 20px;

  overflow-y: auto;

  flex: 1;

}
```

```
.modalUserInfo {

  background-color: var(--color-buttonColor);

  border-radius: 10px;

  padding: 15px;

  margin-bottom: 20px;

}
```

```
.modalUserInfo p {
```

```
margin: 8px 0;

color: var(--color-text);

}
```

```
.answersSection h4 {

    font-size: 18px;

    color: var(--color-text);

    margin-bottom: 15px;

    padding-bottom: 10px;

    border-bottom: 1px solid var(--color-border-dark);

}
```

```
.answersList {

    display: flex;

    flex-direction: column;

    gap: 15px;

    max-height: 400px;

    overflow-y: auto;

    padding-right: 10px;

}
```

```
.answerItem {

    background-color: rgba(255, 215, 157, 0.5);

    border-radius: 10px;

    padding: 15px;
```



```
}
```

```
.questionNumber {  
    font-size: 12px;  
    color: var(--color-text-muted);  
    margin-bottom: 5px;  
}
```

```
.questionText {  
    font-size: 16px;  
    font-weight: bold;  
    color: var(--color-text);  
    margin-bottom: 10px;  
}
```

```
.answerLabel {  
    font-size: 13px;  
    color: var(--color-text);  
    margin-right: 8px;  
}
```

```
.answerRow {  
    margin-bottom: 8px;  
}
```

```
/* Текст ответов - чёрный */

.correctAnswer {
    color: var(--color-text);
    font-weight: bold;
}

.wrongAnswer {
    color: var(--color-text);
    font-weight: bold;
}

.correctAnswerText {
    color: var(--color-text);
}

.answerStatusCorrect {
    margin-top: 10px;
    padding: 6px 12px;
    background-color: var(--color-correct);
    border-radius: 6px;
    font-size: 13px;
    font-weight: bold;
    color: var(--color-text);
    display: inline-block;
}
```

```
.answerStatusWrong {  
    margin-top: 10px;  
    padding: 6px 12px;  
    background-color: var(--color-wrong);  
    border-radius: 6px;  
    font-size: 13px;  
    font-weight: bold;  
    color: var(--color-text);  
    display: inline-block;  
}
```

```
.loadingContainer {  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    min-height: 60vh;  
    gap: 20px;  
}
```

```
.loadingImage {  
    width: 200px;  
    height: auto;  
    object-fit: contain;
```

```
}
```

```
.loadingText {  
    font-size: 24px;  
    font-weight: bold;  
    color: var(--color-text);  
    letter-spacing: 2px;  
}
```

```
.error, .empty {  
    text-align: center;  
    padding: 60px;  
    font-size: 18px;  
    color: var(--color-text);  
}
```

```
.backButton {  
    display: block;  
    margin: 20px auto;  
    padding: 10px 20px;  
    background-color: var(--color-buttonColor);  
    border: none;  
    border-radius: 8px;  
    color: var(--color-text);  
    cursor: pointer;
```

```
    font-size: 16px;
}
```

```
.backButton:hover {
    background-color: var(--color-hover);
}
```

```
.answersList::-webkit-scrollbar {
    width: 6px;
}
```

```
.answersList::-webkit-scrollbar-track {
    background: var(--color-scrollbar-track);
    border-radius: 10px;
}
```

```
.answersList::-webkit-scrollbar-thumb {
    background: var(--color-scrollbar-thumb);
    border-radius: 10px;
}
```

```
.answersList::-webkit-scrollbar-thumb:hover {
    background: var(--color-scrollbar-thumb-hover);
}
```

```
@media (max-width: 768px) {  
  
  .header {  
  
    flex-direction: column;  
  
    text-align: center;  
  
  }  
  
  .title {  
  
    font-size: 24px;  
  
  }  
  
  .statsCards {  
  
    grid-template-columns: repeat(2, 1fr);  
  
  }  
  
  .statValue {  
  
    font-size: 28px;  
  
  }  
  
  .attemptsTable th,  
  .attemptsTable td {  
  
    padding: 8px;  
  
    font-size: 12px;  
  
  }  
}
```

```
@media (max-width: 480px) {
  .statsCards {
    grid-template-columns: 1fr;
  }
}
```

```
.userLink span {
  display: none;
}
```

```
.userCell {
  min-width: auto;
}
}
```

```
===== \hooks\useAuthForm.ts =====
```

```
import { useState } from 'react';
```

```
import axios from 'axios';
```

```
const API_URL = 'http://localhost:8000/api'; // Замени на свой URL бэкенда
```

```
export const useRegisterForm = () => {
  const [formData, setFormData] = useState({
    login: "",
    email: "",
    password: "",
    confirmPassword: "",
```

```

    subscribe: false

  });

  const [error, setError] = useState("");
  const [isSubmitting, setIsSubmitting] = useState(false);

  const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
    const { name, value, type, checked } = e.target;
    setFormData(prev => ({
      ...prev,
      [name]: type === 'checkbox' ? checked : value
    }));
  };

  const handleSubmit = async (e: React.FormEvent) => {
    e.preventDefault();
    setError("");
    setIsSubmitting(true);

    try {
      const response = await axios.post(`${API_URL}/auth/register/`, {
        login: formData.login,
        email: formData.email,
        password: formData.password,
        password2: formData.confirmPassword,
        subscribe: formData.subscribe

```



```
});
```

```
// Автоматически логиним после регистрации
```

```
const loginResponse = await axios.post(`${API_URL}/auth/login/`, {  
  login: formData.login,  
  password: formData.password  
});
```

```
localStorage.setItem('access_token', loginResponse.data.access);  
localStorage.setItem('refresh_token', loginResponse.data.refresh);  
localStorage.setItem('user', JSON.stringify(loginResponse.data.user));
```

```
window.location.href = '/profile'; // Перенаправление после успеха
```

```
} catch (err: any) {  
  if (err.response?.data) {  
    const errors = err.response.data;  
    if (typeof errors === 'object') {  
      setError(Object.values(errors).flat().join(', '));  
    } else {  
      setError(errors.error || 'Ошибка регистрации');  
    }  
  } else {  
    setError('Ошибка соединения с сервером');  
  }  
}
```

```

    } finally {
        setIsSubmitting(false);
    }
};

return { formData, error, isSubmitting, handleChange, handleSubmit };
};

export const useLoginForm = () => {
    const [formData, setFormData] = useState({
        login: "",
        password: ""
    });

    const [error, setError] = useState("");
    const [isSubmitting, setIsSubmitting] = useState(false);

    const handleChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        setFormData(prev => ({
            ...prev,
            [e.target.name]: e.target.value
        }));
    };

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
    };
};

```

```

setError("");

setIsSubmitting(true);

try {

    const response = await axios.post(`${API_URL}/auth/login/`, formData);

    localStorage.setItem('access_token', response.data.access);
    localStorage.setItem('refresh_token', response.data.refresh);
    localStorage.setItem('user', JSON.stringify(response.data.user));

    window.location.href = '/profile';

} catch (err: any) {

    if (err.response?.data) {

        setError(err.response.data.error || 'Ошибка входа');

    } else {

        setError('Ошибка соединения с сервером');

    }

} finally {

    setIsSubmitting(false);

}

};

return { formData, error, isSubmitting, handleChange, handleSubmit };

};

```

```
===== \hooks\usePools.ts =====
```

```
import { useState, useEffect } from 'react';
```

```
import { getPools, createPool, addQuestionToPool, createQuestion, uploadImage } from  
'../api/tests';
```

```
export interface PoolQuestion {
```

```
  id: number;
```

```
  questionText: string;
```

```
  questionImage: string | null;
```

```
  rightAnswer: string;
```

```
  wrongAnswers: string[];
```

```
  isTextInput: boolean;
```

```
  textAnswer: string;
```

```
}
```

```
export interface QuestionPool {
```

```
  id: number;
```

```
  title: string;
```

```
  description: string;
```

```
  questions: PoolQuestion[];
```

```
}
```

```
export interface Question {
```

```
  id: number;
```

```
  questionText: string;
```

```

questionImage: File | null;

rightAnswer: string;

wrongAnswers: string[];

isTextInput: boolean;

textAnswer: string;
}

```

```

export const usePools = () => {

  const [questionPools, setQuestionPools] = useState<QuestionPool[]>([]);

  const [loading, setLoading] = useState(false);


  const loadPools = async () => {

    setLoading(true);

    try {

      const pools = await getPools();

      const formattedPools = pools.map((pool: any) => ({

        id: pool.id,

        title: pool.name,

        description: pool.description,

        questions: (pool.questions || []).map((q: any) => ({

          id: q.id,

          questionText: q.text,

          questionImage: q.image,

          rightAnswer: q.answers?.find((a: any) => a.is_correct)?.text || "",

          wrongAnswers: q.answers?.filter((a: any) => !a.is_correct).map((a: any) => a.text) ||

```

```

[, ", "],

    isTextInput: q.is_text_input || false,

    textAnswer: q.text_answer || "

    )))

  ));

  setQuestionPools(formattedPools);

} catch (error) {

  console.error('Ошибка загрузки пулов:', error);

} finally {

  setLoading(false);

}

};

```

```

const createNewPool = async (title: string, description: string): Promise<QuestionPool |
null> => {

  try {

    const newPool = await createPool({ name: title, description });

    const formattedPool = {

      id: newPool.id,

      title: newPool.name,

      description: newPool.description,

      questions: []

    };

    setQuestionPools(prev => [...prev, formattedPool]);

    return formattedPool;

```

```

    } catch (error) {

        console.error('Ошибка создания пула:', error);

        return null;

    }

};

```

```

const addQuestionToPoolById = async (poolId: number, question: Question):
Promise<boolean> => {

    try {

        let answers: any[] = [];

        let textAnswer = "";

        let isTextInput = question.isTextInput;

        if (question.isTextInput) {

            textAnswer = question.textAnswer;

            answers = [{ text: question.textAnswer, is_correct: true, order_index: 0 }];

        } else {

            answers.push({ text: question.rightAnswer, is_correct: true, order_index: 0 });

            question.wrongAnswers.forEach((answer, idx) => {

                if (answer.trim()) {

                    answers.push({ text: answer, is_correct: false, order_index: idx + 1 });

                }

            });

        }

    }

};

```

```

// Картинку загружаем через uploadImage

let questionImageUrl = null;

if (question.questionImage) {
    questionImageUrl = await uploadImage(question.questionImage, 'question');
}

const questionData = {
    text: question.questionText,
    image: questionImageUrl,
    difficulty: 1,
    explanation: "",
    is_text_input: isTextInput,
    text_answer: textAnswer,
    answers: answers
};

const createdQuestion = await createQuestion(questionData);
await addQuestionToPool(poolId, createdQuestion.id);
await loadPools();
return true;
} catch (error) {
    console.error('Ошибка добавления вопроса в пул:', error);
    return false;
}
};

```



```

useEffect(() => {
    loadPools();
}, []);

return { questionPools, loading, loadPools, createNewPool, addQuestionToPoolById };
};

===== \pages\admin\AdminPanel.tsx =====

import React from 'react';
import Header from '../components/common/Header/Header';
import AdminMain from '../components/common/Admin/AdminPanelMain';

const AdminPanel = () => {
    return (
        <div>
            <Header />
            <AdminMain />
        </div>
    );
};

export default AdminPanel;

===== \pages\create\Create.tsx =====

import React, { useState, useEffect } from 'react';
import { Link, useNavigate } from 'react-router-dom';

```

```

import Header from '../components/common/Header/Header';
import Footer from '../components/common/Footer/Footer';
import api from '../api/axios';
import choiceStyles from './CreateChoice.module.css';

```

```

const Create = () => {
  const navigate = useNavigate();
  const [isBanned, setIsBanned] = useState(false);
  const [banReason, setBanReason] = useState("");
  const [banUntil, setBanUntil] = useState<string | null>(null);
  const [banPermanent, setBanPermanent] = useState(false);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    checkBanStatus();
  }, []);

```

```

const checkBanStatus = async () => {
  try {
    const profile = await api.get('/auth/profile/');
    if (profile.data.is_banned) {
      setIsBanned(true);
      setBanReason(profile.data.ban_reason || "");
      setBanUntil(profile.data.ban_until);
      setBanPermanent(profile.data.ban_permanent || false);
    }
  }

```

```

    }
  } catch (error) {
    console.error('Ошибка проверки бана:', error);
  } finally {
    setLoading(false);
  }
};

```

```

const getBanUntilText = () => {
  if (!banUntil) return "";
  const date = new Date(banUntil);
  return date.toLocaleString('ru-RU');
};

```

```

if (loading) {
  return (
    <div className={choiceStyles.createPage}>
      <Header />
      <div className={choiceStyles.createContainer}>
        <div className={choiceStyles.loading}>Загрузка...</div>
      </div>
      <Footer />
    </div>
  );
}

```

```

if (isBanned) {
  return (
    <div className={choiceStyles.createPage}>
      <Header />
      <div className={choiceStyles.createContainer}>
        <div className={choiceStyles.bannedMessage}>
          <div className={choiceStyles.bannedText}>Ваш аккаунт забанен</div>
          {banReason && <div className={choiceStyles.bannedReason}>
            >Причина: {banReason}</div>}
          {banUntil && !banPermanent && (
            <div className={choiceStyles.bannedUntil}>До: {getBanUntilText()}
          </div>
          )}
          {banPermanent && <div className={choiceStyles.bannedPermanent}>
            >Перманентно</div>}
          <button
            className={choiceStyles.backButton}
            onClick={() => navigate('/')}>
            Вернуться на главную
          </button>
        </div>
      </div>
    <Footer />
  </div>

```

```
);  
}
```

```
return (
```

```
<div className={choiceStyles.createPage}>
```

```
<Header />
```

```
<div className={choiceStyles.createContainer}>
```

```
<h2>выберите тип</h2>
```

```
<div className={choiceStyles.createMainBlockContainer}>
```

```
<Link to="/create/test" className={choiceStyles.createMainBlock}>
```

```
<div className={choiceStyles.createMainBlockTitle}>тест</div>
```

```
<div className={choiceStyles.createMainBlockDescription}>
```

```
<p>проверка знаний с оценкой</p>
```

```
<p>ограничение по времени</p>
```

```
</div>
```

```
</Link>
```

```
<Link to="/create/survey" className={choiceStyles.createMainBlock}>
```

```
<div className={choiceStyles.createMainBlockTitle}>опрос</div>
```

```
<div className={choiceStyles.createMainBlockDescription}>
```

```
<p>сбор мнений без оценки</p>
```

```
<p>без ограничения времени</p>
```

```
</div>
```

```
</Link>
```

```
</div>
```

```

        </div>

        <Footer />

    </div>

    );

};

export default Create;

===== \pages\create\CreateChoice.module.css =====

.createPage {

    min-height: 100vh;

    background-color: #EAE7DC;

    display: flex;

    flex-direction: column;

}

.createContainer {

    max-width: 900px;

    margin: 0 auto;

    padding: 20px;

    flex: 1; /* Добавь - заставляет контент растягиваться */

    display: flex;

    flex-direction: column;

    justify-content: center; /* Центрирует контент по вертикали */

}

```

```
.createMainBlockContainer {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    gap: 35px;  
    flex-direction: column;  
    width: 100%;  
    margin: 30px 0;  
}
```

```
.createMainBlock {  
    display: flex;  
    flex-direction: column;  
    align-items: center;  
    justify-content: center;  
    text-decoration: none;  
    width: 1213px;  
    height: 160px;  
    background-color: #D8C3A5;  
    border-radius: 15px;  
    transition: all 0.3s ease;  
    gap: 10px;  
}
```

```
.createMainBlock:hover {
```

```
background-color: #BE8F4C;

transform: translateY(-5px);

}
```

```
.createMainBlockTitle {

    font-size: 50px;

    font-weight: bold;

    color: black;

    text-transform: uppercase;

    font-family: Arial, Helvetica, sans-serif;

}
```

```
.createMainBlockDescription p {

    color: black;

    font-size: 16px;

    margin: 3px 0;

    font-family: Arial, Helvetica, sans-serif;

    text-align: center;

}
```

```
h2 {

    text-align: center;

    margin-bottom: 40px;

}
```



```
@media (max-width: 1300px) {  
  
  .createMainBlock {  
  
    width: 90%;  
  
    height: 140px;  
  
  }  
  
  .createMainBlockTitle {  
  
    font-size: 40px;  
  
  }  
  
}
```

```
@media (max-width: 768px) {  
  
  .createMainBlockContainer {  
  
    margin: 20px 0;  
  
    gap: 25px;  
  
  }  
  
  .createMainBlock {  
  
    width: 95%;  
  
    height: 120px;  
  
  }  
  
  .createMainBlockTitle {  
  
    font-size: 32px;  
  
  }  
  
}
```

```
.createMainBlockDescription p {  
    font-size: 12px;  
}  
  
h2 {  
    margin-bottom: 30px;  
    font-size: 28px;  
}  
}
```

```
.bannedMessage {  
    background-color: #D8C3A5;  
    border-radius: 10px;  
    padding: 30px;  
    text-align: center;  
    max-width: 500px;  
    margin: 0 auto;  
}
```

```
.bannedText {  
    font-size: 24px;  
    font-weight: bold;  
    color: #000000;  
    margin-bottom: 15px;
```

```
}
```

```
.bannedReason {  
    font-size: 16px;  
    color: #000000;  
    margin-bottom: 10px;  
}
```

```
.bannedUntil {  
    font-size: 14px;  
    color: #000000;  
    opacity: 0.7;  
    margin-bottom: 10px;  
}
```

```
.bannedPermanent {  
    font-size: 14px;  
    color: #000000;  
    font-weight: bold;  
    margin-bottom: 10px;  
}
```

```
.backButton {  
    background-color: #BE8F4C;  
    border: none;
```

```
border-radius: 8px;

padding: 10px 20px;

font-size: 14px;

font-weight: bold;

cursor: pointer;

color: #000000;

margin-top: 15px;

}
```

```
.backButton:hover {

    background-color: #a47a3f;

}
```

```
.loading {

    text-align: center;

    padding: 50px;

    font-size: 18px;

    color: #000000;

}
```

```
===== \pages\create\CreateMain.tsx =====
```

```
import React from 'react';

import { useParams } from 'react-router-dom';

import Header from '../components/common/Header/Header';

import Footer from '../components/common/Footer/Footer';

import CreateTestOrSurvey from '../components/common/Create/CreateTestOrSurvey';
```

```

const CreateMain: React.FC = () => {

  const { type } = useParams<{ type: string }>();

  const validType = type === 'survey' ? 'survey' : 'test';

  return (

    <div style={{ minHeight: '100vh', backgroundColor: '#EAE7DC' }}>

      <Header />

      <div style={{ maxWidth: '900px', margin: '0 auto', padding: '20px' }}>

        <CreateTestOrSurvey type={validType} />

      </div>

      <Footer />

    </div>

  );
};

```

```

export default CreateMain;

```

```

===== \pages\home\Home.module.css =====

```

```

.hideOnMobile {

  display: block;

}

```

```

@media (max-width: 768px) {

  .hideOnMobile {

    display: none;

  }

}

```

```

    }
}

===== \pages\home\Home.tsx =====

import React from 'react';

import Header from '../components/common/Header/Header';

import SearchSection from
'../components/common/Search/SearchSection/SearchSection';

import TestSection from '../components/common/Home/TestSection/TestSection'

import ActiveUsers from '../components/common/Home/ActiveUsers/UsersSection'

import Footer from '../components/common/Footer/Footer';

import styles from './Home.module.css';

const Home = () => {

  return (

    <div>

      <Header />

      <main>

        { /* Блок поиска */ }

        <div className={styles.hideOnMobile}>

          <SearchSection showTypeSelector={false} />

        </div>

        { /* Блок тесты - только тесты */ }

```

```

<TestSection title="ТЕСТЫ" type="recent" />

{ /* Блок опросы - только опросы */ }

<TestSection title="ОПРОСЫ" type="surveys" />

{ /* Блок авторы */ }

<ActiveUsers title="АКТИВНЫЕ ПОЛЬЗОВАТЕЛИ"/>


</main>


<Footer />

</div>

);

};


export default Home;

===== \pages\home\Home.types.ts =====

// Типы для главной страницы

export interface HeroSectionProps {

  title?: string;

  subtitle?: string;

  ctaText?: string;

  ctaLink?: string;

}

```

```

export interface FeatureItem {
  id: string;
  title: string;
  description: string;
  icon?: string; // Можно добавить иконки позже
}

export interface HomeProps {
  // Можно добавить пропсы, если нужно
}

===== \pages\login\Login.tsx =====

import React from 'react';
import Header from '../components/common/Header/Header';
import Footer from '../components/common/Footer/Footer';

import LoginMain from '../components/common/Auth/LoginMain'

const Register = () => {
  return (
    <div>
      <Header />
      <LoginMain />
      <Footer />
    </div>
  );
};

```



```
};
```

```
export default Register;
```

```
===== \pages\profile\Profile.tsx =====
```

```
import React, { useEffect, useState } from 'react';
```

```
import { useParams } from 'react-router-dom';
```

```
import Header from '../components/common/Header/Header';
```

```
import Footer from '../components/common/Footer/Footer';
```

```
import ProfileMain from '../components/common/Profile/ProfileMain';
```

```
import { authAPI } from '../services/api';
```

```
const Profile = () => {
```

```
  const { identifier } = useParams(); // Получаем ID или логин из URL
```

```
  const [isOwnProfile, setIsOwnProfile] = useState(true);
```

```
  const [loading, setLoading] = useState(true);
```

```
  const [profileUserId, setProfileUserId] = useState<number | null>(null);
```

```
  useEffect(() => {
```

```
    checkProfileOwner();
```

```
  }, [identifier]);
```

```
  const checkProfileOwner = async () => {
```

```
    try {
```

```
      // Получаем текущего пользователя
```

```
      const currentUser = await authAPI.getProfile();
```

```

if (!identifier) {

    // Нет identifier - это свой профиль

    setIsOwnProfile(true);

    setProfileUserId(currentUser.data.id);

} else {

    // Есть identifier - пытаемся найти пользователя

    try {

        const targetUser = await authAPI.getUserProfile(identifier);

        // Сравниваем ID

        setIsOwnProfile(currentUser.data.id === targetUser.data.id);

        setProfileUserId(targetUser.data.id);

    } catch (err) {

        // Пользователь не найден

        setIsOwnProfile(false);

        setProfileUserId(null);

    }

}

} catch (err) {

    console.error('Ошибка:', err);

} finally {

    setLoading(false);

}

};

```

```

if (loading) {
    return <div>Загрузка...</div>;
}

```

```

if (!profileUserId && identifier) {
    return (
        <div>
            <Header />
            <div style={{ textAlign: 'center', padding: '60px' }}>
                <h2>Пользователь не найден</h2>
            </div>
            <Footer />
        </div>
    );
}

```

```

return (
    <div>
        <Header />
        <ProfileMain isOwnProfile={isOwnProfile} userId={profileUserId} />
        <Footer />
    </div>
);
};

```

```
export default Profile;
```

```
===== \pages\register\Register.tsx =====
```

```
import React from 'react';
```

```
import Header from '../components/common/Header/Header';
```

```
import Footer from '../components/common/Footer/Footer';
```

```
import RegisterMain from '../components/common/Auth/RegisterMain';
```

```
const Register = () => {
```

```
  return (
```

```
    <div>
```

```
      <Header />
```

```
      <RegisterMain />
```

```
      <Footer />
```

```
    </div>
```

```
  );
```

```
};
```

```
export default Register;
```

```
===== \pages\test\questions\QuestionRouter.tsx =====
```

```
import React, { useState, useEffect } from 'react';
```

```
import { useParams, useLocation } from 'react-router-dom';
```

```
import { getQuestion } from '../api/tests';
```

```

import QuestionPage from '../../components/common/Question/QuestionPage';

import QuestionInputPage from
'../../components/common/Question/QuestionInputPage';

const QuestionRouter = () => {
  const { testId, questionIndex } = useParams();

  const location = useLocation();

  const [questionType, setQuestionType] = useState<'choice' | 'input'>('choice');
  const [loading, setLoading] = useState(true);
  const [timeLeft, setTimeLeft] = useState<number | null>(null);

  useEffect(() => {
    const state = location.state as any;

    if (state?.timeLeft) {
      setTimeLeft(state.timeLeft);
    }
  }, [location]);

  useEffect(() => {
    loadQuestionType();
  }, [testId, questionIndex]);

  const loadQuestionType = async () => {
    setLoading(true);

    try {

```

```

    const data = await getQuestion(Number(testId), Number(questionIndex));

    if (data.is_text_input) {

        setQuestionType('input');

    } else {

        setQuestionType('choice');

    }

} catch (error) {

    console.error('Ошибка загрузки типа вопроса:', error);

} finally {

    setLoading(false);

}

};

if (loading) {

    return <div className="loading">Загрузка...</div>;

}

if (questionType === 'input') {

    return <QuestionInputPage initialTimeLeft={timeLeft} />;

} else {

    return <QuestionPage initialTimeLeft={timeLeft} />;

}

};

export default QuestionRouter;

```

```

===== \pages\test\result\TestResultPage.tsx =====

import React from 'react';

import Header from '../../components/common/Header/Header';

import Footer from '../../components/common/Footer/Footer';


import TestResult from '../../components/common/Result/TestResult'


const TestPage = () => {

  return (

    <div>

      <Header />

      <TestResult />

      <Footer />

    </div>

  );

};


export default TestPage;

===== \pages\test\testpage\TestPage.tsx =====

import React from 'react';

import Header from '../../components/common/Header/Header';

import Footer from '../../components/common/Footer/Footer';


import TestMain from '../../components/common/Test/TestMain'

```

```

const TestPage = () => {
  return (
    <div>
      <Header />
      <TestMain />
      <Footer />
    </div>
  );
};

```

```

export default TestPage;

```

```

===== \pages\testlist\TestList.tsx =====

```

```

import React from 'react';
import Header from '../components/common/Header/Header';
import Footer from '../components/common/Footer/Footer';
import TestSection from '../components/common/Home/TestSection/TestSection'
import TestList from '../components/common/TestList/TestsList';

```

```

const Home = () => {
  return (
    <div>
      <Header />

      <TestList />

```



```

        <Footer />

    </div>

);

};

export default Home;

===== \pages\teststatspage\TestStatsPage.tsx =====

import React from 'react';

import Header from '../components/common/Header/Header';

import Footer from '../components/common/Footer/Footer';

import TestStatsPageMain from
'../components/common/TestStatsPage/TestStatsPageMain';

const TestStatsPage = () => {

    return (

        <div>

            <Header />

            <TestStatsPageMain />

            <Footer />

        </div>

    );

};

```

```

export default TestStatsPage;

===== \services\api.ts =====

import axios from 'axios';

const API_URL = 'http://localhost:8000/api';

const api = axios.create({
  baseURL: API_URL,
});

// Добавляем токен к каждому запросу
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('access_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// Обновление токена при 401
api.interceptors.response.use(
  (response) => response,
  async (error) => {
    const originalRequest = error.config;

```

```

    if (error.response?.status === 401 && !originalRequest._retry) {
      originalRequest._retry = true;
      try {
        const refresh = localStorage.getItem('refresh_token');
        const response = await axios.post(`${API_URL}/auth/refresh/`, { refresh });
        localStorage.setItem('access_token', response.data.access);
        originalRequest.headers.Authorization = `Bearer ${response.data.access}`;
        return api(originalRequest);
      } catch (e) {
        localStorage.clear();
        window.location.href = '/login';
      }
    }
    return Promise.reject(error);
  }
);

```

```

export const authAPI = {
  getProfile: () => api.get('/auth/profile/'),
  updateProfile: (data: any) => api.patch('/auth/profile/', data),
  getUserProfile: (userId: number) => api.get(`/auth/profile/${userId}/`),
  logout: (refresh: string) => api.post('/auth/logout/', { refresh }),
  updateLastSeen: () => api.post('/auth/update-last-seen/'),
};

```

```
export default api;
```

```
===== \style\variables.css =====
```

```
:root {  
  --color-background: #EAE7DC;  
  --color-blockColor: #D8C3A5;  
  --color-buttonColor: #BE8F4C;  
  --color-success: #28a745;  
  --color-danger: #dc3545;  
  --color-warning: #ffaa00;  
  --color-text: #000000;  
  --color-text-muted: #666;  
  --color-border: rgba(0, 0, 0, 0.1);  
  --color-border-dark: rgba(0, 0, 0, 0.2);  
  --color-avatar: #4C4C4C;  
  --color-hover: #a47a3f;  
  --color-overlay: rgba(0, 0, 0, 0.7);  
  --color-scrollbar-track: rgba(0, 0, 0, 0.1);  
  --color-scrollbar-thumb: rgba(0, 0, 0, 0.3);  
  --color-scrollbar-thumb-hover: rgba(0, 0, 0, 0.5);  
}
```

```
===== \types\test.ts =====
```

```
// Ответы
```

```
export interface Answer {  
  id: number;  
  text: string;
```

```
is_correct: boolean;
order_index: number;
}
```

```
export interface AnswerCreate {
  text: string;
  is_correct: boolean;
  order_index: number;
}
```

```
// Вопросы
```

```
export interface Question {
  id: number;
  text: string;
  author: number;
  author_info?: {
    id: number;
    login: string;
    full_name: string;
  };
  difficulty: number;
  explanation: string;
  usage_count: number;
  answers: Answer[];
  created_at: string;
```

```
    updated_at: string;
}
```

```
export interface QuestionCreate {
    text: string;
    difficulty: number;
    explanation: string;
    answers: AnswerCreate[];
}
```

// Пулы

```
export interface Pool {
    id: number;
    name: string;
    description: string;
    user: number;
    user_info?: {
        id: number;
        login: string;
        full_name: string;
    };
    questions_count: number;
    questions?: Question[];
    created_at: string;
    updated_at: string;
```

```
}
```

```
export interface PoolCreate {  
  name: string;  
  description: string;  
}
```

```
// Тесты и опросы
```

```
export interface Test {  
  id: number;  
  title: string;  
  description: string;  
  author: number;  
  author_info?: {  
    id: number;  
    login: string;  
    full_name: string;  
  };  
  is_open: boolean;  
  is_survey: boolean;  
  time_limit: number | null;  
  passing_score: number;  
  attempts_limit: number;  
  image: string | null;  
  topics: string[];
```

```
questions_count: number;

questions?: Question[];

created_at: string;

updated_at: string;

}
```

```
export interface TestCreate {

  title: string;

  description: string;

  is_open: boolean;

  is_survey: boolean;

  time_limit: number | null;

  passing_score: number;

  attempts_limit: number;

  image: string | null;

  topics: string[];

  question_ids: number[];

}
```

===== \types\user.ts =====

```
export interface User {

  id: number;

  login: string;

  email: string;

  status: string;

  created_at: string;
```



```
}
```

```
export interface LoginRequest {
```

```
  login: string;
```

```
  password: string;
```

```
}
```

```
export interface LoginResponse {
```

```
  access: string;
```

```
  refresh: string;
```

```
  user: User;
```

```
}
```

```
export interface RegisterRequest {
```

```
  login: string;
```

```
  email: string;
```

```
  password: string;
```

```
  password2: string;
```

```
}
```

```
export interface RegisterResponse {
```

```
  id: number;
```

```
  login: string;
```

```
  email: string;
```

```
  status: string;
```

```
    created_at: string;
}

export interface RefreshResponse {
    access: string;
    refresh?: string;
}

export interface User {
    id: number;
    login: string;
    email: string;
    full_name: string;
    bio: string;
    avatar: string | null;
    status: string;
    is_subscribe: boolean;
    created_at: string;
    last_seen: string;
    friends_count?: number;
}

export interface LoginResponse {
    access: string;
    refresh: string;
```

```
    user: User;
}
```

```
export interface RegisterResponse {
    message?: string;
    user?: User;
    access?: string;
    refresh?: string;
}
```

```
===== \utils\imageUtils.ts =====
```

```
// Конвертация изображения в base64
```

```
export const convertToBase64 = (file: File): Promise<string> => {
    return new Promise((resolve, reject) => {
        const reader = new FileReader();
        reader.readAsDataURL(file);
        reader.onload = () => resolve(reader.result as string);
        reader.onerror = error => reject(error);
    });
};
```

```
// Сжатие изображения
```

```
export const compressImage = (file: File, maxSizeMB: number = 0.5): Promise<string>
=> {
    return new Promise((resolve, reject) => {
        const reader = new FileReader();
```

```

reader.readAsDataURL(file);

reader.onload = (event) => {

  const img = new Image();

  img.src = event.target?.result as string;

  img.onload = () => {

    const canvas = document.createElement('canvas');

    let width = img.width;

    let height = img.height;


    const maxWidth = 500;

    const maxHeight = 500;


    if (width > height) {

      if (width > maxWidth) {

        height = (height * maxWidth) / width;

        width = maxWidth;

      }

    } else {

      if (height > maxHeight) {

        width = (width * maxHeight) / height;

        height = maxHeight;

      }

    }


    canvas.width = width;

```

```

    canvas.height = height;

    const ctx = canvas.getContext('2d');

    ctx?.drawImage(img, 0, 0, width, height);

    const base64 = canvas.toDataURL('image/jpeg', 0.7);

    resolve(base64);

  };

  img.onerror = reject;

};

reader.onerror = reject;

});

};

```

===== \App.css =====

```

* {

  margin: 0;

  padding: 0;

  box-sizing: border-box;

}

```

```

#root {

  width: 100%;

  min-height: 100vh;

}

```

```

.logo {

```

```
height: 6em;

padding: 1.5em;

will-change: filter;

transition: filter 300ms;
}

.logo:hover {

  filter: drop-shadow(0 0 2em #646cffaa);
}

.logo.react:hover {

  filter: drop-shadow(0 0 2em #61dafbaa);
}


@keyframes logo-spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}


@media (prefers-reduced-motion: no-preference) {
  a:nth-of-type(2) .logo {
    animation: logo-spin infinite 20s linear;
  }
}
```

```
}
```

```
.card {
```

```
padding: 2em;
```

```
}
```

```
.read-the-docs {
```

```
color: #888;
```

```
}
```

```
===== \App.tsx =====
```

```
import React from 'react';
```

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
```

```
import { isAuthenticated } from './api/auth';
```

```
import Home from './pages/home/Home';
```

```
import Register from './pages/register/Register';
```

```
import Login from './pages/login/Login';
```

```
import Profile from './pages/profile/Profile';
```

```
import Create from './pages/create/Create';
```

```
import CreateMain from './pages/create/CreateMain';
```

```
import TestPage from './pages/test/testpage/TestPage';
```

```
import AdminPanel from './pages/admin/AdminPanel';
```

```
import TestList from './pages/testlist/TestList';
```

```
import TestResult from './pages/test/result/TestResultPage';
```

```
import TestStatsPage from './pages/teststatspage/TestStatsPage';
```

```
import QuestionRouter from './pages/test/questions/QuestionRouter';
```

```
import './App.css';
```

```
const PrivateRoute = ({ children }: { children: React.ReactNode }) => {  
  return isAuthenticated() ? <>{children}</> : <Navigate to="/login" />;  
};
```

```
function App() {
```

```
  return (  
    <BrowserRouter>
```

```
    <Routes>
```

```
      <Route path="/" element={<Home />} />
```

```
      <Route path="/register" element={<Register />} />
```

```
      <Route path="/login" element={<Login />} />
```

```
      <Route path="/profile" element={<PrivateRoute><Profile  
/></PrivateRoute>} />
```

```
      <Route path="/profile/:identifier" element={<Profile />} />
```

```
      { /* Страница выбора типа */ }
```

```
      <Route path="/create" element={<PrivateRoute><Create /></PrivateRoute>} />
```

```
      { /* Страница создания теста/опроса с параметром типа */ }
```

```
      <Route path="/create/:type" element={<PrivateRoute><CreateMain  
/></PrivateRoute>} />
```



```

    <Route path="/test/:testId" element={<TestPage />} />

    <Route path="/survey/:testId" element={<TestPage />} />

    <Route path="/admin" element={<PrivateRoute><AdminPanel
/></PrivateRoute>} />

    <Route path="/testlist" element={<TestList />} />

    <Route path="/test/:testId/result" element={<TestResult />} />

    <Route path="/test/:testId/question/:questionIndex"
element={<QuestionRouter />} />


    <Route path="/test/:testId/stats" element={<TestStastPage />} />

  </Routes>

</BrowserRouter>

);

}

export default App;

===== \index.css =====

@import url('https://fonts.googleapis.com/css2?
family=Inter:wght@400;500;600;700&display=swap');

@import './style/variables.css';

* {

  font-family: 'Inter', sans-serif;

}

:root {

```

font-family: system-ui, Avenir, Helvetica, Arial, sans-serif;

line-height: 1.5;

font-weight: 400;

color-scheme: light dark;

color: rgba(255, 255, 255, 0.87);

background-color: #EAE7DC;

font-synthesis: none;

text-rendering: optimizeLegibility;

-webkit-font-smoothing: antialiased;

-moz-osx-font-smoothing: grayscale;

}

a {

font-weight: 500;

text-decoration: inherit;

}

body {

margin: 0;

display: flex;

background-color: var(--color-background);

place-items: center;

min-width: 320px;

```
    min-height: 100vh;
}

h1 {
    font-size: 3.2em;
    line-height: 1.1;
}

button {
    border-radius: 8px;
    border: 1px solid transparent;
    padding: 0.6em 1.2em;
    font-size: 1em;
    font-weight: 500;
    font-family: inherit;
    background-color: #1a1a1a;
    cursor: pointer;
    transition: border-color 0.25s;
}

button:hover {
    border-color: #646cff;
}

button:focus,
button:focus-visible {
    outline: 4px auto -webkit-focus-ring-color;
```

```
}
```

```
@media (prefers-color-scheme: light) {
```

```
  :root {
```

```
    color: #213547;
```

```
    background-color: #ffffff;
```

```
  }
```

```
  a:hover {
```

```
    color: #747bff;
```

```
  }
```

```
  button {
```

```
    background-color: #f9f9f9;
```

```
  }
```

```
}
```

```
===== \main.tsx =====
```

```
import { StrictMode } from 'react'
```

```
import { createRoot } from 'react-dom/client'
```

```
import './index.css'
```

```
import App from './App.tsx'
```

```
createRoot(document.getElementById('root')).render(
```

```
  <StrictMode>
```

```
    <App />
```

```
  </StrictMode>,
```

)

===== vite-env.d.ts =====

```
declare module '*.module.css' {  
  const classes: { [key: string]: string };  
  export default classes;  
}
```

```
declare module '*.module.scss' {  
  const classes: { [key: string]: string };  
  export default classes;  
}
```

```
declare module '*.module.sass' {  
  const classes: { [key: string]: string };  
  export default classes;  
}
```